Sentaurus™ Calibration Workbench User Guide

Version W-2024.09, September 2024

SYNOPSYS®

Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Conventions 12

Customer Support 12

Statement on Inclusivity and Diversity 13

1. Introduction to Sentaurus Calibration Workbench 15

Overview of Functionality 15

Basic Concepts and Terminology 16

Syntax Conventions 17

Running Sentaurus Calibration Workbench 18

Command-Line Options 19

2. Calibration Flows 20

Structure of a Calibration Flow 20

Calibration Modules 20

Setting Up a Calibration Flow 21

Calibration Input 22

Parameters 22

Calibration Parameters 22

Excluding and Including Parameters in Calibration Tasks 23

Attributes for Parameters 23

Targets 24

Scalar Targets 24

XY Profile Targets 24

Split Conditions 25

r^2 Targets 26

R^2 Targets 27

Delta Targets 27

Function Targets 28

Target Constraints 28

Excluding and Including Targets in Calibration Tasks 30

Setting a Weight for Targets 30

Setting an Error Method for Targets 30

Attributes for Targets 31

Parents . . . 32
Parent Projects . . . 32
Setting Options for Calibrations . . . 33
Defining the Parent Project and Simulation Tool Flow . . . 33
Defining Extra Tool Parameters . . . 34
Defining the Child Project Directory . . . 34
Child Project Splitting . . . 35
Setting the Simulation Job Queue . . . 35
Calculating the Total Weighted Root Mean Square Error . . . 35
Setting the Maximum Number of Iterations . . . 36
Setting the Number of Threads . . . 36
Performing Calibrations . . . 36
Simple Evaluation . . . 36
Gradient-Based Local Calibration . . . 37
Setting Controls for the Backend . . . 38
Terminating Calibration Loops . . . 38
Changing the Backend . . . 39
Design-of-Experiments Search . . . 39
Sensitivity Analysis . . . 40
Local Sensitivity Analysis . . . 40
Global Sensitivity Analysis . . . 41
Primary Parameters . . . 42
Setting Controls for Parameters . . . 42
Calibration Based on Machine-Learning Models . . . 42
Calibration by Exploring Machine-Learning Models . . . 43
Calibration by Refining Machine-Learning Models . . . 43
Validating Parameter Ranges . . . 44
Calibration Output . . . 45
Sentaurus Workbench Child Projects . . . 45
Calibrated Parameters . . . 45
Calibrated Targets . . . 46
Calibration History . . . 46
Calculating the Root Mean Square Error . . . 46

3. Sentaurus Calibration Workbench Machine-Learning Module . . . 48

Overview of Machine-Learning Models in Sentaurus Calibration Workbench . . . 48
Setting Parameters and Targets for ML Models . . . 49
Parameters . . . 50
Targets . . . 51

Contents

Function Targets . . . 51
Supported Special Characters . . . 52
Machine Learning Calibration Module: mlcal . . . 52
Modules to Be Loaded . . . 52
Setting up the Models . . . 52
Model Hyperparameters . . . 54
Model Type and Hyperparameters . . . 55
Parameter and Target Scaling . . . 57
Feature Interaction . . . 57
Feature Selection . . . 58
Hyperparameter Optimization . . . 60
Encoding XY Profile Targets . . . 61
Saving and Loading Models . . . 62
Custom Models . . . 63
General MLCal Task Use and Format . . . 63
Handling Conditions . . . 64
Reporting . . . 66
Plotting Data . . . 66
Generating Model Training/Test Data . . . 66
Using generate_data_ml With a Specific DoE . . . 67
Using generate_data_ml by Passing Experimental Parameters . . . 72
Data Generation Using create_ml . . . 73
Re-Running Data Generation . . . 73
Creating the Machine Learning Model . . . 73
Training the Machine Learning Model . . . 74
Data Trimming . . . 74
Using the Machine Learning Model . . . 75
predict_ml . . . 75
Reverse Search . . . 76
Additional Topics . . . 80
Validating Parameter Ranges . . . 80
Sensitivity Analysis and Primary Parameters . . . 80
Merge Tables . . . 82
Examples and Use Cases . . . 85
Basic Workflow . . . 85
Search for High Peak-Gain HBT Device Geometries . . . 86
Fitting for Etching Profile From Data File . . . 90
High-Level Machine Learning Functions . . . 92
explore_ml and refine_ml . . . 93
calibrate_ml . . . 95
References . . . 96

4. Modules for Calibrating Device Simulations 99

General Device Calibration: devcal Module 99

Hot-Carrier Stress Degradation Calibration 99

Input Data 100

Calibration Strategies 102

References 104

5. Modules for Calibrating Process Simulations 105

General Process Calibration: procal Module 105

Implantation Dual-Pearson Moments Calibration 105

Target 106

Parameters 106

Required Sentaurus Workbench Project Tool Database Setup 107

Required Sentaurus Workbench Variables 108

Calibration Strategy 108

Legendre Polynomial Implantation Model Calibration 109

Parameters 110

6. Modules for Calibrating Power Device Simulations 111

IGBT Device Calibration: igbtcal Module 111

IGBT DC Characteristics Calibration 113

IGBT AC Characteristics Calibration 113

7. Modules for Calibrating Topography Simulations 115

PMC Topography Calibration: pmccal Module 115

Reevaluation due to Variability 116

Sentaurus Topography Target Extraction 116

HARC Etching Calibration 117

Calibration Strategy by Gradient-Based Local Calibration Algorithm 118

Parameter Naming Schemes 118

Target Naming Schemes 119

Calibration Strategy by Machine Learning 120

Machine-Learning Model Exploration 121

Machine-Learning Model Refinement 122

Creating a User-Defined Machine-Learning Strategy 122

Validating Parameter Ranges . . . 124
Saving and Analyzing Machine-Learning Model Inference Data . . . 124

8. Graphical User Interface . . . 126

Features of the User Interface . . . 126
Menu Bar . . . 127
Projects Panel . . . 128
Explorer Panel . . . 131
Tables Panel . . . 132
Task Tab . . . 133
Parameters Tab . . . 135
Conditions Tab . . . 136
Targets Tab . . . 137
Function Targets Tab . . . 139
Models Tab . . . 140
Plots Panel . . . 143
Log Panel . . . 144
Customizing GUI Preferences . . . 145
Cleaning Up Projects and Workflows . . . 145

Setting Up a Calibration Flow . . . 146
Creating a Project and Selecting the Calibration Module . . . 147
Setting Up the Calibration Input . . . 148
Options . . . 148
Parameters . . . 148
Conditions . . . 148
Targets . . . 148
Function Targets . . . 149
Tasks . . . 149
Importing Data From Sentaurus Workbench Projects . . . 150
Converting Condition Parameters to Calibration Parameters, and Calibration Parameters to Condition Parameters . . . 152
Executing the Calibration Flow . . . 152

Plots . . . 153
Creating Curve Plots From Scalar Data . . . 153
Creating Curve Plots From Curve Data . . . 160
Creating Parallel Coordinate Plots . . . 167
Creating Quick Plots . . . 173
Creating Plots in Other Tools . . . 174

9. Advanced Image Analysis and Metrology (AImage) 175

Introduction to AImage 175
Overview of Functionality 175
Concepts 175
Features 176
Running AImage 177
Sentaurus Calibration Workbench 177
Command Line 178
Command Line Options 178

Introduction to Image Analysis 178
Image Intensity 178
Image Calibration 180
Image Segmentation 181
Contour Extraction 182

Features of the User Interface 182
Menus 183
File Menu 183
Image Menu 184
View Menu 186
Help Menu 187
Toolbars 187
File Toolbar 187
Measurement Toolbar 187
Intensity Toolbar 188
Binary Toolbar 188
Segmentation Toolbar 188
Objects Panel 188
Context Menu 189
Viewer Area 190
Interaction With Images and Segmentations 191
Interaction With Measurements 191
Interaction With Distance and Angle Objects 192
Interaction With Polygons 192
Interaction With Bounding Boxes 192
Properties Panel 193
Image Processing Panel 193
Developer Panel 194
Log 194
Script Record 194
Python Console 195

Image Processing With AImage 195
User Interaction Workflow 195
Working With Images 196
Working With Projects 197
Working With Measurements 197
Working With Segmentations 198
Image Metrology 199
Applying Image Algorithms 202
Image Segmentation 203
Manual Segmentation 203
Thresholding Segmentation 204
Segment Anything 205
Training Segmentation Models 206
Contour Extraction 208
Interaction With Sentaurus Calibration Workbench 209
Scripting 209
Scripting Example 209
Script With Arguments 210
References 211

A. Commands 212
Syntax Conventions 212
calculate_r2 214
calibrate_ac 215
calibrate_dc 217
calibrate_har_etch 219
calibrate_har_etch_ml 221
calibrate_hcsd_nmos 224
calibrate_hcsd_pmos 225
calibrate_implant_dualpearson 226
calibrate_implant_legendre 228
calibrate_ml 231
create_ml 233
determine_primary_parameters 237
determine_primary_parameters_ml 238

Contents

evaluate 239
explore_har_etch_ml 241
explore_ml 243
generate_data_ml 245
generate_pairplot 248
get_rms_error 249
get_sensitivity 250
get_step_signature 250
infer_ml_models. 251
load_parameters 252
merge_tables 253
Model 254
optimize 256
Parameter 257
Parent 259
plot_data 260
plot_error_history 262
plot_sensitivity 263
predict_ml 264
print_parameters 266
print_targets 267
refine_har_etch_ml 268
refine_ml 270
reverse_search_ml 271
run_sensitivity_analysis 275
run_sensitivity_analysis_ml 276
save_doe 277
save_parameters 278
save_sensitivity 278
search 279
set_backend_control 281

Contents

set_backend_method 283
set_option 284
set_step_signature 287
Target 288
TargetDelta 293
TargetFunction 294
TargetPenalty 296
Targetr2 297
TargetR2 298
train_ml 299
validate_range_ml 301

About This Guide

This user guide describes the Synopsys® Sentaurus™ Calibration Workbench tool.

For additional information, see:

- The TCAD Sentaurus release notes, available on the Synopsys SolvNetPlus site (see Accessing SolvNetPlus)
- Documentation available on the SolvNetPlus site

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Bold text	Identifies a selectable icon, button, menu, or tab. It also indicates the name of a field or an option.
`Courier font`	Identifies text that is displayed on the screen or that the user must enter. It identifies the names of files, directories, paths, parameters, keywords, and variables.
Italicized text	Used for emphasis, the titles of books and journals, and non-English words. It also identifies components of an equation or a formula, a placeholder, or an identifier.
Menu > **Command**	Indicates a menu command, for example, **File** > **New** (from the **File** menu, choose New).

Customer Support

Customer support is available through the SolvNetPlus site.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The site also gives you access to a wide

range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site:

1. Go to https://solvnetplus.synopsys.com.
2. Enter your user name and password. (If you do not have a Synopsys user name and password, follow the instructions to register.)

Contacting Synopsys Support

If you have problems, questions, or suggestions, contact Synopsys support in the following ways:

- Go to the Synopsys Global Support Centers site on www.synopsys.com to find email addresses and telephone numbers for Synopsys support centers throughout the world.
- Go to the SolvNetPlus site and open a case (Synopsys user name and password required).

Contacting Your Local TCAD Support Team Directly

Send an email message to:

- support-tcad-us@synopsys.com from within North America and South America
- support-tcad-eu@synopsys.com from within Europe
- support-tcad-ap@synopsys.com from within Asia Pacific (China, Taiwan, Singapore, Malaysia, India, Australia)
- support-tcad-kr@synopsys.com from Korea
- support-tcad-jp@synopsys.com from Japan

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our

software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction to Sentaurus Calibration Workbench

This chapter introduces the functionality of Sentaurus Calibration Workbench.

Overview of Functionality

Sentaurus Calibration Workbench is a tool designed to perform efficient calibrations for TCAD simulation tools. It provides the following functionality:

- *Evaluation* allows you to run the nominal or current parameter set for given target specifications, for validation of the simulation setup and extraction of the current accuracy.
- *Calibration* allows you to find an optimal parameter set for given target specifications. Multiple calibration targets can be combined, with weights specifying their relative importance. Several algorithms are available to solve calibration problems.
- *Sensitivity analysis* determines how small changes to a given parameter affect simulation responses and determines primary parameters.
- *Search* allows you to run different designs-of-experiments to explore the parameter space.
- *Machine learning* allows you to train models and use them to explore the parameter space to find an optimal parameter set for given target specifications.
- *Predefined or user-defined calibration strategies* including several calibration tasks can be applied to ensure efficient and reproducible calibration processes.

Sentaurus Calibration Workbench provides a Python environment where you can define and use custom algorithms that address specific tasks. All the components of Sentaurus Calibration Workbench are available in this environment and can be combined to develop a calibration strategy. For example, you can combine a sensitivity analysis with a calibration algorithm: through the sensitivity analysis, the primary parameters for calibration are found, and then only these are calibrated.

Sentaurus Calibration Workbench consists of a batch tool and a graphical user interface (GUI) tool that communicate with each other through a *facade*, which uses a database (MongoDB) for the communication.

Although Sentaurus Calibration Workbench is a standalone tool, it interacts with the batch tools of Sentaurus Workbench for setting up and running simulations (Sentaurus Workbench experiments). It can take advantage of the job scheduler of Sentaurus Workbench to speed up simulations by using distributed heterogeneous computing resources.

Sentaurus Calibration Workbench features are available in different tiers, as follows:

Feature	Base	Elite	Apex
Sentaurus Workbench integration: automatic project creation, job submission, and data exchange	X	X	X
Sensitivity analysis and calibration strategies based on traditional techniques; all functions that require machine learning are excluded	X	X	X
Machine-learning functions, including calibration strategies based on machine learning; machine-learning API is excluded		X	X
Machine-learning API that allows you to include custom-defined machine-learning models			X

Basic Concepts and Terminology

Common terms relevant to understanding Sentaurus Calibration Workbench are:

- A *parameter* is a scalar variable, usually defined in a Sentaurus Workbench project, that modifies the simulation flow. It has a finite value that is defined inside a certain domain. There are different parameter categories:
  - *Calibration parameters* are calibrated during the calibration process.
  - *Condition parameters* are not calibrated but define split conditions or states of experiments (for example, conditions in which the target data was obtained experimentally). Their values must not be modified while a calibration is running. However, depending on their purpose, you can vary *state parameters* so Sentaurus Calibration Workbench can calculate specific targets. For example, you can change the drain voltage to calculate separate I_d–V_g curves.
  - *Primary parameters* are defined as parameters with high sensitivity.
- A *response* is a variable or simulation output that describes, for example, device or circuit behavior. A response could be the error of the threshold voltage for the target device with respect to the required value.

- A *target* is a response to be matched against a reference by calibration. The reference could be based on measurements or calculations.
- *An experiment or a parameter set* is a tuple with one value for each calibration parameter and condition parameter that defines a particular instance of the simulation flow.
- *A family of simulations* is a set of split simulations that have the same set of calibration parameters but a different set of condition parameters.
- An *evaluation* is defined as all the simulations required to compute a given family of simulations.
- An *iteration* is an evaluation for a given calibration parameter set as part of a calibration task.
- A *task* is a specific action that Sentaurus Calibration Workbench performs. For example, a task could be a sensitivity analysis, a gradient-based local calibration of a set of parameters, or a machine-learning model generation. Tasks can consist of *subtasks*.
- A *calibration strategy* or *calibration flow* is a sequence of tasks used to obtain information about the relationship between the parameters and responses under consideration.
- A *step* or *calibration step* is a numbered task of a calibration strategy.
- A *trial* or *calibration trial* is the execution of a calibration strategy. If there is variability in the simulator results, then calibration trials can give different results as well.
- A *parent project* is a Sentaurus Workbench project that contains all of the tool steps to be included in the calibration, but not Sentaurus Calibration Workbench itself. Moreover, it includes the variable extractions that are linked to calibration targets. In general, it also includes all calibration parameters as Sentaurus Workbench parameters, with nominal values.
- A *child project* is a Sentaurus Workbench project that Sentaurus Calibration Workbench generates. A child project is based on a parent project and includes all experiments of the calibration.

Syntax Conventions

For details about the syntax conventions used for the Python commands in this user guide, see Syntax Conventions on page 212.

Running Sentaurus Calibration Workbench

Sentaurus Calibration Workbench consists of a batch tool and a GUI tool.

You can run Sentaurus Calibration Workbench in different ways:

- From within Sentaurus Workbench by introducing Sentaurus Calibration Workbench as a tool in the simulation flow of a Sentaurus Workbench project.

  You can open the Sentaurus Calibration Workbench GUI for the Sentaurus Calibration Workbench tool by right-clicking its tool icon and choosing **Edit Input** > **SCW GUI**.

  The Sentaurus Calibration Workbench Python command file or script can be edited for the Sentaurus Calibration Workbench tool by right-clicking its tool icon and choosing **Edit Input** > **SCW Script**. The Python script contains the description of the calibration process to be performed.

- From the command line, you can run the Sentaurus Calibration Workbench GUI by using the command:

  ```
  scwgui <options> <file>
  ```

  where `<options>` are command-line options (see Command-Line Options), and `<file>` is the name of a Sentaurus Calibration Workbench GUI project. All Sentaurus Calibration Workbench GUI project files are defined by the file extension `.scw`. For details about using the GUI, see Chapter 8 on page 126.

- From the command line, you can run the Sentaurus Calibration Workbench batch tool by using the command:

  ```
  scw <options> <inputfile>
  ```

  where `<options>` are command-line options (see Command-Line Options), and `<inputfile>` is a Sentaurus Calibration Workbench input file.

  The Sentaurus Calibration Workbench tool instance has one input file, which is a Python script describing the calibration setup. This script is described in Chapter 2 on page 20. Within a Sentaurus Workbench project, this file can contain Sentaurus Workbench preprocessor directives.

Command-Line Options

Table 1 lists the command-line options available for Sentaurus Calibration Workbench.

Table 1 Command-line options

Option	Description
`-h[elp]`	Displays the help message.
`-v[ersion]`	Displays version information.
For Sentaurus Calibration Workbench batch tool only	
`--max_threads <int>`	Sets the maximum number of threads to use.
`-node <int>`	Launches calibration for the given node.
`-prevnode <int>`	Specifies the previous node of the given node.
`-queue <string>`	Sets the default queue to use.
`--threads <int>`	Sets the number of threads to use.
`-verbose`	Displays additional initialization and loading information.
For Sentaurus Calibration Workbench GUI only	
`-b[atch]`	Runs export in batch mode (requires `-export`).
`-export <string>`	Specifies a script file to export.
`-mesa`	Runs Sentaurus Calibration Workbench GUI with the Mesa driver
`-queue <string>`	Specifies the queue to use to run jobs. Default: `local:default`

2

Calibration Flows

This chapter explains how to set up calibration flows.

Structure of a Calibration Flow

The calibration flow or strategy of Sentaurus Calibration Workbench is defined in its input command file or script and is the main object that links all relevant elements. A calibration flow usually requires the evaluation of several combinations of different parameter sets and acts in accordance to different response values. These evaluations are performed through an external simulation process invoked and coordinated by Sentaurus Calibration Workbench.

Multiple tasks can be performed in one strategy. Strategies use the evaluations and information obtained from different tasks and responses, and organize them to achieve the required result. For example, if a sensitivity analysis task selects some calibration parameters, then a strategy can instruct subsequent tasks to use only those parameters.

If many calibration parameters should be calibrated, then a sequence of calibration tasks, each of them for only a subset of the calibration parameters, is often a recommended strategy.

In summary, a calibration flow or strategy is a sequence of tasks used to obtain information about calibrated parameters for a given set of targets. Sentaurus Calibration Workbench allows the development of user-defined calibration strategies for different calibration problems.

Calibration Modules

A calibration flow is based on an imported Sentaurus Calibration Workbench calibration module. Two types of calibration module are provided:

- A *basic* calibration module provides a basic application programming interface (API), which allows you to set up your own calibration solutions for any application.

The available basic calibration modules are:

- Sentaurus Calibration Workbench module, described in this chapter
- Machine-learning module, described in Chapter 3 on page 48

- An *application-specific* calibration module is designed for a specific application and has predefined calibration strategies developed by calibration experts. These expert calibration strategies are contained in a function with a specific list of arguments and some prerequisites. They are included in application-specific calibration modules.

  The application-specific calibration module is described in Chapter 7 on page 115.

  The expert calibration strategy is described in HARC Etching Calibration on page 117.

Setting Up a Calibration Flow

First, you import a calibration module. For example:

```
from scw.apps.scwcal import SCWCal
```

This loads the basic Sentaurus Calibration Workbench module, described in this chapter.

Second, you instantiate a calibration object from the imported module class. For example:

```
cal = SCWCal()
```

Now, you can use this calibration object to manage the calibration flow.

For each calibration module, you then must define the calibration input (see Calibration Input on page 22) and the details for performing the calibration (see Performing Calibrations on page 36).

Some calibration modules also include predefined expert calibration strategies.

Finally, the calibration output can be further processed (see Calibration Output on page 45).

In general, the goal of calibration strategies is to minimize the root mean square (RMS) error of the target evaluations (see Calculating the Root Mean Square Error on page 46).

Note:

The Sentaurus Calibration Workbench input command file follows the syntax of Python scripts, which means the command file is case sensitive and indentation must be consistent. Comments start with the hash character (#) and can be appended to any line. The script is interpreted sequentially, so every variable must be defined before it is used. Apart from this, the order in which parameters, targets, and strategies are defined is arbitrary.

Calibration Input

The calibration input includes calibration parameters, targets, the simulation flow, and settings. The simulation flow is contained in a Sentaurus Workbench project and is called the *parent project*. The parent project as well as the settings are defined as options.

The calibration input is usually defined at the beginning of the calibration flow, but in principle, it can be defined at any position.

Moreover, parameters and targets can be included and excluded individually for each task of the calibration strategy. In addition, options can be set anytime and multiple times.

The following sections explain the parameters, targets, and options.

Parameters

Calibration parameters are represented by the `Parameter` class, which is imported automatically by:

```
from scw.core.parameter import Parameter
```

Calibration Parameters

You can add calibration parameters by using the following command:

```
p1 = Parameter(name=<s>, value=<f>, minimum=<f>, maximum=<f>,
               [transform=<o>])
```

A parameter is defined by a `name`, which must correspond to the name of a parameter as defined in the Sentaurus Workbench parent project (case sensitive) and the parameter in the simulator command file or parameter file. The Sentaurus Workbench parameter syntax (`@name@`) must be used for parameters in parameterized files.

Note:

If the parameter is not yet defined in the Sentaurus Workbench parent project, then it can be added implicitly (see Defining Extra Tool Parameters on page 34).

The initial value of the parameter is specified with `value`. The range of values that the parameter can take is specified by a lower value (`minimum`) and an upper value (`maximum`). You can optionally specify a transformation object for the parameter value (`transform`). By default, `transform=Linear`, but it can also be specified as `Log10`, `Log`, `Neg`, `Square`, or `Sqrt` (see Table 2). Transform classes can be combined using a tuple, and they are applied from right to left, for instance (`Log10`, `Neg`).

See Parameter on page 257.

Table 2 Available parameter transforms

Transform	Description
`Linear`	Applies a linear identity transformation to the parameter, $x_t = x$
`Log10`	Applies a base 10 logarithm to the parameter, $x_t = \log10(x)$
`Log`	Applies a natural logarithm to the parameter, $x_t = \log(x)$
`Neg`	Applies a negative transformation to the parameter, $x_t = -x$
`Square`	Applies a signed square transformation to the parameter, $x_t = \text{sign}(x) \cdot x*x$
`Sqrt`	Applies a signed square root transformation to the parameter, $x_t = \text{sign}(x) \cdot \sqrt{\lvert x \rvert}$

Excluding and Including Parameters in Calibration Tasks

When parameters are defined, they should be assigned to a Python variable. To be included in subsequent calibration tasks, the parameter (object) variable should be included in a parameter (object) list:

```
parameters = [p1]
```

Here, `parameters` is the name of the list.

Multiple parameters can be included in a parameter list:

```
parameters = [p1, p2, p3]
```

Parameters that are not included in that list retain their value in the calibration task. Therefore, if a parameter should *not* be calibrated in a task, then it should not be included in the parameter list of the subsequent calibration tasks.

Attributes for Parameters

You can retrieve and set the attributes of a `Parameter` object. The `Parameter` class has the following attributes:

- `name`: Name of a parameter as defined in the Sentaurus Workbench project (case sensitive)
- `value`: Initial value of the parameter
- `minimum`: Minimum value that the parameter can accept
- `maximum`: Maximum value that the parameter can accept
- `transform`: Transformation function applied to the parameter value

For example, to retrieve the current value of the parameter `par`, use the variable:

```
par.value
```

To update the current value, use the assignment:

```
par.value = <f>
```

Targets

Calibration targets are represented by the `Target` class, which is imported automatically by:

```
from scw.core.target import Target
```

Two different types of target can be defined: scalar targets and xy profile targets. See Target on page 288.

Scalar Targets

The most basic targets are scalar targets, which correspond to numeric entities that are extracted variables in Sentaurus Workbench.

An extracted variable in Sentaurus Workbench and its extracted numeric value appear in a simulator log file in the following expression on a new line:

```
DOE: <variable> <value>
```

For such extracted variables present in the parent project, a target is defined by:

```
t1 = Target(name=<s>, value=<f>)
```

where `name` is the name of a Sentaurus Workbench extracted variable of the scalar target, and `value` is the numeric goal value of the scalar target.

For example, for a topography calibration, an extracted variable of a critical dimension (CD) called `BottomZ` appears in the log file of Sentaurus Topography with a value of 3.4567 µm as follows:

```
DOE: BottomZ 3.4567
```

The corresponding target object named `t_BottomZ` for the extracted variable `BottomZ` with a goal value of 4 µm must then be defined by:

```
t_BottomZ = Target(name="BottomZ", value=4.0)
```

XY Profile Targets

The xy profile targets correspond to xy curves or profiles. For such xy profiles present in the parent project, a target is defined by:

```
t2 = Target(name=<s>, value=<s>, simulation_variables=<t>,
            reference_variables=<t>)
```

where `name` is the name of a Sentaurus Workbench extracted variable of the simulated output file of the profile target, and `value` is the data file including the reference target xy profile. The supported file formats are CSV, PLT, and PLX. The argument `simulation_variables` requires a list of two arguments, which specify the x- and y-variables of the simulation output profile. Similarly, the argument `reference_variables` requires a list of two arguments, which specify the x- and y-variables of the reference profile target.

Note:

Only xy data sorted in ascending x-axis order is supported.

For example, for a device calibration, the target named `t_IdVg` for an I_d–V_g curve in the file `IdVg.plt` with x-variable `Vg` and y-variable `Id`, which correspond to the Sentaurus Device PLT output file extracted to the variable `IdVgFile` and with x-variable `Gate OuterVoltage` and y-variable `Drain TotalCurrent`, can be defined by using:

```
t_IdVg = Target(name="IdVgFile", value="IdVg.plt",
       reference_variables=['Vg', 'Id'],
       simulation_variables=['Gate OuterVoltage', 'Drain TotalCurrent'])
```

Moreover, for a 1D process calibration, the target named `t_B_SIMS` for a single SIMS profile in the file `b_sims.plx`, which corresponds to the Sentaurus Process output PLX file extracted to the variable `BTotalFile` and with y-variable `BTotal`, can be defined by using:

```
t_B_SIMS = Target(name='BTotalFile', value='/path/b_sims.plx',
                  simulation_variables=['', 'BTotal'])
```

In this case, when `reference_variables` is not specified for a PLX file, the first dataset in the PLX file is used.

Note:

To create the extracted variable of the simulated output file of a profile target, you should write it explicitly to the log file by using a logging command in the simulator command file.

For Tcl input command files (for Sentaurus Process, Sentaurus Topography, and Sentaurus Device in Tcl mode), you can do this, for example, by specifying:

```
puts "DOE: PltFile n@node@_des.plt"
```

For typical Sentaurus Device command files, you can do this, for example, by specifying:

```
Solve {
   System("echo \"DOE: PltFile n@node@_des.plt\" ")
   ...
}
```

Split Conditions

If you define split conditions, that is, condition parameters that can have different values, then the target definition must be linked to specific conditions.

For both scalar and xy profile targets, you must specify the argument `conditions` as follows:

```
t1 = Target(name=<s>, value=<f>, conditions=<d>)

t2 = Target(name=<s>, value=<s>, conditions=<d>,
            simulation_variables=<t>, reference_variables=<t>)
```

The argument `conditions` specifies a dictionary of condition parameter names and their values.

For example, for a topography calibration with different etching times to be calibrated, the target `t_BottomZ` for a CD with the extracted variable name `BottomZ` has the goal value of 4 µm in the case of an etching time of 30 minutes. This must be specified by:

```
t_BottomZ = Target(name="BottomZ", value=4.0,
                   conditions={"time": "30"})
```

In this case, it is also recommended to specify a unique ID for the target, which is used as the identifier internally and for logging purposes. This is specified by:

```
t_BottomZ = Target(name="BottomZ", id="t30_BottomZ", value=4.0,
                   conditions={"time": "30"})
```

Moreover, for example, for a device calibration with different drain voltages and device types, the target `t_IdVg` for an I_d–V_g curve in the file `IdVg.plt` is for the device type `NMOS` and a drain voltage of 2.0 V. This must be specified by:

```
t_IdVg = Target(name="IdVgFile", value="IdVg.plt",
    conditions={'device': 'NMOS', 'Vd': '2,0'}
    reference_variables=['Vg', 'Id'],
    simulation_variables=['Gate OuterVoltage', 'Drain TotalCurrent'])
```

In these cases, the specified values for the condition parameters are used for the simulations corresponding to these targets. If no conditions are specified in the target definition, then the default values are used for the condition parameters. The default values are taken from the nominal experiment of the parent project. The nominal experiment of the parent project is specified by the `parent_experiment` argument (see Defining the Parent Project and Simulation Tool Flow on page 33).

r^2 Targets

The r^2 target is one type of post-evaluated target that you can define after specifying multiple scalar target objects. An r^2 target can be defined by:

```
t1_BottomZ = Target(name="BottomZ", value=2.0,
                    conditions={"time": "10"})
t2_BottomZ = Target(name="BottomZ", value=3.0,
                    conditions={"time": "20"})
```

```
t3_BottomZ = Target(name="BottomZ", value=4.0,
                    conditions={"time": "30"})

t_BottomZ_r2 = Targetr2(id="t_BottomZ_r2", value=1.0,
                        targets = [t1_BottomZ, t2_BottomZ, t3_BottomZ],
                        weight=1.0)
```

In this example, `Targetr2` has a list of targets as an argument. `Targetr2` calculates r^2 of the given targets, and the calculated r^2 is also considered during the calibration process. It can be useful if both the tolerance of each target and the correlation of targets are important.

r^2 is the square of the Pearson correlation coefficient (r), which is a linear correlation between the reference values and the evaluated values. (It is not a coefficient of determination.) In `Targetr2`, r^2 is defined by:
$r^2 = \frac{C_{1,2}|C_{1,2}|}{C_{1,1}C_{2,2}}$ where C is the covariance matrix Cov[X,Y].

R^2 Targets

The R^2 target is another post-evaluated target that represents the coefficient of determination. It is a measure that reflects the proportion of the variance in the evaluated values by the reference values. In simple linear regression, R^2 is equal to the square of the Pearson correlation coefficient (r^2). R^2 is defined by $R^2 = 1 – (SSR / SST)$, where SSR is the sum of squared residuals and SST is the total sum of squares.

This target can be defined by:

```
t_BottomZ_R2 = TargetR2(id="t_BottomZ_R2", value=1.0,
                        targets= [t1_BottomZ, t2_BottomZ, t3_BottomZ],
                        weight=1.0)
```

Delta Targets

Like *r2* and R2 targets, a delta target is also a calibration target that you can define after specifying multiple target objects. This delta target can be defined by:

```
t1_BottomZ = Target(name="BottomZ", value=2.0,
                    conditions={"time": "10"})
t2_BottomZ = Target(name="BottomZ", value=3.0,
                    conditions={"time": "20"})
t_BottomZ_Delta = TargetDelta(id="t_BottomZ_Delta", value=1.0,
                              targets= [t2_BottomZ, t1_BottomZ],
                              weight=2.0)
```

In this example, `TargetDelta` calculates the difference (delta) between the two evaluated values of the given targets and compares it to the reference difference specified by `value`. The difference can be calculated only from two scalar targets and not from profile targets.

Function Targets

Like r^2 targets, a function target is also a target that you can define after specifying multiple target objects. You must provide a post-evaluation function. This target can be defined by:

```
t1_BottomZ = Target(name="BottomZ", value=2.0,
                    conditions={"time": "10"})
t2_BottomZ = Target(name="BottomZ", value=3.0,
                    conditions={"time": "20"})
t3_BottomZ = Target(name="BottomZ", value=4.0,
                    conditions={"time": "30"})

def calculate_distance(targets):
  a = targets[0]
  b = targets[1]
  distance = a.eval - b.eval
  return distance

t1_BottomZ_Eval = TargetFunction(id="t1_BottomZ_Eval", value=1.0,
                                 targets=[t2_BottomZ, t1_BottomZ],
                                 transform=calculate_distance,
                                 weight=1.0)
t2_BottomZ_Eval = TargetFunction(id="t2_BottomZ_Eval", value=2.0,
                                 targets=[t3_BottomZ, t1_BottomZ],
                                 transform=calculate_distance,
                                 weight=2.0)
t3_BottomZ_Eval = TargetFunction(id="t3_BottomZ_Eval", value=1.0,
                                 targets=[t3_BottomZ, t2_BottomZ],
                                 transform=calculate_distance,
                                 weight=1.0)
```

In this example, the command `calculate_distance()` calculates the distance between the two values of the given targets, and the command is provided as a transformer of the three `TargetFunction`.

Like this example, you can define your own post-evaluation commands. In addition, the `TargetFunction` is processed and considered during the calibration process.

Target Constraints

In some situations, you might be interested in setting a target reference in a given range, but not with a specific scalar value. In such a scenario, you can supply a constraint to a calibration target. Constraints on targets can be applied through a penalty function or a constraint range.

In case of penalty functions, for which the constraint is violated, a penalty is added to the target, directing the calibration away from the value.

Penalties are defined using the `TargetPenalty` object. The signature of the constructor is:

```
p = TargetPenalty(filters=<t>, penalty=<c>, [ignore_value=<b>])
```

where:

- `filters` represents a list of conditions defining the constraint for the target
- `penalty` is a function that calculates a penalty, given the difference between the target and the threshold of the constraint
- `ignore_value [False]` specifies whether to ignore the actual value of the target, keeping only the penalty

This can be passed to any scalar target by using the `penalty` keyword. For example:

```
def quad_penalty(delta):
  return delta**2
p = TargetPenalty([("lt", 1.0), ("gt", 0.5)], quad_penalty)
t = Target("vt", penalty=p)
```

Here, the target range of the variable `vt` is between 0.5 and 1.0. If it is not satisfied, then a penalty is applied to the scalar target.

The condition given in `TargetPenalty` refers to the same target given in `OptTarget` in which it is used. For example, consider a case where you want to optimize two values of a threshold voltage, which are output in the Sentaurus Workbench variables `vt_low` and `vt_high`:

```
def quad_penalty(delta):
  return delta**2
p = TargetPenalty([("lt", 0.2), ("gt", -0.1)], quad_penalty)
t1 = OptTarget("vt_low", target=0.25, penalty=p)
t2 = OptTarget("vt_high", target=0.3, penalty=p)
```

This penalty definition penalizes simulation output values if they are greater than 0.2 or less than –0.1 with respect to their target value. In other words, if the value of `vt_low` is greater than 0.45 (0.25 + 0.2) or lesser than 0.15 (0.25 – 0.1), then the penalty is applied to it.

Similarly, if the value of `vt_high` is greater than 0.5 (0.3 + 0.2) or lesser than 0.2 (0.3 – 0.1), then the penalty is applied to it.

As alternative to penalty functions, hard constraints can be specified by the range argument, which can be passed to any scalar target instead of a single target value. For example:

```
t = Target("vt", range=[0.5, 1.0])
```

Targets with hard constraints can be passed to optimize or search tasks together with the regular targets.

Note:

Not all optimize backends support hard constraints. Only the `minimize` backend with the `"COBYLA"` or `"SLSQP"` method can be used.

Excluding and Including Targets in Calibration Tasks

When targets are defined, they should be assigned to a Python variable. To be included in subsequent calibration tasks, the target (object) variable should be included in a target (object) list:

```
targets = [t1]
```

Multiple targets can be included in a target list:

```
targets = [t1, t2, t3]
```

If a target should *not* be included in a task, then you can exclude it from the target list of the subsequent calibration tasks.

Setting a Weight for Targets

You can set several control options for a target when it is defined. For example, each calibration target can have a weight that is applied when aggregating different targets. The weight is specified as follows:

```
t1 = Target(name=<s>, value=<f>, weight=2.0)
```

In this case, the error of the given target is twice as important as the error of other targets without any weight specification, since the default weight is 1.0.

Setting an Error Method for Targets

Each target has an individual method to calculate the pointwise error between the simulation response and the reference value. The following command sets the error method:

```
t1 = Target(name=<s>, value=<f>, error_method=<s>)
```

For details about the available error methods and other arguments, see Target on page 288.

For example, for a device calibration with a target for an I_d–V_g curve, it can be useful to calibrate against the xy profile with a wide range of magnitude in logarithmic scale. For example:

```
t_IdVg = Target(name="IdVgFile", value="IdVg.plt",
     reference_variables=['Vg', 'Id'],
     simulation_variables=['Gate OuterVoltage', 'Drain TotalCurrent'],
     error_method="log_abs")
```

To restrict the window of a target profile for comparison, use the following command to define the lower and upper bounds of the x– and y–data range:

```
t2 = Target(name=<s>, value=<s>, conditions=<d>,
            simulation_variables=<t>, reference_variables=<t>,
            x_range=<t>, y_range=<t>)
```

For example, for a topography calibration with a target for an etch contour, where a relative error is used, it is necessary to exclude zero values of the reference target xy profile.

Moreover, the focus might be only on a part of the etch contour, for example, the negative values of the reference target xy profile. In this case, the ranges are specified in the following way:

```
XYBowingCD = Target(name='XYBowingCD', value='/reference.csv',
                    simulation_variables=['Z', 'Radius_Avg'],
                    reference_variables=['Z', 'Radius_Avg'],
                    error_method="rel",
                    x_range=[1e30, 0], y_range=[1e-10, 1e30])
```

Attributes for Targets

You can retrieve the following attributes of a `Target` object:

- `name`: Name of the Sentaurus Workbench parameter or variable in which the simulation response is available
- `value`: Value to be matched by the simulation response
- `id`: Unique name or ID of the target
- `type`: Type for target classification
- `dtype`: Datatype of the target
- `conditions`: Condition or split parameters that must be set to calculate the calibration targets
- `simulation_variables`: Specifies a tuple with the names of the independent (x) and dependent (y) variables (in that order) in the simulation file
- `reference_variables`: Specifies a tuple with the names of the independent (x) and dependent (y) variables (in that order) in the reference file

You can retrieve and set the following attributes of a `Target` object:

- `weight`: Weight to be applied to the calibration target
- `scale`: Scaling factor used to normalize the target value
- `transform`: Optional transform function to be applied to the target
- `penalty`: Penalty to be applied to the target if the constraints are violated
- `x_range`: Range of interest for the independent (x) variable
- `y_range`: Range of interest for the dependent (y) variable
- `x_scale`: Scaling factor used to multiply by the independent (x) variable
- `y_scale`: Scaling factor used to multiply by the dependent (y) variable

- `error_method`: Method used to calculate the pointwise error between the simulation response and the reference value

For example, to retrieve the current value of the target `tar`, use the variable:

```
tar.value
```

To update the current weight, use the assignment:

```
tar.weight = <f>
```

Parents

Sentaurus Workbench parent projects can be represented by the `Parent` class, which is imported automatically by:

```
from scw.core.parent import Parent
```

Parent Projects

You can add parent projects using the following command:

```
p1 = Parent(name=<s>, path=<s|o>, tools=<t>, [experiment=<i>,]
            [files_to_copy=<t>,] [extra_tool_parameters=<s|d>])
```

A parent is defined by a `name`, by a `path`, and by a the simulation tool flow `tools`. The simulation tool flow is specified as a list of tool labels. For example, for a flow with three tool labels, `sprocess`, `sdevice`, and `svisual`, it is defined by:

```
tools=["sprocess", "sdevice", "svisual"]
```

Only the relevant tool steps need to be included; all others are *bridged* to include their parameters, but they are not executed. However, the Bridge tool cannot be placed in between tools.

If the parent project consists of more than one experiment, then the nominal experiment must be specified by the `parent_experiment` argument. The nominal experiment defines all the condition values unless they are specified explicitly in the target definition (see Split Conditions on page 25).

After the creation of the child project, the command and preference files of the specified tool labels are copied from the parent project. A list of additional files and folders of the parent project to be copied to the child project (such as sourced files with target data or simulation parameters) is defined, for example, for the file `file.txt` and the folder `folder`, by:

```
files_to_copy=["file.txt", "folder"]
```

The file and folder names can include wildcards, for example, if all files with a specific file extension must be copied:

```
files_to_copy=["*.fps"]
```

In the case of selected calibration parameters that are not included in the parent project, they can be added to the child project by specifying the `extra_tool_parameters` argument, which specifies the tool labels to which missing Sentaurus Workbench parameters are added.

If it specifies a single string, then all parameters are added to this tool label. If it specifies a dictionary, with tool labels as keys and lists of parameter objects as values, then the parameters are added to individual tool labels.

Note:

If there is only one parent project and not multiple ones, there is no need to define a Parent object. In this case, you can specify the global parent project in the options (see Defining the Parent Project and Simulation Tool Flow on page 33).

Setting Options for Calibrations

You can define several optional calibration controls by using the `set_option()` command. Some controls are explained here. For details about this command and its arguments, see set_option on page 284.

Defining the Parent Project and Simulation Tool Flow

The global parent project, which includes the simulation flow, is defined by:

```
cal.set_option(parent="/path/of/parent/project")
```

If no parent project is specified for a particular calibration task, then the global parent project is used.

If the parent project consists of more than one experiment, then the nominal experiment must be specified by:

```
cal.set_option(parent_experiment=<i>)
```

The nominal experiment defines all the condition values unless they are specified explicitly in the target definition (see Split Conditions on page 25).

The simulation tool flow is specified as a list of tool labels. For example, for a flow with three tool labels, `sprocess`, `sdevice`, and `svisual`, it is defined by:

```
cal.set_option(tools=["sprocess", "sdevice", "svisual"])
```

Only the relevant tool steps need to be included; all others will be *bridged* to include their parameters, but they are not executed. However, the Bridge tool cannot be placed in between tools.

After creation of the child project, the command and preference files of the specified tool labels are copied from the parent project.

A list of additional files and folders of the parent project to be copied to the child project (such as sourced files with target data or simulation parameters) is defined, for example, for the file `file.txt` and the folder `folder`, by:

```
cal.set_option(files_to_copy=["file.txt", "folder"])
```

The file and folder names can include wildcards, for example, if all files with a specific file extension must be copied:

```
cal.set_option(files_to_copy=["*.fps"])
```

The list can also include files and folders outside of the parent project, for which the path should be specified:

```
cal.set_option(
   files_to_copy=["/path/of/file.txt", "/path/of/folder"])
```

Defining Extra Tool Parameters

In the case of selected calibration parameters that are not included in the parent project, they can be added to the child project by specifying:

```
cal.set_option(extra_tool_parameters=<s|d>)
```

The `extra_tool_parameters` argument specifies the tool labels to which missing Sentaurus Workbench parameters are added.

If it specifies a single string, then all parameters are added to this tool label. If it specifies a dictionary, with tool labels as keys and lists of parameter objects as values, then the parameters are added to individual tool labels.

This feature is usually used in combination with copying a calibration or parameter file, which is parameterized with the Sentaurus Workbench parameters to be added, for example, a Sentaurus Device parameter file `sdevice.par`, which includes model parameterization. This file must be copied by using the argument `files_to_copy` (see Defining the Parent Project and Simulation Tool Flow on page 33).

Defining the Child Project Directory

The default child project directory in which Sentaurus Calibration Workbench generates all child projects is defined by:

```
cal.set_option(child_dir="/path/to/child/project/directory")
```

If no child project directory is specified for a particular calibration task, then the default child directory is used.

Child projects are named after parent projects and are placed into child subdirectories following the name of the Sentaurus Calibration Workbench project. The final subdirectory will be named `<child_dir>/<scw_project>/<parent_name>`.

Child Project Splitting

For some calibration projects, the child projects can become very large (with more than 1000 experiments), which can lead to a slowdown in run time. Using several smaller child projects instead of one large, you can overcome this limitation. To split the child project automatically into smaller ones, you can enable child project splitting by number using the following option:

```
cal.set_option(split_child=True)
```

The maximum number of experiments per child project can be defined by:

```
cal.set_option(split_child_max_experiments=250)
```

In this case, the standard child project is not run but converted into a bookkeeper child project. Instead, several split child projects (with not more than 250 experiments) are generated and run.

Note:

The `split_child` option can only be set initially before the first step and cannot be changed once the child project is generated.

Setting the Simulation Job Queue

The job queue to which the simulation should be submitted is defined by:

```
cal.set_option(queue=<s>)
```

If it is not defined, then the default job queue is `"local:default"` or the one defined by the `-queue` command-line option.

Calculating the Total Weighted Root Mean Square Error

The way of calculating the total weighted RMS error is defined by:

```
cal.set_option(use_scalib_weights=<b>)
```

The total weighted RMS error for *N* individual target errors e_i and weights w_i is calculated as follows:

- If `use_scalib_weights=True`: $\mathrm{RMS} = \sqrt{\frac{\sum (w_i \times e_i)^2}{\sum (w_i)^2}}$
- If `use_scalib_weights=False`: $\mathrm{RMS} = \sqrt{\frac{\sum w_i \times e_i^2}{N}}$

Setting the Maximum Number of Iterations

For a gradient-based calibration algorithm, the maximum number of iterations per calibration task can be specified by:

```
cal.set_option(max_iteration=50)
```

Setting the Number of Threads

To set the number of threads for Sentaurus Calibration Workbench execution, specify the option:

```
cal.set_option(num_threads=1)
```

The number of threads cannot be greater than the maximum number of threads, which is defined by the Sentaurus Calibration Workbench command-line option `--max_threads`. If you define the command-line option `--threads`, then the `num_threads` option is ignored.

Performing Calibrations

To perform a calibration, you execute one or more calibration tasks, including:

- Simple Evaluation
- Gradient-Based Local Calibration
- Design-of-Experiments Search
- Sensitivity Analysis
- Calibration Based on Machine-Learning Models
- Calibration by Exploring Machine-Learning Models
- Calibration by Refining Machine-Learning Models
- Validating Parameter Ranges

In the case of a sequence of calibration tasks, each task is assigned a step signature. To retrieve the current calibration step or task signature, use the command:

```
cal.get_step_signature()
```

Simple Evaluation

To run a simple evaluation task for all included targets with the current parameter values without any calibration, use the command:

```
cal.evaluate(parameters=<t>, targets=<t>)
```

This command is required for an initial evaluation needed to register all nominal or initial parameter values and to validate the simulation setup as well as the initial accuracy. Moreover, the command is useful for an intermediate evaluation or a final evaluation in the calibration flow.

To set the global parent project and the child project directory of the calibration task, either use the optional arguments `parent` and `child_dir`, for example:

```
cal.evaluate(parameters=<t>, targets=<t>, parent=<s>, child_dir=<s>)
```

or use the command:

```
cal.set_option(parent=<s>, child_dir=<s>)
```

For details about these commands and their arguments, see evaluate on page 239 and set_option on page 284.

Gradient-Based Local Calibration

In the simplest situation, you have a set of parameters that you know is a good initial guess for the calibration process. You can then use a gradient-based local calibration algorithm (also known as *optimization*) to find the best set of parameters by calling:

```
cal.optimize(parameters=<t>, targets=<t>)
```

This command runs a calibration of all included parameters against all included targets. For details about how to determine the termination of calibration loops, see Terminating Calibration Loops on page 38.

When it is not possible to obtain a good calibration result for a single calibration task, multiple calls of `cal.optimize()` are possible as a sequence of calibration tasks. The following calibration task starts with the calibrated parameter values from the previous calibration task. You can change the included parameters and targets in between the calibration tasks.

To set the maximum number of iterations per calibration task, either use the optional argument `max_iteration`, for example:

```
cal.optimize(parameters=<t>, targets=<t>, max_iteration=50)
```

or use the command:

```
cal.set_option(max_iteration=50)
```

To set the global parent project and the child project directory of the calibration task, either use the optional arguments `parent` and `child_dir`, for example:

```
cal.optimize(parameters=<t>, targets=<t>, parent=<s>, child_dir=<s>)
```

or use the command:

```
cal.set_option(parent=<s>, child_dir=<s>)
```

For details about these commands and their arguments, see optimize on page 256 and set_option on page 284.

Setting Controls for the Backend

The default gradient-based local calibration algorithm is called `scalib`, which is a backend specific to Sentaurus Calibration Workbench and is based on the Levenberg–Marquardt algorithm.

Optionally, you can set controls for calibration parameters as follows:

```
cal.set_backend_control([initdx=<f>,] [mindx=<f>,] [maxdx=<f>])
```

These controls include `initdx` for the initial normalized parameter update for calibration, `mindx` for the minimum of the normalized parameter update, and `maxdx` for the maximum of the normalized parameter update.

In addition, you can optionally set calibration convergence criteria for the `scalib` backend as follows:

```
cal.set_backend_control([ssq_error_tolerance=<f>,]
                        [parameter_tolerance=<f>,]
                        [target_tolerance=<f>])
```

These controls for convergence criteria include `ssq_error_tolerance` for the relative change in the sum of squares (ssq), `parameter_tolerance` for the change in the norm of the parameter vector, and `target_tolerance` for the tolerance of the total target error (see Terminating Calibration Loops on page 38).

Terminating Calibration Loops

The termination of calibration loops for the `scalib` backend is determined, in order, by the following:

1. When the total weighted RMS error is less than `target_tolerance` (default: 0.0). See Setting Controls for the Backend on page 38.
2. When the ssq value of the target errors increases or the decremental ratio of the ssq value for targets is less than `ssq_error_tolerance` (default: 1e-5), and the ssq value of parameter variations is less than `parameter_tolerance` (default: 2e-3). See Setting Controls for the Backend on page 38.
3. When the looping number exceeds the maximum number (`max_iteration`) specified in the `optimize()` command or, if not present, in the `set_option()` command. The default is 50. However, for more challenging calibration problems, a larger `max_iteration` is recommended.

Changing the Backend

You can change to a different gradient-based local calibration algorithm, other than the default `"scalib"`, by using the following command:

```
cal.set_backend_method(name=<s>)
```

The options for `name` are the default `"scalib"`, the SciPy backends `"least_squares"`, `"minimize"`, and `"shgo"`, as well as `"mkl_nls"`, which is based on the Intel MKL bounded trust region implementation for least squares problems.

Optionally, you can set controls for the initial normalized parameter update for calibration as follows:

```
cal.set_backend_control(initdx=<f>)
```

In addition, you can optionally set the tolerance of the total target error as a calibration stopping criterion as follows:

```
cal.set_backend_control(target_tolerance=<f>)
```

In addition, you can optionally set the absolute tolerance as a calibration stopping criterion to be applied as the default to all stopping criteria for the backends `"least_squares"`, `"minimize"`, and `"mkl_nls"` as follows:

```
cal.set_backend_control(parameter_tolerance=<f>)
```

Alternatively, you can set backend-specific options by using:

```
cal.set_backend_control([method=<s>,] [jacobian_options=<d>,]
                        [options=<d>])
```

Here, the argument `method` sets the calibration method used by the backends, `jacobian_options` specifies a dictionary of keyword arguments passed to the Jacobian evaluation (overruling `initdx`), and `options` specifies a dictionary of backend-specific options (overruling `parameter_tolerance` and `max_iteration`).

For details, see *Sentaurus™ Workbench Optimization Framework User Guide*, Method-Specific Optimization Options.

Design-of-Experiments Search

To explore the parameter space, running and evaluating a design-of-experiments (DoE) can be helpful. You can use the following command to explore the input space of parameters and to have a better idea of how the target responses change with the value of the parameters:

```
cal.search(parameters=<t>, targets=<t>, method=<s>,
           options={"Ns": <i>})
```

Here, `method` defines the DoE method, and the option `Ns` defines the number of experiments in the DoE in addition to the current nominal parameter set.

To set the global parent project and the child project directory of the calibration task, either use the optional arguments `parent` and `child_dir`, for example:

```
cal.search(parameters=<t>, targets=<t>, method=<s>,
           options={"Ns": <i>},
           parent=<s>, child_dir=<s>)
```

or use the command:

```
cal.set_option(parent=<s>, child_dir=<s>)
```

For details about these commands and their arguments, see search on page 279 and set_option on page 284.

Sensitivity Analysis

Local and global sensitivity analysis is used to understand how the variability of parameters affects simulation results and accuracy.

To compute the sensitivity for a list of parameters and targets, specify:

```
cal.run_sensitivity_analysis(parameters=<t>, targets=<t>)
```

To set the global parent project and the child project directory of the calibration task, either use the optional arguments `parent` and `child_dir`, for example:

```
cal.run_sensitivity_analysis(parameters=<t>, targets=<t>,
                             parent=<s>, child_dir=<s>)
```

or use the command:

```
cal.set_option(parent=<s>, child_dir=<s>)
```

For details about these commands and their arguments, see the following sections and run_sensitivity_analysis on page 275 and set_option on page 284.

Local Sensitivity Analysis

In particular, one-parameter-at-a-time local sensitivity analysis aims at understanding the response of a model as a function of small changes to one parameter, with all other parameters fixed. Therefore, it reveals only the local gradient of the response surface of the model with respect to a given parameter (see *Sentaurus™ Workbench Optimization Framework User Guide*, Local Sensitivity Analysis).

The increment (positive ∂x) or decrement (negative ∂x) of the normalized parameter value to calculate the local sensitivity is defined by `sensdx` in the `set_backend_control()` command by default (see Setting Controls for Parameters on page 42).

The normalized parameter update can also be set directly by:

```
cal.run_sensitivity_analysis(parameters=<t>, targets=<t>,
                             percentage_range=<f>)
```

By default, the local sensitivity is computed bidirectionally for positive ∂x and negative ∂x, which corresponds to 3 points on the parameter axis. To compute the local sensitivity monodirectionally for positive ∂x only, specify the number of points explicitly:

```
cal.run_sensitivity_analysis(parameters=<t>, targets=<t>,
                             num_points=2)
```

Global Sensitivity Analysis

In contrast to local sensitivity analysis, global sensitivity analysis aims to summarize information about the entire domain, including interactions between parameters to some extent. The implemented methodology, the method of elementary effects (see *Sentaurus™ Workbench Optimization Framework User Guide*, Global Sensitivity Analysis), is based on aggregating data from multiple local one-parameter-at-a-time experimental designs, that is, trajectories.

Two parameters are required to define a global analysis: `num_levels` determines how many levels (namely, values) between the lower bound and the upper bound each parameter can take. The other parameter `num_trajectories` determines how many trajectories are executed.

To use global sensitivity analysis, you must specify `method="global"`. For example:

```
cal.run_sensitivity_analysis(parameters=<t>, targets=<t>,
                             method="global", num_levels=5,
                             num_trajectories=10)
```

Primary Parameters

Based on a computed sensitivity, you can determine the primary parameters by using:

```
primary_parameters = cal.determine_primary_parameters(parameters=<t>,
                                                      num_parameters=<i>,
                                                      min_sensitivity=<f>,
                                                      use_mean=<b>,
                                                      measure=<s>)
```

This command returns a list of only the primary parameters of the preceding sensitivity analysis for a subsequent calibration task, excluding secondary parameters.

You can limit the number of primary parameters by using the argument `num_parameters`, which is 6 by default.

By default, the sensitivity for the total RMS error is used but, optionally, the mean of the individual sensitivities per target can be selected by using the argument `use_mean`.

For local sensitivity analysis, the relative sensitivity is used by default and, for global sensitivity analysis, the measure μ* is used by default. Other sensitivity measures can be selected by using the argument `measure`.

For details about this command and its arguments, see determine_primary_parameters on page 237.

Setting Controls for Parameters

Optionally, you can set controls for the normalized parameter update for sensitivity analysis as follows:

```
cal.set_backend_control(sensdx=<f>)
```

Calibration Based on Machine-Learning Models

To perform a calibration strategy, based on machine learning, on a given set of parameters for a given set of targets, use the command:

```
cal.calibrate_ml(parameters=<t>, targets=<t>)
```

The machine learning–based calibration strategy consists of a sequence of machine-learning model exploration and refinement subtasks, and is designed for general calibration problems.

To set the global parent project and the child project directory of the calibration task, you can use the optional arguments `parent` and `child_dir`, for example:

```
cal.calibrate_ml(parameters=<t>, targets=<t>, parent=<s>, child_dir=<s>)
```

or use the command:

```
cal.set_option(parent=<s>, child_dir=<s>)
```

For details about these commands and their arguments, see calibrate_ml on page 231 and set_option on page 284.

For more information about low-level functions for machine learning, see Chapter 3, Sentaurus Calibration Workbench Machine-Learning Module.

Calibration by Exploring Machine-Learning Models

To execute an exploration of a machine-learning model on a given set of parameters for a given set of targets, use the command:

```
cal.explore_ml(parameters=<t>, targets=<t>, num_iterations=<i>)
```

The calibration parameters are calibrated based on the exploration algorithm, which works as follows:

1. A child project for machine-learning training is created and run.
2. Machine-learning models are created for each target and split condition. A machine-learning model database is populated.
3. The machine-learning database is used to predict calibration candidates. These candidates are simulated, and the best candidate is selected.
4. The exploration is repeated until the maximum number of iterations (`num_iterations`) is reached.

To set the global parent project and the child project directory of the calibration task, either use the optional arguments `parent` and `child_dir`, for example:

```
cal.explore_ml(parameters=<t>, targets=<t>, parent=<s>, child_dir=<s>)
```

or use the command:

```
cal.set_option(parent=<s>, child_dir=<s>)
```

For details about these commands and their arguments, see explore_ml on page 243 and set_option on page 284.

For more information about low-level functions for machine learning, see Chapter 3, Sentaurus Calibration Workbench Machine-Learning Module.

Calibration by Refining Machine-Learning Models

To execute a refinement of a machine-learning model on a given set of parameters for a given set of targets, use the command:

```
cal.refine_ml(parameters=<t>, targets=<t>, num_iterations=<i>)
```

The calibration parameters are calibrated based on the refinement algorithm, which works as follows:

1. A child project for machine-learning training is created and run.
2. A machine-learning model is trained on data defined around the current nominal value and is used to predict new candidates. These candidates are simulated, and the best candidate (experiment) is selected.
3. Several iterations are performed until the maximum number of iterations (`num_iterations`) is reached or the convergence criteria are met.

To set the global parent project and the child project directory of the calibration task, either use the optional arguments `parent` and `child_dir`, for example:

```
cal.refine_ml(parameters=<t>, targets=<t>, parent=<s>, child_dir=<s>)
```

or use the command:

```
cal.set_option(parent=<s>, child_dir=<s>)
```

For details about these commands and their arguments, see refine_ml on page 270 and set_option on page 284.

For more information about low-level functions for machine learning, see Chapter 3, Sentaurus Calibration Workbench Machine-Learning Module.

Validating Parameter Ranges

Parameter ranges are critical for the creation of a machine-learning model. If the parameter exploration space is too large, then the combination of parameters could lead to failures in simulations and, subsequently, to insufficient data for model training.

The following command executes the validation of the machine learning–based parameter range:

```
cal.validate_range_ml(parameters=<t>, targets=<t>, constraint=<s>)
```

The validation algorithm works as follows:

1. A sensitivity analysis is executed with the full parameter range. Sensitivity includes upper and lower parameter values.
2. The parameter range is decreased or increased if there is a failure in the simulations. If the failure occurs for the lower value, then the parameter lower value is increased. If the failure occurs for the upper value, then the parameter upper value is reduced.
3. Failed experiments include those that match the constraint condition specified by `constraint` or if the RMS error is above a certain threshold.

4. If the parameters have reached a range that is too close to the nominal or if the number of internal iterations is reached, then the algorithm stops.

The log file shows the resulting optimum, lower, and upper values for each parameter. If the value was changed by the validation range task, then the corresponding *previous* value will differ.

To set the global parent project and the child project directory of the calibration task, either use the optional arguments `parent` and `child_dir`, for example:

```
cal.validate_range_ml(parameters=<t>, targets=<t>,
                      parent=<s>, child_dir=<s>)
```

or use the command:

```
cal.set_option(parent=<s>, child_dir=<s>)
```

For details about these commands and their arguments, see set_option on page 284 and validate_range_ml on page 301.

Calibration Output

This section discusses the calibration output.

Sentaurus Workbench Child Projects

Upon execution, Sentaurus Calibration Workbench creates one or more separate Sentaurus Workbench projects that include only the tool instances that are necessary for the calibration, adding all other tool instances to a Bridge tool instance. This automatically created child project includes the experiments for each evaluation or iteration, and it is run in the background.

Note:

By default, child projects are not deleted when cleaning up the Sentaurus Calibration Workbench node or output project in Sentaurus Workbench. If the child project is already present and consistent with the current run, then it will be reused including the simulation results if present.

Calibrated Parameters

The resulting calibration parameter values can be found in the log file of Sentaurus Calibration Workbench. In addition, you can export them to the Sentaurus Workbench variables table or to parameter files.

The following command writes the calibrated parameters to the log file in the syntax for Sentaurus Workbench variable extraction:

```
cal.print_parameters(parameters=<t>, [suffix=<s>])
```

The optional argument `suffix`, which is `_cal` by default, specifies the suffix added to parameter names for Sentaurus Workbench variable extraction, in order to differentiate them from Sentaurus Workbench parameter names. See print_parameters on page 266.

Calibrated Targets

The log file of Sentaurus Calibration Workbench contains the resulting target evaluation values and RMS errors. In addition, you can export them to the Sentaurus Workbench variables table or to parameter files.

The following command writes the calibrated target values to the log file in the syntax for Sentaurus Workbench variable extraction:

```
cal.print_targets(targets=<t>, [suffix=<s>,] [include_path=<b>])
```

The optional argument `suffix`, which is `_cal` by default, specifies the suffix added to parameter names for Sentaurus Workbench variable extraction, in order to differentiate them from preexisting Sentaurus Workbench variable names. Finally, in the case of files as target values, the full path of that file can be included by setting the `include_path` argument to `True`. See print_targets on page 267.

Calibration History

To create a history plot named `error.png`, which includes the error history of a calibration, use the following command *after* the calibration tasks:

```
cal.plot_error_history(file="error.png")
```

If you also want to include the individual target errors in the history plot, then include the target list in the command:

```
cal.plot_error_history(file="error.png", targets=<t>)
```

See plot_error_history on page 262.

Calculating the Root Mean Square Error

Sentaurus Calibration Workbench calculates the RMS error between each evaluated value and target value. For details about the calculation of the target RMS error, see Targets on page 24.

In addition, the total weighted RMS error is calculated, taking into account the weights of each target.

RMS errors are printed to the log file for each evaluation. In addition, RMS errors can be accessed for each target by specifying:

```
cal.get_rms_error(target=<s>)
```

To retrieve the current total weighted RMS error of all targets, use the command without any arguments:

```
cal.get_rms_error()
```

See get_rms_error on page 249.

3

Sentaurus Calibration Workbench Machine-Learning Module

This chapter describes how to use the Sentaurus Calibration Workbench machine-learning module to create and train machine-learning models.

Overview of Machine-Learning Models in Sentaurus Calibration Workbench

A model in Sentaurus Calibration Workbench takes a set of parameters and conditions as inputs and predicts the values of responses (target variables) as outputs. At a high level, working with almost any model consists of five steps:

1. Generate/Collect data.
2. Define/Create the model.
3. Train/Fit the model to the data.
4. Use the model to make predictions.
5. (Optional) Check predictions.

While there may be iterations and/or loops between these different steps (and their boundaries are not always clear), they generally hold whether you are using a linear best fit or a neural network for machine learning (ML).

Sentaurus Calibration Workbench contains capabilities to perform each of these steps, both in individual tasks that perform one or a small set of the steps listed, or in high-level ML tasks which perform all the listed steps automatically. For Sentaurus Calibration Workbench, granular control of the models and the settings for each of these steps is possible only in the `mlcal` module. High-level ML tasks can be found in all modules.

In general, the high-level ML tasks require less steps and settings to be specified but have limited customizability. For the individual tasks, more granular control is possible, but it is more time consuming to set up and requires a greater machine-learning background.

Table 3 Primary mlcal tasks for granular control of machine-learning and associated process type

Function/Task	Generate Data	Create	Train/Fit	Use	Check
`generate_data_ml`	•				•
`create_ml`	•	•			
`train_ml`			•		
`predict_ml`				•	
`reverse_search_ml`				•	

Table 4 High-level ML tasks and associated Sentaurus Calibration Workbench modules

Function/Task	SCW Module
`explore_ml`	All
`refine_ml`	All
`calibrate_ml`	All
`explore_har_etch_ml`	`pmccal`
`refine_har_etch_ml`	`pmccal`
`calibrate_har_etch_ml`	`pmccal`

In the following sections, you will learn how to set up parameters and targets for the machine-learning tasks, how to use the `mlcal` module, and then how to use the high-level ML tasks.

Setting Parameters and Targets for ML Models

The machine learning models in Sentaurus Calibration Workbench also use the Parameter and Target objects described in Chapter 2 on page 20. Parameter objects set the parameters to be varied during calibration/optimization and Target objects specify calibration/optimization goals. Using parameters and targets with machine learning models

in Sentaurus Calibration Workbench is largely the same as using the parameters and targets in the `scwcal` module.

Parameters

Parameters to be included as machine-learning model inputs are defined by:

```
param_obj = Parameter(name=<s>, value=<s>|<i>|<f>,
                      [minimum=<i>|<f>, maximum=<i>|<f>,]
                      [transform=<o>|None,] [choices=<t>,]
                      [sort=<b>,] [dtype=<k>,] [force_pass=<b>,]
                      [is_extracted=<b>])
```

This is the same as for the `scwcal` module (see Parameters on page 22), but with some additional possible arguments:

- `choices` is a list of valid values for a discrete parameter. A minimum and maximum is not required when a parameter is defined using this argument.
- `sort` specifies whether to sort the discrete values before passing them to the machine-learning model.
- `dtype` sets the Python data type of the parameter. Valid options include `float` (general default), `bool`, `int`, or `str` (default for conditions in Sentaurus Calibration Workbench GUI).
- `force_pass` specifies whether, during data sampling, instead of using the DoE method, the experiments are forced to pass every value given by choices for this parameter. `force_pass` parameters are sometimes referred to as *conditions*.

The use of `force_pass` is best explained through an example. Consider a project with `temperature` and `time` as discrete parameters defined by choices. The parameter `time` can be either 10 or 30, and `temperature` can be either 300 or 400. If you want to explore all combinations of these two parameters, you can define the parameters as:

```
p_time = Parameter(name="time", value="10", choices=["10", "30"],
                   dtype=str)
p_temp = Parameter(name="temperature", value="300",
                   choices=["300", "400"], dtype=str)
```

In such a case, these are treated as two independent parameters. However, if we only wanted to consider the combinations of:

- time=10, temperature=300
- time=10, temperature=400
- time=30, temperature=300

where the combination of `time=30`, `temperature=400` is not required, then you can set `force_pass=True` and `choices` to match the desired sequence:

```
p_time = Parameter(name="time", value="10",
                   choices=["10", "10", "30"], dtype=str,
                   force_pass=True)
p_temp = Parameter(name="temperature", value="300",
                   choices=["300", "400", "300"], dtype=str,
                   force_pass=True)
```

This passes each pair of choices together in index order.

Parameters are passed to the main machine-learning object during its initialization. At data generation, the non-`force_pass` parameters are sampled using a DoE method (Box–Behnken or random sampling, for example), while each of the generated combinations of parameter values takes all three time–temperature condition splits together as simulation inputs. See **Using generate_data_ml With a Specific DoE on page 67** for more details on how `force_pass` impacts data generation.

Please see Parameter on page 257 for more details.

Targets

Targets, the output of the simulation, can be scalar values or a set of values representing XY curves. When using machine learning models, targets are defined by:

```
target_obj = Target(name=<s>, values=<t>, dimension=<i>,
                    [conditions=<d>,] [simulation_variables=<t>,]
                    [reference_variables=<t>,] [error_method=<s>,]
                    [range=<t>,] [abs_value_range=<t>,]
                    [x_range=<t>,] [y_range=<t>,] [weight=<f>,]
                    [tolerance=<f>,] [penalty=<o>,] [id=<s>,]
                    [dtype=<k>])
```

Most of these arguments are the same as those in the `scwcal` module (seeAttributes for Targets on page 31). The following arguments are unique to the `mlcal` module:

- `dimension` must be set to `0` for scalar targets and to `2` for XY curve targets.
- `tolerance` is used by adaptive reverse search. Solutions in which all targets are within tolerance are preferred to other solutions.

See Target on page 288 for more details.

Function Targets

Definition of function targets is same as the description in Function Targets on page 51 in Chapter 2. However, in `mlcal`, only `reverse_search_ml` and subsequent

`generate_data_ml` support the function targets. Other tasks, such as `create_ml`, `train_ml` and `predict_ml` ignore the function targets if provided and display a warning.

Supported Special Characters

When defining the names of parameters and targets, the special characters “-” and “_” are allowed. Other special characters are not supported.

Machine Learning Calibration Module: mlcal

The core machine learning module of Sentaurus Calibration Workbench is `mlcal`. While high-level ML tasks exist in all Sentaurus Calibration Workbench modules, granular control of the machine-learning models and their use can only be found in `mlcal`. The `mlcal` module extends the basic Sentaurus Calibration Workbench module `scwcal`. That is, it contains all the tasks included in `scwcal`, plus additional ones unique to `mlcal`.

Because it is intended to allow fine control in terms of machine learning model creation and setup, the intended user of `mlcal` is one who has at least some familiarity with machine learning and machine learning models.

Modules to Be Loaded

The following modules should be loaded when using `mlcal`:

```
from scw.core.parameter import Parameter
from scw.core.target import Target
from scw.apps.mlcal import MLCal
from scw.core.model import Model
from scw.core.scwglobal import *
```

An instance of an `mlcal` application can be created with:

```
mlcal_obj = MLCal()
```

The `MLCal` object has the same options as `SCWCal` object of the `scwcal` module for many items such as parent project, child project directory, queue, and so on. For more information, see Setting Options for Calibrations on page 33 and set_option on page 284.

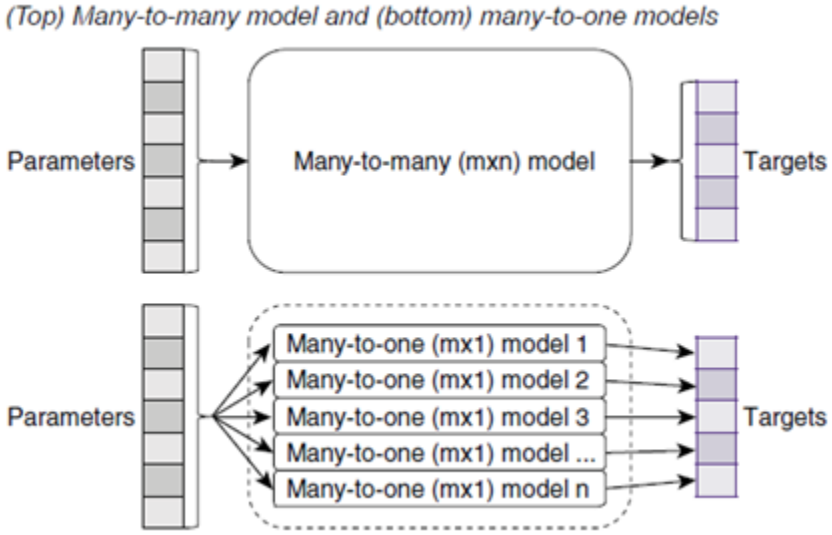
Setting up the Models

In Sentaurus Calibration Workbench, a single `Model` object maps input parameters to targets. These come in two possible configurations: many-parameters-to-many-targets (MxN) or many-parameters-to-one-target (MX1). In the former (MxN), a single specific

machine learning model is used to predict a group of targets based on a set of input parameters. In the latter (Mx1), each target has its own machine learning model associated with it and this model predicts this target value based on the inputs. In the Mx1 configuration, this entire collection of ML models is a single `Model` object in Sentaurus Calibration Workbench.

Note: It can sometimes be confusing as to which “model” is being referred to, the Sentaurus Calibration Workbench `Model` object or the machine learning model for a specific target. In general, `mlcal` functions take the `Model` object (or list of `Model` objects) as an input.

Figure 1 Sentaurus Calibration Workbench machine-learning model configurations



Instantiating a new `Model` object for a given target and the hyperparameters associated with the contained machine learning model(s) is relatively simple:

```
ml_0 = Model(name="ml_0", hyperparameters=hyperparameters)
```

where “hyperparameters” is a dictionary containing all the model details. The setup of this dictionary is covered in the following section. The configuration is set with the `framework` argument. An example of setting the framework to MxN is:

```
ml_0 = Model(name="ml_0", hyperparameters=hyperparameters,
             framework=MxN)
```

By default, `framework=Mx1`.

Model Hyperparameters

Hyperparameters are the parameters that define the specifics of the machine learning models (as opposed to the input “parameters”). They must be declared as a dictionary:

```
hyperparameters = dict()
```

For an Mx1 configuration, hyperparameters must be defined for each target, for example:

```
hyperparameters["Y0"] = {"model_type": MLPR, … }
hyperparameters["Y1"] = {"model_type": ETR, … }
```

For an MxN configuration, hyperparameters are defined for a single model that predicts all targets:

```
hyperparameters["All"] = {"model_type": MLPR, … }
```

All models have the following possible hyperparameters:

- `model_type (keyword)`: Specifies the machine learning model type
- `model_hyperparameters (dict)`: Specifies the hyperparameters for the machine learning model selected in `model_type`. These hyperparameters are unique to each model.
- `parameter_scaler (keyword)`: Specifies the scaling to apply to input parameters
- `target_scaler (keyword)`: Specifies the scaling to apply to output targets
- `feature_interaction_type (keyword)`: Specifies the type of feature interaction terms to be considered
- `feature_interaction_params (dict)`: Specifies the feature interaction settings
- `feature_selection_type (keyword)`: Specifies the type of feature selection algorithm to use
- `feature_selection_parameters (dict)`: Specifies the feature selection settings
- `retrain (bool)`: Specifies whether to retrain the model every time `train_ml` is called
- `encoder_type (keyword)`: Specifies the type of encoder to use for profile targets
- `encoder_params (dict)`: Specifies the profile encoder settings
- `optimizer_type (keyword)`: Specifies the optimizer to use when optimizing the `model_hyperparameters`
- `optimizer_params (dict)`: Specifies the hyperparameter optimizer settings

Each of these settings is covered in more detail in later sections. Most of these are required only if the use of that particular feature is needed. For example, if feature selection is not desired, then the feature selection entries can be omitted from the dictionary.

Model Type and Hyperparameters

A variety of different machine learning models have been included in Sentaurus Calibration Workbench and can be specified by keyword. For example, if you want to specify the multilayer perceptron regressor model for target `Y0`, then the hyperparameter setting is:

```
hyperparameters["Y0"] = {"model_type": MLPR, … }
```

To specify the hyperparameters of the model, the `model_hyperparameter` dictionary must be set:

```
hyperparameters["Y0"] = {"model_type": MLPR,
                          "model_hyperparameters":
                         {"hidden_layer_sizes": [64, 64, 64],
                          …}, …}
```

The keywords and associated models can be found in the following table. Most of these models can be found in the scikit-learn library [1]. As of right now, only regression models can be used. The model_hyperparameters vary significantly between model types. Please check the associated reference in the table to look-up the hyperparameters associated with each model.

Table 5 Hyperparameter model types

Keyword	Regressor	Reference
ANNR	Artificial neural network regressor (Pytorch version of multilayer perceptron regressor)	See below
BgGBR	Bagging regressor with gradient boosting regressor	[2][3]
BgHGBR	Bagging regressor with histogram-based gradient boosting regressor	[2][4]
ETR	Extra trees regressor	[5]
GBR	Gradient boosting regressor	[3]
GPR	Gaussian process regressor	[6]
HGBR	Histogram-based gradient boosting regressor	[4]
Linear	Linear regressor	[7]
MLPR	Multilayer perceptron regressor	[8]
RFR	Random forest regressor	[9]
Quadratic	Quadratic regressor	[7][10]

The information about the possible hyperparameters for ANNR is given in the following list:

- `activation_function`: Activation function for hidden layers (Default `"relu"`)
- `activation_func_args`: Arguments for activation functions.
- `batch_size`: Batch size (Default `100`)
- `batch_normalization`: Batch normalization function
- `batch_norm_args`: Arguments for batch normalization
- `dropout`: Dropout for hidden layers (Default `0.0`)
- `hidden_layer_sizes`: Size of hidden layers. (Default `(16, 16)`)
- `kept_model`: The model to be kept. `"best_validated"` means the model which has the best validation score. `"best_trained"` means the model which has the best training score. `"last_trained"` means the model at the end of the training iteration. (Default `"best_validated"`)
- `learning_rate`: Initial learning rate (Default `1e-3`)
- `loss_function`: Loss function (Default `"mse"`)
- `loss_func_args`: Arguments for `loss_function`.
- `model_filename`: Model file name.
- `model`: Model name to be loaded
- `monotonic_layers`: List of bool to apply monotonic layers. For the first layer, instead of a single bool, a list of bool is available to control components individually. Number of layers should be number of hidden layers + 1. For example, [[False, True, True], True, True]
- `num_epochs`: Number of epoch (Default `100`)
- `num_threads`: Number of threads
- `num_workers`: Number of subprocesses to use for data loading. `0` means that the data is loaded in the main process. (Default `0`)
- `optimizer`: Optimizer (Default `"adam"`)
- `opt_args`: Arguments for optimizer
- `patience`: Patience for early stop
- `random_state`: Random state to split validation set (Default `42`)
- `report_filename`: Report file name

- `report_interval`: Report interval of epoch (Default `100`)
- `set_flush_denormal`: Denormal floating numbers on CPU. If `True`, 1e-323 become 0. If `False`, 1e-323 become 1e-323. (Default `True`)
- `validation_fraction`: Validation fraction (Default `0.1`)
- `verbose_level`: Verbose level (Default `1`)
- `warm_start`: If `True`, warm starting (transfer learning) is applied. (Default `False`)
- `weight_decay`: Weight decay (L2 regularization) (Default `1e-4`)
- `weight_initialization`: Weight initialization function
- `weight_init_args`: Arguments for `weight_initialization`.

Parameter and Target Scaling

To improve training speed and quality of a machine learning model, it is usually better to scale the parameters or targets to values closer to unity. The scaling methods available in Sentaurus Calibration Workbench are shown in Table 6. The default value of this hyperparameter is Standard.

Table 6 Supported parameter and target scalers

Method	Description
`Identity`	No scaling applied
`Standard`	Standard scaler
`MinMax`	Min-max scaler
`Quantile`	Quantile transformer
`Robust`	Robust scaler

As an example, standard scaling of the input parameters is set in the hyperparameters by:

```
hyperparameters["Y0"] = {…, "parameter_scaler": Standard, …}
```

Feature Interaction

Feature interaction provides a method of increasing input parameters by including the interaction terms between different input parameters. Currently, only polynomial features[10] are supported. They can be added to the model hyperparameters using:

```
hyperparameters["Y0"] = {…, "feature_interaction_type":
                         PolynomialFeatures,
                         "feature_interaction_params": {…},
```

```
…}
```

where `"feature_interaction_params"` is a dictionary that specifies the settings for the feature interaction. The possible dictionary terms are the following:

- `degree`: Specifies the degree for the polynomial feature interaction (default: `2`).
- `create_only_interaction`: Specifies whether interaction features are generated and terms with the power of degree of the same input feature are excluded. If set to `True`, then only interaction features are generated and terms with the power of degree of the same input feature are excluded (default: `False`).
- `include_bias`: Specifies whether to generate bias terms as new features (default: `False`).
- `scale`: Specifies the scale for new polynomial features (default: `Linear`).
- `save_new_features`: Specifies whether to save a new data frame for input parameters as a CSV file (default: `False`).
- `csv_filename`: Sets the file name for saving the data set as a CSV file (default: `"polynomialfeatures_dataset"`).
- `check_correlations`: Specifies whether to analyze the correlation between parameters using a heat map (default: `False`).
- `correlation_image_filename`: Sets the file name for saving an image of feature correlations as a PNG file (default: `"feature_correlations"`).
- `outfile_prefix`: Sets the prefix of the output CSV file and image file (default: `"n@node@_"`).

For example, you can generate second-order polynomial terms from the initial input parameters without bias terms and save the result as a CSV file and PNG file:

```
hyperparameters["Y0"] = {…, "feature_interaction_type":
                         PolynomialFeatures,
                         "feature_interaction_params":
                         {"degree": 2,
                         "create_only_interaction": False,
                         "include_bias": False,
                         "save_new_features": True},
                         …}
```

Feature Selection

Feature selection, which operates independently or in combination with feature interaction, is a function that selects only features with higher importance by comparing the importance of numerous parameters in various ways. Depending on the statistical method, the results of selectively applying only a portion of the total parameters to the modeling can show higher or similar performance compared to those using all parameters. Increasing the number of

parameters does not always guarantee performance improvement, and it can save time when modeling with fewer parameters. Feature selection can be defined in the model hyperparameters:

```
hyperparameters["Y0"] = {…, "feature_selection_type": SHAP,
                         "feature_selection_parameters": {…}, …}
```

where `feature_selection_parameters` is a dictionary that specifies the settings for the feature interaction. The possible values for `feature_selection_type` are:

- `FI`: Simple selection based on feature importance
- `SFS`: Sequential feature selection along the `"forward"` or `"backward"` direction [11]
- `RFE`: Recursive feature elimination [12].
- `SHAP`: Shapley additive explanations [13].

SFS and RFE work only for models that have the attribute `coef_` or `feature_importances_`. Refer to the documentation for the specific model to determine this.

You can set the model to perform recursive feature elimination using the full data set on all targets with a 10-fold cross-validation with the hyperparameter setting:

```
hyperparameters["Y0"] = {…, "feature_selection_type": RFE,
                         "feature_selection_parameters":
                         {"data_set": Full,
                         "target": All,
                         "cv": 10}, …}
```

Alternatively, you can use a sequential feature selection method for all data and a target named "tar_1" with the "forward" direction, by generating five cross-validation sets with a 15% test size:

```
hyperparameters["Y0"] = {…, "feature_selection_type": SFS,
                         "feature_selection_parameters":
                         {"data_set": Full,
                         "target": "tar_1",
                         "cv": 5,
                         "test_size": 0.15,
                         "sfs_direction": "forward"}, …}
```

SHAP, a newly added option, is used as a feature selection by calculating statistical marginal contributions of features based on cooperative game theory:

```
hyperparameters['Y0'] = {…, "feature_selection_type": SHAP,
                         "feature_selection_parameters":
                         {"data_set": Full,
                         "target": All,
                         "cv": 10,
                         "threshold": 0.01,
                         "shap_summary_type": "shap_summary",
```

```
"save_summary_figure": True},…}
```

The possible `feature_selection_parameters` dictionary terms are:

- `data_set`: Specifies the subset of data on which the model is trained. Options are `Full`, `Passed`, `PassedTrimmed`, and `Trimmed` (default: `Full`)
- `target`: Specifies the target to select features by importance (default: `All`)
- `cv`: Sets the number of cross-validation sets generated by the basic cross-validator ShuffleSplit (default: `10`)
- `test_size`: Sets the fraction between the test set and the total training set for cross-validation (default: `0.1`)
- `scoring`: Sets the score indicator to evaluate feature importance (default: `"r2"`)
- `verbose`: Sets the verbosity level (default: `1`)
- `sfs_direction`: Specifies the optimization direction for sequential feature selection. This argument works only for `method=SFS`. Options are `"backward"` (default) and `"forward"`
- `threshold`: Sets the threshold with which to select valid features (default: `0.01`)
- `random_seed`: Sets the random seed for cross-validation (default: `1`)
- `val_fraction`: Sets the fraction between the validation set and the total dataset (default: `0.1`)
- `retrain`: Specifies whether to allow redefinition of an already trained model (default: `False`)
- `shap_summary_type`: Sets the type of the `shap` summary plot. The available options are `"shap_summary"`, `"shap_beeswarm"`, and `"heatmap"`. Used only for `method=SHAP` (default: `"shap_summary"`).
- `save_summary_figure`: Specifies whether to save the figure of feature selection summary. Used only for `method=SHAP` (default: `True`).

Hyperparameter Optimization

To increase model accuracy, the model hyperparameters often need to be optimized. Sentaurus Calibration Workbench includes two methods with which to accomplish this optimization: grid search and Bayesian search. A grid search tests every specified combination of hyperparameters, while a Bayesian search performs a search within the maximum and minimum bounds of the hyperparameters. In both cases, n-fold cross-validation is used with the training set data to determine the best set of hyperparameters. This optimization occurs during the training of the model.

To use grid search, first the model hyperparameter values must be declared as lists. Then the grid search settings can be specified. For example:

```
hyperparameters = dict()
hyperparameters["Y0"] = {"model_type": MLPR,
                         "model_hyperparameters":
                         {"hidden_layer_sizes":
                         [[64, 64, 64], [128,128,128]],
                         "alpha": [0.1, 0.3, 1.0],
                         "max_iter": [5000]},
                         "optimizer_type": GridSearch,
                         "optimizer_params": {'cv': 10}
                         }
```

In this case, the MLPR hyperparameters `hidden_layer_sizes` and `alpha` are to be optimized using a grid search. During training, six different combinations are tested: [64, 64, 64] and 0.1; [64, 64, 64] and 0.3; [64, 64, 64] and 1.0; [128, 128, 128] and 0.1; [128, 128, 128] and 0.3; and [128, 128, 128] and 1.0. For each combination, 10-fold cross-validation is used (cv of 10) and then the accuracies are compared, with the best selected.

For a Bayesian search, you only need to specify the maximum and minimum values of the hyperparameters to search between. However, only scalar values can be optimized. For example:

```
hyperparameters["Y0"] = {"model_type": MLPR,
                         "model_hyperparameters":
                         {"hidden_layer_sizes": [[64, 64, 64]],
                         "alpha": [0.1, 1.0],
                         "max_iter": [500, 5000]},
                         "optimizer_type": Bayesian,
                         "optimizer_params": {"cv": 5}}
                         }
```

This optimizes the `alpha` and `max_iter` hyperparameters between the maximum and minimum values specified. A 5-fold cross-validation is used to assess which combination of hyperparameters is best.

Each of the optimizer types has several parameters. For GridSearch, information about the parameters can be found at the scikit learn website [14]. For Bayesian, these can be found at [15].

Encoding XY Profile Targets

Curve data from different simulations might not share the same size of data points or the same range. To fit them with a single machine-learning training and prediction scheme, preprocessing is usually necessary. In addition to format considerations, preprocessing also contributes to the quality of a machine-learning model by extracting curve features or by reducing the signal bandwidth. The preprocessing of curve targets is performed by using encoders in the scwml module.

Currently, the only encoder that can be used for XY profile targets is the point sampling encoder. The point sampling encoder converts curve data into a certain number of data points by sampling at regular intervals. The encoder is set using the `encoder_type` model hyperparameter. A point sampling encoder that takes Y values at 50 evenly spaced X coordinates is set with:

```
hyperparameters["XY_Profile"] = {…,
                                 "encoder_type": PointSamplingEncoder,
                                 "encoder_params": {"num_data_points": 50}
```

The point sampling encoder has other parameters that can also be set:

- `num_data_points` is the number of data points to sample from each curve.
- `use_floating_range` specifies whether to clip off leading and trailing zeroes for the curves, potentially making the effective range of various curves different. By default, `use_floating_range=False`. If the X axis is not the exact same for all samples, then `use_floating_range` must be set to `True`.
- `sort_axis` sorts data using a specified axis; (–1, 0, 1) is available. The default `sort_axis=-1` means that data is not sorted.
- `remove_spikes` removes singular point spikes such as [0, nonzero, 0]. By default, `remove_spikes=False`.

Note: Only XY data that can be represented as a function y(x) can be encoded using the PointSamplingEncoder.

Note: XY Profiles can only be used as targets in the Mx1 configuration.

Saving and Loading Models

While some of the tasks in `mlcal` automatically save and/or load `Model` objects, it is also possible to save or load models manually. A model can be saved using the save method:

```
model_obj.save("model_filename.ml")
```

A `Model` object can be loaded using the load method:

```
model_obj.load("model_filename.ml")
```

However, if `model.obj.validate = True`, then all the hyperparameters of the model to be loaded must match those of the current model. Otherwise, an error occurs. To avoid this, set `model_obj.validate = False`. The validate property can be set during model initialization as well.

Custom Models

In addition to the models provided by the scwmal module, labeled by the keywords, you can pass a customized model as the main machine-learning model, provided that it has methods and interfaces conforming to the scikit-learn syntax:

- A *fit* method takes two 2D NumPy arrays, representing the feature matrix and the output matrix, with a row (first dimension) for each sample/experiment, and a column (second dimension) for each input feature or output target value. The return value for the fit method is optional.
- A *predict* method takes one 2D NumPy array, representing the feature matrix, with a row (first dimension) for each sample/experiment, and a column (second dimension) for each input feature. Its return value is a NumPy matrix containing the predicted target values, with a row (first dimension) for each sample.

Therefore, in Python, a custom model class has a minimum set of methods of:

```
import numpy as np
from numpy.typing import NDArray

class CustomModel:
    def __init__(self, args):
        #Initialize the model
    def fit(self, X: NDArray, Y: NDArray) -> None | NDArray
        #Train the model using feature inputs X and target values Y
    def predict(self, X: NDArray) -> NDArray:
        #Return predicted target values for X features
```

General MLCal Task Use and Format

Tasks in the `mlcal` application are all member functions of a specific instance of the `mlcal` object and are called by:

```
mlcal_obj.task(arguments)
```

Most of these tasks contain the following arguments:

- `models (list)`: Specifies a list of trained machine-learning `Model` objects
- `parameters (list)`: Specifies a list of machine-learning model `Parameter` objects
- `targets (list)`: Specifies a list of machine-learning model `Target` objects
- `parent (string)`: Specifies the parent project. If set to `None`, then option("parent") is used.
- `child_dir (string)`: Specifies the directory of the child projects. If set to None, then option("child_dir") is used.

- `save_report (bool)`: Specifies whether to save a report in the node report folder (*{project_dir}/results/nodes/{node number}/n{node number}_report/*)

For the sake of organization, it is typical to group all the models, parameters, and targets into lists after they have been defined. These lists are then passed as arguments into the individual tasks. For example:

```
models = [ml_0, ml_1]
parameters = [p_0, p_1, p_2]
targets = [t_0, t_1]
mlcal_obj.task(models=models, parameters=parameters,
               targets=targets)
```

This is the format that is used throughout the rest of this chapter.

Handling Conditions

`Parameter`, `Target`, and `Model` each have arguments for defining conditions. For example, `Parameter` with `choice` argument becomes a conditional (discrete) parameter, and `Target` and `Model` have the `conditions` argument. The use cases vary depending on the combination of these three elements, as shown in the two tables that follow. If there are split conditions, it is recommended to use (Conditional parameters, Force pass, Target condition, Model condition) = (T, T, Defined, Undefined) or (T, T, Defined, Defined) in general. It is also recommended that the values of the three conditions should be same.

Table 7 Task dependent condition priority

Task name	Priority of conditions
`create_ml`	Model conditions, conditional parameters
`generate_data_ml`	Conditional parameters
`predict_ml`	Conditional parameters
`reverse_search_ml`	Target conditions, conditional parameters (If `force_pass` is `True`, only the initial value is used. If `force_pass` is `False`, it become a discrete parameter.)

Table 8 Combinations of conditions

Conditional parameter	Force pass	Target condition	Model condition	Description
T	T	Defined	Undefined	`create_ml`, `generate_data_ml`, `predict_ml` use force-passed conditional parameters to generate DoE. `train_ml` treats the conditional parameters as input features. `reverse_search_ml` uses the target condition.
	F	Defined	Undefined	`create_ml`, `generate_data_ml`, `predict_ml` use conditional parameters as discrete parameters to generate DoE. `train_ml` treats the conditional parameters as input features. `reverse_search_ml` uses the target condition.
T	T	Defined	Defined	`create_ml` uses model conditions to generate DoE. `generate_data_ml`, `predict_ml` use force-passed conditional parameters to generate DoE. `train_ml` treats the conditional parameters as model conditions.`reverse_search_ml` uses the target condition.
	F	Defined	Defined	`create_ml` uses model conditions to generate DoE.`generate_data_ml`, `predict_ml` use conditional parameters as discrete parameters to generate DoE. `predict_ml`'s prediction result becomes `None` when the conditions do not match between conditional parameters and model conditions.`train_ml` treats the conditional parameters as model conditions. `reverse_search_ml` uses the target condition.

Reporting

By default, the `mlcal` specific machine-learning commands create a report folder in the node directory. The report folder contains results of created data (for `create_ml` and `generate_data_ml`), prediction results (for `predict_ml` and `reverse_search_ml`), and results for checking accuracy of the ML model (for `train_ml`). The reported items can be visualized using parallel coordinates plot from Sentaurus Calibration Workbench GUI. See Creating Parallel Coordinate Plots on page 167.

In addition, for the commands which include DoE data generation functions, you can activate the `generate_pairplot` epilogue which generates a correlation pairplot image [16] for parameter and target data, respectively. See generate_pairplot on page 248. The image includes a histogram for each target, allowing you to check the approximate distribution. In addition, the figure checks whether there are targets showing positive/negative correlation through linear fitting among different targets.

Plotting Data

The command `plot_data` allows you to plot the data after an `mlcal` task. The supported plot types are `curve`, `hist`, `line`, `parallel_coordinates`, `scatter`, and `surface`, and it generates graphs based on two different backends, `mpl` (for matplotlib), and `plotly`. The `mpl` produces static images, while the `plotly` backend generates interactive graphs, which have characteristics of user interaction. For example,

```
mlcal_obj.plot_data(targets=targets, parameters=parameters,
                    x="PWELL", y=["Ron_at200A", "Eoff_mJ"],
                    plot_type="scatter", backend="plotly",
                    outfile_prefix="MLoptimization")
```

For more information, see plot_data on page 260.

Generating Model Training/Test Data

There are two different ways to generate initial model data. The first is by using the `generate_data_ml` task. The second is by generating the data at the same time as creating the model using `create_ml`. Using `create_ml` is the simplest and most compact way of doing this, but sometimes it is beneficial to explicitly separate out data generation from the other calibration steps and use `generate_data_ml` instead.

There are three ways in which a DoE (design-of-experiments, or set of experimental parameter input values) can be set in `generate_data_ml`:

- By setting a specific DoE and number of experiments
- By passing specific experimental parameters via a csv or DataFrame
- By taking the values from reverse search results

The first and second cases are described in this section. The third will be described in Reverse Search on page 76, because it is meant to be used as a “check” of model predictions rather than for initial data generation.

For the complete set of task arguments, see generate_data_ml on page 245.

Using generate_data_ml With a Specific DoE

The simplest way of using `generate_data_ml` is to run a specific DoE within the parameter space defined in the passed parameters. For example, running 100 experiments (`doe_size=100`) uniformly randomly sampled (`doe_method="uniform"`) is accomplished with:

```
mlcal_obj.generate_data_ml(parameters=parameters, targets=targets,
                           doe_method="uniform", doe_size=100,
                           save_report=True)
```

This creates a child project in the (previously) specified directory which contains these run experiments. It then saves that data as a csv file from which this data can be inspected or loaded. An equivalent method of running this set of experiments is:

```
mlcal_obj.generate_data_ml(parameters=parameters, targets=targets,
                           data_generation={doe_method="uniform",
                           doe_size=100}, save_report=True)
```

This second method is usually used when additional arguments are used to define the DoE. For example, if you want to run the same set of experiments but also include the current values of the parameters and a specific random seed of 2, you can use the following command:

```
mlcal_obj.generate_data_ml(parameters=parameters, targets=targets,
                           data_generation={doe_method="uniform",
                           doe_size=100, include_nominal=True,
                           random_seed=2}, save_report=True)
```

The DoE’s available in Sentaurus Calibration Workbench are "`box-behnken`", "`full-factorial`", "`gaussian`", "`grid`", "`latin-hypercube`", "`uniform`", "`sobol`", and "`adaptive`". The `doe_size` argument is not required for the full-factorial method ("full-factorial") and the Box–Behnken method ("box-behnken"). For the grid DoE, `doe_size` can be either a single integer value (wherein the grid has the same number of points for each parameter) or a list of integer values (wherein the grid has different numbers of points for each parameter). For example, to specify a grid with 3 sample points along the first parameter and 5 for the second parameter (15 samples in total):

```
mlcal_obj.generate_data_ml(parameters=parameters, targets=targets,
                           doe_method="grid", doe_size=[3,5],
                           save_report=True)
```

The length of the list must be equal to the number of parameters.

To include the nominal values of parameters in the DoE also, set `include_nominal=True` in the `data_generation` dictionary. By default, the nominal value of each parameter is the middle point between the minimum and maximum values of the parameter (`use_mean=True`). If `use_mean=False`, then the user-provided initial parameter value is used as the nominal value.

force_pass Parameters and Data Generation

If a Parameter has choices set and has `force_pass=False`, then it is treated just like the Parameters with minimum and maximum values by the DoE. As an example, consider generating data using two parameters as inputs. The first has a maximum and minimum set, while the other uses 2 discrete choices. If the data is generated with a uniformly random DoE with `doe_size=50`, both the continuous parameter and the discrete parameter are randomly selected with 50 experiments in total.

```
p_time = Parameter(name="time", value=5.0, minimum=1.0,
                   maximum=10.0)
p_temp = Parameter(name="temperature", value="300",
                   choices=["300", "400"], dtype=str)
                   ...
mlcal_obj.generate_data_ml(parameters=parameters, targets=targets,
                           doe_method="uniform", doe_size=50,
                           save_report=True)
```

The resulting generated data would have 50 experiments in total and look similar to:

...	time	temperature	...
...	5.3	300	...
...	8.7	400	...
...	6.4	400	...
...	3.3	300	...
...	...	...	...
...	9.8	400	...

However, if `force_pass=True`, then the DoE is run for each set of choices. So, if the DoE is uniformly random with `doe_size=50`, then 50 uniformly random samples are generated for each condition set, which is 100 experiments (2 x 50):

```
p_time = Parameter(name="time", value=5.0, minimum=1.0,
                   maximum=10.0)
p_temp = Parameter(name="temperature", value="300",
                   choices=["300", "400"], dtype=str,
                   force_pass=True)
```

```
                    ...
mlcal_obj.generate_data_ml(parameters=parameters, targets=targets,
                           doe_method="uniform", doe_size=50,
                           save_report=True)
```

The resulting data that is generated has 100 experiments in total and look similar to:

...	time	temperature	...
...	5.3	300	...
...	8.7	300	...
...	6.4	300	...
...	3.3	300	...
...	...		...
...	9.8	300	...
...	5.3	400	...
...	8.7	400	...
...	6.4	400	...
...	3.3	400	...
...	...	...	...
...	9.8	400	...

Adaptive Data Generation

Adaptive data generation is a special DoE in that rather than all experiments being set initially and run in parallel, the data is generated in batches of randomly generated (“uniform”) experiments, with an additional machine learning model (called the “filter”) used to monitor the results and increase the number of “passed” experiments. Adaptive data generation runs until the number of passed experiments matches that of the desired DoE size.

An experiment is considered “passed” for a model if:

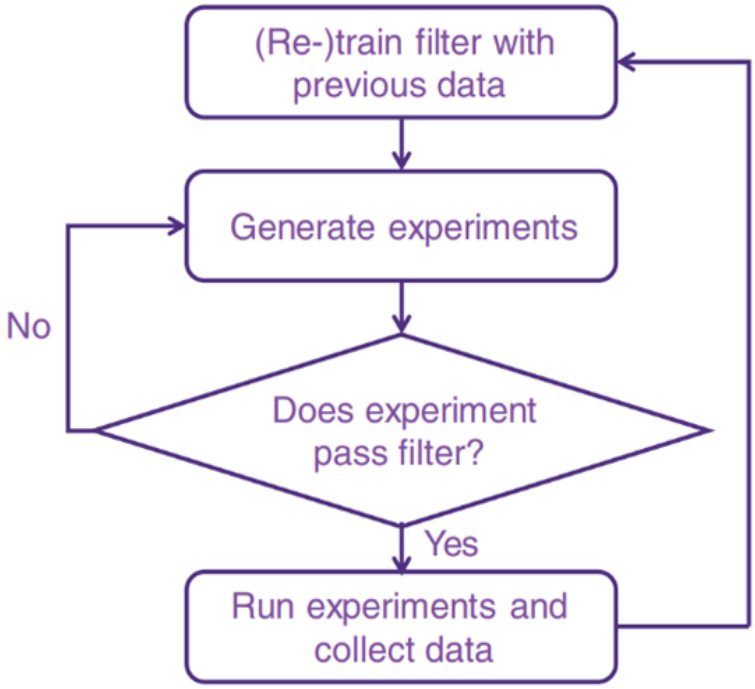
- It completes successfully without error
- All target values are within their specified range

Note:

If using adaptive data generation with `create_ml`, it uses only targets that are contained within the model.

The machine learning model used for the filter must be a classifier type model. In this case, it is classifying experiments as “pass” or “fail” based on their input parameters. After a batch is run, the filter is (re-)trained using all previously generated data. When a new batch of experiments is generated, their parameters are passed through the filter to determine whether they pass or fail. Those experiments that are likely to pass are kept, while those that fail are replaced with new experiments. This continues until all the sets of experimental parameters for that batch are likely to pass. After that, the batch of experiments runs, and additional data is generated. See Figure 2 for a simplified flow diagram.

Figure 2 Simplified flow of adaptive data generation method



If you want to generate 500 passed experiments (`doe_size=500`) by running batches of 50 experiments (`batch_size=50`) and using a decision tree classifier to use previously run experiments to predict pass/fail experiments (`classifier=DecisionTree`), the function call is:

```
mlcal_obj.generate_data_ml(parameters=parameters, targets=targets,
                           data_generation={
                           "doe_method": "adaptive",
```

```
                                "doe_size": 500,
                                "batch_size": 50,
                                "classifier": DecisionTree},
                                save_report=True)
```

In many cases, you might want to specify an initial DoE to run before switching to random generation (`initial_sampling_doe`), the number of batches to use that initial DoE for (`initial_doe_batches`), a certain number of generated samples that must “pass” before using the ML filter, and a transformation to apply to the model inputs (`transformer`). The function call then looks like:

```
mlcal_obj.generate_data_ml(parameters=parameters, targets=targets,
                           data_generation={
                           "doe_method": "adaptive",
                           "doe_size": 500, "batch_size": 50,
                           "threshold_num_pass": 100,
                           "initial_sampling_doe": "sobol",
                           "initial_doe_batches": 2,
                           "transformer": Standard,
                           "classifier": DecisionTree},
                           save_report=True)
```

The supported transformers and classifiers are given in Table 9 and Table 10.

Table 9 Supported transformers

Keyword	Transformer
`Identity`	No transformer applied
`Standard`	Mean-standard scaler
`MinMax`	Min-max scaler
`Quantile`	Quantile transformer
`Robust`	Robust scaler

Table 10 Supported classifiers

Keyword	Classifier	Hyperparameter Ref.
`AdaBoost`	AdaBoost classifier	[17]
`DecisionTree`	Decision tree classifier	[18]
`GPC`	Gaussian process classifier	[19]
`KNNC`	K Nearest neighbors classifier	[20]

Table 10 Supported classifiers

Keyword	Classifier	Hyperparameter Ref.
`MLPC`	Multilayer perception classifier	[21]

Each transformer or classifier has different hyperparameters. The hyperparameters and their default values follow the rules of the respective scikit-learn methods. The hyperparameters can be set in the `data_generation` dictionary:

```
data_generation={"doe_method": "adaptive", …,
                 "scaler_hyperparameters": {…},
                 "classifier_hyperparameters": {…}, …}
```

For example, you can select the neural network classifier (`"classifier": MLPC`), which has hyperparameters such as `hidden_layer_sizes` and `activation`, each with default values [2]. You can change the values of some hyperparameters by specifying:

```
data_generation={"doe_method": "adaptive", …,
                 "classifier": MLPC, …,
                 "classifier_hyperparameters":
                 {"hidden_layer_sizes": (32, 16, 4),
                 "activation": "tanh",
                 "learning_rate": "adaptive"}, …}
```

Using generate_data_ml by Passing Experimental Parameters

Data can also be generated by passing the specific parameter values via csv or pandas DataFrame. This is done by setting the `doe` argument. The columns of the csv/DataFrame should be the parameter names and the rows should be the values of the parameters for each experiment. So, for a function call of:

```
p0 = Parameter(name="Param0", value=0.0, minimum=-5.0, maximum=5.0)
p1 = Parameter(name="Param1", value=1.0, minimum=-5.0, maximum=5.0)
parameters = [p0, p1]
mlcal_obj.generate_data_ml(parameters=parameters, targets=targets,
                           doe=”test_exp.csv”, save_report=True)
```

the structure of “test_exp.csv” should be:

...	Param0	...	Param1	...
...	2.0	...	2.5	...
...	1.5	...	0.3	...
...	-3.0	...	4.5	...
...	...	...	...	...

Running `generate_data_ml` then creates a child project and runs experiments with these specific input parameters and stores them in the child project directory. Note that other columns can be present in the csv/DataFrame without an issue.

Data Generation Using create_ml

The task `create_ml` can either load data from previously generated experiments or it can generate the initial data itself. It contains the `doe_method`, `doe_size`, and `data_generation` arguments, which work the same way as those arguments in `generate_data_ml`. A typical `create_ml` function call to create an ML model and also run a latin hypercube DoE with 100 experiments might be:

```
mlcal_obj.create_ml(models=models, parameters=parameters,
                    targets=targets, doe_method="latin-hypercube",
                    doe_size=100)
```

Note:

This does not train the machine learning model with the generated data. More about this is discussed in later sections.

Re-Running Data Generation

When running an `mlcal` project, experiments are re-run in the child Sentaurus Workbench project only if a successfully completed one with those input parameters does not exist. That is, an experiment is run only if:

- The experiment does not already exist in the child Sentaurus Workbench project
- The experiment exists in the child Sentaurus Workbench project, but it failed

Mlcal does not automatically delete these child projects after the Sentaurus Calibration Workbench project has completed. If cleanup of the child project is desired, this must be performed manually.

Creating the Machine Learning Model

While the `Model` class is used to define the machine learning models used, `create_ml` is the task that creates the (untrained) Sentaurus Calibration Workbench machine learning object. This is saved as `{ml name}_untrained.ml` in the project directory. In Sentaurus Calibration Workbench, this object also contains the training/test data associated with it and so this data is required as an input. If the experimental data is provided in “exp_data.csv”, the function call is:

```
mlcal_obj.create_ml(models=models, parameters=parameters,
                    targets=targets, train_data="exp_data.csv")
```

Alternatively, as described in Data Generation Using create_ml on page 73, `create_ml` can also generate the data to be used for training the model. For the complete set of task arguments, see create_ml on page 233.

Training the Machine Learning Model

After the model has been created, it needs to be trained. This is performed by `train_ml`. A simple function call for training the models for a given set of parameters and targets is:

```
mlcal_obj.train_ml(models=models, parameters=parameters, targets=targets
```

The trained model is saved as `{ml name}_trained.ml` in the project directory. When training, the data is randomly split into two sets: train data and test data. 90% of the model data set becomes the train data set, which is used for training the model. The remaining 10% is the test data set used to assess the quality of the model predictions on a set of data that it has yet to see. The `train_ml` task also outputs the performance of the model on both these sets, including plots comparing the predicted values to the actual values.

Normally, if a trained model already exists, it is not be re-trained. To re-train an existing trained model (rather than just loading it), the retrain argument must be set to `True`:

```
mlcal_obj.train_ml(models=models, parameters=parameters,
                   targets=targets, retrain=True)
```

This retrain is different from the retrain model hyperparameter. A summary of how these work in together is given in Table 11.

Table 11 Model and train_ml retrain combinations

train_ml “retrain” argument	Model “retain” Hyperparameter	Model “warm_start” Hyperparameter	Effect if trained model exists
True	---	---	Untrained model loaded and training performed.
False	---	---	Existing trained model loaded
False	True	True	Existing trained model loaded and `warm_start` applied.

For the complete set of task arguments, see train_ml on page 299.

Data Trimming

The data used for training can also be trimmed. The criteria for this are the same as for adaptive data generation, in which the target values must all be within specified ranges. For trimming to work, the target being used as a trimmer must be included in the model. To filter

the training set, set the `data_set` argument to `"trimmed"`, `"passed"`, or `"passedtrimmed"`. The target range defined in `create_ml` becomes `"passed"` and the target range defined in `train_ml` becomes `"trimmed"`. To select only trimmed data:

```
mlcal_obj.train_ml(models=models, parameters=parameters,
                   targets=targets, data_set="trimmed")
```

To have the full data set used, set `data_set` to `"full"`:

```
mlcal_obj.train_ml(models=models, parameters=parameters,
                   targets=targets, data_set="full")
```

Even with the `"full"` data set, failed experiments and experiments for which the profile target file does not exist are automatically trimmed.

Using the Machine Learning Model

Once you have a trained machine learning model, you can use it for either the prediction of targets, given specific inputs (`predict_ml`) or for finding parameters that results in specific target values (`reverse_search_ml`).

predict_ml

`predict_ml` works very similarly to `generate_data_ml` in that it takes input parameters and outputs a csv file of results. To have the models predict the value of a list of targets based on specific input parameters using a uniformly random DoE with 100 experiments, the `predict_ml` function call is:

```
mlcal_obj.predict_ml(models=models, parameters=parameters,
                     targets=targets, doe_method="uniform",
                     doe_size=100)
```

In addition to using specific DoEs, the `predict_ml` task can be run on the data used to train and test the model. There are three options for the `doe_method` that specify this data: `"model_data"`, `"model_train_data"`, `"model_validate_data"` (which is the test data set). If you want to see the model predictions for the test data set, the function call is:

```
mlcal_obj.predict_ml(models=models, parameters=parameters,
                     targets=targets,
                     doe_method="model_validate_data")
```

Like `generate_data_ml`, `predict_ml` can also take as an input argument a csv file or pandas DataFrame that defines specific parameter values to be run:

```
mlcal_obj.predict_ml(models=models, parameters=parameters,
                     targets=targets, doe="test_exp.csv")
```

For information about the required csv table format, see **Using generate_data_ml by Passing Experimental Parameters on page 72**.

Normally, making predictions outside of the parameter bounds of the model (extrapolation) is not allowed and results in an error. However, if you set `check_boundary=False`, then a only warning is issued:

```
mlcal_obj.predict_ml(models=models, parameters=parameters,
                     targets=targets, doe=”test_exp.csv”,
                     check_boundary=False)
```

For the complete set of task arguments, see predict_ml on page 264.

Reverse Search

Reverse search (using a model for finding parameters that result in specific target values) is relatively straightforward but has some idiosyncrasies in how it should and can be used compared to the other functions. It is not possible to work backward from the target values to the specific parameters, so instead the parameter space must be explored using the ML model(s) and the parameters values that result in the closest target values selected. A basic reverse search that uses a uniform random DoE and selects the top 10 candidates is:

```
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            doe_method="uniform", doe_size=100000)
```

Note that `doe_size` should be set quite large because the machine learning models can run very quickly when compared to TCAD simulations. In the this function call, the candidate values are the best ones found using the DoE. To perform a further gradient-based optimization on these candidates, the `optimize` argument should be set to `True`:

```
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            doe_method="uniform", doe_size=100000,
                            optimize=True)
```

In addition to using a DoE, specific values to search can be set using the `doe` argument in the same manner as is used for `predict_ml` or `generate_data_ml`. The parameter ranges can also be specified using a DataFrame or csv file, using the `range` argument.

For the complete set of task arguments, see reverse_search_ml on page 271.

Adaptive Reverse Search

Unlike the other reverse searches, adaptive reverse_search is performed in multiple steps. At each step, a uniform random DoE is run with a size of `doe_size`. From the results of this search, the best candidates and parameter search space are updated. This continues for `num_steps` steps, or until the time limit given by `time_limit` has been reached.

When performing an adaptive reverse search, some of the arguments are given as kwargs. If For example, if you define an adaptive reverse search task with a `doe_size` at each step of 1000, a time limit of 1 hour, and a maximum number of steps of 100, the function call is:

```
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            doe_method="adaptive", doe_size=1000,
                            **{"num_steps": 100,
                            "time_limit": 3600})
```

Some key adaptive reverse search kwargs are:

- `num_steps (int)`: Specifies the number of searching steps
- `time_limit (int)`: Specifies the time limit
- `individual_rms (bool)`: If `True`, include individual rmse in addition to the total rmse
- `search_method (str)`: Specifies the search method: "min_tol" or "min_rms". “min_tol” prioritizes all targets within their tolerances, while “min_rms” considers only the rmse.

Priority targets are given extra weight when assessing the result of an adaptive reverse search. They are specified using the individual target IDs:

```
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            doe_method="adaptive", doe_size=1000,
                            priority_targets=["t_Y0","t_Y1"],
                            **{"num_steps": 100,
                            "time_limit": 3600})
```

For more information, see reverse_search_ml on page 271.

Multi-Modal and Multi-Objective Reverse Search

Unlike the conventional DoE methods, the NSGAIII-based DoE methods, NSGAIII_S and NSGAIII_M, perform in multiple steps. NSGAIII_S is a DoE method designed for multi-modal optimization. Using NSGAIII_S increases the likelihood of identifying multiple minima points in the total RMS error space. On the other hand, NSGAIII_M is used for multi-objective optimization. When there are multiple targets, NSGAIII_M provides candidates by considering not only the total RMS error but also the individual error of each target. These two methods determine the population size and number of generations based on the given DoE size. For more detailed adjustments, additional arguments can be defined in dictionary form like in the following example:

```
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            doe_method="NSGAIII_S", doe_size=10000,
                            args_nsgaIII={"pop_size": 100,
                            "n_points": 100, "n_max_gen":1000,
                            "mutation_prob": 0.1})
```

For more information, see reverse_search_ml on page 271.

Clustering Method for Candidate Selection

`reverse_search_ml` has a clustering feature. With the clustering option enabled, it forms clusters of candidates and extracts the candidates with the lowest total error from each cluster. This increases the likelihood of finding diverse candidate combinations across a broader parameter space. The clustering option includes both a fixed number and optimal number of clusters. With the fixed number option, a specified number of clusters is generated. With the optimal number option, it automatically determines the optimal number of clusters. It generates image files (pair plot and parallel coordinates plot) to visualize the clusters. (The label “0” is the selected best candidates and the others are the cluster numbers.)

Chains of reverse_search_ml

Unlike other tasks, the outputs from `reverse_search_ml` are stored in the `mlcal` object and added to with each subsequent run. So, if you run two reverse searches in a row, such as:

```
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            doe_method="uniform", doe_size=100000)
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            doe_method="sobol", doe_size=100000)
```

The first reverse search saves a csv with 10 candidates and the second reverse search stores a csv with 20 candidates: the results from both `reverse_search_ml` calls. This allows you to build up a series of reverse search results.

To reset the reverse search results prior to performing the search, the `reset_results` argument should be set to `True`. So, if you modify the second `reverse_search_ml` with this argument:

```
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            doe_method="uniform", doe_size=100000)
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            doe_method="sobol", doe_size=100000,
                            reset_results=True)
```

Each of the reverse searches now return 10 candidates, the results from each search.

Updating Parameter Values and Ranges

After performing a reverse search, the parameter space can be updated based on the results of the search. This is set using the `update_parameters` argument. If it is set to `value`, then the parameter values are set to those of the best candidate found using reverse search. If it is set to `range`, then in addition to the parameter values being set, the parameter ranges are to the maximum and minimum values in the set of candidates.

An example use is chaining together a series of reverse searches, narrowing the reverse search space, and then searching again and evaluating the result:

```
#Performs an initial set of reverse searches and narrows the range
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=50,
                            doe_method="uniform", doe_size=100000)
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=50,
                            doe_method="sobol", doe_size=100000,
                            update_parameters="range")

#Resets the results and searches in the narrowed range
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            optimize=True, doe_method="uniform",
                            doe_size=100000,
                            update_parameters="value",
                            reset_results=True)

#Runs an evaluation using the best reverse search candidate
mlcal_obj.evaluate(parameters=parameters, targets=targets)
```

Checking With generate_data_ml

When `generate_data_ml` is run after a reverse search and the DoE is left unspecified, it uses the candidates found using `reverse_search_ml` as the input experiments. This is usually the preferred approach to using evaluate, because even well-trained models can have a certain degree of error in their predictions and it makes sense to evaluate multiple candidates. In such a case, the script of the previous section becomes:

```
#Performs an initial set of reverse searches and narrows the range
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=50,
                            doe_method="uniform", doe_size=100000)
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=50,
                            doe_method="sobol", doe_size=100000,
                            update_parameters="range")

#Resets the results and searches in the narrowed range
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            doe_method="uniform", doe_size=100000,
                            update_parameters="value",
                            reset_results=True)
mlcal_obj.reverse_search_ml(models=models, parameters=parameters,
                            targets=targets, num_candidates=10,
                            doe_method="sobol", doe_size=100000,
                            update_parameters="value")

#Evaluates the 20 candidate parameter sets and selects the best one
```

```
mlcal_obj.generate_data_ml(parameters=parameters, targets=targets,
                           child_dir="@pwd@/../results_child",
                           update_parameters=True)
```

Note that a new child directory is specified in the `generate_data_ml` step. This is because an additional identifying parameter is added to reverse search results and are therefore not compatible with the previous child Sentaurus Workbench projects.

Additional Topics

Validating Parameter Ranges

In many cases, the parameter ranges may have been set without running experiments other than the nominal case. This can mean that when the TCAD simulations are run at the extrema of the parameters, there may be many failures and/or errors. The function `validate_range_ml` checks and adjusts the parameter ranges so that the child experiments pass for the extreme values of each parameter. It is called with:

```
mlcal_obj.validate_range_ml(parameters=parameters, targets=targets)
```

This adjustment is performed by starting with the nominal parameter values, and then setting each individual parameter to their maximum and minimum values, with all other parameters remaining at the nominal values. If an experiment fails at a particular minimum/maximum parameter value, this value is increased (minimum) or decreased (maximum) and then re-tested. This continues until every experiment passes or a maximum number of iterations. The increase/decrease ratio and maximum number of iterations can be set with the `reducing ratio` and `num_validations` arguments:

```
mlcal_obj.validate_range_ml(parameters=parameters, targets=targets,
                            reducing_ratio=0.1, num_validations=7)
```

You can also add additional constraints that must be met via the `constraint` argument. For example, if an output variable of `Fail` must be equal to 0 to be considered a valid experiment, then you call:

```
mlcal_obj.validate_range_ml(parameters=parameters, targets=targets,
                            reducing_ratio=0.1, num_validations=7,
                            constraint="Fail==0")
```

For the complete set of task arguments, see validate_range_ml on page 301.

Sensitivity Analysis and Primary Parameters

In many cases, there are a substantial amount of input parameters, but not all may have a significant effect on the target values. It can therefore be valuable to assess how sensitive the target output response is to the different input parameters and keep only the subset of

parameters that have a significant impact. In `mlcal`, this is accomplished by performing a sensitivity analysis.

A local sensitivity analysis can be run using the following command:

```
mlcal_obj.run_sensitivity_analysis_ml(parameters=parameters,
                                      targets=targets)
```

The result can be found in the report folder for the node. For cases where simulations should be retried upon a failure or results should be averaged from multiple simulations, a `num_retries` argument and a `num_evaluations` argument are available. If you want to have retry upon failure and used sensitivity results averaged from 5 experiments, the function call is:

```
mlcal_obj.run_sensitivity_analysis_ml(parameters=parameters,
                                      targets=targets,
                                      num_retries=2,
                                      num_evaluations=5)
```

Note:

The DoE used to determine sensitivity in `run_sensitivity_analysis_ml` is different from that used in `scwcal`'s `run_sensitivity_analysis`.

To get a list of the primary parameters based on a previously run sensitivity analysis, the `determine_primary_parameters_ml` function should be used:

```
param_list =

           mlcal_obj.determine_primary_parameters(parameters=parameters)
```

This parameter list can then be used in any following step. By default, the number of primary parameters returned is 8. If a different number is desired, this can be set using the `num_parameters` argument. A minimum sensitivity value can also be set using the `min_sensitivity` argument. For example, if only the top 5 parameters than have a sensitivity greater than 0.05 are needed, then the function call is:

```
param_list =
mlcal_obj.determine_primary_parameters(parameters=parameters,
                                       num_parameters=5,
                                       min_sensitivity=0.05)
```

This particular function call returns a list of up to 5 parameters. If less than 5 parameters meet the `min_sensitivity` constraint, then the parameter list includes only those that meet the minimum sensitivity constraint. By setting the number of parameters high, every parameter that meets the minimum sensitivity constraint can be returned.

For more information about each function, seerun_sensitivity_analysis_ml on page 276 and determine_primary_parameters_ml on page 238.

Merge Tables

Sometimes, a single experiment (or row of data) outputs (or contains) multiple data points that should all be captured in a single parameter and target. For example, when varying the RF power as an input parameter for etching, each experiment might output etch depths at several different times:

	power	time_0	depth_0	time_1	depth_1	time_2	depth_2
Exp 1	100	1.0	2.5	2.0	4.5	3.0	5.5
Exp 2	200	1.0	3.5	2.0	6.5	3.0	8.5
...	...	...	...	...	...	...	...

In Sentaurus Calibration Workbench, these are normally set as individual parameters and targets:

```
p_power = Parameter(name="power", value=150.0, minimum=0.0,
                    maximum=300.0)

#time parameters are extracted from the data, not set
p_time0 = Parameter(name="time_0", minimum=0.0, maximum=5.0,
                    is_extracted=True)
p_time1 = Parameter(name="time_1", minimum=0.0, maximum=5.0,
                    is_extracted=True)
p_time2 = Parameter(name="time_2", minimum=0.0, maximum=5.0,
                    is_extracted=True)

t_depth0 = Target(name="depth_0", value=3.0, dimension=0)
t_depth1 = Target(name="depth_1", value=5.0, dimension=0)
t_depth2 = Target(name="depth_2", value=7.0, dimension=0)
```

However, for the ML model, you may want just a “time” parameter and “depth” target that encompasses all the individual times and depth. This is done through a merge table. First, in addition to the above, you must declare the additional time and depth parameters:

```
#time parameters are extracted from the data, not set
p_time = Parameter(name="time", minimum=0.0, maximum=5.0,
                   is_extracted=True)
t_depth = Target(name="depth", value=3.0, dimension=0)
```

Next, you must declare a merge table that specifies which parameters and targets to merge and the names of the merged parameter and target:

```
merge_table = {"time":["time_0", "time_1", "time_2"],
               "depth":["depth_0", "depth_1", "depth_2"]}
```

Note that the length of the lists for each merged parameter and target must match, because they will be paired with each other for training (that is, the depth given by “depth_0” is at “time_0”). When creating the ML object, all of the parameters and targets should be passed

to it. However, for subsequent steps using the ML models, only the merged values should be used:

```
create_params = [p_power, p_time0, p_time1, p_time2, p_time]
create_targets = [t_depth0, t_depth1, t_depth2, t_depth]
ml_params = [p_power, p_time]
ml_targets = [t_depth]

ml_obj.create_ml(models=models, parameters=create_params,
                 targets=create_targets,
                 train_data=["etch_data.csv"],
                 merge_table=merge_table)

ml_obj.train_ml(models=models, parameters=ml_params,
                targets=ml_targets)

ml_obj.predict_ml(models=models, parameters=ml_params,
                  targets=ml_targets, doe_method="sobol",
                  doe_size=100)

ml_obj.reverse_search(models=models, parameters=ml_params,
                      targets=ml_targets, num_candidates=10,
                      doe_method="sobol", doe_size=100000,
                      update_parameters="value")
```

Merge Tables and Reverse Search

One of the challenges with using merge tables is that after performing a reverse search, it is difficult to perform a check using `generate_data_ml` or `evaluate`. This is because the found parameters and targets are the merged ones and are not necessarily inputs and outputs of the parent project.

Because of this, it is recommended to have a setup that can take the merged parameters as input and report the merged output targets. For the previous example then, you may want a parent project that uses “time” as an input that defines the complete etch time and “depth” as the final etch depth. So, you might generate data with:

```
p_power = Parameter(name="power", value=150.0, minimum=0.0,
                    maximum=300.0)

#time parameters are extracted from the data, not set
p_time0 = Parameter(name="time_0", minimum=0.0, maximum=5.0,
                    is_extracted=True)
p_time1 = Parameter(name="time_1", minimum=0.0, maximum=5.0,
                    is_extracted=True)
p_time2 = Parameter(name="time_2", minimum=0.0, maximum=5.0,
                    is_extracted=True)

t_depth0 = Target(name="depth_0", value=3.0, dimension=0)
t_depth1 = Target(name="depth_1", value=5.0, dimension=0)
t_depth2 = Target(name="depth_2", value=7.0, dimension=0)
```

```
#By setting is_extracted=True, the parameter match the parent
p_time = Parameter(name="time", minimum=0.0, maximum=5.0,
                   is_extracted=True)
t_depth = Target(name="depth", value=3.0, dimension=0)

create_params = [p_power, p_time0, p_time1, p_time2, p_time]
create_targets = [t_depth0, t_depth1, t_depth2, t_depth]

ml_obj.generate_data_ml(parameters=create_params,
                        targets=create_targets, doe_method="sobol",
                        doe_size=500)
```

Assuming that the “time” parameter of the parent project is set to 3.0 and it corresponds to the total etch time and also “time_2”, the data in the saved csv should look similar to that in the following table.

	power	**time**	**time_0**	**depth_0**	**time_1**	**depth_1**	**time_2**	**depth_2**	**depth**
Exp 1	100	3.0	1.0	2.5	2.0	4.5	3.0	5.5	5.5
Exp 2	200	3.0	1.0	3.5	2.0	6.5	3.0	8.5	8.5
...	...		...	...	...	...	...	...	...

Now, you may run the same model creation and training as in the previous merge table example:

```
ml_params = [p_power, p_time]
ml_targets = [t_depth]

ml_obj.create_ml(models=models, parameters=create_params,
                 targets=create_targets,
                 train_data="etch_data.csv",
                 merge_table=merge_table)

ml_obj.train_ml(models=models, parameters=ml_params,
                targets=ml_targets)

ml_obj.reverse_search(models=models, parameters=ml_params,
                      targets=ml_targets, num_candidates=10,
                      doe_method="sobol", doe_size=100000,
                      update_parameters="range")
```

Even though the “time” and “depth” parameter are in the data table, they are ignored in favor of the merged parameter and target. To run `generate_data_ml`, you must first set the “time” parameter to no longer be extracted and then run.

```
p_time.is_extracted=False

mlcal_obj.generate_data_ml(parameters=parameters, targets=targets,
                           child_dir="@pwd@/../results_child",
```

```
                                update_parameters=True)
```

Examples and Use Cases

Basic Workflow

The following snippet shows a very basic workflow with a simple set of steps to optimize two parameters to match targets at two different conditions. This is meant to serve as an example of the simplest possible `mlcal` calibration algorithm.

```
from scw.core.parameter import Parameter
from scw.core.target import Target
from scw.apps.mlcal import MLCal
from scw.core.model import Model

# Defining parameters and targets
p0 = Parameter(name="stickingA", value=0.05, minimum=0.01,
               maximum=0.2)
p1 = Parameter(name="yield", value=1.0, minimum=0.05, maximum=2.0)
c1 = Parameter(name="time", value="10", choices=["10","20"],
               force_pass=True, dtype=str)

t_yA = Target(name="depth", value=1.6776,
              conditions={"time": "20"})
t_yB = Target(name="depth", value=0.9386,
              conditions={"time": "10"})

parameters = [p0, p1, c1]
targets = [t_yA, t_yB]

# Declare project options
mlcal0 = MLCal()
mlcal0.set_option(parent="@pwd@/../parent/example_parent")
mlcal0.set_option(child_dir="@pwd@/../child")
mlcal0.set_option(tools=["stopo_step1”])
mlcal0.set_option(queue="lsf:all")

# Evaluate
mlcal0.evaluate(parameters=parameters, targets=targets)

# Define Machine-Learning Model
hyperparameters = dict()
hyperparameters["depth"] = {"model_type": MLPR,
                            "model_hyperparameters":
                            {"hidden_layer_sizes": [8, 8],
                            "solver": "lbfgs"},
                            "retrain": True,
                            "parameter_scaler": Standard}
ml_0 = Model(name="ml_0", hyperparameters=hyperparameters)
models = [ml_0]
```

```
# Generate TCAD Data in child project
mlcal0.generate_data_ml(parameters=parameters, targets=targets,
                        doe_method="sobol", doe_size=128)

# Create machine learning model using generated data
mlcal0.create_ml(models=models, parameters=parameters,
                 targets=targets, train_data=
["@pwd@/results/nodes/1/n1_report/2_ML_generate/2_ML_generate.csv"])

# Train the machine learning model
mlcal0.train_ml(models=models, parameters=parameters,
                targets=targets,
                retrain=True)

# Perform a reverse search using the machine learning model
# parameters that result in the target values
mlcal0.reverse_search_ml(models=models, parameters=parameters,
                         targets=targets, doe_method="sobol",
                         doe_size=100000, num_candidates=10)

# Generate TCAD data for the 10 candidate parameter sets
mlcal0.generate_data_ml(parameters=parameters, targets=targets,
                        child_dir="@pwd@/../results_child",
                        update_parameters=True)

# Report best result from the TCAD data
mlcal0.print_parameters(parameters=parameters)
mlcal0.print_targets(targets=targets)
for target in targets:
    print(f"{target.name} Error:
            {mlcal0.get_rms_error()}")
```

Search for High Peak-Gain HBT Device Geometries

This example studies the DC current–voltage behavior of a heterojunction bipolar transistor (HBT) with respect to the device geometries. One simulation flow consists of three steps: The input parameters include device geometries (width and thickness, shown as arrows in Figure 3), and the doping levels at the emitter and base. There are 15 parameters in total (13 geometric parameters and 2 doping parameters). The device can be of either npn type or pnp type. In this example, the type takes npn as a constant.

Three outputs are extracted from the simulation. The DC Gummel plot includes the change of the base current, the collector current with respect to the base–emitter voltage, and the ratio between the collector current and the base current as the gain (see Figure 4). The

voltage and gain at the peak of the gain–voltage curve is extracted as PeakGain and Vb_PeakGain.

Tool name (tool label)	Description	Command file	Preference file
Sentaurus Structure Editor (`sde`)	Generates device structure	`sde_dvs.cmd`	`sde_dvs.prf`
Sentaurus Device (`Gummel`)	Stimulates device	`Gummel_des.cmd`	`Gummel_des.prf`
Sentaurus Visual (`PlotGummel`)	Generates Gummel plot and extracts measurements	`PlotGummel_vis.cmd`	`PlotGummel_vis.prf`

The collector current at Vb_PeakGain is extracted as Ic_PeakGain. The three extracted values are returned in the Sentaurus Visual step using the `puts "DOE: $name $value"` mechanism.

Usually, the PeakGain is of the order of magnitude of hundreds. The question this example solves is whether it is possible to have an HBT device structure with a peak gain above 1000, while Ic_PeakGain is low (below 1e-5 A/μm).

Figure 3 Device structure with geometric parameters labeled

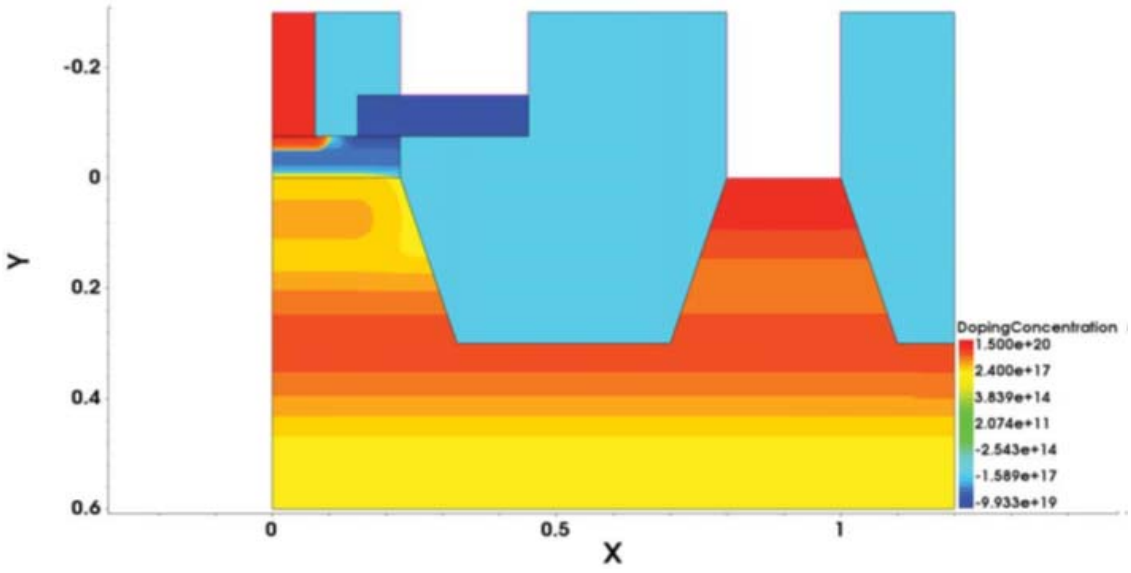
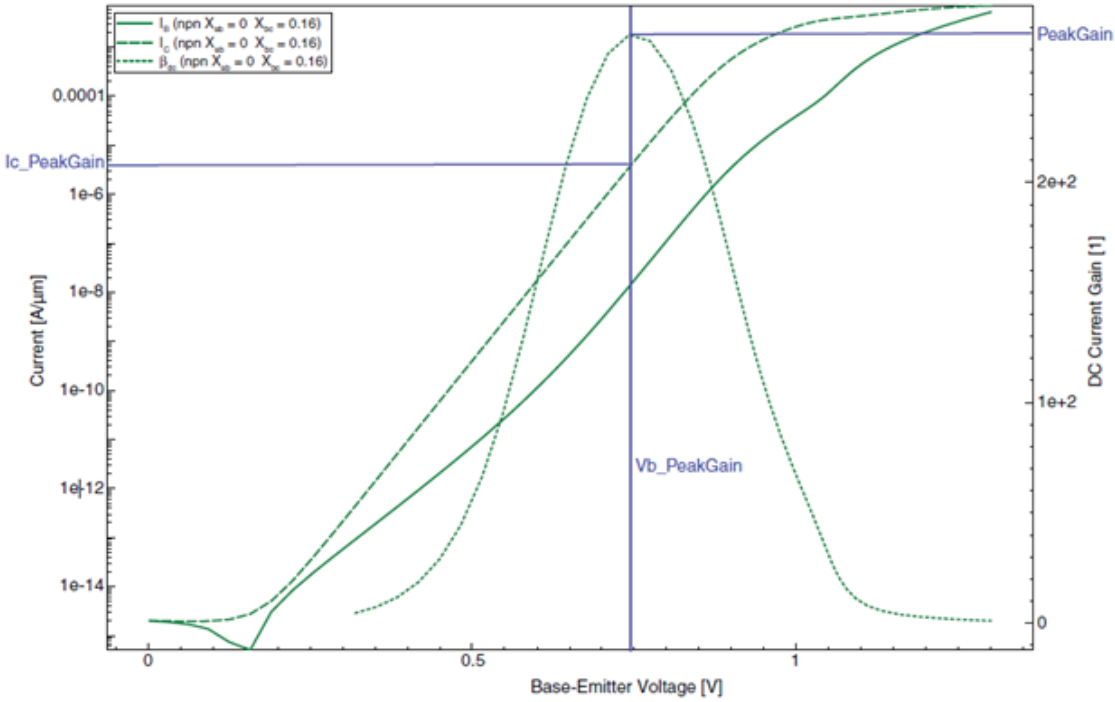


Figure 4 Gummel plot showing extracted targets in example



Sentaurus Calibration Workbench MLCal Script

```
# Import libraries
from scw.core.scwglobal import *
from scw.core.parameter import Parameter
from scw.core.target import Target
from scw.apps.mlcal import MLCal
from scw.core.model import Model

# Definition of parameters and targets
g0 = Parameter("G0", 0.075, minimum=0.0375, maximum=0.1125)
g1 = Parameter("G1", 0.225, minimum=0.1125, maximum=0.3375)
g2 = Parameter("G2", 0.45, minimum=0.225, maximum=0.675)
g3 = Parameter("G3", 0.1, minimum=0.05, maximum=0.2)
wcol = Parameter("Wcol", 0.2, minimum=0.1, maximum=0.3)
wemi = Parameter("Wemi", 0.15, minimum=0.075, maximum=0.225)
dwc = Parameter("dWc", 0.1, minimum=0.05, maximum=0.15)
dwb = Parameter("dWb", 0.075, minimum=0.0375, maximum=0.1125)
xeb = Parameter("Xeb", 0, minimum=0, maximum=0.32)
xbc = Parameter("Xbc", 0.16, minimum=0, maximum=0.32)
hsub = Parameter("Hsub", 0.3, minimum=0.15, maximum=0.45)
hcol = Parameter("Hcol", 0.3, minimum=0.15, maximum=0.45)
hbas = Parameter("Hbas", 0.075, minimum=0.0375, maximum=0.1125)
hbc = Parameter("Hbc", 0.075, minimum=0.0375, maximum=0.1125)
hemi = Parameter("Hemi", 0.225, minimum=0.1125, maximum=0.3375)
np_type = Parameter("Type", "npn", choices=["npn"])
```

```
pg = Target("PeakGain", values=2000, abs_value_range=[750, None])
ic = Target("Ic_PeakGain", values=1e-6,
            abs_value_range=[None, 1.5e-05])

vb = Target("Vb_PeakGain", range=[0.6, 0.8])
dip = Target("HasIbDip", range=[None, 0.5])

parameters = [g0, g1, g2, g3, wcol, wemi, dwc, dwb, xeb, xbc,
              hsub, hcol, hbas, hbc, hemi, np_type]
              targets = [pg, ic, vb, dip]

# Declare project options
mlcal0 = MLCal()
mlcal0.set_option(parent="@pwd@/../parent/HBT_parent")
mlcal0.set_option(child_dir="@pwd@/../child-internal-HBT")
mlcal0.set_option(tools=["sde", "Gummel", "PlotGummel"])
mlcal0.set_option(queue="sge:tcad")

# Evaluate
mlcal0.evaluate(parameters=parameters, targets=targets)

# Define Machine-Learning Model
hyperparameters = dict()
hyperparameters["PeakGain"] = {"model_type": MLPR,
                               "model_hyperparameters":
                              {"hidden_layer_sizes":
                               (512,256,128,64,32),
                               "max_iter": 10000,
                               "tol": 5e-4,
                               "alpha": 1e-3},
                               "retrain": True}
hyperparameters["Ic_PeakGain"] = {"model_type": MLPR,
                                  "model_hyperparameters":
                                 {"hidden_layer_sizes":
                                  (512,256,128,64,32,16),
                                  "max_iter": 10000,
                                  "tol": 5e-4,
                                  "alpha": 1e-2} ,
                                  "retrain": True}
hyperparameters["Vb_PeakGain"] = {"model_type": Linear,
                                  "retrain": True}
hyperparameters["HasIbDip"] = {"model_type": Linear,
                               "retrain": True}
ml_0 = Model(name="ml_0", hyperparameters=hyperparameters)
             models = [ml_0]

# Create the ML model and generate data
mlcal0.create_ml(models=models, parameters=parameters,
                 targets=targets,
                 data_generation={"doe_method"="adaptive",
                                  "doe_size": 100,
                                  "random_seed": 1,
```

```
                                    "batch_size": 50,
                                    "pass_rate": 0.9,
                                    "threshold_num_pass": 10,
                                    "transformer": Standard,
                                    "classifier": DecisionTree}
                  )

# Adjustment target ranges for narrower training data set
pg.range=[None, 10000]
ic.range=[None, 4e-5]
vb.range=[0.5, 0.8]

# Train model
mlcal0.train_ml(models=models, parameters=parameters,
                targets=targets, retrain=True, data_set="trimmed")

# Reverse Search
targets = [pg, ic]
mlcal0.reverse_search_ml(models=models, parameters=parameters,
                         targets=targets, doe_method="uniform",
                         doe_size=100000, num_candidates=20,
                         update_parameters="value")

# Generate TCAD data for the 20 candidate parameter sets
mlcal0.generate_data_ml(parameters=parameters, targets=targets,
                        child_dir="@pwd@/../results_child",
                        update_parameters=True)

# Report best result from the TCAD data
mlcal0.print_parameters(parameters=parameters)
mlcal0.print_targets(targets=targets)
for target in targets:
    print(f"{target.name} Error: {mlcal0.get_rms_error()}")
```

Fitting for Etching Profile From Data File

This example demonstrates how to load data from a file, apply name-merging rules and encoding methods for a high-dimensional profile, and perform training, prediction, and reverse search. The data is prepared before the machine-learning practice and has been extracted into `data.csv` files.

The features include process parameters (gas flow, C4F8 gas ratio, power, …) and the times at which different profiles are taken (time_01, time_02, and time_03). The outs are three profile files for each of those time steps: profile_01, profile_02, and profile_03. The profile files are CSV files containing the Z and AverageRadius columns, which define the profile.

During the training, the three time steps and profiles are merged, so that the machine-learning model can predict the resulting profile with the given experiment conditions and time.

Sentaurus Calibration Workbench MLCal Script

```
# Import libraries
from scw.core.scwglobal import *
from scw.core.parameter import Parameter
from scw.core.target import Target
from scw.apps.mlcal import MLCal
from scw.core.model import Model

# Define parameters and targets (initial value required for all)
p_gf = Parameter(name="Gas_flow_sccm", value=0.)
p_cfr = Parameter(name="C4F8_ratio", value=0.)
p_o2r = Parameter(name="O2_ratio", value=0.)
p_pres = Parameter(name="Pressure_mTorr", value=0.)
p_pow = Parameter(name="Power_W", value=0.)
p_ac_cur = Parameter(name="AC_current_A", value=0.)
p_bias = Parameter(name="Bias_frequency_MHz", value=0.)
p_time_1 = Parameter(name="time_01", value=0.)
p_time_2 = Parameter(name="time_02", value=0.)
p_time_3 = Parameter(name="time_03", value=0.)
p_time_all = Parameter(name="time", value=0.)

t_prf_1 = Target("profile_01", values="target.csv",
                 simulation_variables=["Z", "AverageRadius"])
t_prf_2 = Target("profile_02", values="target.csv",
                 simulation_variables=["Z", "AverageRadius"])
t_prf_3 = Target("profile_03", values="target.csv",
                 simulation_variables=["Z", "AverageRadius"])
t_prf_all = Target("profile", values="target.csv",
                   simulation_variables=["Z", "AverageRadius"])

param_all = [p_gf, p_cfr, p_o2r, p_pres, p_pow, p_ac_cur, p_bias,
             p_time_all]
param_create_only = [p_time_1, p_time_2, p_time_3]

tar_all = [t_prf]
tar_create_only = [t_prf_1, t_prf_2, t_prf_3]

#Create merge table for the profiles and times
merge_table = {"time":["time_01", "time_02", "time_03"],
               "profile":["profile_01", "profile_02",
                          "profile_03"]}

# Declare project options
# The parent project is the project from which the data was generated
mlcal0 = MLCal()
mlcal0.set_option(parent="@pwd@/../parent/etch_parent")
mlcal0.set_option(child_dir="@pwd@/../child-internal-etch”)
mlcal0.set_option(tools=["etch_step"])
mlcal0.set_option(queue="sge:tcad")

# Define Machine-Learning Model
hyperparameters = dict()
```

```
hyperparameters["profile"] = {"model_type": MLPR,
                               "model_hyperparameters":
                              {"hidden_layer_sizes": (256,256,256),
                               "max_iter": 10000,
                               "early_stopping": True,
                               "n_iter_no_change": 1000,
                               "alpha": 0,
                               "learning_rate": "adaptive"},
                               "retrain": False,
                               "parameter_scaler": Standard,
                               "target_scaler": Standard,
                               "encoder_type": PointSamplingEncoder,
                               "encoder_params":
                              {"use_floating_range": True ,
                               "num_data_points": 50}}
ml_0 = Model(name="ml_0", hyperparameters=hyperparameters)
models = [ml_0]

# Create machine learning model using existing data
parameters = param_all + param_create_only
targets = tar_all + tar_create_only
mlcal0.create_ml(models=models, parameters=parameters,
                 targets=targets, train_data=["data.csv"],
                 merge_table=merge_table)

# Train model
parameters = param_all
targets = tar_all
mlcal0.train_ml(models=models, parameters=parameters,
                targets=targets, retrain=True)

# Perform reverse search
mlcal0.reverse_search_ml(100000, doe="latin-hypercube",
                         num_candidates=50,
                         update_parameters="range")
mlcal0.reverse_search_ml(100000, doe="uniform", num_candidates=5,
                         optimize=True, update_parameters="value")

# Predict profile for best reverse search result
mlcal0.predict_ml(models=models, parameters=parameters,
                  targets=targets, doe_method="grid", doe_size=1)
```

High-Level Machine Learning Functions

The high-level machine learning functions include all the elements of a machine-learning based calibration or optimization algorithm as described earlier in this chapter. However, the machine learning models used by these algorithms are predefined and need not be set by you. Whereas the functions of the previous section are unique to `mlcal`, the following functions are included in all modules. They consist of:

- `explore_ml`: Explores the parameter space using a DoE and then finding the best candidates using reverse search
- `refine_ml`: Performs a more local search and DoE than `explore_ml` but also trains a machine learning model and uses reverse search to find the best candidates
- `calibrate_ml`: Uses both `explore_ml` and `refine_ml` as part of a general calibration strategy

explore_ml and refine_ml

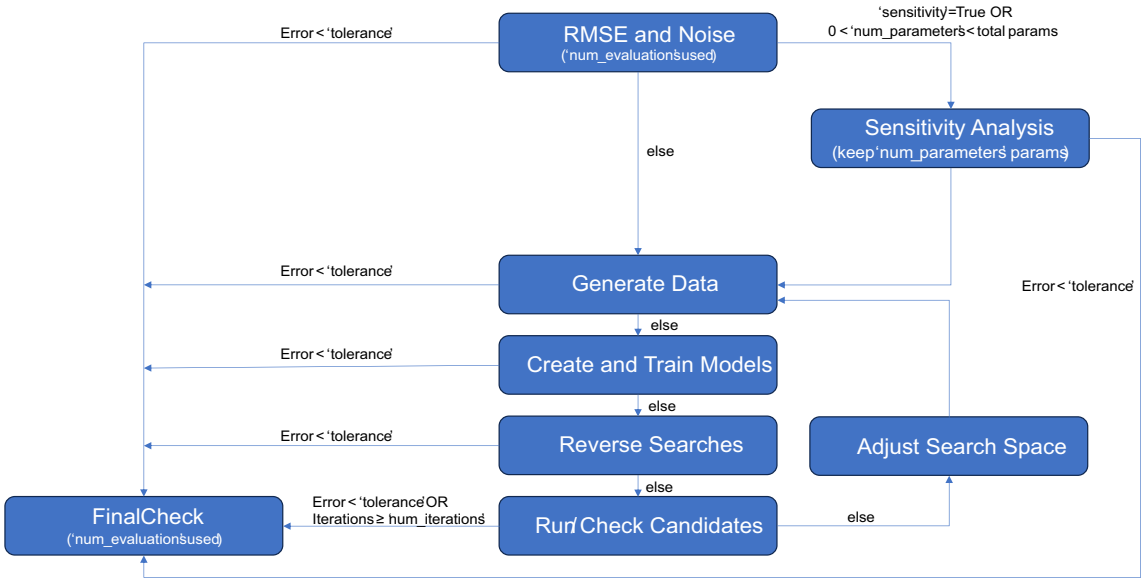
Both `explore_ml` and `refine_ml` use very similar high-level algorithms. They both have relatively large function signatures:

```
mlcal_obj.explore_ml(parameters=<t>, targets=<t>, [parent=<s>,]
                     [child_dir=<s>,] [task_name=<s>,]
                     [tolerance=<f>,] [num_iterations=<i>,]
                     [num_parameters=<i>,]
                     [explore_doe_range=<f|i>,]
                     [explore_doe_method=<k>,]
                     [explore_doe_size=<i>,] [explore_doe=<s|p>,]
                     [explore_doe_sep=<s>,] [sensitivity=<b|s|d>,]
                     [constraint=<s>,] [priority_targets=<t>,]
                     [num_retries=<i>,] [num_evaluations=<i>])

mlcal_obj.refine_ml(parameters=<t>, targets=<t>, [parent=<s>,]
                     [child_dir=<s>,] [task_name=<s>,]
                     [tolerance=<f>,] [num_iterations=<i>,]
                     [num_parameters=<i>,] [sensitivity=<b|s|d>,]
                     [constraint=<s>,] [num_retries=<i>,]
                     [num_evaluations=<i>])
```

To assist with understanding all of the arguments, a flowchart that summarizes the algorithm used in both functions is shown in Figure 5. The details of each step can differ between `refine_ml` and `explore_ml`. After every step, all of the experiments previously run are assessed to determine which result is the best and whether the tolerance has been met. If the tolerance is met, then a final evaluation is performed.

Figure 5 Flowchart description of explore_ml and refine_ml tasks



RMSE and Noise: The parent project is evaluated `num_evaluations` times. The RMSE is taken from the average and the variation between the simulated values is assessed to see whether a calibration within tolerance is likely possible. This is mostly a concern for TCAD simulations where there can be run-to-run variation.

Sensitivity Analysis: If only a subset of parameters are to be used during calibration, then a sensitivity analysis is run to determine which parameters have the greatest impact on the target values. Based on this analysis, the top `num_parameters` parameters are used during the following calibration/optimization.

Generate Data: Child Sentaurus Workbench projects are run and data generated. This experiment differs between `explore_ml` and `refine_ml`. In `explore_ml`, the experiment is defined by the user, while `refine_ml` uses a DoE predefined in Sentaurus Calibration Workbench. The key arguments used to define the DoE used by `explore_ml` are:

- `explore_doe_range (float or filename)`: Sets the parameter range scaling used for the DoE. A value of 1.0 means the full range. If a file name is provided, then it sets the name of a CSV file that specifies the parameter ranges for the DoE.
- `explore_doe_method (keyword)`: Sets the DoE method. Options are "`box-behnken`", "`full-factorial`", "`gaussian`", "`latin-hypercube`", "`uniform`", "`sobol`", and "`custom`".
- `explore_doe_size (int)`: Sets the number of samples for the DoE.

- `explore_doe (filename or DataFrame)`: Sets the DoE of the CSV file path or pandas data frame when `explore_doe_method` is "custom".

Create and Train Models: Machine learning models are created and trained on the generated data.

Reverse Searches: A series of reverse searches are run using the trained models to try and find the best set of parameters to match the targets. Several candidate parameter sets are suggested. The accuracy of `priority_targets` is given extra weight during the reverse searches.

Run/Check Candidates: Sentaurus Workbench experiments are run using the candidate parameter sets and the best selected. If the result is within tolerance or the maximum number of iterations has been exceeded, then the algorithm moves to a final check of the best results.

Adjust Search Space: If the result is not within tolerance and the maximum number of iterations has not been reached, the search space is adjusted based on the results so far. This differs between `explore_ml` and `refine_ml`, with `explore_ml` changing the parameter bounds while keeping a relatively broad search space and `refine_ml` focusing more on moving the center point of the parameter space. Higher `num_iterations` tend to be more useful for `refine_ml` than for `explore_ml`. A single iteration for `explore_ml` and then moving on to `refine_ml` or another method is often the most efficient course of action.

Final Check: The Sentaurus Workbench project is evaluated using the best parameter set it found `num_evaluations` times. The RMSE is taken from the average.

For more information, see explore_ml on page 243 and refine_ml on page 270.

calibrate_ml

The `calibrate_ml` function iterates through the `explore_ml` and `refine_ml` steps to find parameters that result in the desired target values. Like `explore_ml` and `refine_ml`, it has many possible arguments:

```
calibrate_ml(parameters=<t>, targets=<t>, [parent=<s>,]
             [child_dir=<s>,] [task_name=<s>,] [tolerance=<f>,]
             [num_iterations=<i>,] [num_validations=<i>,]
             [num_parameters=<i>,] [num_explore_parameters=<i>,]
             [num_refine_parameters=<i>,] [explore_doe_range=<f>,]
             [explore_doe_method=<k>,] [explore_doe_size=<i>,]
             [tolerance_refine=<f>,] [strategy=<s>,]
             [validate_range=<b>,] [sensitivity=<b|s|d>,]
             [constraint=<s>,] [priority_targets=<t>,]
             [num_retries=<i>,] [num_evaluations=<i>])
```

Either `explore_ml` or `refine_ml` is called until the RMSE error is less than `tolerance` or `num_iterations` has been reached. The decision to run `refine_ml` or `explore_ml` or `validate_range` is based on several factors.

In Table 12, the parameters which affect each step within the `calibrate_ml` task are shown. They are equivalent to the same setting for the individual function(s). In addition, there are arguments that adjust the flow of the `calibrate_ml` algorithm:

- `tolerance (float)`: Sets the total relative RMS error tolerance
- `tolerance_refine (float)`: Sets the tolerance at which all subsequent iterations must be refine_ml. Note that refine_ml may be called even if this tolerance is not yet met as it only defines one case in which it would be called.
- `validate_range (bool)`: Specifies whether to perform an initial parameter range validation analysis.
- `strategy (string)`: Specifies the strategy. For good initial parameter values and ranges, specify "standard". “standard” strategy performs the range validation and the sensitivity analysis at the beginning of the workflow. If the initial parameters are far from optimal or the nominal simulation failed, then specify "validate". The “validate” strategy performs several times of the range validations and the sensitivity analysis during while running the calibration.

Table 12 calibrate_ml sub-tasks and their relevant arguments

explore_ml Steps	refine_ml Steps	Validate Range Steps	calibrate_ml Flow
num_explore_parameters	num_refine_parameters	num_validations	tolerance
explore_doe_range	constraint		tolerance_refine
explore_doe_method	num_retries		validate_range
explore_doe_size	num_evaluations		strategy
priority_targets	tolerance		
constraint	sensitivity		
num_retries			
num_evaluations			
tolerance			
sensitivity			

For more information, see calibrate_ml on page 231.

References

[1] For more information about the scikit-learn package, go to https://scikit-learn.org/stable/index.html.

[2] For more information about bagging regressors, go to https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingRegressor.html#sklearn.ensemble.BaggingRegressor

[3] For more information about the gradient boosting regressor, go to https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html#sklearn.ensemble.GradientBoostingRegressor

[4] For more information about histogram gradient boosting regressor, go to https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html#sklearn.ensemble.HistGradientBoostingRegressor

[5] For more information about the extra trees regressor, go to https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.ExtraTreesRegressor.html#sklearn.ensemble.ExtraTreesRegressor

[6] For more information about the Gaussian process regressor, go to https://scikit-learn.org/stable/modules/generated/sklearn.gaussian_process.GaussianProcessRegressor.html#sklearn.gaussian_process.GaussianProcessRegressor

[7] For more information about the linear regressor, go to https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html#sklearn.linear_model.LinearRegression

[8] For more information about multilayer perceptron regressor, go to https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html#sklearn.neural_network.MLPRegressor

[9] For more information about the random forest regressor, go to https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html#sklearn.ensemble.RandomForestRegressor

[10] For more information about polynomial feature interaction, go to https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PolynomialFeatures.html

[11] For more information about sequential feature selection, go to https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SequentialFeatureSelector.html

[12] For more information about recursive feature elimination, go to https://scikit-learn.org/stable/modules/feature_selection.html#recursive-feature-elimination.html

[13] For more information about SHAP, go to https://shap.readthedocs.io/

[14] For more information about GridSearch optimization, go to https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

[15] For more information about Bayesian optimization, go to https://scikit-optimize.github.io/stable/modules/generated/skopt.BayesSearchCV.html

[16] For more information about the correlation pairplot, go to https://seaborn.pydata.org/generated/seaborn.pairplot.html

[17] For more information about the AdaBoost classifier, go to https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html

[18] For more information about the decision tree classifier, go to https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

[19] For more information about the Gaussian process classifier, go to https://scikit-learn.org/stable/modules/generated/sklearn.gaussian_process.GaussianProcessClassifier.html

[20] For more information about the K nearest neighbors classifier, go to https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

[21] For more information about the multilayer perceptron classifier, go to https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

4

Modules for Calibrating Device Simulations

This chapter describes the application-specific calibration modules for device simulations.

This chapter presents the calibration modules for devices:

- General Device Calibration: devcal Module
- Hot-Carrier Stress Degradation Calibration

General Device Calibration: devcal Module

The `devcal` module allows you to calibrate device simulators by defining your own calibration strategy as well as application-specific predefined calibration strategies.

At the beginning of the Sentaurus Calibration Workbench command file, import the module. For example:

```
from scw.apps.devcal import DevCal
```

Then, instantiate a calibration object from the imported module class. For example:

```
devcal = DevCal()
```

Now, this calibration object can be used to manage the calibration flow, including parameter, target, option, and task specifications (see Chapter 2 on page 20).

Hot-Carrier Stress Degradation Calibration

With the trend toward technology miniaturization and the need for higher on-chip integration densities, device reliability issues are a major concern. Leakage and reliability are especially critical when designing input and output (IO) devices or memory peripheral devices, which operate at high voltage. The charged carriers in the channel of such transistors, accelerated by the large electric field, can gain enough high energy to damage the semiconductor–insulator interface.

The term *hot-carrier degradation* in modern devices comprises several microscopic physical mechanisms, such as interface Si-H bond dissociation by interaction with single high energetic (*hot*) carriers or with a series of *cold* carriers, bond breakage due to field-enhanced thermal interaction with the lattice, and the impact of bond energy dispersion on interface traps generation.

The hot-carrier stress (HCS) degradation model in Sentaurus Device combines all these microscopic physical aspects into a simple analytic description, suitable for modeling time-dependent interface trap generation. For details, see *Sentaurus™ Device User Guide*, Hot-Carrier Stress Degradation Model.

However, the HCS degradation model, based on [1], was originally designed for IO devices, more specifically for lateral double-diffused metal–oxide–semiconductor (LDMOS) transistors with shallow trench isolation. Even though the essential physics of hot-carrier degradation is accounted for in the HCS degradation model, major adjustments to the model parameters might be needed, when applying it to the degradation study of a different technology. Such a parameter calibration process can be complex and time-consuming, since it requires a deep understanding of the models involved.

The `devcal` module is designed to overcome this limitation, by providing a tool for fast experimental data calibration or, at least, for a significant support to it. In addition to interface trap formation, another important aspect of the hot-carrier degradation analysis consists of modeling the detrimental effect that generated traps have on carrier mobility. Therefore, to provide a complete tool for the calibration of all the I_d–V_g characteristics at different stress times, which can be used to extract the threshold voltage or the drain current variations as a function of stress time, the `devcal` module also takes into account, in the calibration process, the parameters of the Coulomb scattering mobility model that depends on the interface trap concentration as well as the parameters to define the energy distribution of the generated traps. For details, see *Sentaurus™ Device User Guide*, Mobility Degradation Components due to Coulomb Scattering.

The `devcal` module has been tested mostly on measurements of memory peripheral MOSFETs. However, it can help the calibration process of MOS-based devices in general.

Input Data

Two calibration commands are available: one for NMOS devices and one for PMOS devices. The degraded NMOS experimental curves can be calibrated with acceptor-type traps only. For PMOS devices, both donor- and acceptor-type traps are needed. This implies some minor differences in the required inputs to the calibration functions that are presented here.

You must provide the following inputs:

- As part of the parent project, the Sentaurus Device command file, which includes all the commands needed to perform the time-dependent degradation simulation followed by

the I_d–V_g simulation post-degradation, must be provided. In detail, quasistationary ramps to the stress biases, followed by the transient degradation simulation with different final times, must be performed. The transient final time must be defined as a Sentaurus Workbench parameter, for example, `time`, and added as a condition parameter for targets in Sentaurus Calibration Workbench. The values correspond to the stress times at which the experimental curves are extracted.

The HCS degradation model must be activated in the `Traps` statement of the `Physics` section relative to the material or region interface where the traps are generated, with the options `SHE` and `BondDispersion` (see *Sentaurus™ Device User Guide*, Hot-Carrier Stress Degradation Model).

For NMOS devices, this would be:

```
Traps(
   (Acceptor Gaussian EnergyMid=@EmidA@ EnergySig=@EsigA@
    FromMidBandGap eHCSDegradation(SHE BondDispersion))
)
```

Both trap types are needed for PMOS devices:

```
Traps (
   (Acceptor Gaussian EnergyMid=@EmidA@ EnergySig=@EsigA@
    FromMidBandGap
    eHCSDegradation(SHE BondDispersion))
   (Donor Gaussian EnergyMid=@EmidD@ EnergySig=@EsigD@
    FromMidBandGap
    hHCSDegradation(SHE BondDispersion))
)
```

As shown here, a Gaussian energy distribution is assumed for the generated traps, with the arbitrary initial values of `EnergyMid` (position of the Gaussian peak) and `EnergySig` (standard deviation) defined by Sentaurus Workbench parameters.

The corresponding calibration parameters must be defined for Sentaurus Calibration Workbench in the following way. For NMOS devices, this is:

```
EmidA = Parameter('EmidA', 0, -0.5, 0.5)
EsigA = Parameter('EsigA', 0.1, 0.1, 0.3)
```

For PMOS devices, this is:

```
EmidA = Parameter('EmidA', -0.35, -2.0, 0.0)
EsigA = Parameter('EsigA', 0.5, 0.05, 1.0)
EmidD = Parameter('EmidD', -0.55, -0.7,-0.3)
EsigD = Parameter('EsigD', 0.2, 0.1, 0.4)
```

The Coulomb scattering mobility model that depends on the interface trap concentration must be activated in the semiconductor `Physics` section of the Sentaurus Device input file for the material or region interface where the traps are generated (see *Sentaurus™ Device User Guide*, Mobility Degradation Components due to Coulomb Scattering).

- In the Sentaurus Calibration Workbench input file, the Sentaurus Device parameter file must also be specified in the `set_option()` command in the list of `files_to_copy`. This file must not include the parameters of the HCS degradation model and the Coulomb scattering mobility model. They are calibrated by the `devcal` calibration module starting from the default values and provided as output of the simulation.
- The `.plt` file contains the gate voltage and drain current data to generate the I_d–V_g characteristics.
- If the degradation simulation and the post-degradation I_d–V_g simulation are run separately, then the `.tdr` file is required, which is used to transfer the generated interface trap distribution.
- The targets must be defined in the following way:

  If you want to directly compare the measured and simulated I_d–V_g characteristics, then the I_d–V_g data must be defined as profile targets. In this case, Sentaurus Calibration Workbench automatically calculates internally and minimizes the root mean square (RMS) error of such curves.

  However, if you want to define a different type of error to be optimized (for example, linear or saturation drain current variation at a specific gate voltage, or threshold voltage variation), then you must provide a Sentaurus Visual script for the error and variable extraction, and define the corresponding scalar targets.

  In either case, for the classification of the provided targets, you must follow a specific target type for I–V plots:

  - Include `type=IdVgLin` in the target definition for linear drain currents.
  - Include `type=IdVgSat` in the target definition for saturation drain currents.
  - Include `type=IdVgOffSat` in the target definition for off-state saturation drain currents.

Calibration Strategies

The calibration strategies are implemented in the `devcal` calibration functions. They consist of a sequence of parameter set evaluations, conditional operations, and gradient-based calibration steps, which follows the logic of calibration work performed by expert engineers.

To develop and test the calibration strategies, experimental degraded I_d–V_g characteristics in linear and saturation regimes at different stress times, different stress bias conditions (on-state and off-state degradation) for both n- and p-type devices at two different temperatures have been used. In general, the experimental data shows not only a threshold voltage shift, but also a significant subthreshold slope degradation and, in some cases, a nonmonotonic current variation with stress time. To reproduce these effects, the parameters

of the generated trap energy distribution and the mobility degradation due to traps must be calibrated, in addition to the HCS degradation model parameters.

Due to its complexity and the large number of calibration parameters, the calibration task is divided into subtasks, which are performed in a specific order. Starting from the physics-limited range of variation for each parameter, the idea is to restrict the variation range with evaluation steps before performing a gradient-based calibration step, thereby increasing the chances of success for the calibration. The calibration strategy exploits the characteristics of the HCS degradation model by allowing the separate analysis of its three interface trap formation contributions, that is, the single-particle process, the multiple-particle process, and the field-enhanced thermal process (see *Sentaurus™ Device User Guide*, Hot-Carrier Stress Degradation Model).

Due to the different behavior shown by the experimental data and the formation of both acceptor and donor traps, the calibration strategy for PMOS devices differs from the one for NMOS devices, and it requires a longer turnaround time.

The `devcal` calibration command for NMOS devices is:

```
devcal.calibrate_hcsd_nmos(parameters=<t>,
                           targets=<s>,
                           use_thermal=<b>,
                           condition_variable=<s>,
                           parent=<s>,
                           child_dir=<s>)
```

The argument `parameters` includes a list of exactly two user-defined Sentaurus Workbench parameters. The first one is the Sentaurus Device acceptor trap distribution central energy `EmidA`, and the second one is the Sentaurus Device acceptor trap distribution energy width `EsigA`. For details about the definition of these parameters, see Input Data on page 100.

The argument `use_thermal` specifies whether to use the field-enhanced thermal component of the HCS degradation model in Sentaurus Device, which is set to `True` by default.

The target condition parameter for the transient final time, for example, `time`, must be specified by the argument `condition_variable`.

For a description of all arguments, see calibrate_hcsd_nmos on page 224.

The `devcal` calibration command for PMOS devices is:

```
devcal.calibrate_hcsd_pmos(parameters=<t>,
                           targets=<s>,
                           condition_variable=<s>,
                           parent=<s>,
                           child_dir=<s>)
```

The argument `parameters` includes a list of exactly four user-defined Sentaurus Workbench parameters:

- The first one is the Sentaurus Device acceptor trap distribution central energy `EmidA`.
- The second one is the Sentaurus Device acceptor trap distribution energy width `EsigA`.
- The third one is the Sentaurus Device donor trap distribution central energy `EmidD`.
- The fourth one is the Sentaurus Device donor trap distribution energy width `EsigD`.

For details about the definition of these parameters, see Input Data on page 100.

The target condition parameter for the transient final time, for example, `time`, must be specified by the argument `condition_variable`.

For a description of all arguments, see calibrate_hcsd_pmos on page 225.

Finally, both calibration commands `devcal.calibrate_hcsd_nmos` and `devcal.calibrate_hcsd_pmos` have as output a Sentaurus Device Parameter file `sdevice.par`, which adds the calibrated HCS degradation model and interface charge mobilities to the base calibration of the parent project.

References

[1] S. Reggiani *et al.*, “TCAD Simulation of Hot-Carrier and Thermal Degradation in STI-LDMOS Transistors,” *IEEE Transactions on Electron Devices*, vol. 60, no. 2, pp. 691–698, 2013.

[2] G. Torrente *et al.*, “Hot Carrier Stress modeling: from degradation kinetics to trap distribution evolution,” in *IEEE International Integrated Reliability Workshop (IIRW)*, South Lake Tahoe, CA, USA, pp. 134–137, October 2015.

5

Modules for Calibrating Process Simulations

This chapter describes the application-specific calibration modules for process simulations.

This chapter describes the calibration modules for process simulations:

- General Process Calibration: procal Module
- Implantation Dual-Pearson Moments Calibration
- Legendre Polynomial Implantation Model Calibration

General Process Calibration: procal Module

The `procal` module allows you to calibrate process simulators by defining your own calibration strategy as well as application-specific predefined calibration strategies.

At the beginning of the Sentaurus Calibration Workbench command file, import the module. For example:

```
from scw.apps.procal import ProCal
```

Then, instantiate a calibration object from the imported module class. For example:

```
procal = ProCal()
```

Now, this calibration object can be used to manage the calibration flow, including parameter, target, option, and task specifications (see Chapter 2 on page 20).

Implantation Dual-Pearson Moments Calibration

Ion implantation is an essential technique that introduces dopant (ion) atoms into semiconductor material. Sentaurus Process simulates the spatial distribution of ions by using either analytic implantation or Monte Carlo methods.

Analytic implantation functions can be described by the dual-Pearson (`DualPearson`) model, and Sentaurus Process provides different implantation tables for a wide range of

implantation conditions and various species. However, sometimes, you need to calibrate `DualPearson` moments to obtain accurate implantation profiles that match experimental SIMS profile data or to obtain the moments for implantation conditions that are outside the range of the implantation tables.

The implantation calibration command (`calibrate_implant_dualpearson`) executes a predefined calibration strategy and then saves the calibrated moments in a file, which Sentaurus Process can use.

The implantation moments calibration strategy supports the following:

- Moments calibration for the dual-Pearson model
- Species in silicon

Target

The target for this calibration is a single 1D SIMS profile data. If no experimental data is available, then you can run a Monte Carlo simulation to be used as the target data.

An example of a target definition is:

```
target = Target("tcad_profile", id="imp_Anode",
                value="./SIMS/Nimp_Anode.plx",
                conditions={"sims_name" : "imp_Anode",
                            "field" : "BTotal",
                            "species" : "Boron",
                            "energy" : "60",
                            "dose" : "2E13",
                            "tilt" : "0",
                            "Tox_nm" : "1.5"},
                reference_variables=["", "BTotal"],
                simulation_variables=["", "BTotal"],
                x_range=[0.07, 0.67],
                y_range=[1E14, 1E22],
                error_method="log_abs",
                tolerance=0.0)
```

The PLX data file format does not have the x-variable name; therefore, in this example, both the reference and simulation x-variables are set as `""`. In addition, `"log_abs"` is selected for the method to calculate the root mean square (RMS) error between the target and TCAD simulation.

Parameters

The strategy calibrates the implantation moments as parameters to fit SIMS target data.

For the dual-Pearson model, the calibration parameters are:

- `beta`
- `beta2`
- `gamma`
- `gamma2`
- `ratio`
- `rp`
- `rp2`
- `stdev`
- `stdev2`

The strategy automatically generates the parameters, so you do not need to define them. It extracts the values of the default moments corresponding to the species and implantation conditions (such as dose, energy, tilt, and oxide thickness) and sets them as the initial values of the parameters for the calibration.

Required Sentaurus Workbench Project Tool Database Setup

The implantation moments calibration strategy writes the parameter file including commands to extract the initial moments or to update the moments during calibration.

The parameter file is named `calib.fps`, so you must modify the tool database of the loaded parent project.

To access the tool database in Sentaurus Workbench, choose **Edit** > **Tool DB** > **Project**. Then, add the following commands in the tool database:

```
set WB_tool(sprocess,input,calibration,file) "calib.fps"
set WB_tool(sprocess,input,calibration,label) "Calibration..."
```

In addition, in the Sentaurus Process command file, you must source the parameter file before the implantation condition definition. For example:

```
## Source parameter file named calib.fps
source @calibration@

implant @species@ dose=@dose@ energy=@energy@ tilt=@tilt@
```

Required Sentaurus Workbench Variables

The parameter file (`calib.fps`) for the initial moments extraction includes Sentaurus Workbench variables to be replaced by values as follows:

```
set mom [implant @species@ dose=@dose@ energy=@energy@ tilt=@tilt@
         oxide.thickness=@Tox_nm@<nm> get.moments]
```

The following Sentaurus Workbench variables must be defined in the parent project:

- `dose`
- `energy`
- `species`
- `Tox_nm` (screen oxide thickness in nm)

The following Sentaurus Workbench variable is optional:

- `multrot` (no period; it corresponds to `mult.rot` in the `implant` command)

Note:

If the implantation condition includes `mult.rot` in the `implant` command, then the `multrot` variable must be set up in the project.

Calibration Strategy

The implantation moments calibration strategy is as follows:

1. Perform the initial evaluation with the default implantation moments.
2. Run a sensitivity analysis on the parameters.
3. Perform the first calibration with primary parameters selected from the sensitivity analysis.
4. Perform the second calibration with all parameters.
5. Save the calibrated moments in the `{outfile_prefix}implant_calib.fps` file.

The command that executes the implantation `DualPearson` moments calibration strategy is:

```
procal.calibrate_implant_dualpearson(targets=targets)
```

To run only the initial evaluation, specify:

```
procal.calibrate_implant_dualpearson(targets=targets,
                                      evaluate_only=True)
```

If `run_sensitivity_only=True`, then the strategy stops after the sensitivity analysis. In addition, you can control the number of primary parameters by using the `num_parameters` argument.

For details about the `calibrate_implant_dualpearson` command, see calibrate_implant_dualpearson on page 226.

Legendre Polynomial Implantation Model Calibration

The analytic model of ion implantation based on Legendre polynomials implemented in the Sentaurus Process simulator is a computationally efficient and accurate method. The `procal` module can perform the calibration of the Legendre polynomial implantation model (L-PIM).

The calibration strategy for L-PIM calibration is as follows:

1. Create and run a grid design-of-experiments (DoE) for the specified material and species, for the defined implantation parameters (energy, dose, tilt, rotation, and oxide cap-layer thickness), based on a predefined extraction parent project designed to simulate primary and secondary implantation profiles. The implantation simulations are based on Sentaurus MC.
2. Extract the Legendre coefficients for doping and damage from the simulated primary and lateral implantation profiles, and write the coefficients into L-PIM table files.

The corresponding calibration command is:

```
procal.calibrate_implant_legendre(material=<s>, species=<s>,
                                  parameters=<t>)
```

For details about the `calibrate_implant_legendre` command and its arguments, see calibrate_implant_legendre on page 228.

The argument `parameters` is a list of parameter objects for the grid DoE, as described in Parameters on page 110.

To set the child project directory of the calibration task, either use the optional argument `child_dir`, for example:

```
procal.calibrate_implant_legendre(material=<s>, species=<s>,
                                  parameters=<t>, child_dir=<s>)
```

or use the command:

```
cal.set_option(child_dir=<s>)
```

To set the directory in which the output L-PIM tables are written, use the optional argument `table_dir`. By default, the tables are written into the subdirectory `./PIM` in the project directory.

The Legendre polynomial table format is used for the L-PIM tables and has the extension .pim. For amorphous materials such as oxide, it is recommended to convert the L-PIM tables to Pearson tables by selecting the Dios table format with the extension .tab, using the optional `table_format` argument.

In addition, you can also set the following arguments:

- `temperature`: Sets the implantation temperature (in degree Celsius). Default: 25
- `notch_dir`: Sets the wafer notch direction or orientation. The orientation is described by Miller indices (string), which can be separated by "_", for example "1_-1_0". Default: “110"
- `miscut_toward`: Sets the wafer miscut toward orientation. The orientation is described by Miller indices (string), which can be separated by "_", for example "-1_-1_0". Default: "110"
- `miscut_angle`: Sets the positive wafer miscut tilt angle (in degree). Default: `0.0`
- `beam_div`: Sets the implantation beam divergence. Default: `0.5`

Parameters

The input parameter list defines the TCAD simulation DoE to be used for the extraction of the Legendre coefficients. The Legendre polynomial implantation tables are only accurate within the parameter space of this DoE.

Since the extraction parent project is predefined, the parameter names of the implantation parameters are also predefined. Therefore, the parameters passed to the calibration commands are as follows:

- `dose`: Parameter for the implantation dose (positive, in cm^{-2})
- `energy`: Parameter for the implantation energy (positive, in keV)
- `rotation`: Parameter for the implantation rotation (positive, in degree)
- `tilt`: Parameter for the implantation tilt (positive, in degree)
- `screen`: Parameter for the screening cap-layer thicknesses (unitless)

You can define these parameters either as calibration parameters with a range defined by minimum and maximum or as condition parameters, which allow you to choose different discrete values.

6

Modules for Calibrating Power Device Simulations

This chapter describes the application-specific calibration modules for power device simulations.

Sentaurus Device is an essential simulator for the development of power electronics technology. You can use Sentaurus Calibration Workbench to calibrate the parameters of power device models in AC and DC characteristics.

The `igbtcal` module provides calibration strategies for insulated gate bipolar transistor (IGBT) AC and DC characteristics.

This chapter describes the calibration module for power devices:

- IGBT Device Calibration: igbtcal Module

IGBT Device Calibration: igbtcal Module

The IGBT is one of the key components in various power applications due to its low conductivity and switching losses. It is introduced to take advantage of both power MOSFET and BJT devices to achieve optimal device characteristics. Typically, the Sentaurus Process and Sentaurus Device simulators are used to characterize the optimal conditions for achieving the required performance of an IGBT. However, the accuracy of the simulation depends on experience, knowledge of model selection, and parameter calibration for the appropriate models. The selection of a physical model according to each device characteristic and a well-defined calibration strategy is essential for the success of IGBT calibrations.

The `igbtcal` module is designed to guide the calibration strategy of model parameters for IGBT device characteristics using Sentaurus Device. This module provides two calibration strategies: DC and AC characteristics. The DC calibration strategy optimizes the model parameters by calibrating the I_c–V_g, I_c–V_c, and breakdown voltage data automatically. The AC calibration strategy includes steps to calibrate the model and parameters to fit C–V, gate charging (Q_g), and switching characteristics. Here, the measured data (target) should be

curve or point data to be compared with the calibrated simulation results. In addition, it is important to select the parameters most sensitive to the target characteristics. As the number of parameters increases, the number of iterations increases.

At the beginning of the Sentaurus Calibration Workbench command file, import the module. For example:

```
from scw.apps.igbtcal import IGBTCal
```

Then, instantiate a calibration object from the imported module class. For example:

```
igbtcal=IGBTCal()
```

Now, this calibration object can be used to manage calibration inputs including parameter, target, option, and task specifications (see Chapter 2 on page 20).

`Target` and `Parameter` for the `igbtcal` module require you to specify the type of the simulation data, so that the module can classify the parameters and targets for each calibration step:

```
Parameter("C", 5.8e2, 5.8e1, 5.8e3, Log10, type=IcVg)

Target("sIcVg", id="IcVg", value="IGBT_25C_IcVg.plt",
   conditions={"IcVg": "1", "IcVc": "0", "BV":"0"},
   reference_variables=["x", "y"],
   simulation_variables=["gate OuterVoltage", "collector TotalCurrent"],
   error_method="log_abs", x_range=[0.5, 15.0],
   type=IcVg
)
```

The available types are as follows:

- DC calibration:
  - IcVg
  - IcVc
  - BV
- AC calibration:
  - CV
  - Qg
  - Switching

In `Target`, if the `error_method` is not specified, then the default error method is applied based on each type of simulation. For details, see Target on page 288.

IGBT DC Characteristics Calibration

The IGBT calibration module `igbtcal` allows you to calibrate IGBT DC device characteristics – I_c–V_g, I_c–V_c, and breakdown voltage – using interface charge and mobility, resist and high-field saturation, V_t shift, breakdown voltage, and so on.

To run an evaluation task for all included targets with the initial parameter values, specify:

```
igbtcal.calibrate_dc(parameters, targets, evaluate_only=True)
```

The calibration strategy performs multi-step calibration for the targets and parameters classified by types: IcVg, IcVc, and BV. If `run_initial_doe=True`, then each calibration step performs the design-of-experiments search first, then it starts the calibration at parameters with the minimum root mean square (RMS) error obtained from the initial search. The BV calibration step always runs the initial DoE search regardless of the value of `run_initial_doe`. The DoE options for `igbtcal` are `"latin-hypercube"`, `"uniform"`, and `"sobol"`. If targets are not found for a specific type, then the calibration step for that type is omitted. In addition, the target tolerance (one of the calibration stopping criteria) for each step can be controlled separately.

To increase accuracy, some parameters are recommended to include for calibration: *fixed charge* at the silicon–oxide interface (`Qit`) for IcVg and *contact resistance* (`resist`) for IcVc. In particular, if the `igbtcal` module cannot find a parameter named `resist` (case insensitive) in the IcVc parameter list, then the module omits the contact resistance calibration step.

The command that executes the DC calibration strategy is:

```
igbtcal.calibrate_dc(parameters=parameters, targets=targets)
```

For details about the `calibrate_dc` command, see calibrate_dc on page 217.

IGBT AC Characteristics Calibration

The IGBT calibration module `igbtcal` allows you to calibrate IGBT AC device characteristics – C–V, gate charge (Q_g), and switching simulations – using parasitic components, lifetime, recombination parameters, and so on.

In the AC calibration strategy, `Parameter` and `Target` are classified as `type=CV`, `Qg`, and `Switching`.

To run an evaluation task for all included targets with the initial parameter values, specify:

```
igbtcal.calibrate_ac(parameters, targets, evaluate_only=True)
```

The command that executes the AC calibration strategy is:

```
igbtcal.calibrate_ac(parameters=parameters, targets=targets)
```

This command runs the initial evaluation with all targets and then proceeds to the C–V, Q_g, and switching calibrations in a sequential order. When the calibration is finished, it proceeds to the final evaluation with the calibrated parameters and saves the parameter file as `{outfile_prefix}sdevice.par`.

In the same way as with the DC calibration, you can switch on the initial DoE search at each calibration step by specifying `run_initial_doe=True`.

For details about the `calibrate_ac` command, see calibrate_ac on page 215.

7

Modules for Calibrating Topography Simulations

This chapter describes the application-specific calibration modules for topography simulations.

Sentaurus Topography simulates deposition, etching, and simultaneous etching and deposition processes using physical and geometric models. Two simulation methods are used to implement the physical models: the level set method and the particle Monte Carlo method.

This chapter describes the calibration modules for topography:

- PMC Topography Calibration: pmccal Module
- HARC Etching Calibration

PMC Topography Calibration: pmccal Module

The `pmccal` module allows you to calibrate the particle Monte Carlo (PMC) method by defining your own calibration strategy. It also includes application-specific predefined calibration strategies as well as specific features to take into account the variability of the PMC method.

Note:

For the level set method, the `pmccal` module can be used as well, but so can the Sentaurus Calibration Workbench (`scwcal`) module.

At the beginning of the Sentaurus Calibration Workbench command file, import the module. For example:

```
from scw.apps.pmccal import PMCCal
```

Then, instantiate a calibration object from the imported module class. For example:

```
pmccal = PMCCal()
```

Now, this calibration object can be used to manage the calibration flow, including parameter, target, option, and task specifications (see Chapter 2 on page 20).

Reevaluation due to Variability

PMC and Monte Carlo flux calculations can show numeric variability from run to run due to numeric noise, even for the same parameter set. As a consequence, the evaluation and also the parameter calibration can show variability.

By default, simulations for a specific parameter and target set is only evaluated once, and results are reused in the following tasks if the parameter set and the target set are identical. However, for PMC calibration, it is beneficial to reevaluate it to prevent the calibration converging in a local optimum due to numeric noise and to continue to find a global optimum. Therefore, the evaluation and gradient-based local calibration tasks will reevaluate all simulations by default.

To run a simple evaluation task for all included targets with the current parameter values without reevaluation, use the command:

```
cal.evaluate(parameters=<t>, targets=<t>, reevaluate=False)
```

To run a gradient-based local calibration task for all included targets with the current parameter values without reevaluation, use the command:

```
cal.optimize(parameters=<t>, targets=<t>, reevaluate=False)
```

See evaluate on page 239 and optimize on page 256.

Sentaurus Topography Target Extraction

To extract the evaluated critical dimensions (CDs) of a Sentaurus Topography simulation, the Sentaurus Topography command file should include the extraction part at the end of the topography simulation. There is no need for an external extraction tool such as Sentaurus Visual.

For the PMC method, it is recommended to use the `extract type=shape_analysis` command of Sentaurus Topography, because it provides averaging as well as smoothing, which is beneficial for calibration.

For the extraction of a CD for an etched hole, shape analysis extraction based on a cylinder hole should be used. For example, in the case of the necking CD and position, it is used to extract the z-coordinates, polymer surface coverage, and average radius:

```
set FHL [extract name=Full_Hole \
   type=shape_analysis reference_shape=cylinder_hole \
   output_type={ } \
   point1={0.1 0.1 @structure_bottom@} \
   point2={0.1 0.1 @structure_top@} \
   max_radius=0.1 smoothing_order=25 top_shift=0.005]

set Z_coords [lindex [lsearch -regexp -inline [lindex $FHL 1] Z] 1]
set SurfCov_P_Avg [lindex [lsearch -regexp -inline [lindex $FHL 1]
```

```
   SurfaceCoverage_P_Avg] 1]
set Radius_Avg [lindex [lsearch -regexp -inline [lindex $FHL 1]
   Radius_Avg] 1]
```

Based on these quantities, the extraction of the specific CD and its location is performed:

```
set NeckingCD 1.0e5
set NeckingZ  0.0

foreach zcoord $Z_coords surfcover $SurfCov_P_Avg radiusavg $Radius_Avg
{
   if {$radiusavg<$NeckingCD && $zcoord>$BowingZ && $radiusavg>0 &&
   $surfcover>0.98} {
      set NeckingCD $radiusavg
      set NeckingZ  $zcoord
   }
}

set NeckingCD [expr 2*$NeckingCD]
```

Finally, variable extraction in the style of Sentaurus Workbench is needed:

```
puts "DOE: NeckingZ  [format %.4g $NeckingZ]"
puts "DOE: NeckingCD [format %.4g $NeckingCD]"
```

For each extracted variable, a scalar target must then be defined in Sentaurus Calibration Workbench (see Scalar Targets on page 24).

HARC Etching Calibration

For the fabrication of DRAM and 3D NAND memory devices, a crucial process step is the formation of high-aspect ratio contact (HARC) holes by plasma etching. To calibrate HARC formation, it is important to study the influence of process parameters on the contact hole shape, including the hole depth and diameter, as well as the necking and bowing of the sidewalls (see Figure 6 on page 119). To utilize the PMC method of Sentaurus Topography for such studies, it is critical to calibrate the corresponding plasma etching models based on specific process parameters and their measured shapes.

The `pmccal` module provides two predefined comprehensive calibration strategies for high-aspect ratio (HAR) etching:

- One based on a gradient-based local calibration algorithm (see Calibration Strategy by Gradient-Based Local Calibration Algorithm on page 118)
- One based on machine learning (see Calibration Strategy by Machine Learning on page 120)

In addition, the calibration module allows you to define your own calibration strategies.

Calibration Strategy by Gradient-Based Local Calibration Algorithm

The predefined calibration strategy for HARC etching, based on the gradient-based local calibration algorithm, includes many calibration tasks, which are grouped in the initial, middle, and final parts. When the total root-mean-square (RMS) error is within the tolerance or the allowed number of tasks is exceeded, the calibration strategy switches from the initial part to the middle part, and then from the middle part to the final part, which ends the calibration.

The following command executes the calibration strategy:

```
pmccal.calibrate_har_etch(parameters=<t>,
                          targets=<t>,
                          parent=<s>,
                          child_dir=<s>,
                          tolerance_end=<f>,
                          tolerance_mid=<f>,
                          tolerance_init=<f>,
                          max_iteration_end=<i>,
                          max_iteration_mid=<i>,
                          max_iteration_init=<i>,
                          weight_factor=<f>,
                          reevaluate=<b>,
                          use_bi_sensitivity=<b>)
```

For a description of the arguments, see calibrate_har_etch on page 219.

The calibration strategy is based on a classification of calibration parameters and targets. Since Sentaurus Calibration Workbench does not know the Sentaurus Topography API, it does not automatically recognize the role of each model parameter being parameterized or the type of extracted variables. Therefore, the naming scheme of calibration parameters needs to differentiate between different models and materials. In addition, targets and extracted variable names must follow strict naming schemes. See Parameter Naming Schemes and Target Naming Schemes on page 119.

Parameter Naming Schemes

This parameter naming scheme is case insensitive. Use the following naming scheme for correct parameter classification:

- Yield functions: `yield_<type>_<material>`
- Sputtering: `p_sputter_<type>_<material>`
- Reflection: `p_reflect_<type>_<material>`
- Deposition (sticking): `sc_<type>_<material>`
- Redeposition: `sc_redep_<type>_<material>`

- Chemical etching: `rad_<type>_<material>`
- Flux: `flux_<type>`

The `<type>` can be any arbitrary string you have defined.

The supported options for `<material>` are:

- For hard masks: `mask, photoresist, polymer`
- For the bulk: `bulk, nitride, oxide, silicon`

Target Naming Schemes

This target naming scheme is case insensitive. Use the following naming scheme for correct target classification:

```
t_<condition>_<extraction>
```

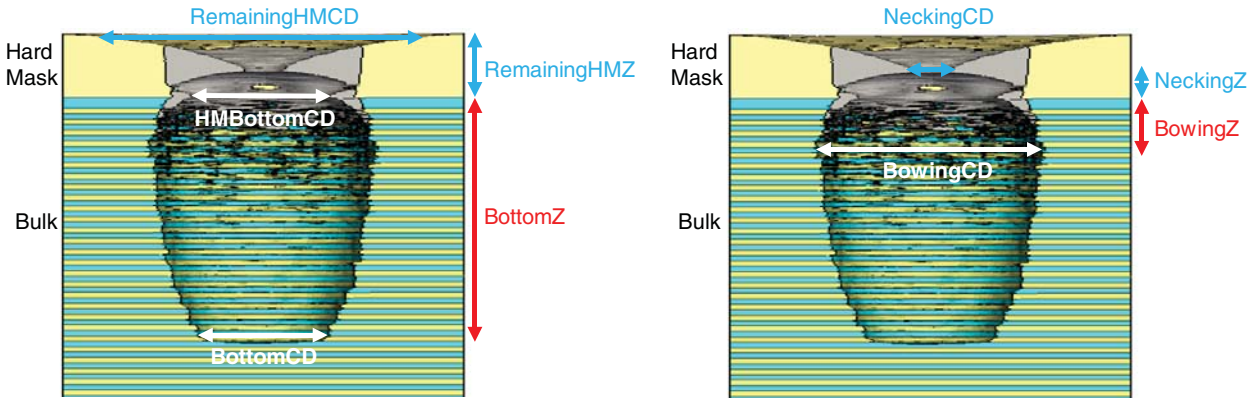
The string `<condition>` describes the conditions unambiguously.

The supported extracted variables used for the string `<extraction>` are:

- Depth and bottom width of bulk hole: `BottomCD, BottomZ`
- Thickness and top width of remaining hard mask: `RemainingHMCD, RemainingHMZ`
- Width at hard mask–bulk interface position: `HMBottomCD`
- Position and width of smallest opening (by deposition): `NeckingCD, NeckingZ`
- Position and width of largest opening of bulk hole: `BowingCD, BowingZ`

Additional targets can also be used, but the proposed targets are required to allow for a successful calibration result.

Figure 6 Illustration of extracted variables



Calibration Strategy by Machine Learning

The predefined calibration strategy for HARC etching, based on machine learning, includes parameter sensitivity analysis, training data computation, training the machine-learning model, and finally searching for the best candidates by machine-learning inference. This machine-learning calibration strategy, by default, performs a calibration for user-specified parameters and targets and has been validated for different HARC etching examples. However, you can create user-defined strategies by using a set of machine-learning base functions (see Creating a User-Defined Machine-Learning Strategy on page 122).

Unlike the calibration strategy for gradient-based local calibration, this strategy does not require using a specific naming scheme for calibration parameters, targets, and extracted variables. This means you can assign any arbitrary names to parameters, targets, and extracted variables, because they do not need to be classified. However, the same naming schemes as for the gradient-based local calibration can still be used.

As usual, calibration parameters are specified by `Parameter()` with nominal, minimum, and maximum values, where the minimum and maximum values define the parameter range. The machine-learning strategy task performs a sequence of subtasks (`explore` and `refine`). The outcomes of these subtasks are calibrated parameters and a file where the corresponding machine-learning models are saved.

The `calibrate_har_etch_ml()` command executes the machine learning–based calibration strategy for HARC etching (see calibrate_har_etch_ml on page 221).

For example, a typical use case would be:

```
pmccal.calibrate_har_etch_ml(parameters=parameters, targets=targets,
                             parent="../parent/HARC",
                             child_dir="../ML1",
                             tolerance=0.07, num_parameters=0,
                             constraint="Fail==0")
```

Machine-learning projects are created in the subdirectory `../ML1` as specified by the `child_dir` argument.

The argument `parent` specifies the global parent Sentaurus Workbench project.

By specifying `num_parameters`, you can control the number of features used for training. If you specify `num_parameters=0`, then all available parameters are used as features.

The constraint `"Fail==0"` means that only experiments with the Sentaurus Workbench extracted variable `Fail` equal to 0 are selected. Moreover, any Sentaurus Workbench variable such as `"Valid==1"`, as well as more complex expressions such as `"Fail==0 and split==1"`, can be used for `constraint`. Usually, `constraint` is used to discard poor data from machine-learning training.

The calibration strategy performs a sequence of `explore` and `refine` subtasks until either the `tolerance` is reached or the maximum number of internal iterations is reached.

Some additional control is allowed by setting the argument `tolerance_refine`:

- If the weighted RMS error in the current internal iteration is less than `tolerance_refine`, then the `refine` subtask is executed.
- Otherwise, the `explore` subtask is executed.

In addition, you can define target-specific tolerances. The target-specific tolerance should be set to a value that considers the overall global tolerance, the required accuracy for that specific target, and noise levels.

The following example shows how to set tolerances for individual targets:

```
t_BottomZ = Target(name='BottomZ', value=<f>, id='t_BottomZ',
   tolerance=0.01)
t_RemainingHM = Target(name='RemainingHM', value=<f>,
   id='t_RemainingHM', tolerance=0.05)
t_BowingZ = Target(name='BowingZ', value=<f>, id='t_BowingZ',
   tolerance=0.1)
t_NeckingZ = Target(name='NeckingZ', value=<f>, id='t_NeckingZ',
   tolerance=0.1)
t_BottomCD = Target(name='BottomCD', value=<f>, id='t_BottomCD',
   tolerance=0.08)
t_RemainingHMCD = Target(name='RemainingHMCD', value=<f>,
   id='t_RemainingHMCD', tolerance=0.08)
t_BowingCD = Target(name='BowingCD', value=<f>, id='t_BowingCD',
   tolerance=0.05)
t_NeckingCD = Target(name='NeckingCD', value=<f>, id='t_NeckingCD',
   tolerance=0.08)
t_HMBottomCD = Target(name='HMBottomCD', value=<f>, id='t_HMBottomCD',
   tolerance=0.08)
```

In addition to the predefined calibration strategy command `calibrate_har_etch_ml()`, there are other machine-learning base commands for the following applications:

- Machine-Learning Model Exploration
- Machine-Learning Model Refinement

Machine-Learning Model Exploration

The `explore_har_etch_ml()` command executes the exploration of a machine-learning model (see explore_har_etch_ml on page 241). The arguments of this command have the same meaning as those for `calibrate_har_etch_ml()` as described in Calibration Strategy by Machine Learning on page 120.

The calibration task of `explore_har_etch_ml()` corresponds to the `explore` subtask of `calibrate_har_etch_ml()`.

The exploration algorithm works as follows:

1. A child project for machine-learning training is created in the directory `child_dir` and is run.
2. Machine-learning models are created for each target and split condition. A machine-learning model database is populated.
3. The machine-learning database is used to predict calibration candidates. These candidates are simulated, and the best candidate is selected.
4. The exploration is repeated until the maximum number of iterations (`num_iterations`) is reached.

Machine-Learning Model Refinement

The `refine_har_etch_ml()` command executes the refinement of a machine-learning model (see load_parameters on page 252). The arguments of this command have the same meaning as those for `calibrate_har_etch_ml()` as described in Calibration Strategy by Machine Learning on page 120.

The calibration task of `refine_har_etch_ml()` corresponds to the `refine` subtask of `calibrate_har_etch_ml()`.

The refinement algorithm works as follows:

1. A child project for machine-learning training is created in the directory `child_dir` and is run.
2. A machine-learning model is trained in proximity of the current nominal parameter values and is used to predict new experiments. These experiments are simulated, and the best experiment is selected.
3. Several iterations are performed until the maximum number of iterations (`num_iterations`) is reached or the convergence criteria are met.

Creating a User-Defined Machine-Learning Strategy

The simplest way to run the predefined strategy is *as is*. In this case, the machine-learning calibration is performed until either the `tolerance` is achieved or the maximum number of internal iterations is reached.

If the achieved calibration is not accurate enough, then you can extend the calibration strategy or even create a user-defined one. For this purpose, you can use the machine-learning base commands for machine-learning model exploration (see Machine-Learning Model Exploration) and machine-learning model refinement (see Machine-Learning Model Refinement).

When combining several calibration strategy and base commands, task-specific names for the machine-learning tasks are required to distinguish the different tasks. Task names are defined by setting the argument `task_name`.

The following example shows how to use the `refine_ml()` command to fine-tune the calibration parameters after a `calibrate_har_etch_ml()` task:

```
pmccal.calibrate_har_etch_ml(parameters=parameters, targets=targets,
                             parent="../parent/HARC",
                             child_dir="../ML1",
                             task_name="PMC_HAR", tolerance=0.07,
                             num_parameters=0, constraint="Fail==0")

pmccal.refine_ml(parameters=parameters, targets=targets,
                 parent="../parent/HARC", child_dir="../ML1",
                 task_name="PMC_HAR_1", tolerance=0.07,
                 num_parameters=0,
                 sensitivity="PMC_HAR",
                 constraint="Fail==0")
```

Here, the machine-learning tasks `PMC_HAR` and `PMC_HAR_1` are executed sequentially. The `PMC_HAR` task performs a calibration with `tolerance=0.07`. The calibrated parameter values from the `PMC_HAR` task are then used as the initial values for the `PMC_HAR_1` task, which performs a `refine` task with target `tolerance=0.07`.

The argument `sensitivity="PMC_HAR"` specifies that the sensitivity analysis from task `PMC_HAR` is used in task `PMC_HAR_1`, reusing existing simulations and reducing the total number of simulations.

The following example shows how to perform a `refine` task after an initial calibration using only a subset of the targets:

```
pmccal.calibrate_har_etch_ml(parameters=parameters,
                             targets=all_targets,
                             parent="../parent/HARC",
                             child_dir="../ML1",
                             task_name="PMC_HAR", tolerance=0.07,
                             num_parameters=0, constraint="Fail==0")

pmccal.refine_ml(parameters=parameters,
                 targets=targets_wo_BottomCD_NeckingCD,
                 parent="../parent/HARC", child_dir="../ML1",
                 task_name="PMC_HAR_1", tolerance=0.07,
                 num_parameters=0,
                 sensitivity="PMC_HAR",
                 constraint="Fail==0")
```

The task `PMC_HAR_1` performs a `refine` task including all targets except `t_BottomCD` and `t_NeckingCD`. This can be done to fine-tune the targets without `t_BottomCD` and `t_NeckingCD` that did not match the expected accuracy after the execution of task `PMC_HAR`.

In the early stages of the calibration, it might be interesting to explore the calibration parameter space using the `explore` task and to analyze the inferred machine-learning data:

```
pmccal.explore_ml(task_name="PMC_HAR_1", tolerance=0.07,
                  strategy=0, num_parameters=0, constraint="Fail==0",
                  parent="../parent/HARC", child_dir="../ML1")

pmccal.infer_ml_models("PMC_HAR", child_dir="../ML1",
                       doe_type="uniform", doe_size=30000)
```

The task `PMC_HAR_1` performs an `explore` task. In this example, you might not be interested in the calibrated parameters from the `PMC_HAR_1` task but prefer to generate data using model inference (see Saving and Analyzing Machine-Learning Model Inference Data) and to analyze this data using a tool such as Sentaurus PCM Studio.

Validating Parameter Ranges

Parameter ranges are critical for the creation of a machine-learning model. If the parameter exploration space is too big, then the combination of parameters could lead to failures in simulations and, subsequently, to insufficient data for model training. The `validate_range_ml()` command executes the validation of the machine learning–based parameter range (see Validating Parameter Ranges on page 44).

The new ranges can be used in a subsequent task, since the parameter ranges are reset automatically, as shown in the following example:

```
pmccal.validate_range_ml(parent="../parent/HARC",
                         child_dir="../ML1",
                         task_name="PMC_HAR", constraint="Fail==0")

pmccal.calibrate_har_etch_ml(parent="../parent/HARC",
                             child_dir="../ML1",
                             task_name="PMC_HAR_1", tolerance=0.07,
                             num_parameters=0, constraint="Fail==0")
```

The task `PMC_HAR_1` uses the parameter ranges computed in task `PMC_HAR`.

Saving and Analyzing Machine-Learning Model Inference Data

The following command executes the model inference (see infer_ml_models on page 251):

```
pmccal.infer_ml_models(child_dir=<s>,
                       task_name=<s>,
                       doe=<s|d|p>,
                       doe_size=<i>,
                       doe_type=<s>,
                       doe_levels=<i>,
                       doe_sep=<s>,
                       file_prefix=<s>)
```

All exploration type of models, created during the execution of `task_name`, can be used to infer data. The design-of-experiments (DoE) used to generate the data can have the following properties:

- `doe` defines the range of features. If not specified, then the range is taken from the minimum and maximum values of the model features used during model training and creation. These values can be:
  - A dictionary specifying the minimum and maximum values for all or some features
  - A `.csv` file containing feature names as columns and corresponding values in the rows (in this case, minimum and maximum values are extracted from the values in the `.csv` file)
  - A pandas data frame, specifying the DoE range or the actual DoE if `doe_type="custom"`
- `doe_size` sets the number of rows of the DoE.
- `doe_type` sets the type of the DoE.
- `doe_levels` applies only to a full-factorial DoE (`doe_type="factorial"`). It defines the levels used for each model feature (two levels mean `min` and `max`).

The file created by `infer_ml_models()` is saved in the `child_dir` directory and has the following name format:

```
<file_prefix>_<task_name>_infer_<doe_type>_<index>.csv
```

The `index` depends on the number of available models created during the execution of `task_name`. Inference is performed on each model and the corresponding file is saved using a progressive index. The `.csv` file can be analyzed by a third-party application or Sentaurus PCM Studio.

Example

```
pmccal.calibrate_har_etch_ml(parent="../parent/HARC",
                             child_dir="../ML1",
                             task_name="PMC_HAR", tolerance=0.07,
                             num_parameters=0, constraint="Fail==0")

pmccal.infer_ml_models(task_name="PMC_HAR", child_dir="../ML1",
   doe_type="nominal")

pmccal.infer_ml_models(task_name="PMC_HAR", child_dir="../ML1",
   doe_type="uniform", doe_size=30000)

pmccal.infer_ml_models(task_name="PMC_HAR", child_dir="../ML1",
   doe_type="uniform", doe_size=30000,
   doe="ML_Calibrate_range_1.csv",
   file_prefix="test_")
```

8

Graphical User Interface

This chapter describes how to use the Sentaurus Calibration Workbench graphical user interface to create calibration flows.

The Sentaurus Calibration Workbench graphical user interface (GUI) is a specialized tool designed to provide an interface to the batch Sentaurus Calibration Workbench tool and to efficiently implement calibration strategies without involving the Python environment. It also can visualize the intermediate and final results.

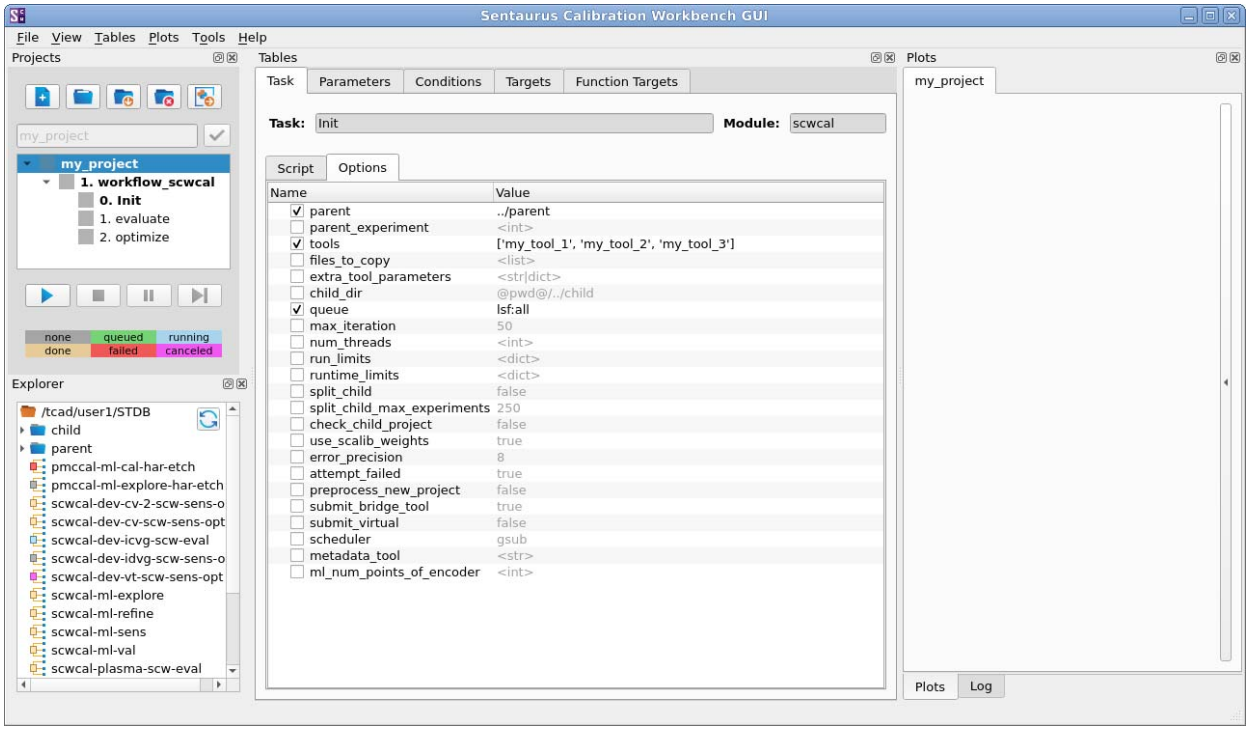
Features of the User Interface

To run the Sentaurus Calibration Workbench GUI, see Running Sentaurus Calibration Workbench on page 18.

The GUI has different panels (see Figure 7):

- The Projects panel, upper-left side of the main window, shows all the projects created or loaded, including the control and status of their execution (see Projects Panel).
- The Explorer panel, lower-left side of the main window, shows all the available Sentaurus Calibration Workbench projects with their corresponding state (see Explorer Panel).
- The Tables panel, in the middle of the main window, has different tabs (see Tables Panel).
- The Plots panel, right side of the main window, lists all the plots you have generated, separated into tabs, one tab per project (see Plots Panel).
- The Log panel, right side of the main window, shows all the generated files at the project and workflow levels of the execution of the Sentaurus Calibration Workbench batch tool, which is executed by the Sentaurus Calibration Workbench GUI (see Log Panel).

Figure 7 Main window of graphical user interface



Menu Bar

You can use the menu bar to access the main operations.

Table 13 Available menus

Menu	Description
File	Creates, loads, cleans up, saves, closes, and exports Sentaurus Calibration Workbench GUI projects, and imports Sentaurus Workbench parameters.
View	Shows and hides panels; resets the layout; edits preferences.
Tables	Adds and removes rows from tables; imports and export items (CSV files).
Plots	Creates, removes, and exports plots; sets up the number of columns.
Tools	Opens the Advanced Image Analysis and Metrology tool. See Chapter 9: Advanced Image Analysis and Metrology.
Help	Provides information about the Sentaurus Calibration Workbench GUI.

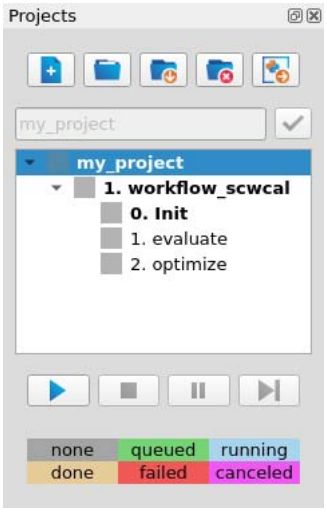
Projects Panel

This panel shows information about the different projects loaded and provides buttons to manage projects and their execution. This panel is divided into different parts:

- The File toolbar buttons allow you to interact with project files, for example, to create, open, save, close, and export files.
- The list of projects shows the strategy description of each calibration project, by dividing projects into workflows, and workflows into steps. Each of these elements shows the status of the calibration run by using different colors in the box next to each element.
- Under the list of projects, the Execution toolbar buttons allow you to control the execution of the calibration of each project. The status of the buttons depends on which project is selected.
- The color legend explains the status of a calibration run.

Bold text indicates the selected step and which step is represented in the Tables, Plots, and Log panels. If you select a project or workflow, then the last executed step is bold (selected). When a project has not been executed, the Init step is selected.

Figure 8 Projects panel



Each element in the list of projects has a corresponding option in a context menu, which varies depending on the element selected. Table 14 lists the possible options.

Table 14 Options available from context menu related to Projects panel

Option	Description
Project options	
Add New Workflow to Project	Creates a new workflow and appends it to the selected project.
Run Project Calibration	Runs the selected project (all workflows).
Stop Calibration	Stops the current running project.
New Project	Creates a new project and appends it to the list of projects.
Save Project	Saves the selected project.
Copy Project	Copies the selected project and appends it to the list of projects.
Close Project	Closes the selected project.
Move Project Up	Moves the selected project one position up in the list of projects.
Move Project Down	Moves the selected project one position down in the list of projects.
Export Script	Exports the Sentaurus Calibration Workbench template script to a user-defined file.
Get Latest Database Data	Imports the latest information from the database.
Open Child in SWB	Opens a single Sentaurus Workbench child project from the list of all the possible child projects generated by the project.
Import All SWB Data	Imports all Sentaurus Workbench data from the parent project.
Clean Up	Opens the Cleanup dialog box.
Workflow options	
Add New Step to Workflow	Creates a new step and appends it to the selected workflow.
Run Workflow Calibration	Runs the selected workflow.

Table 14 Options available from context menu related to Projects panel (Continued)

Option	Description
Stop Calibration	Stops the current running workflow.
Copy Workflow	Copies the selected workflow and appends it to the selected project.
Remove Workflow	Removes the selected workflow from the workflows list of the selected project.
Move Workflow Up	Moves the selected workflow one position up in the workflows list from the selected project.
Move Workflow Down	Moves the selected workflow one position down in the workflows list from the selected project.
Get Latest Database Data	Imports the latest information from the database.
Open Child in SWB	Opens a single Sentaurus Workbench child project from the list of all the possible child projects generated by the project.
Import All SWB Data	Imports all Sentaurus Workbench data from the parent project.
Clean Up	Opens the Cleanup dialog box.
Step options	
Insert New Step	Creates a new step and inserts it at the selected location.
Copy Step	Copies the selected step and appends it to the selected workflow.
Remove Step	Removes the selected step from the steps list of the selected workflow.
Move Step Up	Moves the selected step one position up in the steps list from the selected workflow.
Move Step Down	Moves the selected step one position down in the steps list from the selected workflow.
Open Child in SWB	Opens a single Sentaurus Workbench child project from the list of all the possible child projects generated by the workflow where the step belongs to.

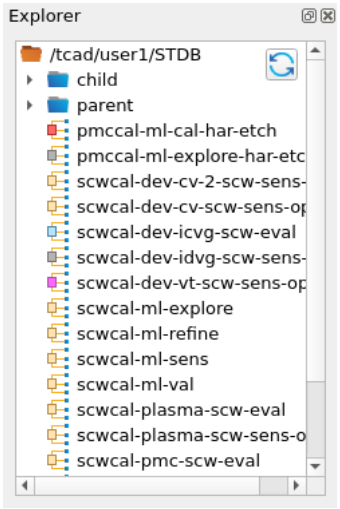
Explorer Panel

This panel shows all the Sentaurus Calibration Workbench projects available in the directory defined by the environment variable `STDB`. The icon to the left of a project name shows the running state of the project.

Click the upper-right button to refresh the view.

Bold text indicates projects that are opened in the latest session of the Sentaurus Calibration Workbench GUI.

Figure 9 Explorer panel



Each element in the Explorer panel has a corresponding option in a context menu. Table 15 lists the possible options.

Table 15 Options available from context menu related to Explorer panel

Option	Description
Open	Opens a Sentaurus Calibration Workbench project, only if the directory is a valid Sentaurus Calibration Workbench project. Otherwise, the action is omitted.
Cut	Places the path of the directory on the internal clipboard. When the path is pasted, the directory is deleted.
Copy	Copies the selected directory. It places the path of the directory on the internal clipboard.

Table 15 Options available from context menu related to Explorer panel (Continued)

Option	Description
Paste	Pastes the directory represented by the path stored on the internal clipboard.
Delete	Deletes the selected directory.
Rename	Renames the selected project.
Command prompt	Opens a command prompt in the selected directory.

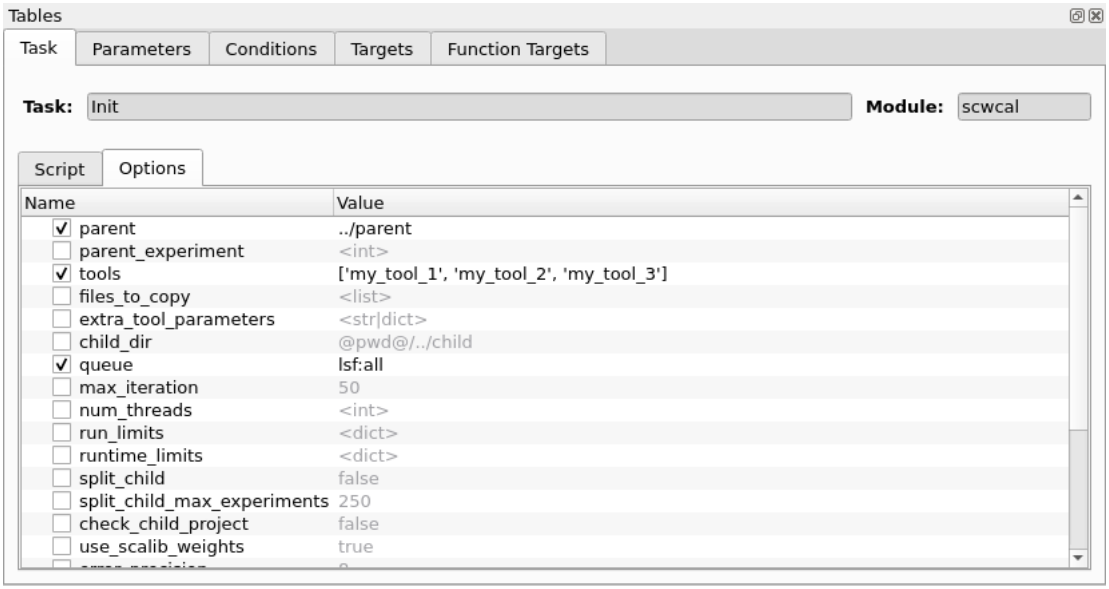
Tables Panel

This panel presents information on the following tabs:

- The **Task** tab shows the *task definition* for a specific step (see Task Tab).
- The **Parameters** tab shows the *calibration parameters* and their attributes (see Parameters Tab).
- The **Conditions** tab shows the *condition parameters* (see Conditions Tab).
- The **Targets** tab shows the *extracted variable targets* and their attributes (see Targets Tab).
- The **Function Targets** tab shows the *function targets* and their attributes (see Function Targets Tab).
- The **Models** tab shows all the machine-learning models in the calibration project (see Models Tab). This tab is available only when the machine-learning calibration module (`mlcal`) is selected.

The information defined on these tabs is specific to each *calibration step*. If the text color of a value is gray, then it means that the value is inherited from a previous step. If a cell is orange, then it means that cell contains a calculated value as the result of the execution of the selected calibration step.

Figure 10 Tables panel showing Options tab for selected task



The tables on the **Parameters**, **Conditions**, **Targets**, **Function Targets**, and **Models** tabs have inline validation. Conflicting cells are pale red. The following are common conflicts:

- A cell has no value but a value is mandatory.
- A cell has a value that conflicts with another cell, for example, two cells colored pale red and both with values could mean that it is not valid for those two cells to have data.

Table cells might also be invalid for a specific condition, usually defined by the Type property. Such cells are colored dark gray.

Task Tab

This tab provides details to define the task. It is defined by additional subtabs:

- The first subtab differs depending on the task used in the step:
  - *Init and custom tasks* show the **Script** subtab, which allows you to define custom Python scripts in the following sections: The initial script is executed before the instantiation of the module selected in the workflow. The task script is executed after the definition of parameters and targets (see Figure 11).
  - *Other tasks* show the **Arguments** subtab, which allows you to define the arguments for a given task and to set its prologue and epilogue commands with their arguments (see Figure 12).

- The **Options** subtab (second subtab) lists a set of defined options to be applied to a task. If none is defined, then the task uses the options from the previous step, or the default options if it is the Init step (see Figure 10).

  If there is any conflict with the options in the Init step, this subtab is displayed.

Figure 11 Tables panel showing Script subtab for selected task

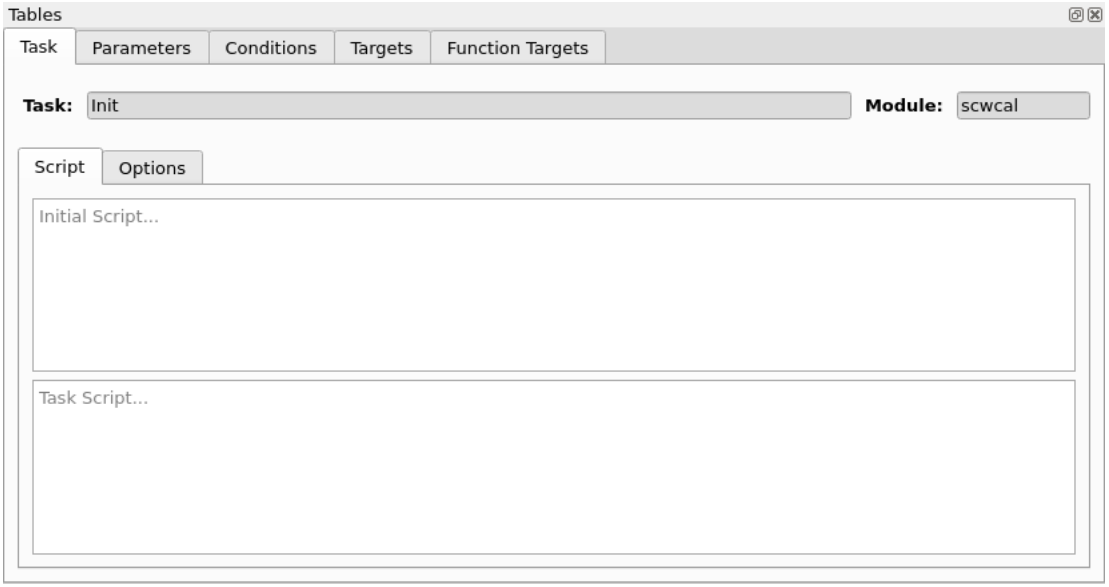
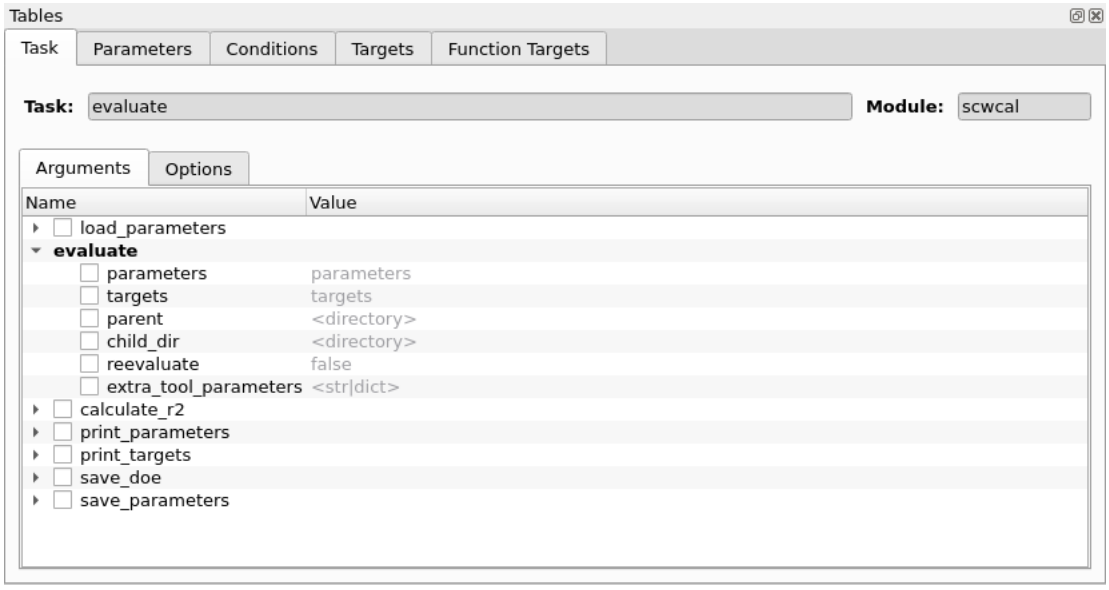


Figure 12 Tables panel showing Arguments subtab for selected task



Parameters Tab

The table lists all the *calibration parameters* used in the calibration project. The list must be defined in the Init step and cannot be changed in any of the consecutive steps. The table also lists the attributes of each calibration parameter, such as their lower and upper bounds.

Each element in the list (calibration parameter) can be included (checked) or excluded (unchecked) in each *calibration step*. It is possible to select or clear all elements from the list by clicking the *Name* column header.

The Value column shows the current value of the parameter as a result of the execution of that step.

To add a parameter:

1. Click **Add** or press Ctrl+M.

   A new calibration parameter is added to the end of the list, with the focus on the first column of the new element.

2. Define the name of the parameter.

To remove parameters:

1. Select the required parameters.
2. Click **Remove** or press Ctrl+Shift+M.

To import parameters:

1. Click **Import**.

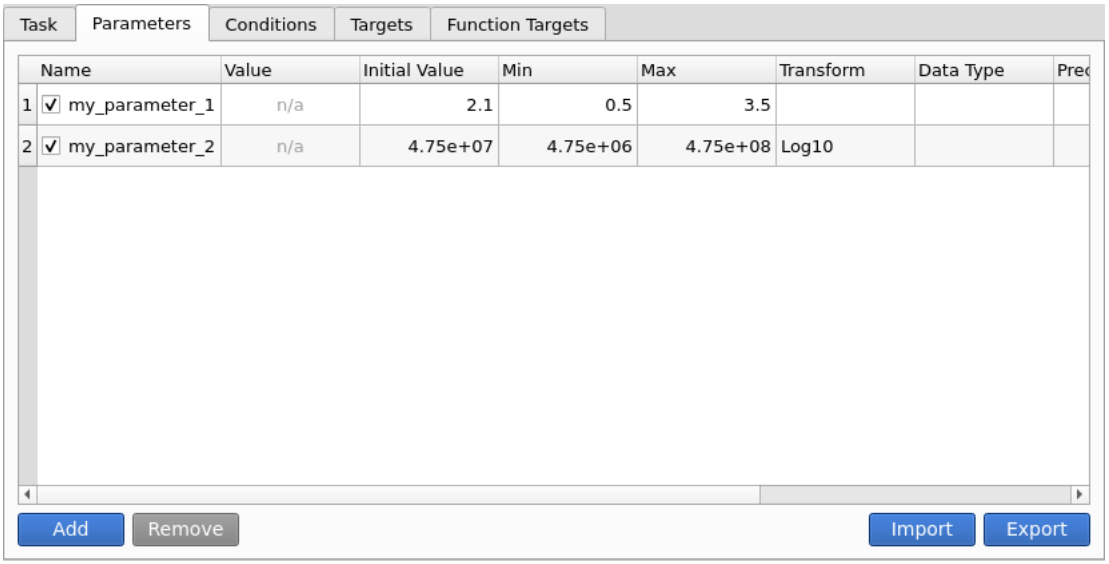
   The Import CSV File dialog box opens.

2. Navigate to the required CSV file.
3. Click **Open**.

To export parameters to a CSV file:

1. Select the required parameters.
2. Click **Export**.

Figure 13 Tables panel showing Parameters tab



Conditions Tab

The table lists all the *condition parameters* used in the calibration project with a defined value. The list must be defined in the Init step and cannot be changed in any of the consecutive steps, but their values can be changed.

Each element of the list (condition parameter) can be included (checked) or excluded (unchecked) in each *calibration step*. It is possible to select or clear all elements from the list by clicking the *Name* column header.

To add a condition:

1. Click **Add** or press Ctrl+M.

   A new condition parameter is added to the end of the list, with the focus on the first column of the new element.

2. Define the name of the parameter.

To remove conditions:

1. Select the required conditions.
2. Click **Remove** or press Ctrl+Shift+M.

To import conditions:

1. Click **Import**.

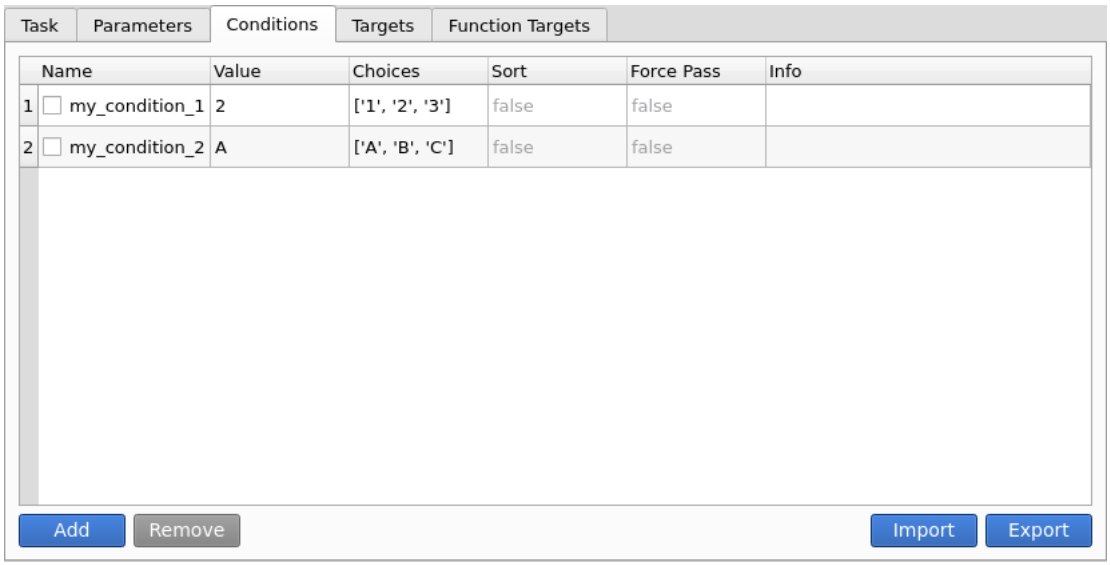
   The Import CSV File dialog box opens.

2. Navigate to the required CSV file.
3. Click **Open**.

To export conditions to a CSV file:

1. Select the required conditions.
2. Click **Export**.

Figure 14 Tables panel showing Conditions tab



Targets Tab

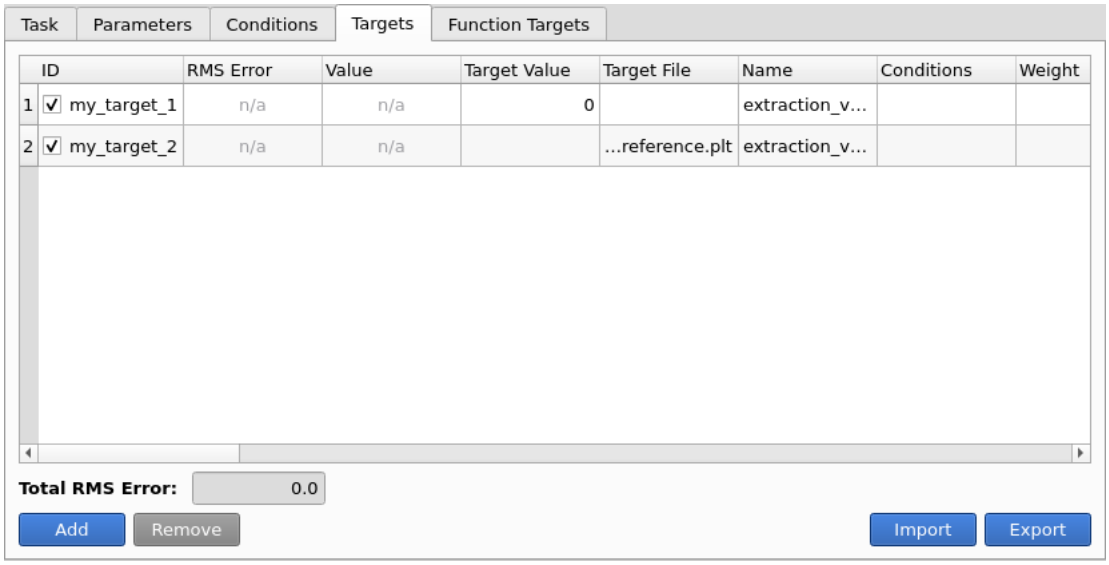
The table lists all the extracted variable targets used in the calibration project. The list must be defined in the Init step and cannot be changed in any of the consecutive steps. The table also lists the attributes of each target, such as their weight and scale.

Each element of the list (target) can be included (checked) or excluded (unchecked) in each *calibration step*. It is possible to select or clear all elements from the list by clicking the *ID* column header.

The RMS Error and Value columns show the current RMS error and current simulated value (if a scalar target value has been defined) of the target as a result of the execution of that step.

The **Total RMS Error** field shows the total RMS error of all the executed targets (extracted variable targets and function targets) for the selected step.

Figure 15 Tables panel showing Targets tab



To add a target:

1. Click **Add** or press Ctrl+M.

   A new target is added to the end of the list, with the focus on the first column of the new element.

2. Define the name of the target.

To remove targets:

1. Select the required targets.
2. Click **Remove** or press Ctrl+Shift+M.

To import targets:

1. Click **Import**.

   The Import CSV File dialog box opens.

2. Navigate to the required CSV file.
3. Click **Open**.

To export targets to a CSV file:

1. Select the required targets.
2. Click **Export**.

Function Targets Tab

The table lists all the function targets in the calibration project. The list must be defined in the Init step and cannot be changed in any of the consecutive steps. The table also lists the attributes of each function target, such as a list of the extracted variable targets and functions.

Each element of the list (function target) can be included (checked) or excluded (unchecked) in each calibration step. It is possible to select or clear all elements from the list by clicking the *ID* column header.

The **Total RMS Error** field shows the total RMS error of all the executed targets (extracted variable targets and function targets) for the selected step.

To add a function target:

1. Click **Add** or press Ctrl+M.

   A new target is added to the end of the list, with the focus on the first column of the new element.
2. Define the name of the function target.

To remove function targets:

1. Select the required function targets.
2. Click **Remove** or press Ctrl+Shift+M.

To import function targets:

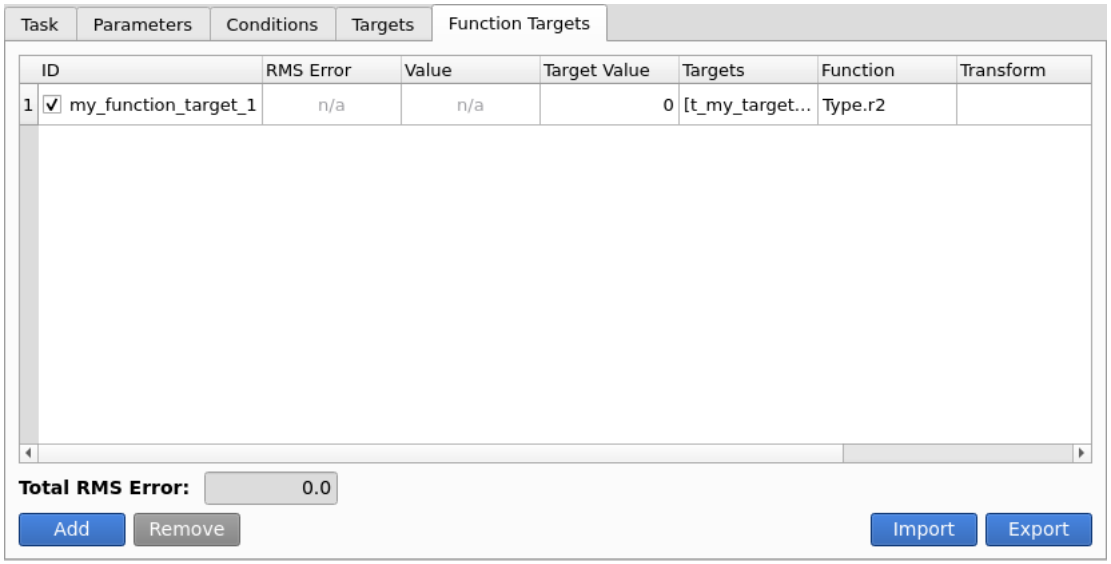
1. Click **Import**.

   The Import CSV File dialog box opens.
2. Navigate to the required CSV file.
3. Click **Open**.

To export function targets to a CSV file:

1. Select the required function targets.
2. Click **Export**.

Figure 16 Tables panel showing Function Targets tab



Models Tab

Note:

This tab is available only when the machine-learning calibration module (`mlcal`) is selected.

The table lists all the machine-learning models in the calibration project. The list must be defined in the Init step and cannot be changed in any of the consecutive steps. The table also lists the attributes of each machine-learning model, such as the model hyperparameters.

Each element of the list (model) can be included (checked) or excluded (unchecked) in each calibration step. It is possible to select or clear all elements from the list by clicking the *Name* column header.

To add a machine-learning model:

1. Click **Add** or press Ctrl+M.

   A new model is added to the end of the list, with the focus on the first column of the new element.

2. Define the name of the model.

To remove models:

1. Select the required model.
2. Click **Remove** or press Ctrl+Shift+M.

To import machine-learning models:

1. Click **Import**.

   The Import CSV File dialog box opens.

2. Navigate to the required CSV file.
3. Click **Open**.

To export models to a CSV file:

1. Select the required models.
2. Click **Export**.

Figure 17 Tables panel showing Models tab

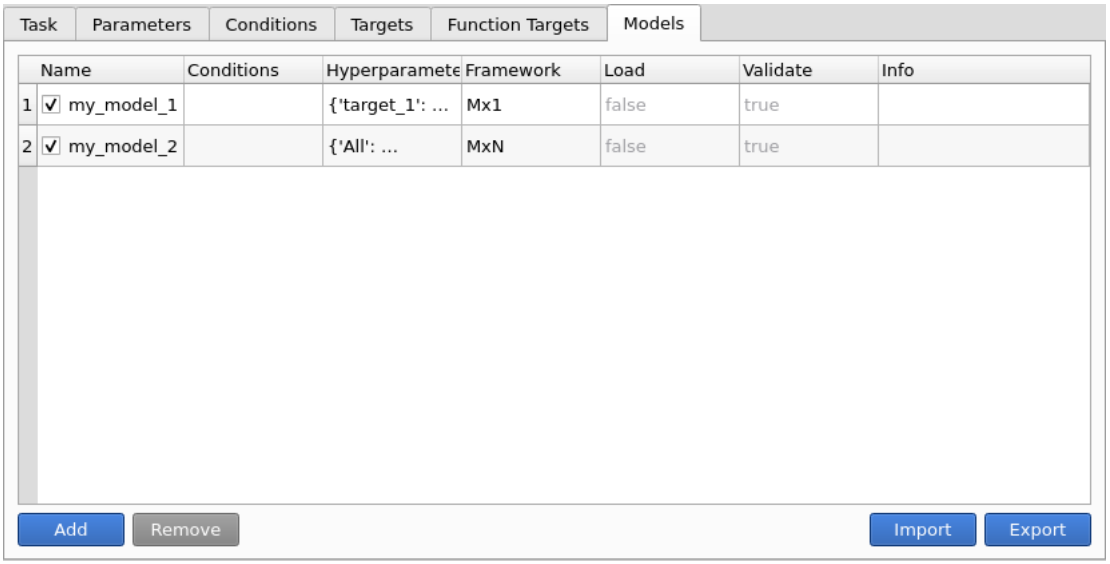


Figure 18 Hyperparameter dialog box setting with GridSearch

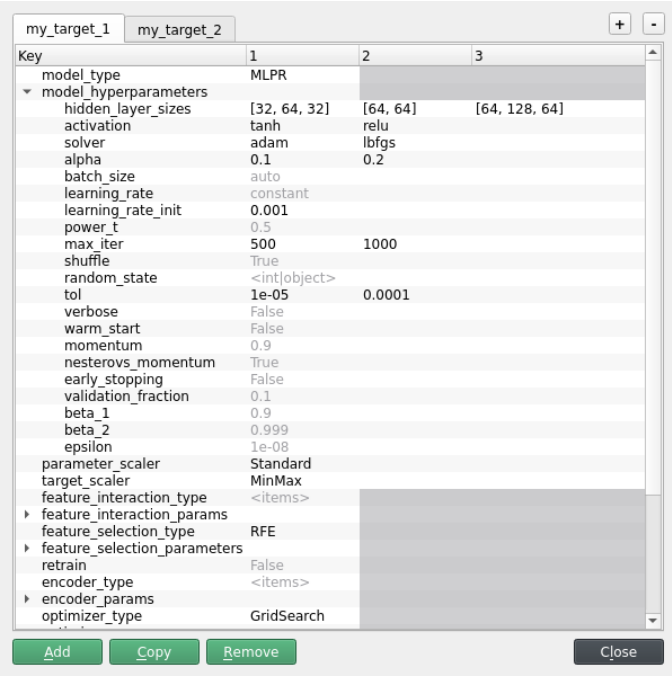
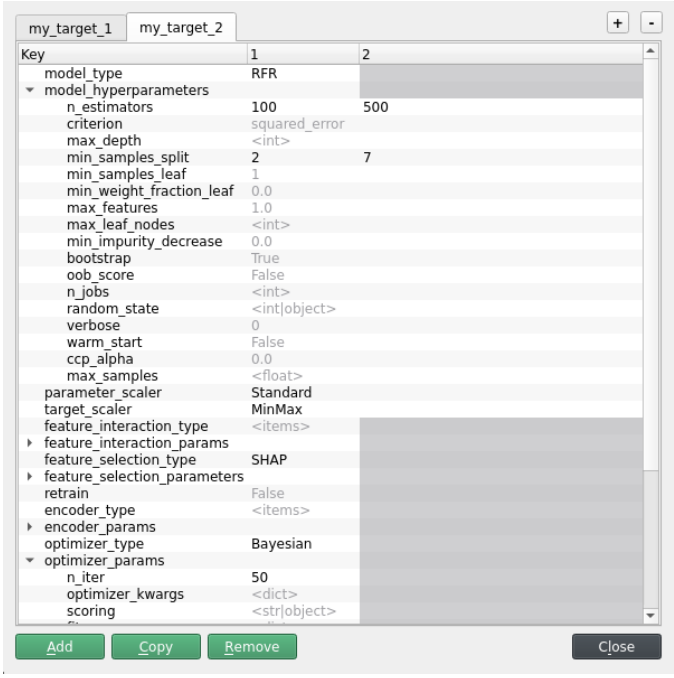


Figure 19 Hyperparameter dialog box setting with Bayesian



You can specify the machine-learning model for each target through the hyperparameter dialog box and set up `GridSearch` or `Bayesian` to optimize model hyperparameters. Additionally, you can set `encoder_type`, `feature_interaction_type`, `feature_selection_type`, and corresponding arguments through this dialog box.

Plots Panel

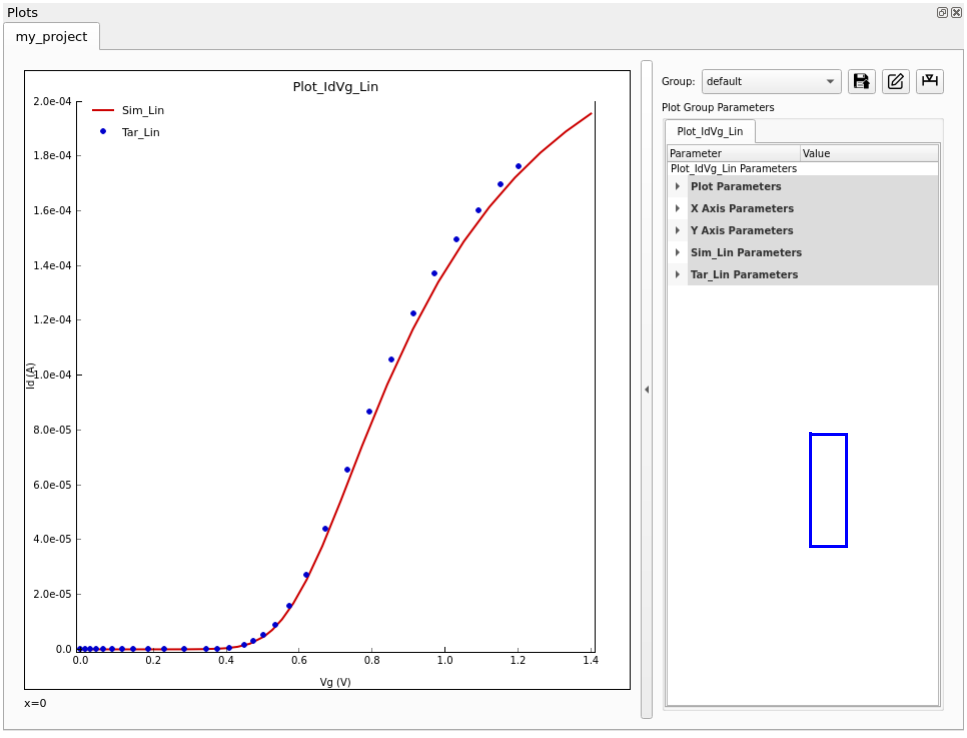
This panel shows all the plots that are generated automatically or manually. The plots are grouped per project using tabs, where each tab is labeled by the project name.

On each tab, plots can have a grid layout or one maximized while others are hidden. Each tab can have an independent layout.

Each plot can be customized by modifying its properties, exported to a file, or closed, all independently of other plots.

Plot properties are shown to the right of the Plots panel, but they are hidden by default. To show plot properties, click the button highlighted in Figure 20. After the properties are displayed, you can hide them by clicking the button again.

Figure 20 Plots panel showing one plot with its properties visible



Plot properties are grouped by type. The first group lists general plot properties, such as title and legend visibility. The next group lists axes properties, including labels, ranges, and axis

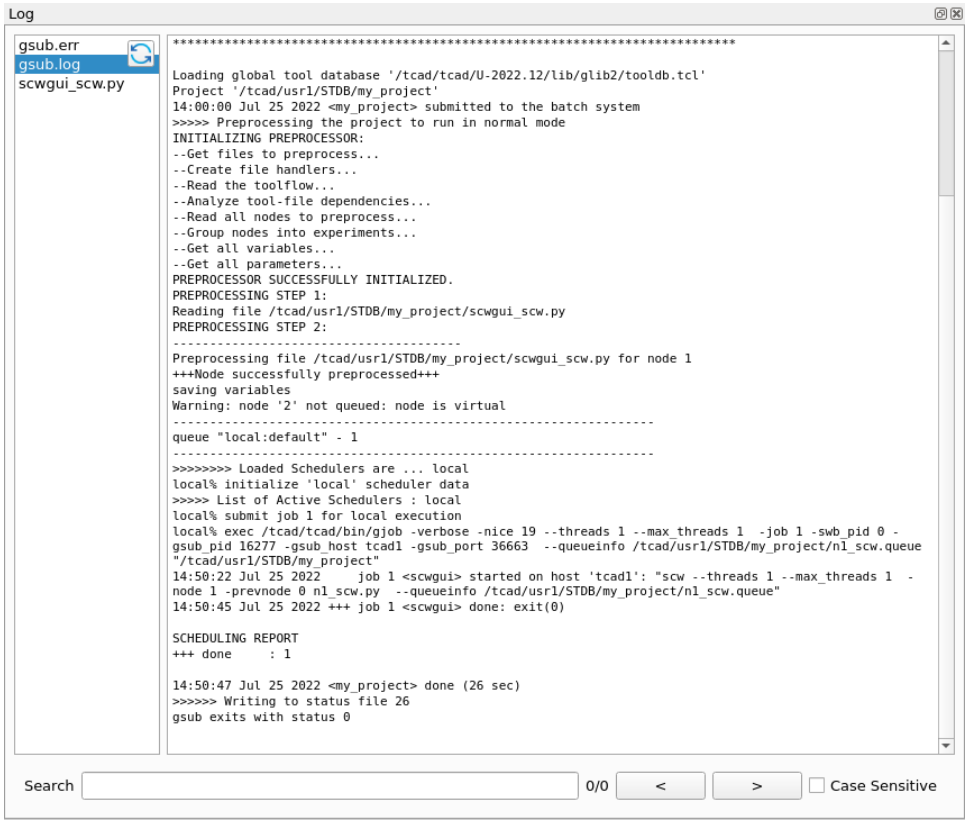
scale properties. The final group lists curve properties, such as color, line style and width, and marker types and size.

Log Panel

This panel lists all the output files of the execution of the Sentaurus Calibration Workbench batch tool. Each project has its own Log panel, which changes according to the project and workflow selection in the Projects panel.

The output captured from the Sentaurus Calibration Workbench batch tool can be saved into a file or can be removed.

Figure 21 Log panel showing the results of an executed project



The Log panel provides a search function that highlights all matched words. It counts the number of matches and shows which is the current selected match. Select **Case Sensitive** if the search requires this option. You can use the keyword to move through the found matches. Press **Return** to move forward and **Shift+Return** to move backward.

Customizing GUI Preferences

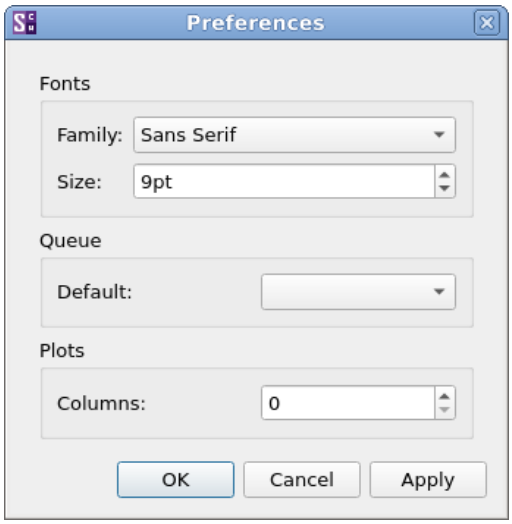
You can customize the Sentaurus Calibration Workbench GUI preferences.

To customize GUI preferences:

1. Choose **View** > **Preferences**.

   The Preferences dialog box opens.

2. Change preferences as required.



3. Click **OK** or **Apply**.

Note:

Some preferences are applied the next time you launch the Sentaurus Calibration Workbench GUI.

On Linux operating systems, the GUI preferences are stored in the following file:

```
~/.config/Synopsys/scwgui.conf
```

Cleaning Up Projects and Workflows

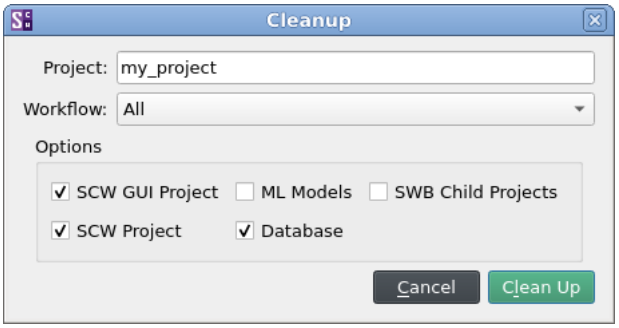
You can clean up projects and workflows.

To clean up a project or workflow:

1. Select a project or workflow in the Projects panel.
2. Choose **File** > **Clean Up Project/Workflow**.

Alternatively, right-click the selected project or workflow and choose **Clean Up**.

The Cleanup dialog box opens.



3. Select the options as required:

Option	Description
SCW GUI Project	Remove all data that has been saved in the Sentaurus Calibration Workbench project: the run state of the project, workflow and steps, and the data obtained from Sentaurus Calibration Workbench (cells in pale orange), and substeps.
SCW Project	Remove all Sentaurus Calibration Workbench project files. This is equivalent to the Sentaurus Workbench cleanup of a Sentaurus Calibration Workbench project.
ML Models	Remove machine-learning model files created by any machine-learning function or module.
Database	Remove data saved by Sentaurus Calibration Workbench to be read by the Sentaurus Calibration Workbench GUI, using a database infrastructure.
SWB Child Projects	Remove all child projects created by Sentaurus Calibration Workbench. These projects are located in the folder specified either by the option `child_dir` (see Defining the Child Project Directory on page 34) or by the task argument `child_dir`.

4. Click **Clean Up**.

Setting Up a Calibration Flow

This section describes how to define a basic calibration flow using the Sentaurus Calibration Workbench GUI, following the setup explained in Chapter 2 on page 20.

Setting up a calibration flow requires the following steps:

1. Create a project and select the calibration module to use. See Creating a Project and Selecting the Calibration Module.
2. Define the main calibration options on the **Options** subtab of the **Task** tab for the Init step. See Setting Up the Calibration Input.
3. Add parameters, conditions, and targets to the Init step. See Setting Up the Calibration Input. In addition:
   a. Import parameters from Sentaurus Workbench projects as condition parameters. See Importing Data From Sentaurus Workbench Projects.
   b. Convert condition parameters to calibration parameters and vice versa. See Converting Condition Parameters to Calibration Parameters, and Calibration Parameters to Condition Parameters.
4. Add new steps, assigning a task to each one. Set up the task arguments and options. See Setting Up the Calibration Input.
5. Execute the calibration flow. See Executing the Calibration Flow.

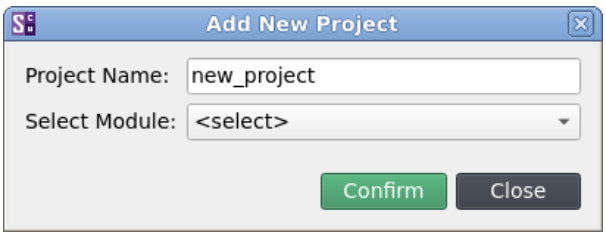
Creating a Project and Selecting the Calibration Module

You can select a calibration module when you create a Sentaurus Calibration Workbench GUI project and when additional workflows are added to an existing project.

To create a project:

1. In the Projects panel, click the **+** button.

   The Add New Project dialog box opens.



2. Enter the name of the project.
3. Select a calibration module for the project to use.
4. Click **Confirm**.

The new project contains one workflow with one step, the Init step (see Figure 8).

Setting Up the Calibration Input

Calibration input includes parameters, targets, and the simulation flow (calibration steps) and its options.

Note:

In the Sentaurus Calibration Workbench GUI, parameters (calibration parameters), conditions (condition parameters), targets (extracted variable targets), and function targets are defined *only* in the Init step.

Options

You must define the **parent** and **tools** task options in the Init step. You also can define several optional calibration controls. All of these options are located on the **Options** subtab of the **Task** tab.

Some controls such as **parent**, **child_dir**, and **tools** are defined in Setting Options for Calibrations on page 33.

Parameters

Each parameter (calibration parameter) is a row in the table. To add parameters on the **Parameters** tab, click **Add**. After a new row is added, you must define a name for the parameter and assign an initial value, which is mandatory. See Parameters Tab on page 135.

The name of the parameter must correspond to a parameter name as defined in the Sentaurus Workbench project (case sensitive) and the parameter in the simulator command file or parameter file. You must use the Sentaurus Workbench parameter syntax (`@name@`) for parameters in parameterized files.

You can include or exclude parameters from a calibration task (step) by selecting or clearing the check box next to the parameter name.

Conditions

Each condition (condition parameter) is a row in the table. To add conditions on the **Conditions** tab, click **Add**. After a new row is added, you must define a name for the condition and assign a value. See Conditions Tab on page 136.

The conditions are used in targets as split conditions.

Targets

Each target is a row in the table. To add targets on the **Targets** tab, click **Add**. After a new row is added, you must define a name for the target and assign a target value, which can be

a scalar for a scalar target or a reference target xy profile (one of them must be defined). See Targets Tab on page 137.

The name of the target must correspond to the name of a Sentaurus Workbench extracted variable (case sensitive).

You can include or exclude targets from a calibration task (step) by selecting or clearing the check box next to the target name.

If split conditions (condition parameters that can have different values) are defined, then you can use them by setting them in the Conditions column.

Function Targets

Each function target is a row in the table. To add targets on the **Function Targets** tab, click **Add**. After a new row is added, you must define a name for the function target and assign a target scalar value, as well as the extracted variable targets and the function to be used (extracted variable targets must be defined before function targets). See Function Targets Tab on page 139.

You can include or exclude function targets from a calibration task (step) by selecting or clearing the check box next to the name of the function target.

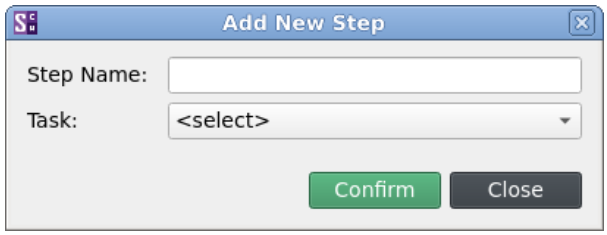
Tasks

Setting up tasks (steps) defines the calibration flow.

To add a task:

1. In the Projects panel, right-click the corresponding workflow and choose **Add New Step to Workflow**.

   The Add New Step dialog box opens.



2. Enter the name of the step.

   If the **Step Name** is not defined, then it takes the same name as the task.

3. Select a task.

   Each module has a list of valid tasks that can be added as calibration steps.

4. Click **Confirm**.

The step is added to the end of the workflow.

After a task is added, you can define its options as follows:

- If the task is custom, then you can define a Python script to be executed in that task.
- For any other task type, you can define the arguments of that task, as well as the prologue and epilogue commands (of that task) together with their arguments.

Importing Data From Sentaurus Workbench Projects

You can import calibration parameters, condition parameters and targets from the data defined in Sentaurus Workbench projects. It is expected that you import this data from the parent project used.

The Sentaurus Workbench parameters can be imported as calibration parameters (Parameters tab) or condition parameters (Conditions tab), with all the values used in the given project. Also, it is possible to use the extracted variables from the Sentaurus Workbench project to create targets (Targets tab) automatically, with the selected condition parameters.

You can import Sentaurus Workbench parameters in one of the following ways.

To import Sentaurus Workbench parameters as condition parameters:

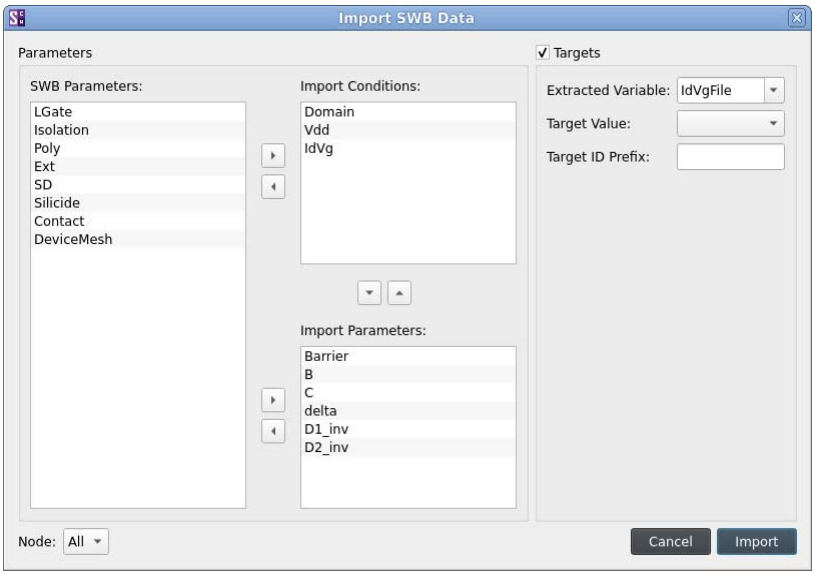
1. In the Sentaurus Calibration Workbench GUI, choose **File** > **Import SWB Data**.

   The Find SWB Project dialog box opens.

   Select the Sentaurus Workbench project to be used and click **Choose**.

   Note:
   If the parent project is defined, then the dialog box has that directory selected. If the parent project is not defined, then the dialog box has the current project directory selected.

The Import SWB Data dialog box opens.



2. Highlight the required Sentaurus Workbench parameters in the left pane and move them to the **Imported Parameters** or **Imported Conditions** pane.
3. If you want to import Targets, enable the Targets section at the right side of the dialog box. If not, skip to Step 7.
4. Set Extracted variable to be used for the Target.
5. Set the Target value, if there is one in the parent project.
6. Optionally, you can define a target prefix for the given set of targets.
7. Click **Import**.

   A dialog box opens, displaying the summary of the operation.

Alternatively, to import all the parameters available in the given Sentaurus Workbench project as condition parameters:

1. In the Projects panel, right-click the selected project and choose **Import All SWB Data**.

   The Find SWB Project dialog box opens.

2. Select the Sentaurus Workbench project to be used.

   Note:
   If the parent project is defined, then the dialog box has that directory selected. If the parent project is not defined, then the dialog box has the current project directory selected.

Converting Condition Parameters to Calibration Parameters, and Calibration Parameters to Condition Parameters

You can convert condition parameters (**Conditions** tab) to calibration parameters (**Parameters** tab) and vice versa.

To do so, right-click any column of a selected set of condition parameters (**Conditions** tab) and choose **Convert Condition Into Parameter**.

Note:

Only numeric condition parameter values can be converted to calibration parameters (**Parameters** tab).

The new calibration parameters do not have their lower and upper bounds set, so you must set them manually.

On the other hand, to convert a calibration parameter (**Parameters** tab) to a condition parameter (**Conditions** tab), you must select the required calibration parameters and then right-click these parameters and choose **Convert Parameter Into Condition**.

The new condition parameters have only one value assigned in the Choices column, with the same value as the initial value of the calibration parameter.

Executing the Calibration Flow

After you have defined the calibration flow, you execute the corresponding calibration by clicking the **Play** button in the Projects panel.

This action launches the Sentaurus Calibration Workbench batch tool (binary `scw`) as a job in the queue defined on the command line (default: `local:default`). The status of the Sentaurus Calibration Workbench GUI is updated at each step, including the last calculated values of parameters and targets.

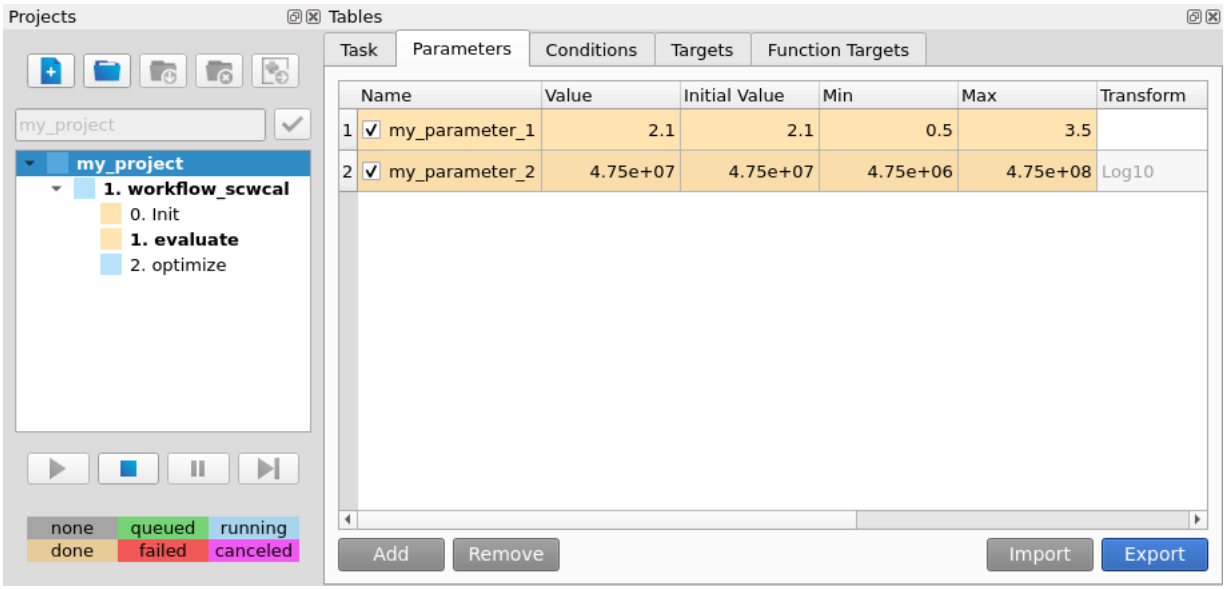
You can also run jobs in a different queue by right-clicking the project or workflow and choosing **Run (Project or Workflow) Calibration**, and then selecting the required queue.

You can also change the queue in the GUI preferences (see Customizing GUI Preferences on page 145).

To check the status of steps, refer the box next to the step name to the legend in the Projects panel.

On the **Parameters**, **Targets**, and **Function Targets** tabs, the last values calculated are shown in cells with an orange background (see Figure 22).

Figure 22 Projects panel and Tables panel showing a running project



Plots

This section describes different types of plot and how to create them in the Sentaurus Calibration Workbench GUI.

You also can create plots directly from the tables (*quick plots*).

Creating Curve Plots From Scalar Data

You can create curve plots from data represented as a single scalar value per step, which means that the bottom (x) axis represents the step or sub-step number, and the y (left) axis represents the scalar data in a single cell from one table.

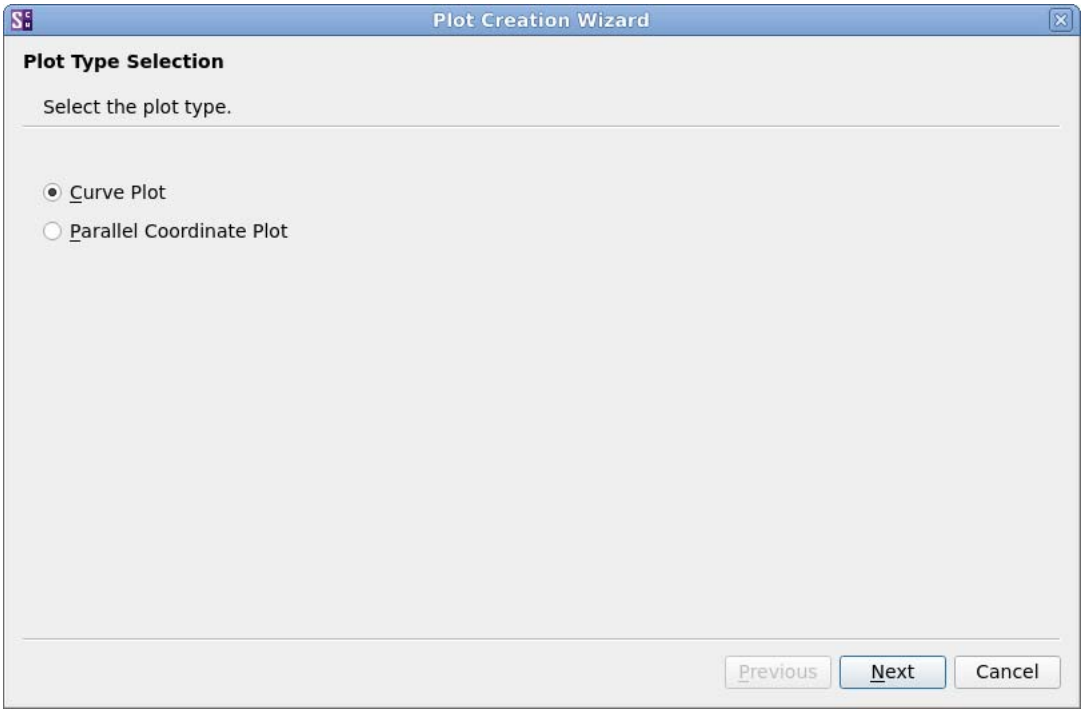
An example of this type of data is the RMS error of the target obtained as the result of the execution of the step (calibration task). You also can create curve plots from the total RMS error (all targets).

To create a curve plot from scalar data:

1. Choose **Plots** > **New Plots**.

   The Plot Creation Wizard opens.

2. On the Plot Type Selection page, select **Curve Plot**.



3. Click **Next**.
4. On the Input Data Selection page, select the data source from which to create the curve plot.

   Select a **Step** or **Sub-Step** (if required) or both.

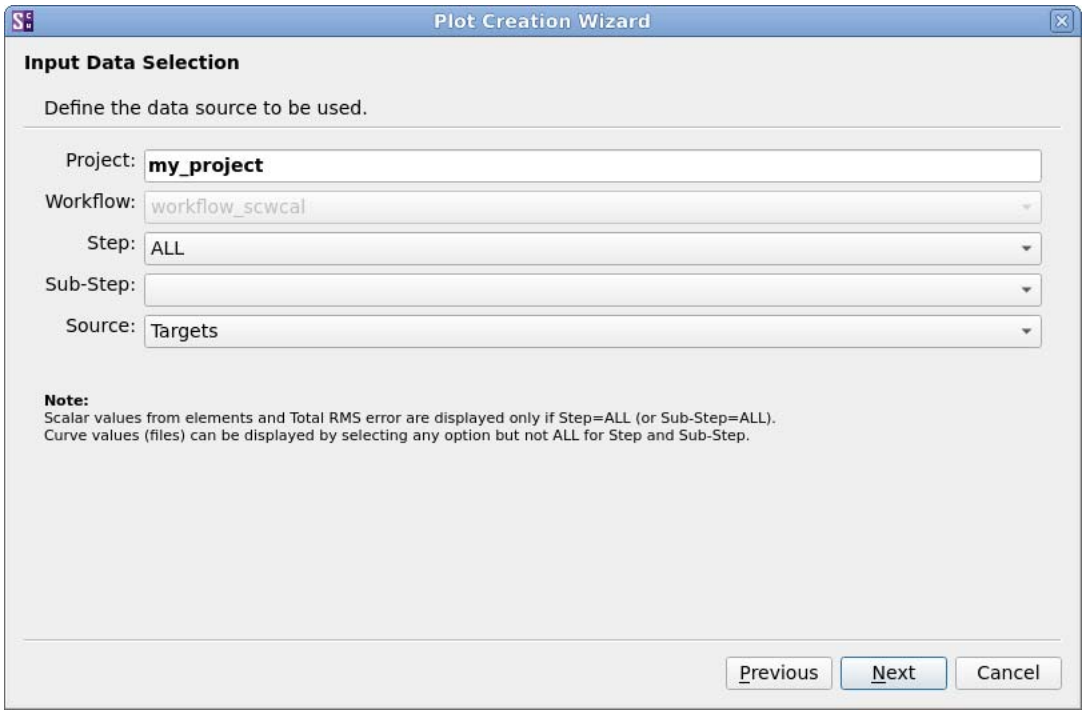
   From the **Step** list, select one of the following:

   - If you select a *specific step*, then the plot will display the same data, independent of the step selected in the Projects panel.
   - If you select **SELECTED** (for curve data), then the plot will show the step data corresponding to the step selected in the Projects panel.
   - If you select **ALL** (only for scalar data), then the plot will use all the steps to create the plot, using a single value per step.

     Scalar data curve plots are not dependent of the step selection in the Projects panel.

   In this case, select **ALL** to create a curve plot from scalar data.

From the **Source** list, select data source: **Parameters** or **Targets**.

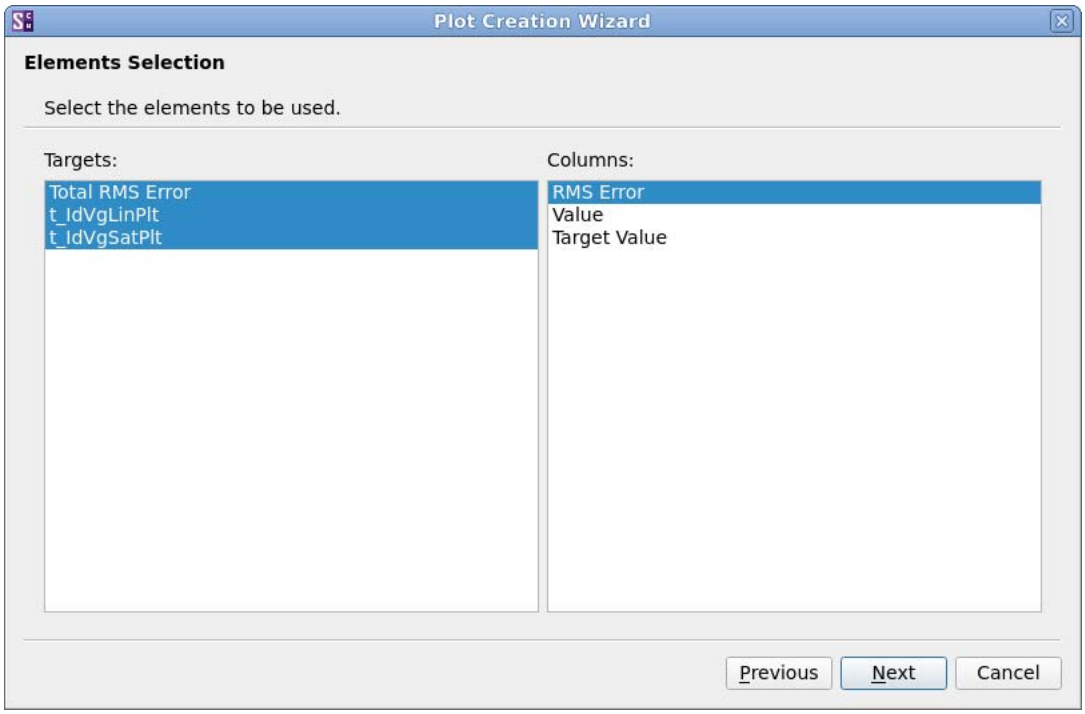


5. Click **Next**.
6. On the Elements Selection page, select the required elements to plot.

   The Targets list always shows the total RMS error and all the available targets. The Columns list shows only the columns or attributes that can be scalar.

   From the Targets list, select all the targets. From the Columns list, select **RMS Error**.

This selection will plot the RMS error of all targets together with the total one in each step.

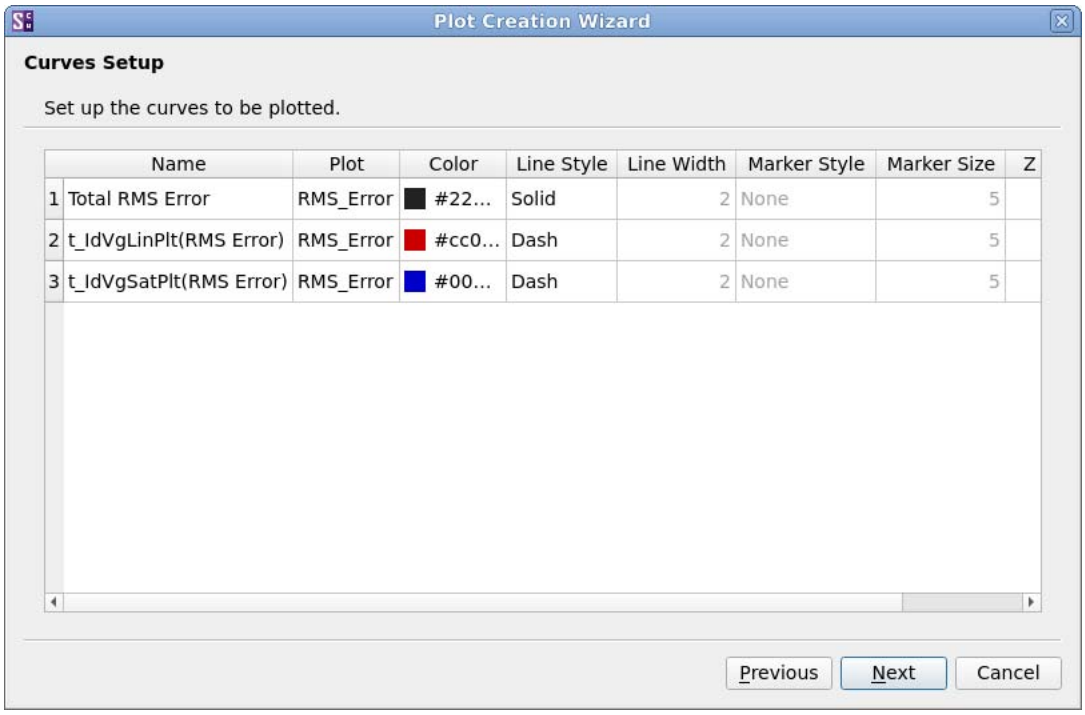


7. Click **Next**.
8. On the Curves Setup page, set up all the curves to be created.

   Usually, the number of curves is the number of combinations of the selections made on the Elements Selection page between the Targets and Columns lists.

   Set the name of the curves, the target plot, the color, the line style and width, and the marker style and size.

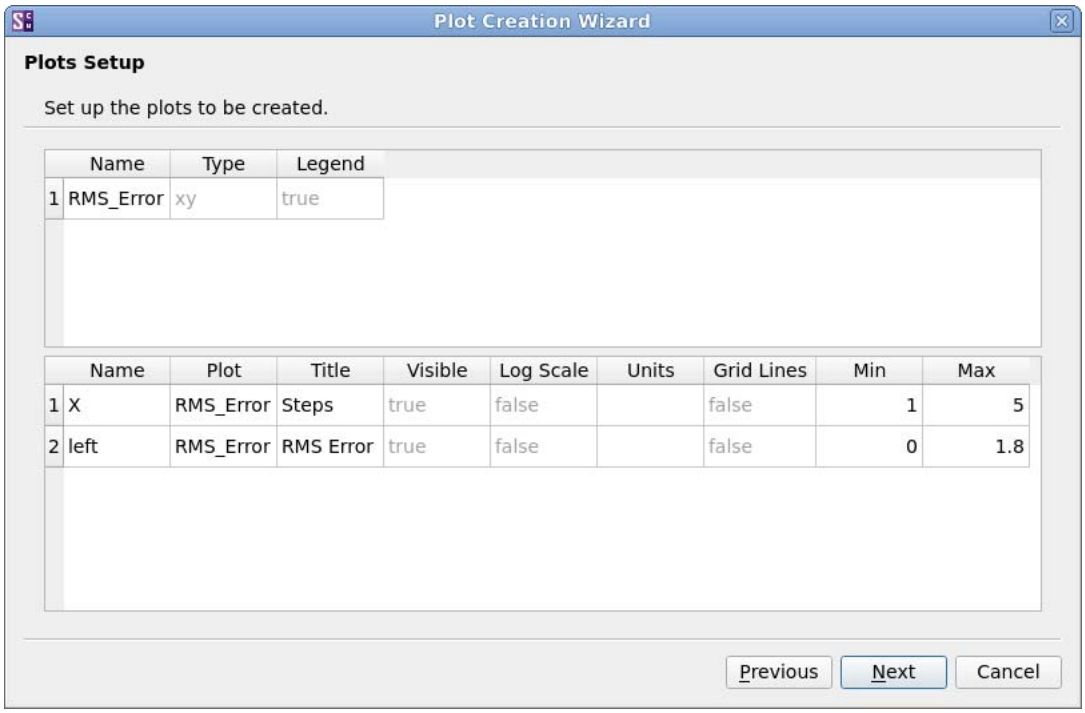
   You can create multiple curves with different curves by changing the name of the plot in the Plot column.



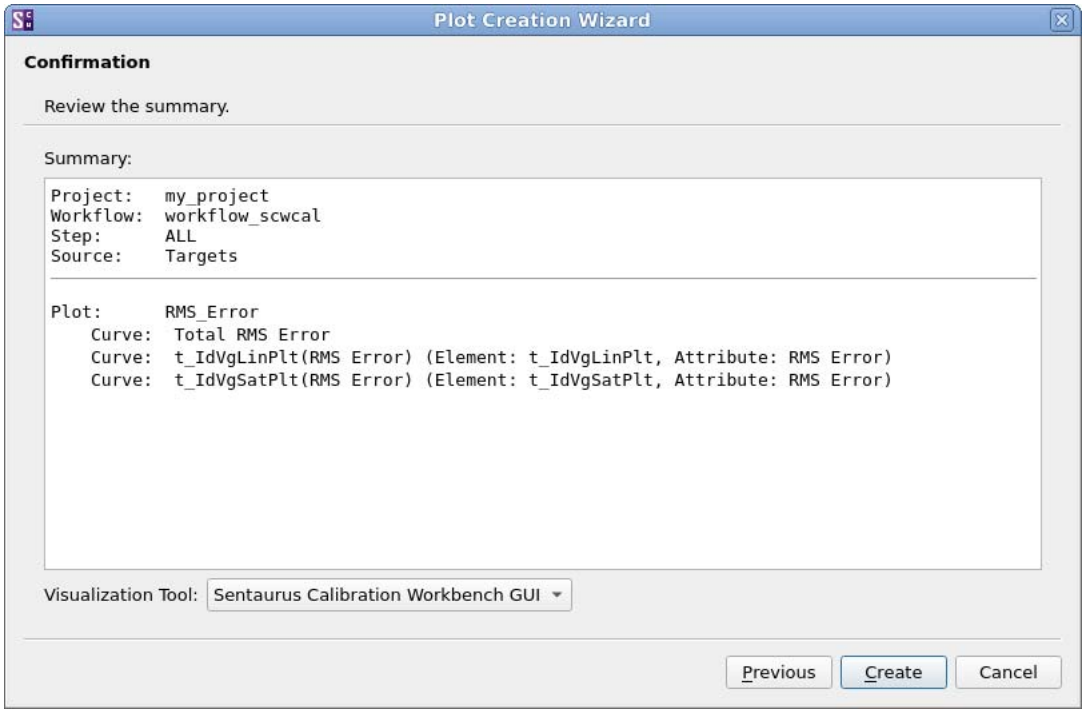
9. Click **Next**.
10. On the Plots Setup page, set the visibility of the curve legend and the axes properties of the plots to be created.

    Axes properties are the title, the visibility, the scaling (log or linear), the units, the grid visibility, and the ranges (minimum and maximum) to be used.

    Note:
    You cannot change the plot name because it is used as a reference for the target plots of the curves to be created from the Curves Setup page.



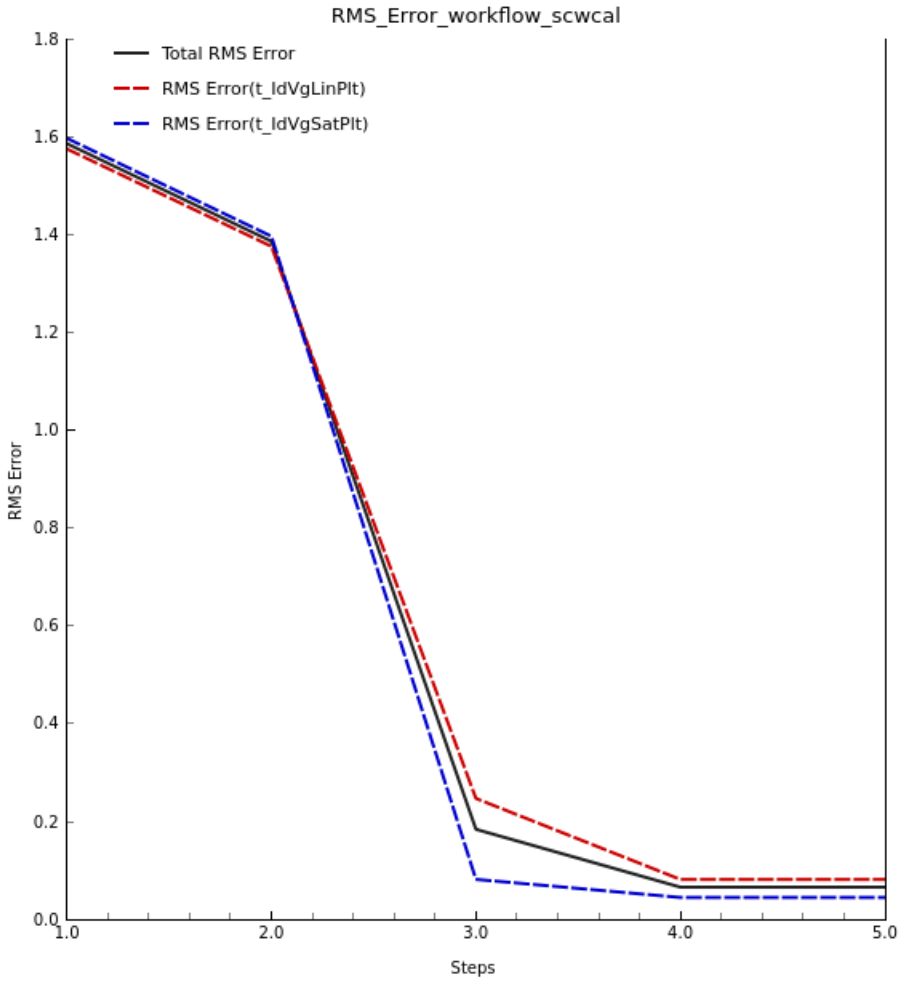
11. Click **Next**.
12. On the Confirmation page, review the summary of the curve plots to be created.



13. From the **Visualization Tool** list, select the tool to use for creating the plot, from either **Sentaurus Calibration Workbench GUI** or **Sentaurus Visual**.
14. Click **Create** to create the plot.

Figure 23 shows the resulting plot.

Figure 23 Curve plots generated using scalar data (total RMS error and RMS error of two targets)



Creating Curve Plots From Curve Data

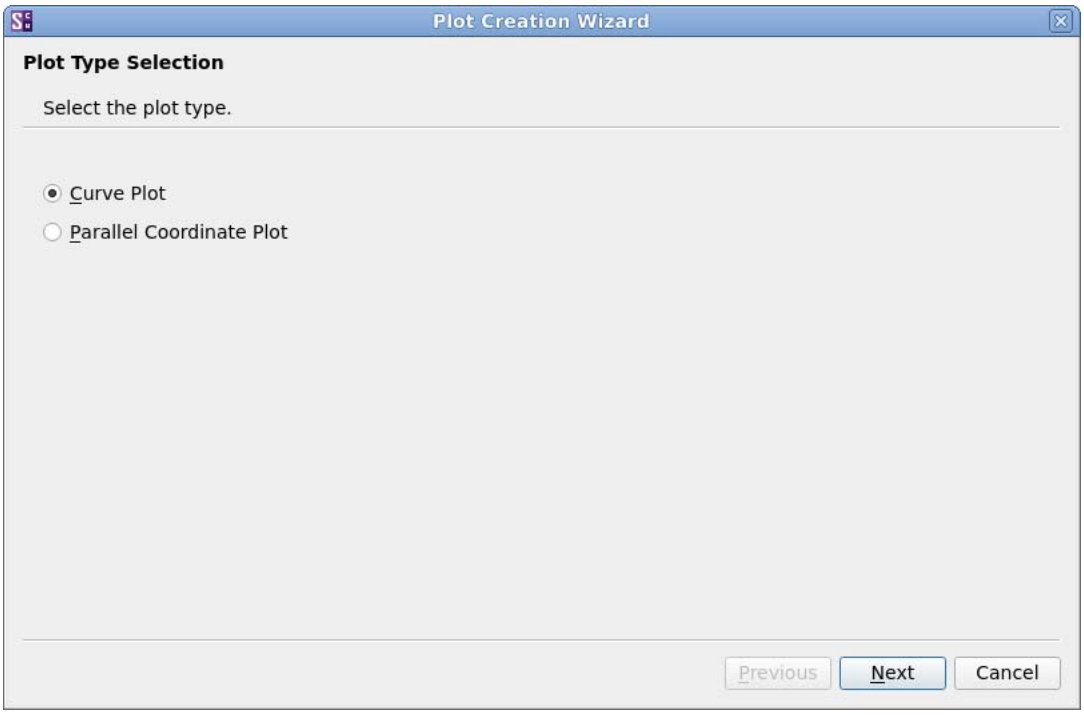
You can create curve plots from data represented as profile file names in a single cell of a table and, optionally, if required, two columns from the same row representing the x- and y-variables to be used from the dataset of the profile file name.

To create a curve plot from curve data:

1. Choose **Plots** > **New Plots**.

   The Plot Creation Wizard opens.

2. On the Plot Type Selection page, select **Curve Plot**.



3. Click **Next**.
4. On the Input Data Selection page, select the data source from which to create the curve plot.

   Select a **Step** or **Sub-Step** (if required) or both.

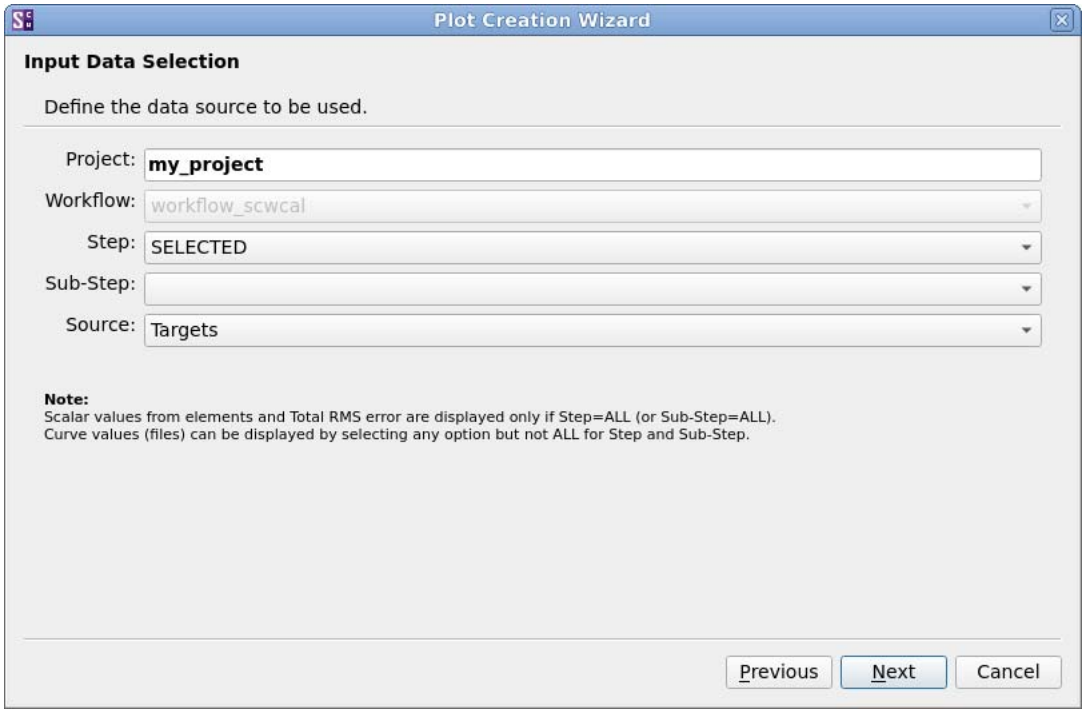
   From the **Step** list, select one of the following:

   - If you select a *specific step*, then the plot will display the same data, independent of the step selected in the Projects panel.
   - If you select **SELECTED** (for curve data), then the plot will show the step data corresponding to the step selected in the Projects panel.
   - If you select **ALL** (only for scalar data), then the plot will use all the steps to create the plot, using a single value per step.

     Scalar data curve plots are not dependent of the step selection in the Projects panel.

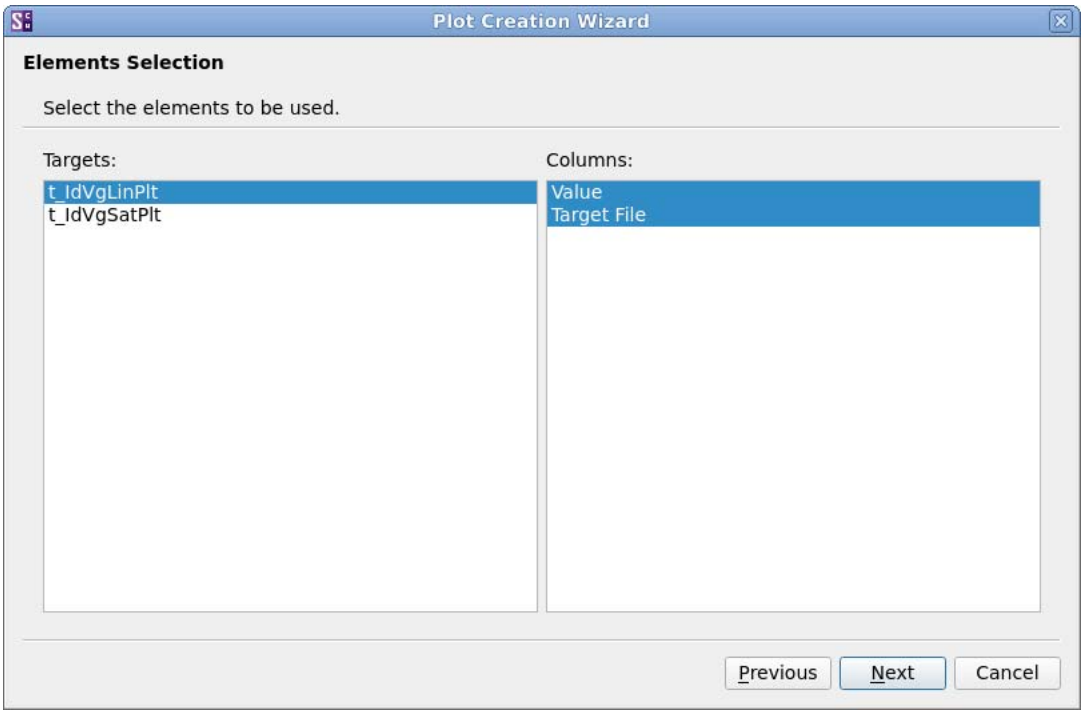
   In this case, select **SELECTED** to create a curve plot from curve data.

   From the **Source** list, select the data source: **Parameters**, **Targets**, or **Function Targets**.



5. Click **Next**.
6. On the Elements Selection page, select the required elements to plot.

   The Targets list always shows all the available targets. The Columns list shows only the columns or attributes that can be defined as curve data (that is, **Value** and **Target File**).

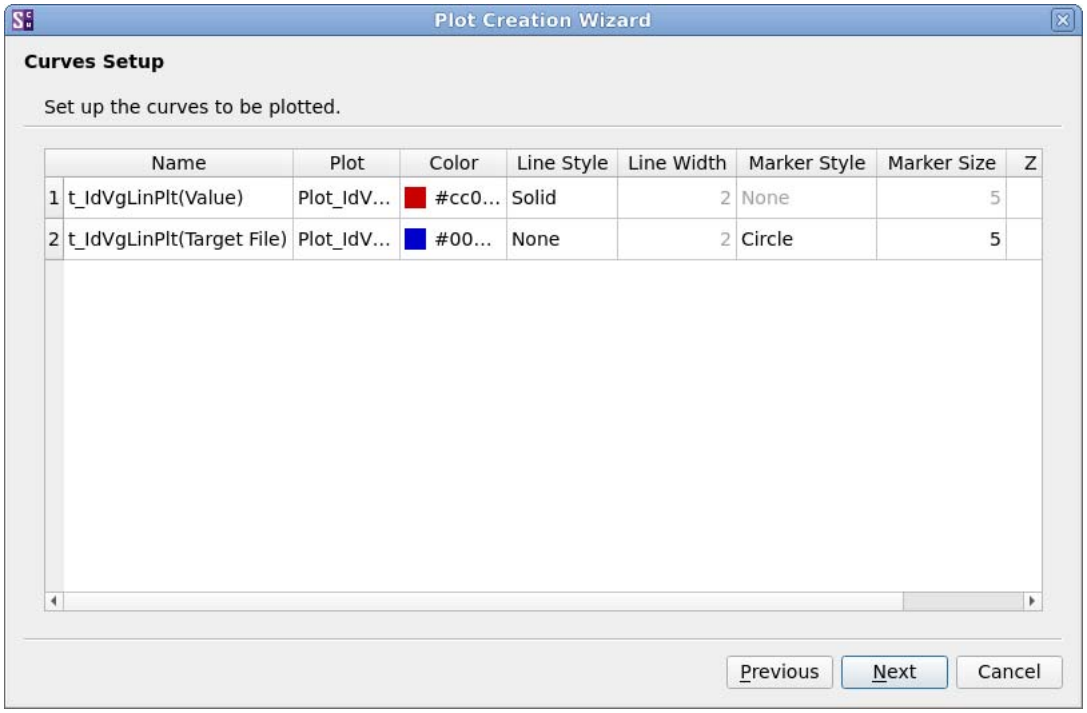


7. Click **Next**.
8. On the Curves Setup page, set up all the curves to be created.

   Usually, the number of curves is the number of combinations of the selections made on the Elements Selection page between the Targets and Columns lists.

   Set the name of the curves, the target plot, the color, the line style and width, and the marker style and size.

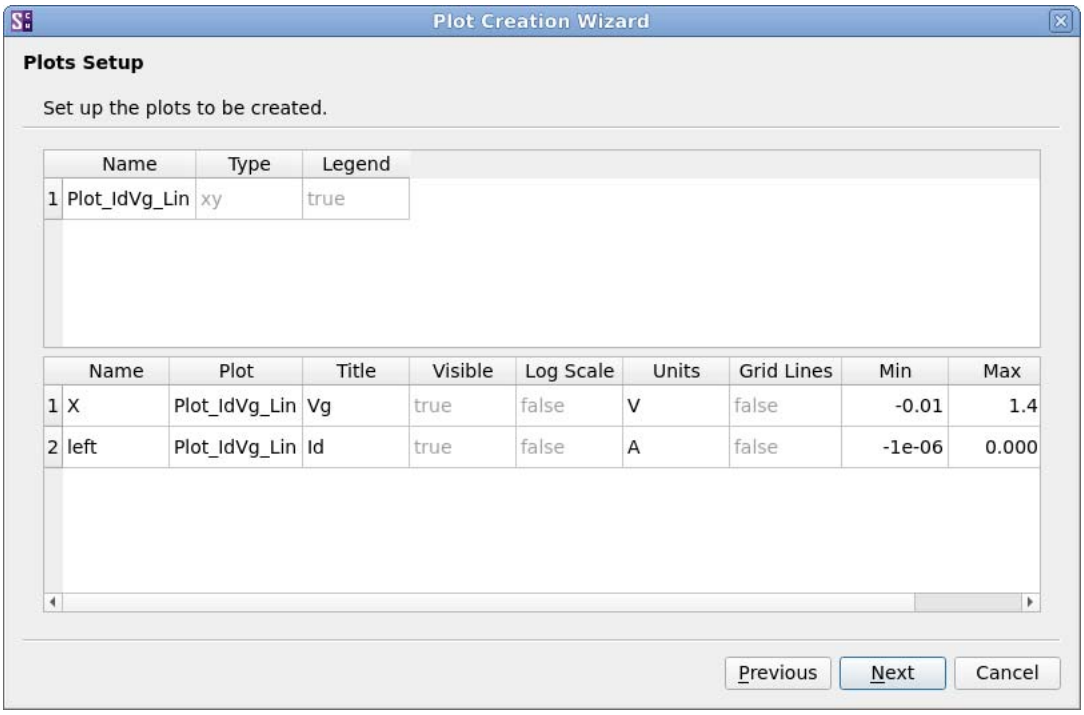
   You can create multiple curves with different curves by changing the name of the plot in the Plot column.



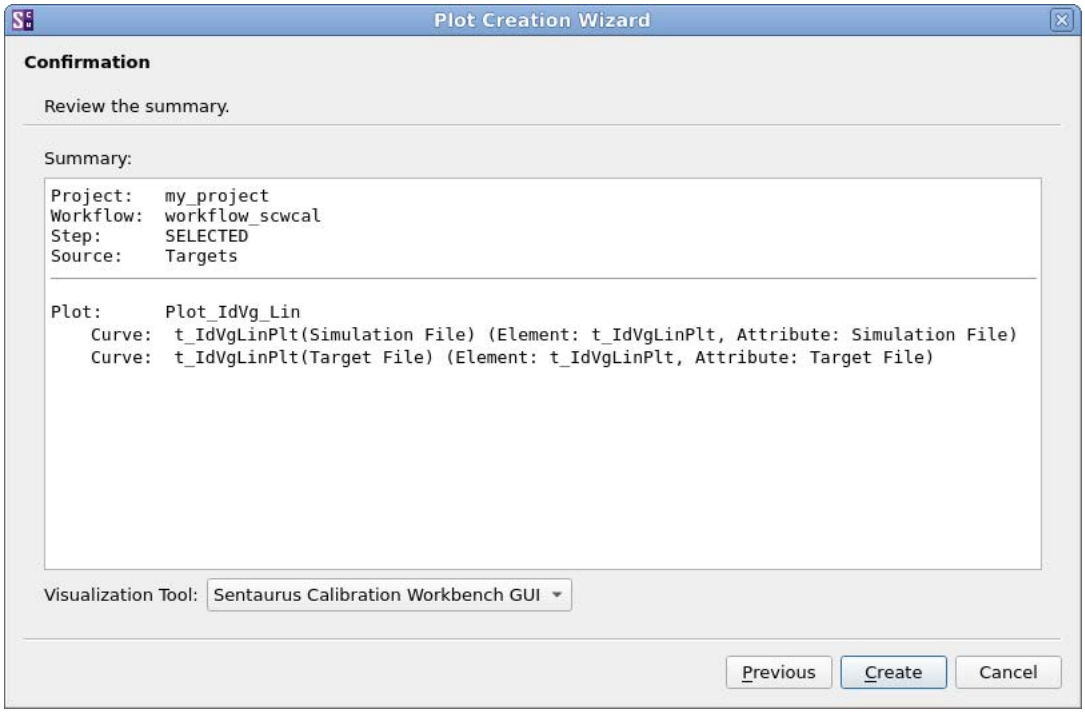
9. Click **Next**.
10. On the Plots Setup page, set the visibility of the curve legend and the axes properties of the plots to be created.

    Axes properties are the title, the visibility, the scaling (log or linear), the units, the grid visibility, and the ranges (minimum and maximum) to be used.

    Note:
    You cannot change the plot name because it is used as a reference for the target plots of the curves to be created from the Curves Setup page.



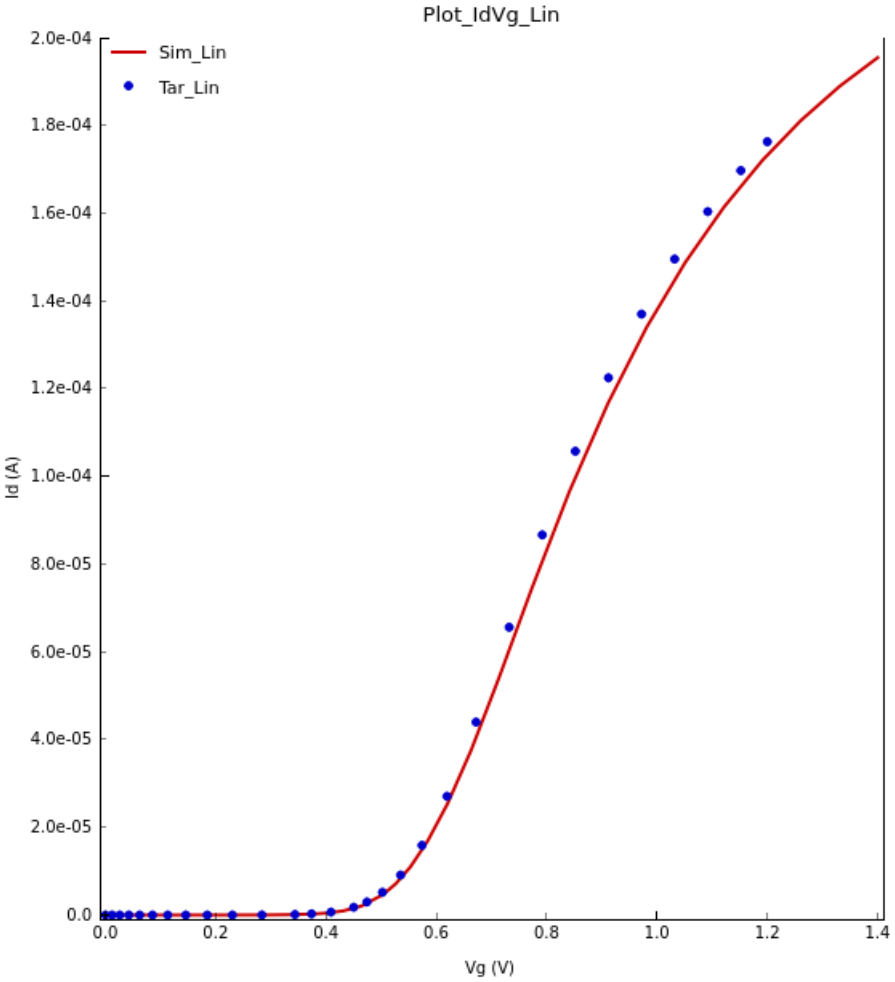
11. Click **Next**.
12. On the Confirmation page, review the summary of the curve plots to be created.



13. From the **Visualization Tool** list, select the tool to use for creating the plot, from either **Sentaurus Calibration Workbench GUI** or **Sentaurus Visual**.
14. Click **Create** to create the plot.

Figure 24 shows the resulting plot.

Figure 24 Curve plots generated using curve data



Creating Parallel Coordinate Plots

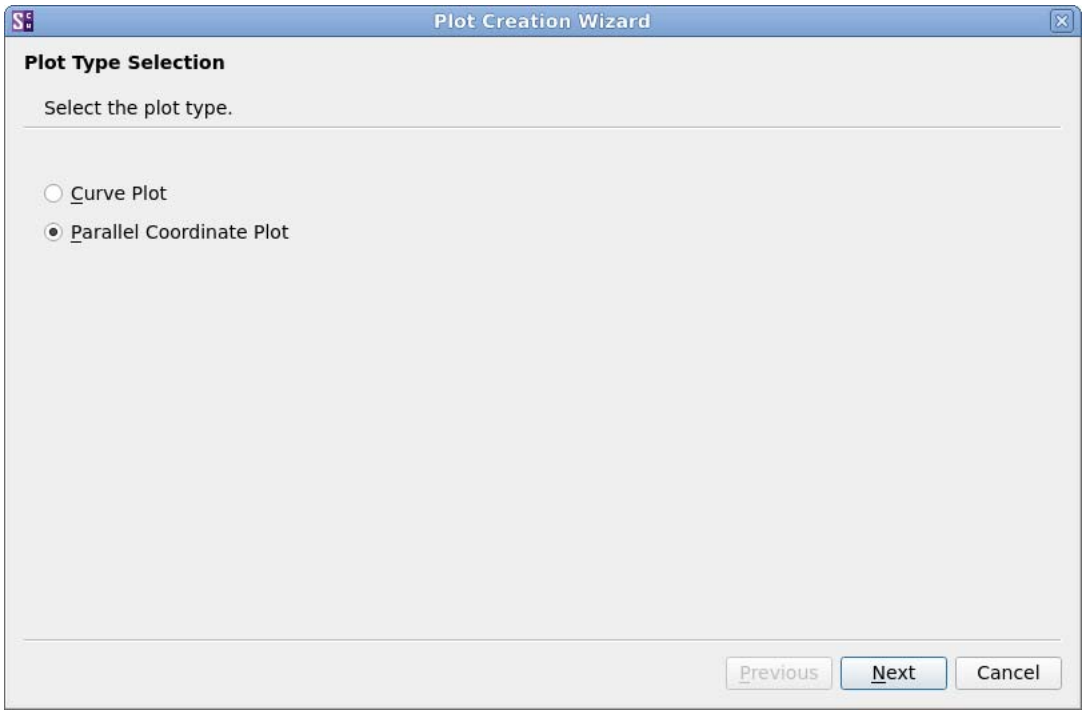
You can create parallel coordinate plots from tables of data generated by the Sentaurus Calibration Workbench batch tool or from data in CSV files.

To create a parallel coordinate plot:

1. Choose **Menu** > **Plots** > **New Plots**.

   The Plot Creation Wizard opens.

2. On the Plot Type Selection page, select **Parallel Coordinate Plot**.

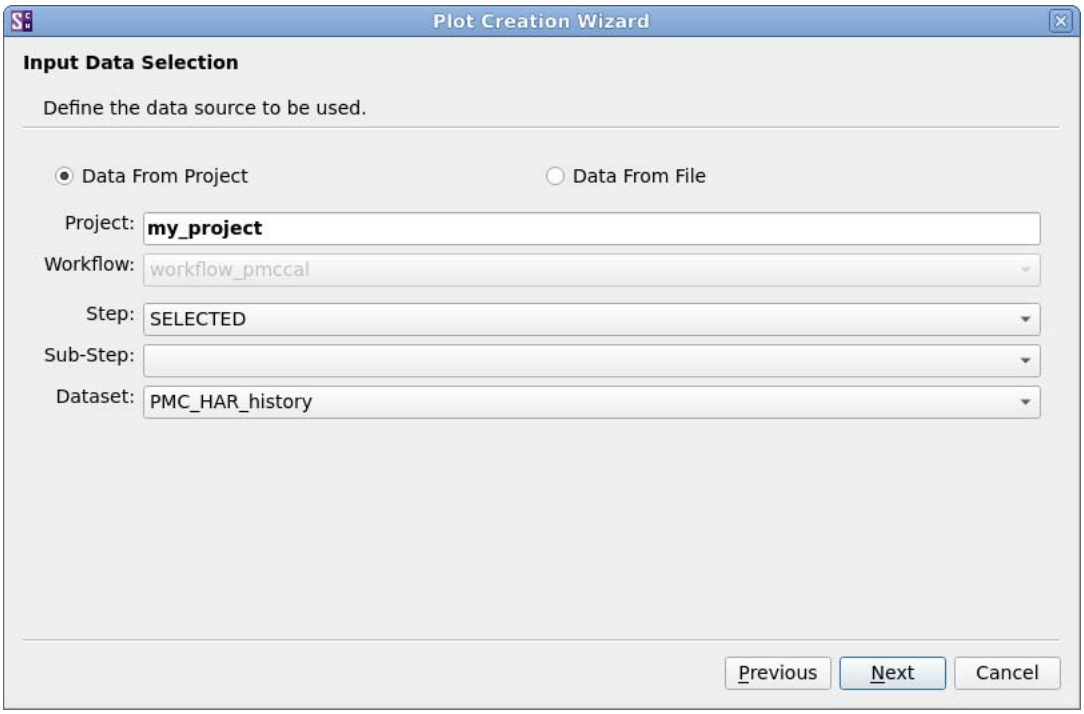


3. Click **Next**.
4. On the Input Data Selection page, select the source of data to create the parallel coordinate plot.

   If the project contains table data in any step (or sub-step), then the **Data From Project** and **Data From File** options are available. However, if there is no table data in the project, then only the **Data From File** option is available.

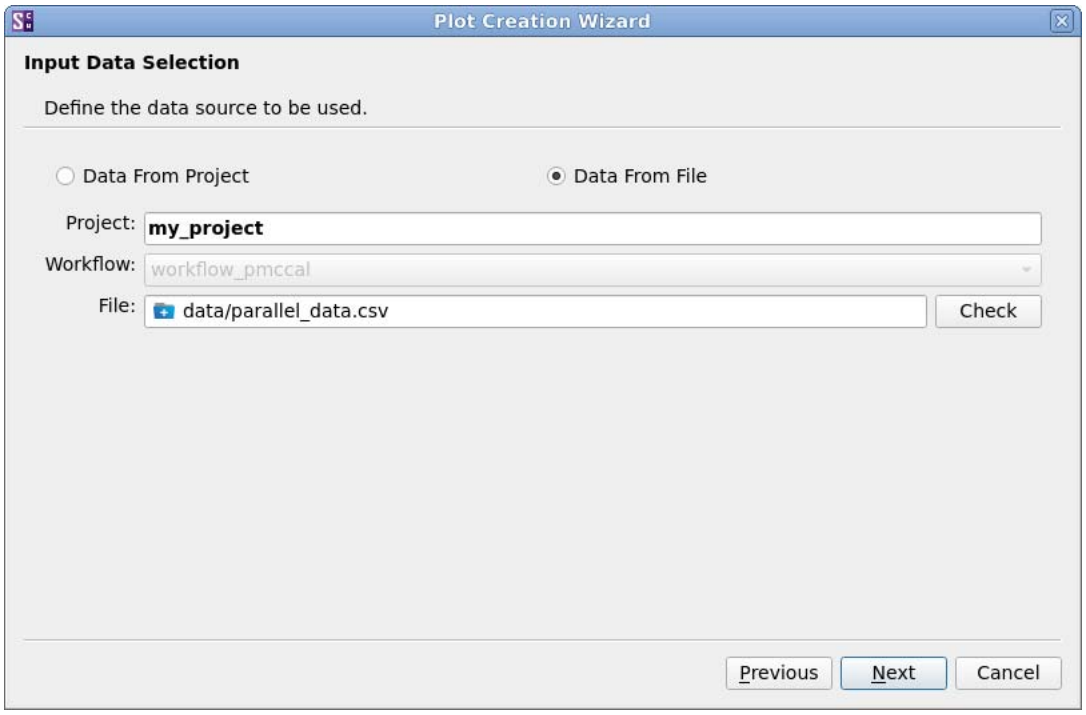
   - If you select **Data From Project**, then you must select the **Step** (and **Sub-Step** if required) that contains a dataset to be shown. The **Dataset** lists only the available datasets (table data) that match the **Step** and **Sub-Step** selection.

     If you select **SELECTED** for **Step** or **Sub-Step**, then the plot will show the data of the selected step or sub-step in the Projects panel.



- If you select **Data From File**, then you must select a CSV file to be plotted in the **File** field.

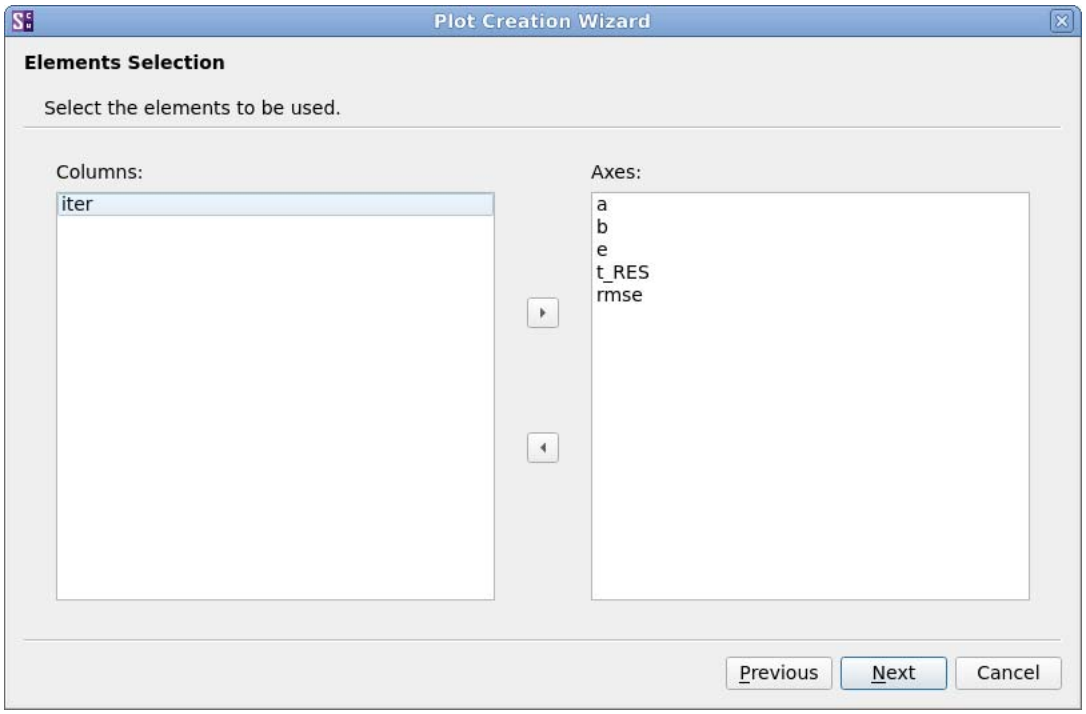
  If you select **SELECTED** for **Step** or **Sub-Step**, then the plot will show the data of the selected step or sub-step in the Projects panel.



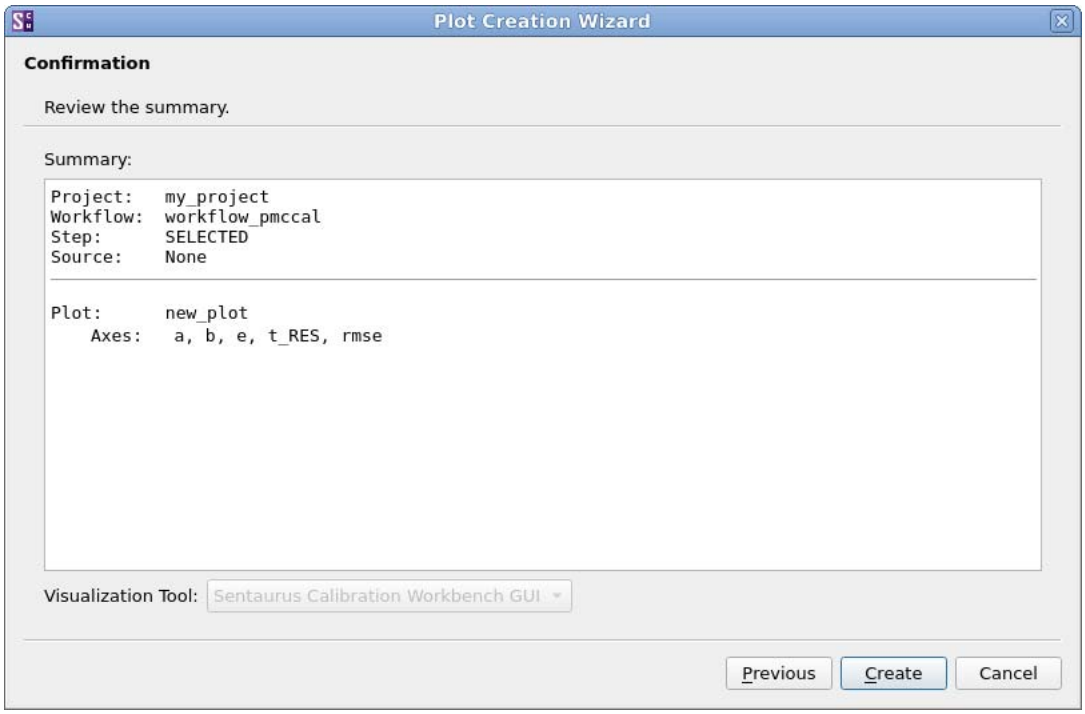
5. Click **Next**.
6. On the Elements Selection page, select the elements to be plotted.

   From the Columns lists, select which columns of the table data must be plotted.

   The Axes list is filled automatically with the columns that match the calibration parameters and target names, and includes the RMS error if it exists.



7. Click **Next**.
8. On the Confirmation page, review the summary of the plot to be created.



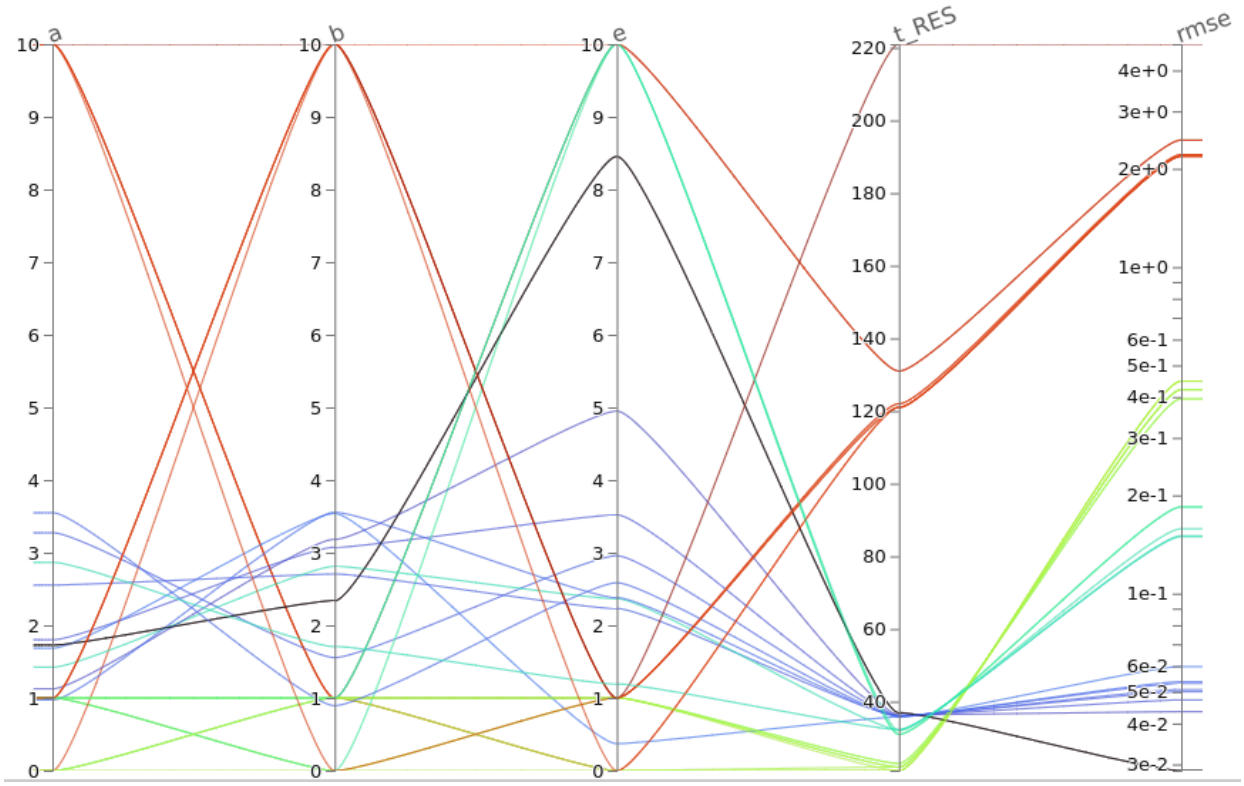
9. From the **Visualization Tool** list, select the tool to use for creating the plot, from either **Sentaurus Calibration Workbench GUI** or **Sentaurus PCM Studio**.

   Note:
   If Sentaurus PCM Studio is not available, then the entry appears gray.

10. Click **Create** to generate the parallel coordinate plot.

Figure 25 shows the resulting plot.

Figure 25 Parallel coordinate plot



Creating Quick Plots

To create a quick plot:

1. In a table, select the cells with the required information to generate a plot.
2. Right-click the selected cells and choose **New Quick Plot**.

   If plots have already been generated, then there is an option to add the requested curves into any of those plots.

You can also right-click in the **Total RMS Error** field, on the **Targets** or **Function Targets** tab, and choose **Quick Plot** to create a plot from this data.

Using this feature, curves are formatted automatically according to the following rules:

- Each color is assigned, in order of curve creation, to the specific element of the tables (row), so every curve created from the same element, but different column, has the same color in a plot.

- On the **Parameters** tab, calibration parameter elements have the following format:
  - The value has a solid line style, with a width of 2.
  - The initial value has a dash line style, with a width of 1.
  - The range (Min and Max columns) has a dot line style, with a width of 2.
- On the **Targets** tab, target elements have the following format:
  - The scalar or simulated profile value has a solid line style, with a width of 2.
  - The source profile has circles as markers of size 7, without any line.
  - The target scalar value has a dot line style, with a width of 1, and circles as markers of size 5.
  - The RMS error value has a dash line style, with a width of 2.
  - The total RMS error has a solid line style, with a width of 2, and is dark gray.
- On the **Function Targets** tab, function target elements have the following format:
  - The scalar value has a solid line style, with a width of 2.
  - The target scalar value has a dot line style, with a width of 1, and circles as markers of size 5.
  - The RMS error value has a dash line style, with a width of 2.
  - The total RMS error has a solid line style, with a width of 2, and is dark gray.

Creating Plots in Other Tools

You can create plots in other tools such as Sentaurus Visual and Sentaurus PCM Studio for files mentioned in the tables. Using this feature allows you to add more plots to the already opened instance of any of these tools.

Note:

Only CSV files can be displayed with Sentaurus PCM Studio.

To create a plot in other tools:

1. In a table, select the cell with the required information to generate a plot.
2. Right-click the cell and choose **Visualize in Sentaurus Visual** or **Visualize in Sentaurus PCM Studio**.

   Note:

   If Sentaurus PCM Studio is not available, then this option appears gray.

9

Advanced Image Analysis and Metrology (AImage)

This chapter describes how to use the Advanced Image Analysis and Metrology tool to analyze images with computer vision and machine learning algorithms.

Introduction to AImage

Overview of Functionality

Advanced Image Analysis and Metrology (AImage) is a software tool integrated with Sentaurus Calibration Workbench that extracts information from images. Information includes measurements, features, segments, profiles. AImage supports advanced computer vision and machine learning algorithms for pattern recognition. AImage is primarily used in TCAD to analyze TEM/SEM images, but it also works with general purpose images (grayscale or color RGB).

Concepts

- **Computer vision**: The field that focuses in acquiring, processing, analyzing, and understanding digital images. The goal is to extract explainable data from the real world. The term *computer vision* is typically interchangeable with Image Analysis or Image Processing.
- **Image**: A visual representation. A digital image is composed of picture elements, also known as pixels, each with finite, discrete quantities of numeric representation for its intensity.
- **Transmission Electron Microscopy (TEM)**: A microscopy technique in which a beam of electrons is transmitted through a specimen to form an image. The specimen is most often an ultrathin section with less than 100-nm thickness or a suspension on a grid.
- **Scanning Electron Microscopy (SEM)**: A type of electron microscope that produces images of a sample by scanning the surface with a focused beam of electrons. The

electrons interact with atoms in the sample, producing various signals that contain information about the surface topography and composition of the sample.

- **TEM Versus SEM**: The main difference between SEM and TEM is that SEM creates an image by detecting reflected or knocked-off electrons, while TEM uses transmitted electrons (electrons that are passing through the sample) to create an image. As a result, TEM offers valuable information on the inner structure of the sample, such as crystal structure, morphology, and stress state information, while SEM provides information on the sample's surface and its composition.
- **Segmentation**: The representation of an image into something that is more meaningful and easier to analyze. The representation tries to classify the pixels in regions that belong to a class.
- **Measurement**: The quantification of attributes of an object or event. In computer vision, measurements quantify the regions or objects inside an image. The measurements are expressed as distances, areas, angles, or polygons.
- **Contour Extraction**: The process that obtains the boundary of a region or object in the image. The boundary is represented as a curve or polygon joining all the continuous points along the boundary.

Features

AImage compress a set of features to solve image analysis based on measurements and segmentations. See Table 16 for a detailed list of features.

Table 16 List of features provided in Advanced Image Analysis and Metrology (AImage)

Group	Description
General	Library for image processing, executable app for GUI and scripting GUI (objects tree, viewer, algorithms, console, log) Open/Save project files Multiple application windows

Table 16 List of features provided in Advanced Image Analysis and Metrology (AImage)

Group	Description
Image	Import/Export image files (formats: jpg, png, tiff, …) Image supports calibration: scaling, origin, and orientation Batch processing of multiple images Segmentations (masks with transparencies) Processing algorithms: • Resample: crop, padding, resize, translate, rotate • Intensity: cast, rescale, RGB to grayscale • Contour extraction: image or segmentation based • Filtering: Gaussian, mean, median, gradient • Arithmetic: add, subtract, multiply, divide • Logic: Not, And, Or, Xor • Additive Noise: Gaussian, salt and pepper, shot, speckle • Statistics: Image intensity, measurements • Box: replicate, crop from image
Measurement	Distances, angles, polygons, and bounding boxes Import/Export as text file (txt, csv, plx) Update according to global units Manually and automatically generated
Segmentation	Manual (Draw, Erase) Thresholding Automatic ML-based (pretrained and custom models) Segment Anything

Running AImage

AImage is a Python library, a graphical user interface and a command line tool part of TCAD Sentaurus™ and Sentaurus Calibration Workbench.

Sentaurus Calibration Workbench

AImage can run from the Sentaurus Calibration Workbench GUI doing the following:

1. Run scwgui from command line: `$ scwgui`
2. Open the Tools menu and click **Open Advanced Image Analysis and Metrology …**.

Command Line

From the command line, you can run AImage GUI by using:

```
$ aimage <options>
```

Command Line Options

AImage allows you to open projects or images and run scripts through the command line. See Table 17 for details.

Table 17 Command-line options of AImage

Option (abbreviated)	Option Name	Description
-h	`--help`	Shows the command line help message and exit
-v	`--version`	Shows the `aimage` version and exits
-i [INPUT ...]	`--input [INPUT ...]`	Input image files
-p PROJECT	`--project PROJECT`	Project file
-s SCRIPT	`--script SCRIPT`	Script file (Python)
-l LOG	`--log LOG`	Log file. Default: `aimage.log`
	`--mesa`	Runs with mesa driver
	`--offscreen`	Offscreen mode
	`--default-window`	Restores the default window view

Introduction to Image Analysis

Image analysis is the process of extracting information from images. This section elaborates on some concepts to better understand the underlying computations and algorithms in image processing.

Image Intensity

An image is a visual representation of something. A digital image is represented as an n-dimensional array of pixel values. In 2D is a functional representation of intensity values. The functional description of a grayscale image is f: [x,y] → [0, 255], with 0 being black and

255 being white (see Figure 26). A color image (RGB) has three channels. Therefore, in a color image f(x,y) is a vector of three values instead of one. See Figure 27

Figure 26 *Gray scale image. The corresponding intensity values are shown in a 2D array. Source: https://ai.stanford.edu/~syyeung/cvweb/tutorial1.html.*

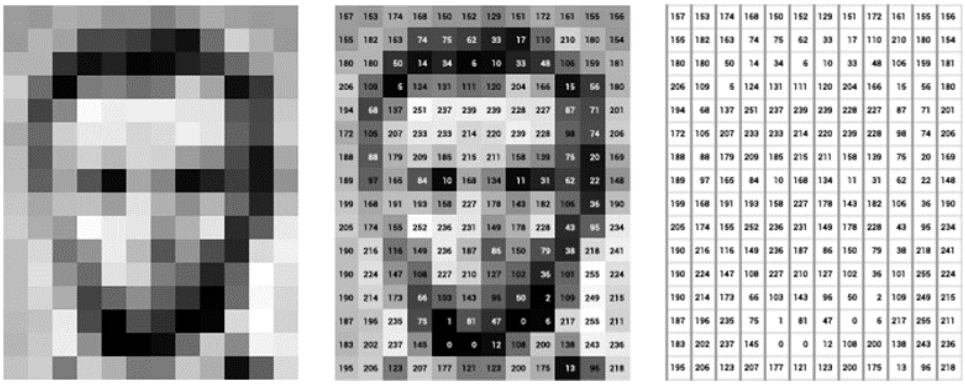


Figure 27 *RGB or color image. (Upper) An image has three components for red, green, and blue colors. (Lower) The corresponding intensity values are shown in a 2D array with a vector representing a single intensity value. Source: https://ai.stanford.edu/~syyeung/cvweb/tutorial1.html.*

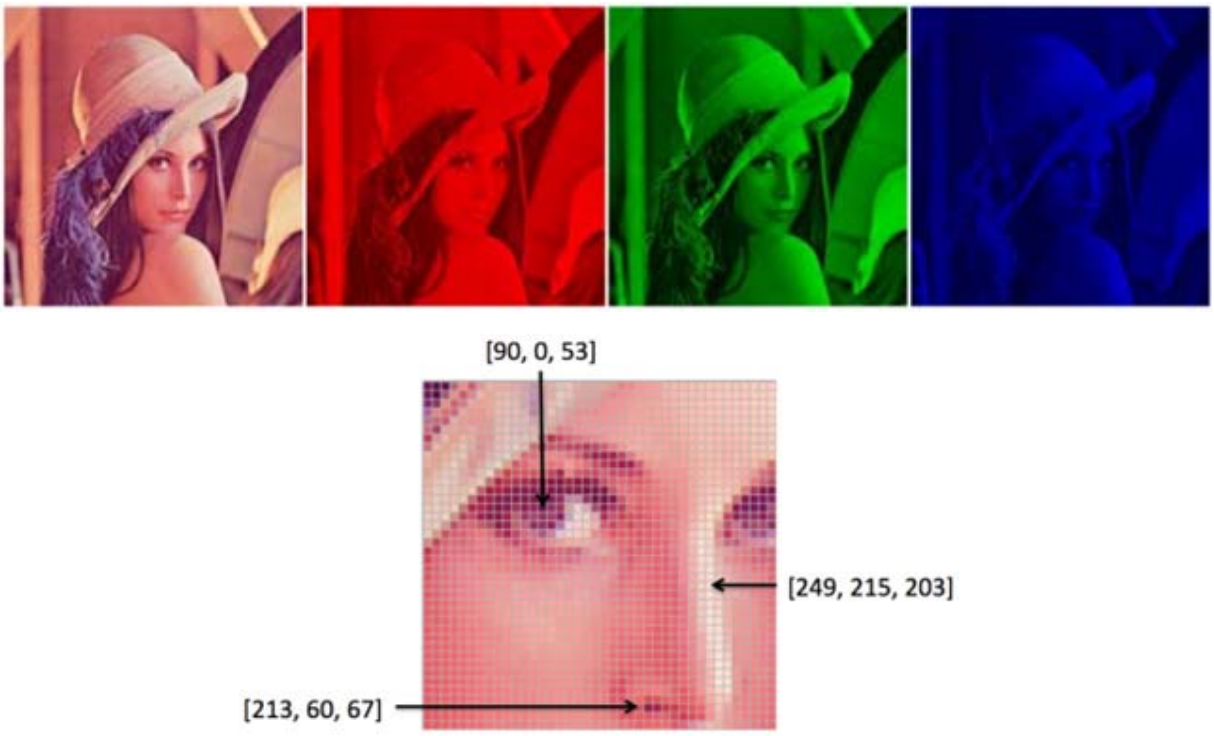
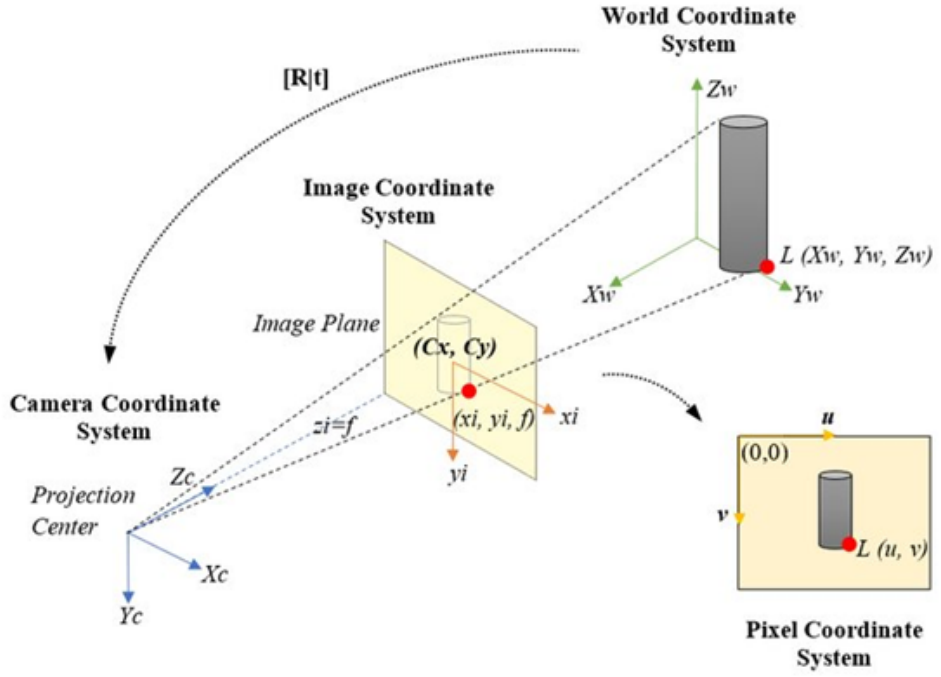


Image Calibration

Image calibration is the process of matching dimensions and coordinates from the image space to some specific units (usually world coordinates with metric units). For 2D calibration, the image is transformed from pixel coordinates to world coordinates with the z coordinate assumed to be 0 (see Figure 28). The transformation requires a scaling 2d-vector, an origin 2d-vector, and an orientation 2x2 matrix. The scaling defines the correspondence pixel per length (meter). The origin represents the zero coordinates for the image. The orientation determines the image rotation with respect to the origin. For simplicity, the calibration process often ignores the orientation.

Figure 28 Correspondence between image coordinate system and world coordinates



The conversion from pixel coordinates (u,v) to world coordinates (x,y) is defined as shown in Equation 1.

Transforming image pixel coordinates to world coordinates (1)

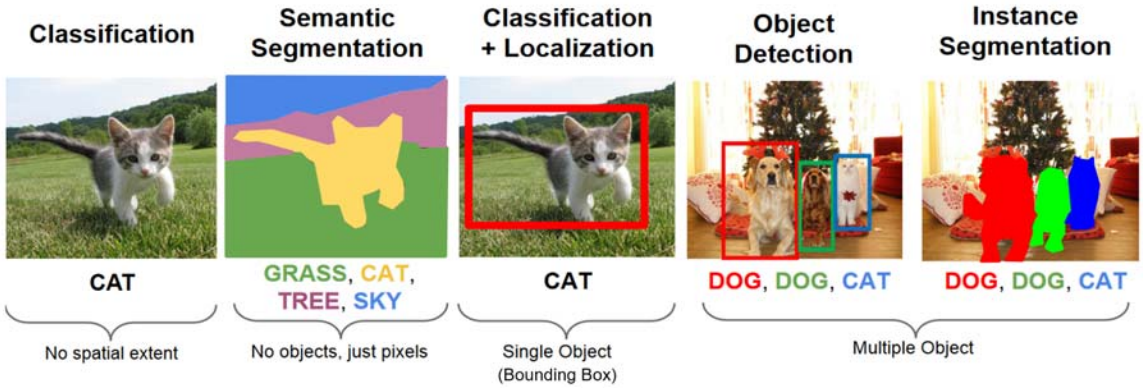
$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} Cos\theta & -Sin\theta \\ Sin\theta & Cos\theta \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Where the scaling factors are expressed as (sx, sy). For digital images, the scaling on each axis is usually the same, that is, a squared area or squared field of view. The orientation matrix depends on the angle θ in 2D and usually takes the value 0. The origin is expressed as (tx, ty).

Image Segmentation

Image segmentation is the process of partitioning a digital image into multiple image segments, also known as image regions or image objects (sets of pixels). The segmentation is also called a mask image. A segmentation is represented as f: [x,y] → [0, 1], with pixel values 1 where the object of interest is and 0 everywhere else. There are multiple terms and approaches shared between object detection and segmentation tasks. Figure 29 compresses and explains the differences between the terminology.

Figure 29 Image segmentation. Source: http://cs231n.stanford.edu/slides/2022/lecture_9_jiajun.pdf



- Classification is the process of labeling the object inside an image.
- Localization refers to the process of defining a bounding box where the classified object is.
- Object detection implies classification and localization of objects inside an image.
- Semantic segmentation is an approach detecting regions, for every pixel, the belonging class.
- Instance segmentation is an approach that identifies, for every pixel, the specific belonging instance of the object and can be described as semantic segmentation combined with classification.

Contour Extraction

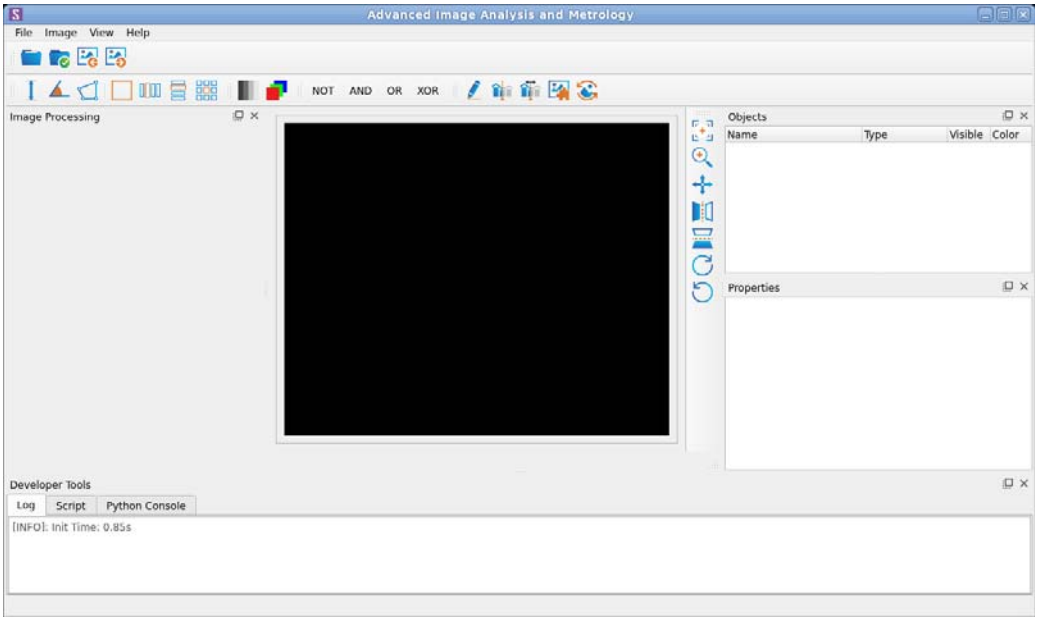
Contours can be explained simply as a curve joining all the continuous points (along the boundary), having the same color or intensity. The contours are a useful tool for shape analysis and object detection and recognition. In computer vision, contour extraction requires to be automated and mathematically well defined. The automation is achieved by first detecting the regions and then extracting the edges, boundaries, or contours. Image segmentation algorithms are a suitable solution for region detection. The contours are better represented by a polygon with a set of points *P*: $[x_1, x_2, \ldots x_f]$.

Features of the User Interface

The graphical user interface (GUI) of AImage offers a main window as shown in Figure 30. The GUI has a menu, toolbars, panels, and a viewer area. The panels' location can be moved around the GUI according to your preferences.

- **Menu**: The menu offers all the functionalities of AImage. The menu is available at the top of the window.
- **Toolbars**: The toolbars contain quick access buttons to create objects and apply algorithms in AImage. The toolbars are in the upper part of the window and below the menu.
- **Image Processing Panel**: This panel allows you to switch image processing algorithms and apply them to image objects. The panel is located on the right side of the window.
- **Object Panel**: It shows the working data objects in AImage. The panel is located to the upper-right side of the window.
- **Properties Panel**: This panel shows the properties of the current object selected in the object panel. The panel is located on the lower-right of the window.
- **Developer Panel**: This panel contains the development tools including a Python console with log and script records. The panel is in the bottom part of the window.
- **Viewer Area**: This is the display area for images and measurements. The panel is in the central area of the window.

Figure 30 Main window of AImage graphical user interface



Menus

The menu bar contains the main functionalities available in the AImage GUI.

Table 18 AImage list of menus with functionality description

Menu	Description
File	Open a new window and quit application. Open, save, and close aimage projects. Import/export image files.
Image	Image algorithms to be shown in the processing toolbar. Includes algorithms related to calibration, segmentation, filter, intensity, arithmetic, resample, additive noise, contour extraction, and statistics.
View	Show and hide window toolbars. Change GUI theme color and restore default view.
Help	Provides information about Advanced Image Analysis and Metrology.

File Menu

The File menu (shown in Figure 31) provides the following actions:

- **New Window**: Open a new window of AImage. The existing window remains with the current working data.

- **Open Project**: Open an AImage project. It removes the current working data.
- **Save Project**: Save the current project data in the selected file (`<project_file>.json`). If called for the first time, it performs the Save As Project action.
- **Save As Project**: Save the current project data and choose the file path and name (`<project_file>.json`). The directory where the project file is located also has additional metadata related to image files.
- **Close Project**: Close and remove the current project file and data.
- **Import Image File**: Open and add an image file to the current project.
- **Export Image File**: Save a selected image in the current project to a file.
- **Quit**: Close the application.

Figure 31 File menu with file options

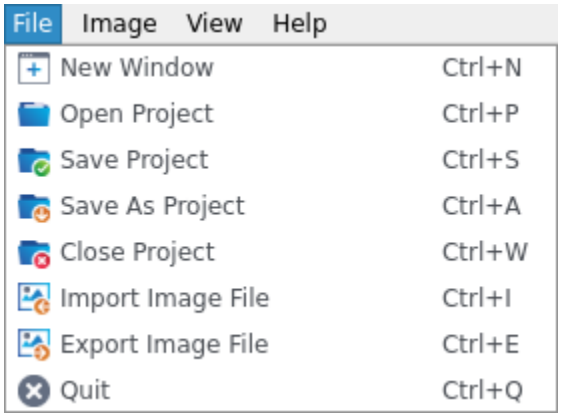


Image Menu

The image menu (shown in Figure 32) allows you to open algorithm forms in the processing panel. The image menu provides the following submenus and actions:

- **Calibration**: Image calibration routines.
  - **Interactive**: Calibration that places objects to define the spacing and the origin.
  - **Manual**: Calibration to set manually spacing and origin for images.
- **Segmentation**: Image segmentation algorithms.
  - **Manual**: Edit segmentation manually.
  - **Otsu**: Automatic thresholding using the Otsu algorithm. Used to generate a segmentation.

    - **Threshold**: Algorithm for setting a threshold value to segment an image.
    - **Segment Anything**: Apply the Segment Anything algorithm to an image.
    - **Train Model**: Method to train a segmentation model using a data set of images and segmentations.
    - **Predict Model**: Use trained model to segment an image.
- **Filter**: Common filters for image processing.
    - **Gaussian**: Apply a Gaussian filter to an image.
    - **Mean**: Apply a mean filter to an image.
    - **Median**: Apply a median filter to an image.
    - **Gradient**: Compute the gradient of an image.
- **Intensity**: Image intensity algorithms.
    - **Cast**: Convert the pixel type for numerical precision.
    - **Rescale**: Rescale image intensity values.
    - **RGB to Gray**: Convert a RGB or RGBA image to gray scale image.
    - **Gray to 3 Channels**: Convert a gray scale image to 3 channels. It replicates the intensity values in the other channels.
- **Arithmetic**: Apply an arithmetic operation between image – constant or between two images.
    - **Addition**: Pixelwise addition of an image with another image or a constant.
    - **Subtract**: Pixelwise subtraction of an image with another image or a constant.
    - **Multiply**: Pixelwise multiplication of an image with another image or a constant.
    - **Divide**: Pixelwise division of an image with another image or a constant.
- **Resample**: Transform images and resample.
    - **Padding**: Pad an image with additional pixels.
    - **Crop**: Crop image size.
    - **Resize**: Change the pixel size of an image.
    - **Translate**: Translate an image defined by a point location.
    - **Rotate**: Image rotation defined by an angle.
    - **Rotate about a point**: Define a central point and an angle of rotation for an image.

- **Noise**: Additive noise filters.
  - **Additive Gaussian**: Add Gaussian noise to an image.
  - **Salt and Pepper**: Add salt-and-pepper noise to an image.
  - **Shot**: Add shot noise to an image.
  - **Speckle**: Add speckle noise to an image.
- **Contour**: Algorithms for contour extraction
  - **Extraction from Segmentation**: Extract contours from a segmentation and create polygons objects to represent the contours.
- **Statistics**: Print statistics report.
  - **Image Intensity**: Show image intensity statistics.
  - **Measurements**: Show measurements statistics. It shows only the statistics for the same type of measurements grouped under the same image.
- **Clear**: Clear the processing panel and remove any algorithm form.

Figure 32 Image menu with image processing algorithms.



View Menu

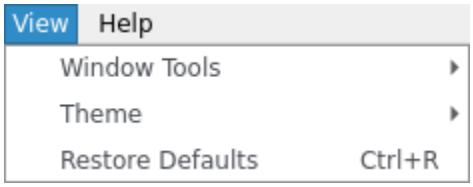
The View menu (shown in Figure 33) can change how the GUI appears.

Window Tools: Check buttons to show/hide toolbars or panels.

Theme: Select a window theme. Options are default, light, dark or auto (light or dark).

Restore Defaults: Change the GUI to the default location of panels and toolbar.

Figure 33 View menu with user interface options



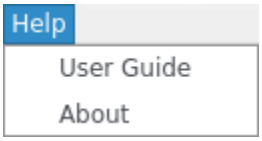
Help Menu

The Help menu is shown in Figure 34.

User Guide: Opens the user guide PDF file

About: Shows information about AImage with the release version

Figure 34 Help menu with information options



Toolbars

File Toolbar

The File toolbar contains shortcut buttons to open/save project files and import/export images.

Figure 35 File toolbar with shortcuts to open/save project and import/export images



Measurement Toolbar

The Measurement toolbar is used to place new measurement objects such as distances, angles, polygons, and bounding boxes. There are some functionalities next to the bounding box button which allow to replicate bounding boxes using pattern recognition in the horizontal, vertical or all directions.

Figure 36 Measurement toolbar with distance, angle, polygon, and bounding box.



Intensity Toolbar

The Intensity toolbar has two actions: one to convert images from RGB to gray and the other one to convert images from gray to 3 channels.

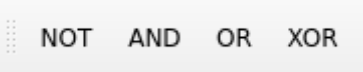
Figure 37 Intensity toolbar to transform RGB to Gray and Gray to 3 Channels



Binary Toolbar

The Binary toolbar is used primarily to apply binary operations to segmentation images. It supports the NOT, AND, OR, and XOR operators. These operators work by selecting the segmentation images on the objects tree (panel) and then pressing the desired operation button.

Figure 38 Binary toolbar with binary operations to be applied to segmentations.



Segmentation Toolbar

The segmentation toolbar allows to create segmentation using manual editing, Otsu algorithm, thresholding, or segment anything. It also includes a functionality to apply inpainting to an area based on a segmentation image.

Figure 39 Segmentation toolbar with segmentation algorithms



Objects Panel

The Objects panel holds an object tree. The tree contains the working and existing objects in AImage. AImage defines a hierarchy where image objects are at the top level and all the measurements (distance, angle, polygon, bounding box) and segmentation objects are

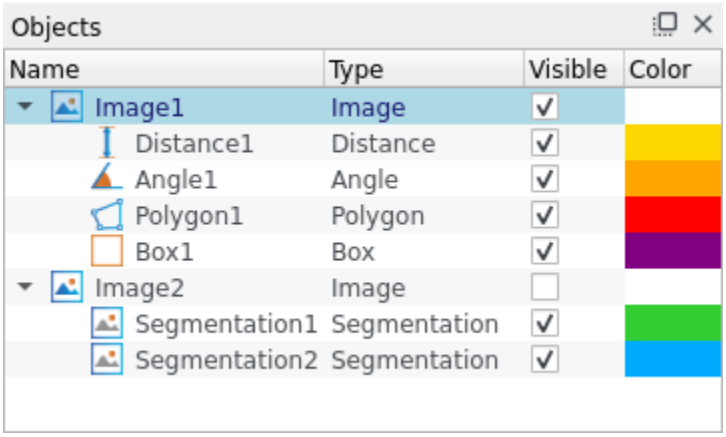
under the image at the lower level. That means that images are the parent object and measurement are child objects.

The object tree has for column is described as:

- **Name**: Object name. Can be changed using the context menu (Right-click > Rename) or by double-clicking the name and then editing the text.
- **Type**: Object type. Fixed parameters.
- **Visible**: Check box to hide or show the object in the viewer.
- **Color**: Object color. Can be changed using left click on the color area. It shows a new window form to select the desired color.

The object tree allows drag and drop.

Figure 40 Objects panel. The listed objects are images, segmentations, distances, angles, polygons, and bounding boxes



Context Menu

The Context menu is the menu that appears when you right-click on an object in the tree. The options in the Context menu depend on the current object. See Table 19 for details.

Table 19 Context menu options in the object tree

Action	Object Type	Description
Rename	All	Change object name.
Delete	All	Delete object from tree.
Import Measurement	Image	Import measurement object from text file. The object is placed as a child object to the image.

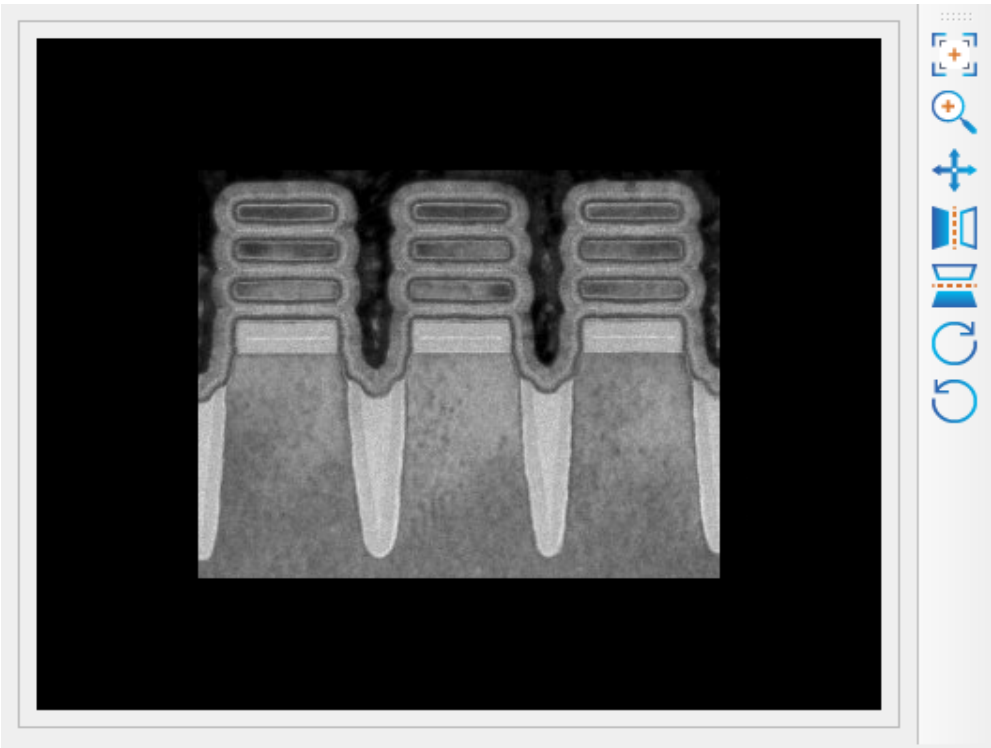
Table 19 Context menu options in the object tree

Action	Object Type	Description
Export Measurement	Measurement (Distance, Angle, Polygon, Bounding box)	Export measurement object to text file. Allowed formats: txt, csv, plx. The txt format is the format that AImage store measurements that are capable to be imported back.
Import Segmentation	Image	Import segmentation files. The segmentation is placed as a child object to the image.
Export Segmentation	Segmentation	Export segmentation files. Segmentations are saved in regular image formats (png, jpg, tiff, …).
Contour Extraction	Segmentation	Extract contours from segmentation. The contours are new polygons objects. If the segmentation has non-connected areas, it creates multiple polygons.
Crop Image to Bounding Box	Bounding Box	Create an image based on the region inside the bounding box.

Viewer Area

The Viewer panel is the central area where images and measurement objects are visualized, see Figure 41. The viewer area has a toolbar to its right side. The Viewer toolbar has buttons to reset the camera view, zooming, panning, horizontal mirror, vertical mirror, rotate clockwise, and rotate anticlockwise.

Figure 41 Viewer panel, containing a toolbar to manipulate the field of view. Source: https://mma.prnewswire.com/media/1504371/Row_of_2_nm_nanosheet_devices.jpg



The viewer allows you to interact with images and measurement objects. When you click an object in the viewer, the object is highlighted in the object panel (tree). The same behavior occurs if an object is selected in the object panel (tree): The object is highlighted in the viewer area. The viewer interaction changes depending on the selected object.

Interaction With Images and Segmentations

The images are zoomed in/out with the mouse wheel or by right-clicking along with up and down movements. The image center can be displaced by holding the middle mouse button and then moving in the desired direction.

Interaction With Measurements

There are two interaction modes for measurement objects. The first mode is placement, and the second mode is editing. The placement is when a new measurement is created using the measurement toolbar. The editing is related to changes of existing objects.

Interaction With Distance and Angle Objects

The distance objects are placed using the **Distance Measurement** button in the measurement toolbar. The distance is created with one left click. Afterward, the created object is displayed in the object panel. Left-clicking again inside the image locates the first point of the distance object. The third left-click finishes the distance placement. During placement, it is possible to perform all the interactions described in Interaction With Images and Segmentations on page 191.

Distance objects allow locking movement in the horizontal direction or in the vertical direction during placement and editing. To lock in the horizontal direction, use the **H** key. For vertical lock, use the **V** key. To reset the locked movement, use the **R** key or the same key that locked the movement.

The editing for distance objects start when the object is selected or highlighted from the Viewer or the Objects panel. Holding the mouse left-click over the end points (displayed as a cross) allows movement editing over the distance object. After releasing the mouse button, the object updates the coordinates values.

You can create angle objects using the **Angle Measurement** button in a similar way to distance objects. In this case, the number of left clicks is four, because the angle objects have three coordinates. The interaction for placement and editing work as with the distance objects.

Interaction With Polygons

The polygons are placed using the **Polygon Measurement** button in the measurement toolbar. The polygon is created with one left click over the viewer area. Then, each point coordinate of the polygon is created with one left click over the desired location. To finish the placement, right-click over the final point coordinate.

You can start editing a polygon by highlighting the object from the Viewer or the Objects panel. The polygon’s points can be moved individually by placing the mouse near a point and holding the left mouse button and dragging over to the new desired location. The entire polygon can shift his location by approaching to one of the polygon’s points and holding the middle mouse button while dragging and dropping to the new polygon location. You can delete a point of the polygon by selecting the desired point in the viewer and then pressing the **Delete** key.

Interaction With Bounding Boxes

You can generate bounding box objects using the **Bounding Box Measurement** button in the Measurement toolbar. The bounding box placement works as drawing a rectangle. Compared to the previously described objects, there is no need of an initial left-click to create the object. The drawing starts left-clicking over the upper-left corner in the image,

holding the mouse button, then dragging to the lower-right corner and dropping in the desired location.

You can edit the bounding box by moving the mouse near the boundaries of the box. The mouse cursor changes its shape, and you can then hold the left mouse button and move to the new desired location. Placing the mouse over the center of the box allows you to shift the box location by dragging and dropping with the mouse middle button.

Properties Panel

The Properties panel shows information about the current object selected in the Objects panel. Figure 42 offers an example with the information about an image object. For measurement objects, it shows the object coordinates.

Figure 42 Properties panel, showing information related to the selected object in the Objects panel. The example shows the information of an image.

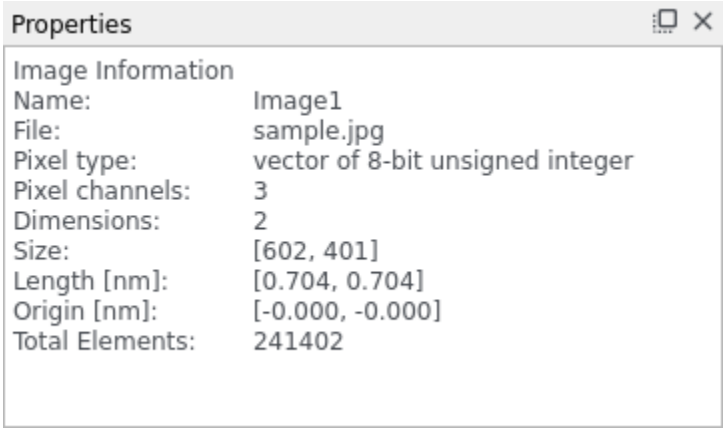
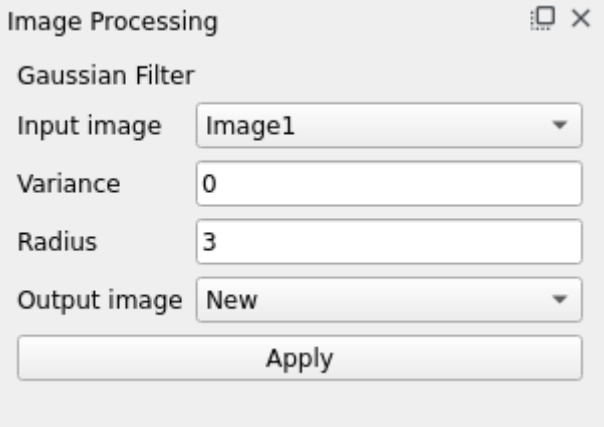


Image Processing Panel

The Image Processing panel is the area where to apply the algorithms to images. The panel changes depending on the selected algorithm using the menu. Figure 43 shows an example of a Gaussian filter and the selectable parameters in the Image Processing panel.

Figure 43 Image Processing panel. The example shows the parameters of a Gaussian filter algorithm.



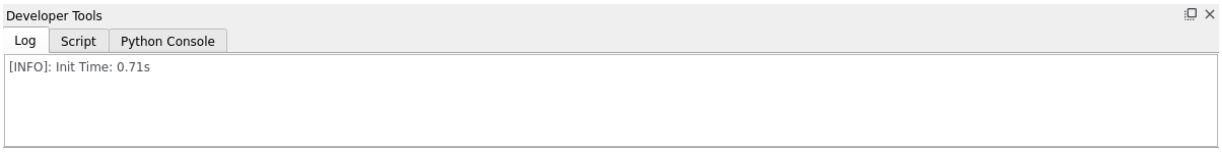
Developer Panel

The Developer panel collects three tabs containing the log, the script record, and the Python console.

Log

The Log tab in the Developer panel contains text of command executions during user interaction with AImage (see Figure 44). If AImage is running from the binary, the log is also available in the command line and in a stored text file (*default location ./aimage.log*).

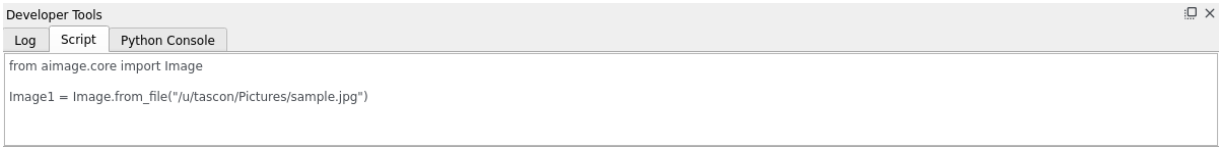
Figure 44 Log tab in Developer panel, which records all the run operations in AImage.



Script Record

The Script record creates Python code while commands and algorithms are executed in AImage (see Figure 45). The text can be copied and saved as a Python file for later reproduction.

Figure 45 Script tab in Developer panel, which records in a Python script all the run operations in AImage



Python Console

The Python Console allows you to enter commands directly to AImage (see Figure 46). The commands can be from any Python package available in the TCAD Sentaurus tool. The Python library name of AImage is “aimage”. The existing objects (images, segmentations, and measurements) in AImage are accessible as variables in the Python interpreter by their name.

Figure 46 Python Console tab in Developer panel, which allows applying Python functions and performing operations over existing objects in AImage

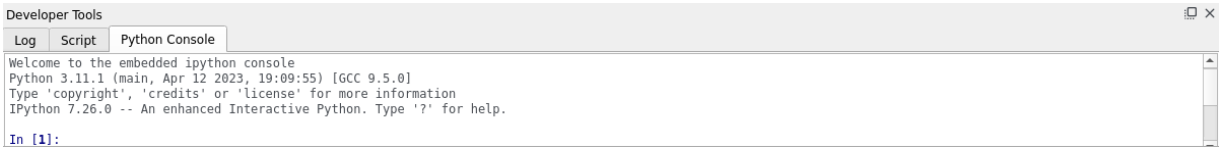


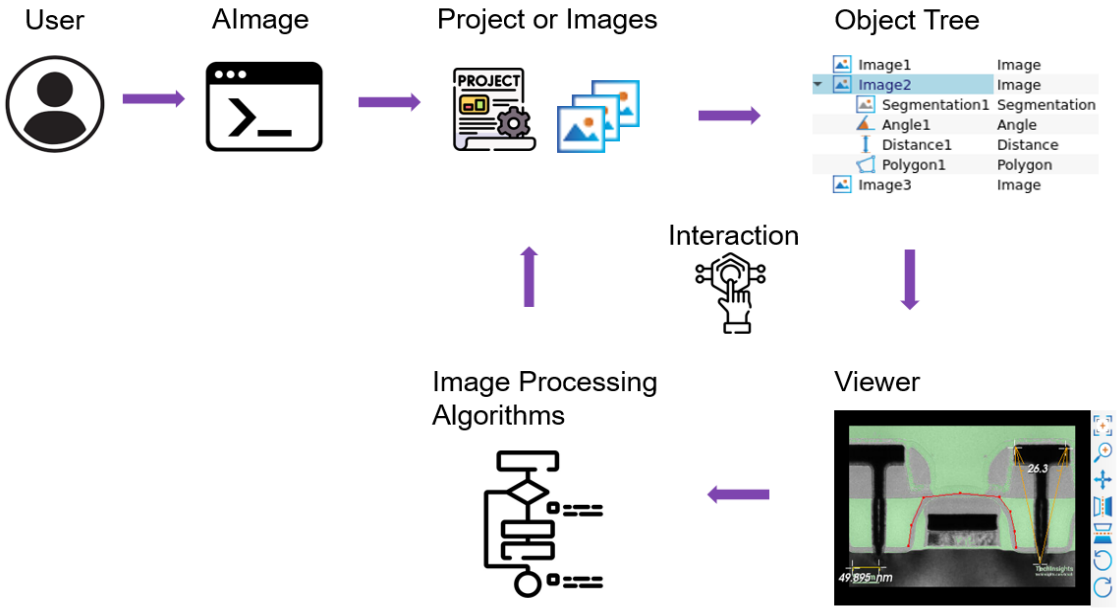
Image Processing With AImage

User Interaction Workflow

Figure 47 shows the user interaction workflow of AImage. The user starts importing images or opening a project. The objects are selected in the Objects panel (tree) and the Viewer area displays the current image with their measurements. Image processing algorithms can be applied to images and the output results are new objects in the tree. This interaction continues iteratively to produce the desired image analysis.

The user interaction is recorded as Python commands in the Script tab of the Developer panel. The image processing method applied during the interaction can then be reproduced as a script.

Figure 47 User interaction workflow for image processing with AImage



Working With Images

Images are the core objects in AImage. AImage supports the common file formats png, jpg, bmp, tiff, among others.

To import an image:

Method	Procedure
Command line	`aimage -i "<file_path>"`
Python console	`from aimage.core import Image` `Image1=Image.from_file("<file_path>")`
GUI	Using the **Import Image File** button, select the image in the file explorer, and click **Open**.

To export an image:

Method	Procedure
Python console	`# Having an image object called Image1` `Image1.to_file("<file_path>")`

Method	Procedure
GUI	Select the image to export in the Objects panel (tree), click the **Export Image File** button, select the folder path and set the file name (with the extension, for example, image.png) in file explorer, and click **Save**.

Working With Projects

A Project is a collection of working objects in AImage. You can save and open projects to restore a working progress with objects. AImage project files are json files with specific metadata defined by the tool.

To open a project:

Method	Procedure
Command line	`aimage -p "<file_path>"`
Python console	`from aimage.core import Project` `Project.open(file="<project_file>.json")`
GUI	Click the **Open Project** button, select the project (json file) in File Explorer, and click **Open**.

To save a project:

Method	Procedure
Python console	`from aimage.core import Project` `Project.save(file="<project_file>.json")`
GUI	Click the **Save Project** button, create an empty folder to store the project in File Explorer, select the created folder and set the project name (with the json extension, for example, `<project_file>.json`), click **Save**.

Working With Measurements

In AImage measurements refer to distance, angles, polygon, or bounding box objects.

To create a measurement:

Method	Procedure
GUI	Select the image in the Objects panel (tree) where the measurement is going to be created, click the **<Object> Measurement** (<Object> refers to Distance, Angle, Polygon, or Bounding Box), left-click once over the viewer to create the object, and place the object as explained in Viewer Area on page 190 (interactions).

To import a measurement:

Method	Procedure

Python console	`# Having an image object called Image1`
	`from aimage.core import Measurement`
	`Measurement.from_file(file = "<file_path>", parent = Image1)`

GUI	Select the image in Objects panel (tree) where the measurement is going to be imported, right-click to open the context menu, left- click on **Import Measurement**, select measurement (txt file) in File Explorer, and click **Open**.

To export a measurement:

Method	Procedure
Python console	`# Having a measurement object called Distance1` `Distance1.to_file("<file_path>")`
GUI	Select the measurement object to export in the Objects panel (tree), right-click to open the Context menu, left-click on **Export Measurement**, select the folder path and set the file name (with the extension, for example, distance.txt), and click **Save**.

Working With Segmentations

Segmentations are images that contain pixel values of 1 where the object of interest is located and 0 everywhere else (semantic segmentation). A segmentation always has an image as a parent object. The segmentation has the same pixel size and coordinate system as the parent image.

To create a segmentation using algorithms or manual editing, see Image Segmentation on page 181.

To import a segmentation:

Method	Procedure

Python console	`# Having an image object called Image1`
	`from aimage.core import Segmentation`
	`Segmentation.from_file(file = "<file_path>", parent = Image1)`

GUI	Select the image in the Objects panel (tree) where the segmentation is going to be imported, right-click to open the Context menu, left-click on **Import Segmentation**, select segmentation (image file) in File Explorer, and click **Open**.

To export an image:

Method	Procedure
Python console	`# Having a segmentation object called Segmentation1` `Segmentation1.to_file("<file_path>")`
GUI	Select the segmentation object to export in the Objects panel (tree), right-click to open the Context menu, left-click on **Export Segmentation**, select the folder path and set the file name (with the extension, for example, mask.png), and click **Save**.

Image Metrology

Image metrology is the process of measuring length (in metric units) of the existing shapes, regions, or objects in an image. Image calibration changes the image coordinates to world coordinates in metric units using Equation 1.

In AImage, the image calibration is the recommended first step before making measurements. However, AImage allows you to perform calibration at any point during processing an image while keeping consistency in the measurement's locations. This means that the measurement object such as distances or polygons can be placed first and then the image can be calibrated.

To manually calibrate an image:

Method	Procedure

Python console	`# Having an image object called Image1`
	`from aimage.core import Image`
	`Image1 = Image.from_file("<file_path>")`
	`Image1.calibrate(unit = "nm", spacing = (2.0, 1.0), origin = (100.0, 50.0))`

GUI	Select the image object to calibrate in the Objects panel (tree), click the Image menu, select **Calibration**, click **Manual**, set the parameters: units, width, height, origin x, and origin y, and click **Apply**.

Microscopic images (TEM/SEM) typically contain a ruler that specify the correspondence between pixel and length. AImage allows the user interactive calibration to place a distance object to calculate the proper calibration values. The procedure is shown in Figure 48 and involves the following:

1. Select the image object to calibrate in the Objects panel (tree).
2. Click the **Image** menu.
3. Click **Interactive**.
4. Select **Calibration**.
5. Set the parameters: units, measure length, length [metric], measure origin.
6. Click **Apply**.

The **Measure length** button changes the viewer interaction similar to when you place a distance object. Click the image to set the first point (usually the left side of the ruler) and the click again to set the final point of the measurement. Edit to fine-tune the measurement points. After the ruler is placed, click the **Measure length** button again to finish the procedure. This updates the **Length [pixels]** according to the measure. See Figure 49.

The origin can be measured interactively as the length. Use the **Measure origin** button to start the interaction. Left-click on the desired location for the image center. The interaction in the viewer creates a green cross object in the image having its center at the desired location. The center can be moved by dragging and dropping the horizontal or the vertical line of the cross. To finish, click the **Measure origin** button again and this updates the parameters “Origin x” and “Origin y”.

After setting the length and the origin (or simply one of them), use the **Apply** button to finish the interactive calibration. Figure 50 shows the measurement objects created after image calibration. These objects reflect the correct measurements in metric units according to the ruler.

Figure 48 Interactive Image Calibration form in Image Processing Panel

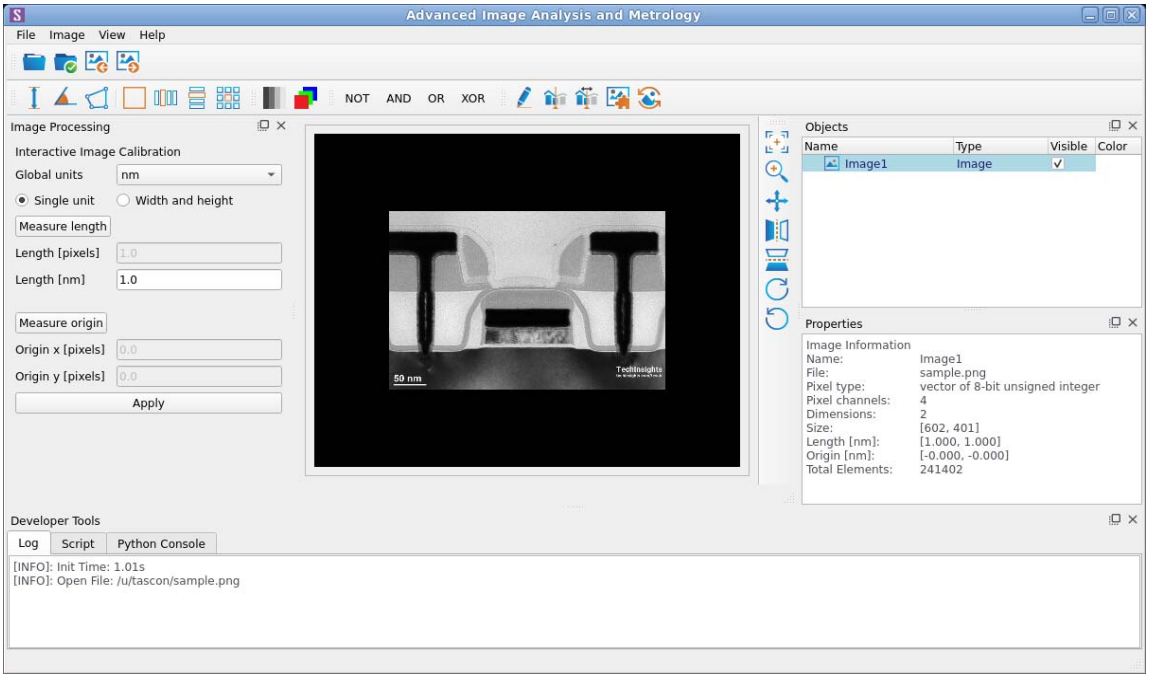


Figure 49 Measuring the length in the interactive image calibration

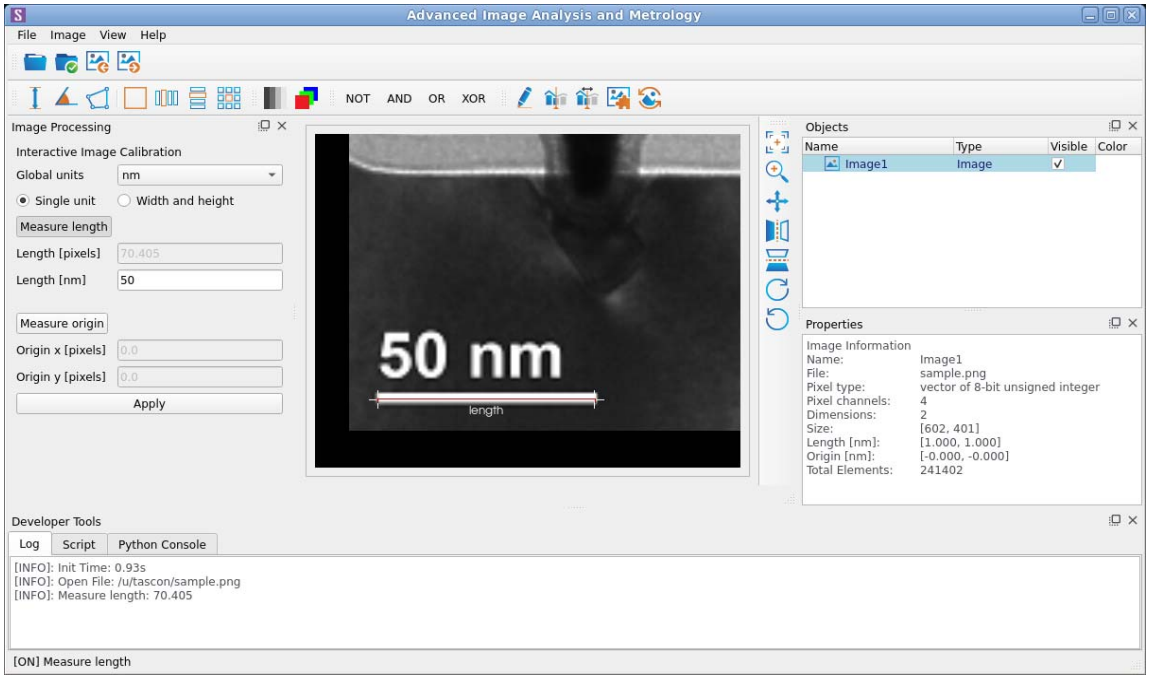
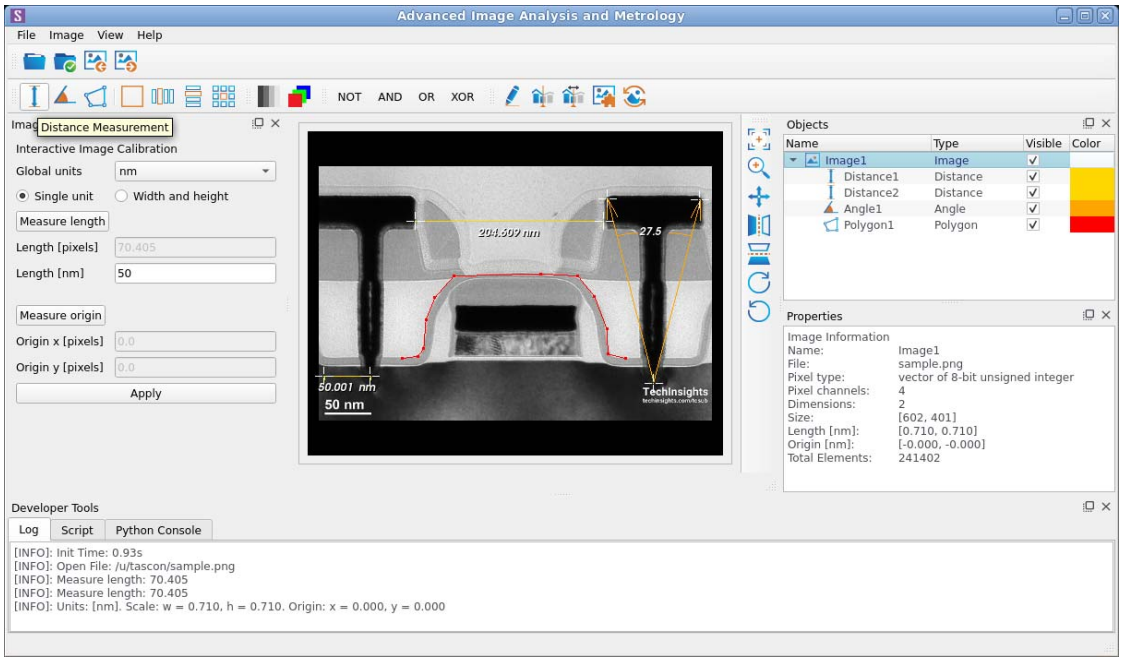


Figure 50 Create measurement objects after image calibration



Applying Image Algorithms

Most of the image algorithms available in AImage work in a similar way. The procedure involves the selection of the algorithm in the Image menu, which displays an algorithm form in the Image Processing Panel. After setting the parameter value, you run the algorithm using the **Apply** button. As an example, the following description summarize how to use a Gaussian filter with an image:

Method	Procedure

Python console	`# Having an image object called Image1`
	`from aimage.vision import filters`
	`Image2 = filters.gaussian(image = Image1, variance = 0, radius = 3)`

GUI	Having images in the Objects panel (tree), click the Image menu, select **Filter**, click **Gaussian**, set the parameters: input image, variance, radius, output image, and click **Apply**.

The input image is a combo box listing each of the images in the Objects panel (tree) and has the option to select as input “All” the images. The option “All” means that if the algorithm is applied, it performs the same operation on every image in the Objects panel. After selecting the option "All", click **Apply** to effectively run the algorithm.

The output image parameter is a combo box with two options: “New”, “Inplace”. The option “New” means that the algorithm generates a new image after being applied while the option “Inplace” means that the input image is going to be replaced after the algorithm runs.

After applying any image algorithm in the GUI, the script record shows the corresponding Python code to perform the same operation. Therefore, reproducing image algorithm workflows is straightforward by saving the script records.

Image Segmentation

Image segmentation generates a mask image. In AImage, the segmentations are child objects to images. There are multiple methods in AImage to achieve manual or automatic image segmentation.

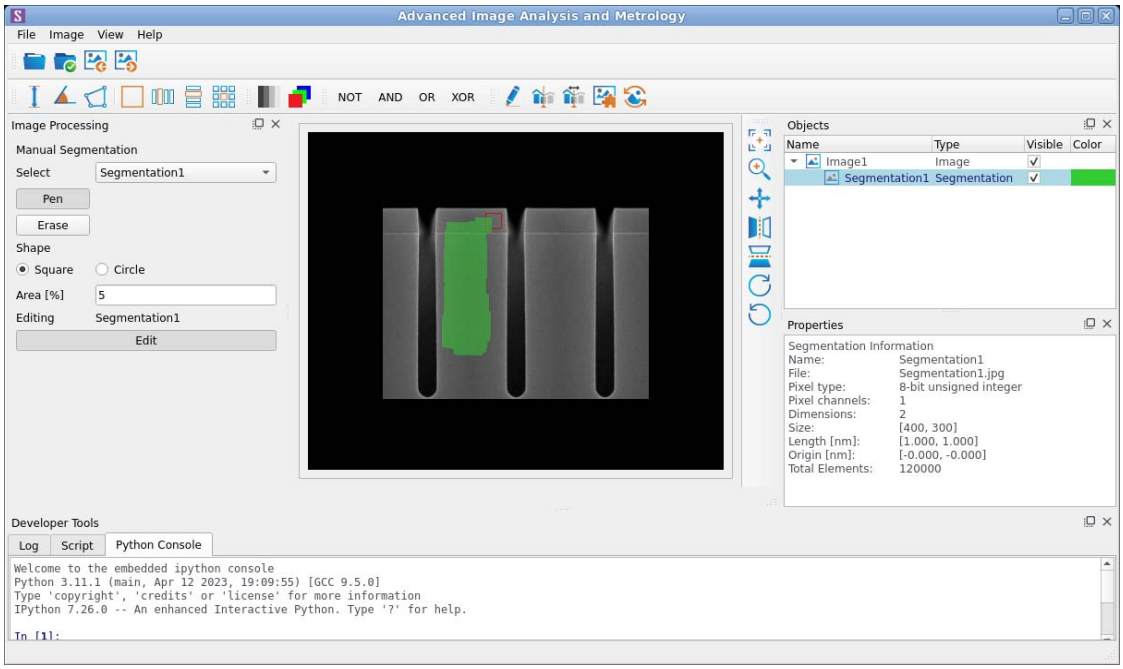
Manual Segmentation

You can perform manual segmentation in AImage using the **Manual Segmentation** button in the Segmentation toolbar. This action opens the Manual Segmentation form in the Image Processing Panel as shown in Figure 51. The manual segmentation operates on the currently selected image in the Objects panel (tree). The parameter options for the manual segmentation are:

- **Select (segmentation)**: Combo box to select the segmentation to edit. It has the option “New segmentation” or the names of existing segmentations in the working image (Image1 in Figure 51).
- **Pen**: Tool to add pixels to the segmentation.
- **Erase**: Tool to remove pixels to the segmentation.
- **Shape (Square or Circle)**: Select the shape of the editing interaction. This changes the mouse drawing shape during the interaction in the viewer.
- **Area [%]**: Select the area proportional to the viewer field of view that covers the pen or erase tool.
- **Editing (display only)**: Show the current segmentation under editing after pressing the edit button.
- **Edit**: Start editing the segmentation.

The manual segmentation process starts after pressing the **Edit** button. The interaction works like other pixel painting software. If the pen tool is activated, the left mouse button adds pixels of the segmentation inside the selected area. If the erase tool is activated, the left mouse button removes pixels of the segmentation inside the selected area. Zooming and the image interactions are enabled during editing (see Interaction With Images and Segmentations on page 191).

Figure 51 Manual Segmentation parameters form in Image Processing Panel



Thresholding Segmentation

Thresholding segmentation uses the image histogram (profile of intensity values) to separate the background from the foreground. The threshold value is the intensity point that splits the image regions and the resulting segment. The algorithm has other parameters related to inside and outside values. The inside value defines the value that the segmentation takes below the threshold and the outside value the above the threshold. AImage also supports an automatic thresholding algorithm called Otsu.

To apply thresholding segmentation:

Method	Procedure

Python console	`# Having an image object called Image1`
	`from aimage.vision import threshold`
	`Segmentation1 = threshold.binary(image = Image1, threshold_value = 128,`
	`inside_value = 0, outside_value = 1)`

GUI	Select the image object to calibrate in the Objects panel (tree), click the **Threshold Segmentation** button in the Segmentation toolbar, set the parameters: inside value, outside value, threshold value, and click **Apply**.

Segment Anything

The Segment Anything Model (SAM) is a segmentation model released by Facebook that is publicly available [1]. SAM is a promptable segmentation system with zero-shot generalization to unfamiliar objects and images, without the need for additional training [2]. SAM refers to the neural network model and the corresponding dataset (SA-1B) of 1B masks and 11M images. SAM performs Instance Segmentation (Bounding box, Classification, Semantic Segmentation). AImage implements SAM with a single button. The input is an image, and the outputs are the semantic segmentations.

To obtain segmentations with SAM:

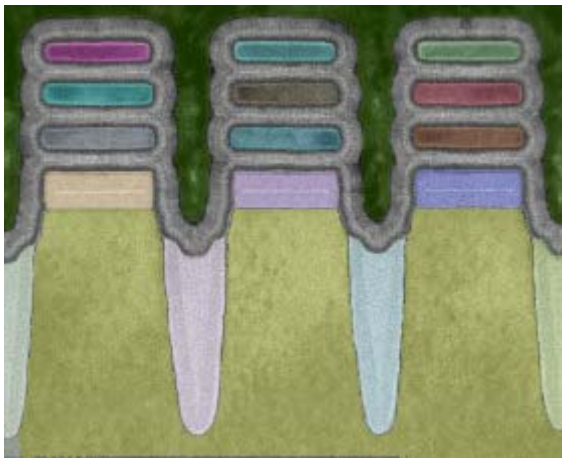
Method	Procedure

Python console	`# Having an image object called Image1`
	`from aimage.vision import segment`
	`list_segmentations = segment.anything(image = Image1)`

GUI	Select the image object to calibrate in the Objectx panel (tree), click the **Segment Anything** button in the Segmentation toolbar. The algorithm starts running and produces the segmentations.

Figure 52 shows the result of applying the segment anything algorithm to the same image shown in **Figure 8**. The image is a Gate All Around (GAA) transistor. The algorithm create automatically segments for all the regions in the image.

Figure 52 Segment Anything algorithm applied to a GAA image. Source: https://mma.prnewswire.com/media/1504371/Row_of_2_nm_nanosheet_devices.jpg



Sometimes, the resulting segments created by segment anything are not representative of the user needs. For this case, you can obtain the desired segmentation by combining or

removing segments using binary operations (see Binary Toolbar on page 188) or manually editing segmentation.

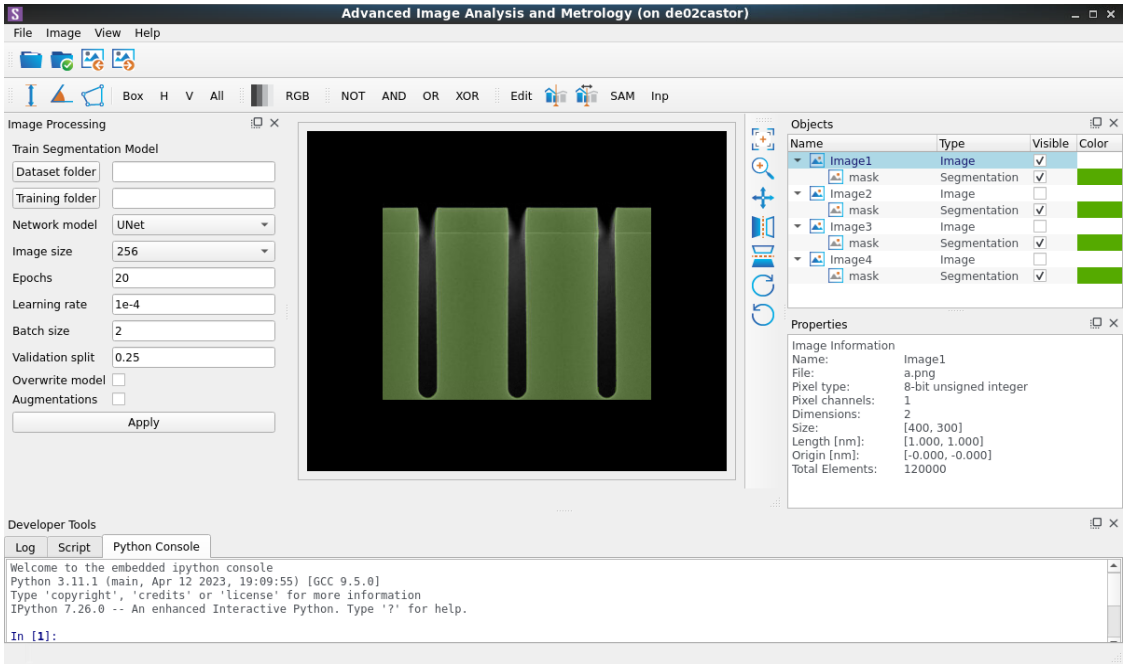
Training Segmentation Models

AImage supports training machine learning models for image segmentation. The process requires a training dataset that needs to be created by the user. The objects panel in Figure 53 shows the dataset structure: each image in the data set must have at least one segmentation with the same name. This segmentation is interpreted as the ground truth data for training.

To train a segmentation model, select the Segmentation option in the Image menu, and then select Train Model. The Image Processing Panel opens a form called Train Segmentation Model (see Figure 53). The parameters for training a segmentation model are:

- **Dataset folder**: Folder to save the input dataset (images and segmentations).
- **Training folder**: Folder to place the training metadata.
- **Network model**: Select the network model UNET, VNET, or UNET-TR.
- **Image size**: Image patch size used for training.
- **Epochs**: Number of iterations during the optimization.
- **Learning rate**: Parameter value to be adjusted during training to achieve convergence.
- **Batch size**: Set the batch size during training.
- **Validation split**: Sets the proportion of images used for validation. Value from 0 to 1 (equivalent to 0% or 100%).
- **Overwrite model**: Select to overwrite the output model file if the selected training folder already has a model.
- **Augmentations**: Enable augmentations during training. This randomly translates and rotates the training images during every epoch.

Figure 53 Train segmentation models with AImage

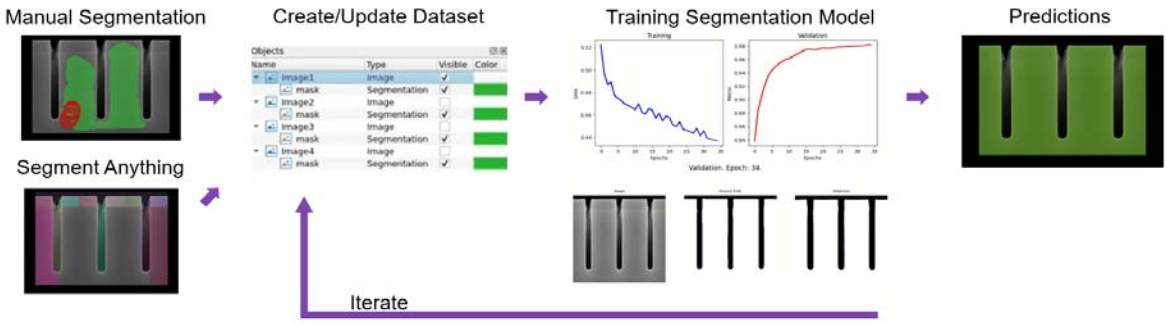


The button **Apply** button starts the training process and two pop-up windows are displayed. The first window shows the optimization curves and the second window shows the prediction images during validation. The input parameters should be tuned to achieve convergence and a good model. The recommended minimum number of images for training a segmentation model is 10.

When the training process finishes, the log prints the output model file path. This model file can be used to predict segmentations for new images. Select the Segmentation option in the Image menu and then select the Predict Model option. This opens a new form in the processing panel. The form only has one parameter “Model file” where the model path is then introduced. The **Apply** button generates a segmentation for the image selected in the object panel (tree).

You can create the training dataset by using a combination of manual segmentation, thresholding, or segment anything. In addition, new images can also be incorporated and use the trained model to generate an initial estimation of their ground truth segmentation. This methodology is represented in Figure 54. The approach speeds up and reduces the burden of the training data collection.

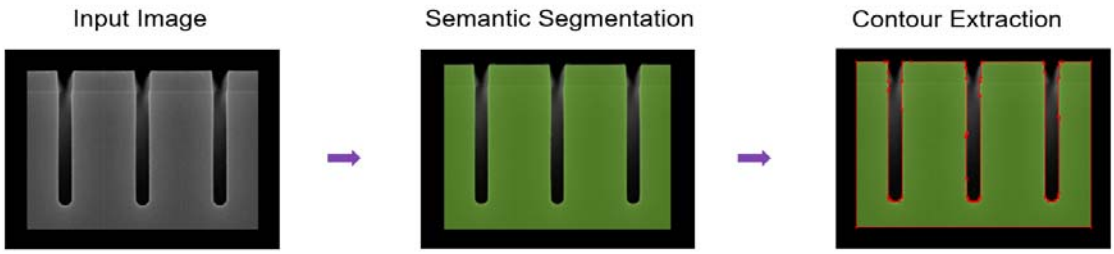
Figure 54 Proposed ML pipeline to train segmentation models with AImage



Contour Extraction

AImage handles contour extraction as a two-step process. First, a semantic segmentation is obtained from an image and then the contour extraction algorithm generates a polygon measurement (see Figure 55). Using the segmentation as the primary source of the contour simplifies the computation due to the inherent binary representation of segmentation images. Furthermore, this procedure means that the most difficult part of the contour extraction process is to achieve a good semantic segmentation of the object.

Figure 55 How to apply contour extraction in AImage



To extract contours from a segmentation object:

Method	Procedure

Python console	`# Having a segmentation object called Segmentation1`
	`from aimage.vision import contours`
	`list_polygons = contours.from_segmentation(Segmentation1)`

Method	Procedure
GUI	Having an image and its segmentation in the Objects panel (tree), click the Image menu, select **Contour**, click **Extraction from Segmentation**, select the input segmentation, and click **Apply**.

Interaction With Sentaurus Calibration Workbench

There are two methods to achieve interaction between AImage and Sentaurus Calibration Workbench. The straightforward way is to share objects between AImage and Sentaurus Calibration Workbench using files. Contours extracted from AImage can be exported as plx files and then imported as targets in Sentaurus Calibration Workbench.

The other alternative is direct sharing and works only when AImage is opened from Sentaurus Calibration Workbench GUI. For an existing measurement in AImage, the context menu (right-click the object) in the Objects panel (tree) has the “Link to SCW variable”. The submenu displays the existing target variables names in Sentaurus Calibration Workbench and select one. This variable linking enables that the AImage measurement shares the coordinate data with Sentaurus Calibration Workbench.

Scripting

You can run python scripts with AImage using the option `-s (--script)`. The algorithm runs first and then the GUI opens. To finish the script after running, use the `--offscreen` option to avoid opening the GUI. The python scripts mainly import aimage components, but it is possible to use any Python library supported in the TCAD Sentaurus tool.

Scripting Example

A simple script to read an image is shown in the following script. AImage can run the script from the command line:

```
$ aimage -s read_image.py
```

Script 1*: read_image.py script. Reads an image and then applies image calibration.*

```
from aimage.core import Image

Image1 = Image.from_file("/user/name/images/sample.jpg")
         # import image from file
Image1.calibrate(unit = "nm", spacing = (0.7, 0.7), origin = (210, 122))
         # calibrate image
```

Script With Arguments

AImage supports having additional inputs through the command line for automation or batch processing. This means that the input files are not hardcoded in the script. For this, the option "`script-args`" is used after the script file name. The input parameters are added to the script code using the `argparse` library. See the example to achieve contour extraction in the script that follows.

AImage can run the script from the command line:

```
$ aimage –s contour_extraction.py script-args –i <input_image> -o
            <output_file_prefix>
```

Script 2*: Script contour_extraction.py. Reads an input image as command line argument, then extracts contours as polygons and saves them as text files.*

```
#!/usr/bin/env python
from argparse import ArgumentParser

from aimage.core import Image, Polygon
from aimage.vision import intensity
from aimage.vision import threshold
from aimage.vision import contours


def parse_arguments():
    """Parse input arguments"""
    parser = ArgumentParser()
    parser.add_argument(
        "-i", "--input", type=str, required=True,
        help="Input image file"
    )
   parser.add_argument(
        "-o", "--output", type=str, default="polygon",
        help="Output file prefix"
   )
   args = parser.parse_args()
   return args


def main():
    args = parse_arguments()

    img1 = Image.from_file(args.input)  # create image from file.
    img2 = intensity.rgb2gray(img1)
           # if image is rgb convert to grayscale
    img3 = threshold.otsu(img2)  # otsu segmentation algorithm
    plgs = contours.from_segmentation(img3)
           # polygons is a list of Polygon

    for k, plgn in enumerate(plgs):
        file_output = args.output + str(k)
        print("Saving file:", file_output)
```

```
            plgn.write_plx(file_output + ".plx")  # export polygon as plx
            plgn.write_text(file_output + ".txt")  # export in text format
        return

    if __name__ == "__main__":
        main()
```

References

[1] https://github.com/facebookresearch/segment-anything

[2] Alexander Kirillov *et al*. "Segment anything." arXiv preprint arXiv:2304.02643 (2023).

A

Commands

This appendix describes the commands of the Sentaurus Calibration Workbench calibration modules.

Syntax Conventions

The following conventions are used in the syntax of Python commands:

- `<a>`: Array of (x,y) type, for example, ((0.0, 1e19), (0.1, 1.3e19), (0.2, 1e18))
- `<b>`: Boolean, either `True` or `False`
- `<c>`: Callable
- `<d>`: Dictionary
- `<f>`: Floating-point number
- `<i>`: Integer
- `<k>`: Keyword or type
- `<m>`: Machine-learning model
- `<o>`: Object
- `<p>`: pandas data frame
- `<s>`: String value
- `<t>`: Tuple or list
- `()` denotes a grouping only
- `[]` denotes optional arguments
- `|` denotes exclusive options

Note:

Standard Python syntax conventions are followed. Command arguments can be written using either `<argument>=<value>` or `<value>`. For example:

```
cal.evaluate(parameters=[p1, p2, p3], targets=[t1, t2, t3],
   parent="../parent/example")

cal.evaluate([p1, p2, p3], [t1, t2, t3], "../parent/example")
```

If only values are used, then the arguments must be written in the order presented in the respective command syntax.

Only the `set_option()` command must be written using the format `<argument>=<value>`. This command does not accept value-only input.

calculate_r2

This command computes the *r*-squared of targets, which is the square of the Pearson correlation coefficient:

If `show_sign=True`, then $r^2 = \frac{C_{1,2}|C_{1,2}|}{C_{1,1}C_{2,2}}$.

If `show_sign=False`, then $r^2 = \frac{C_{1,2}C_{1,2}}{C_{1,1}C_{2,2}}$.

In these expressions, *C* is the covariance matrix Cov[X,Y].

Syntax

```
calculate_r2(targets=<t>, [group_by_name=<b>,] [show_sign=<b>])
```

Argument	Type	Description
`targets`	`<t>`	Specifies a list of targets.
`group_by_name`	`<b>`	Specifies how targets are grouped. If set to `True`, then the given targets are grouped by their names, and r^2 is calculated separately. Default: `False`
`show_sign`	`<b>`	Specifies whether to show negative values if the values have an inverse correlation. See the previous expressions. Default: `False`

Returns

None

Examples

```
cal.calculate_r2(targets=[t1, t2, t3], group_by_name=True)
```

calibrate_ac

This command sets the AC characteristics calibration strategy, specific to the `igbtcal` calibration module (see IGBT AC Characteristics Calibration on page 113).

Syntax

```
calibrate_ac(parameters=<t>, targets=<t>,
             [parent=<s>,] [child_dir=<s>,] [evaluate_only=<b>,]
             [tolerance_cv=<f>,] [tolerance_qg=<f>,]
             [tolerance_switching=<f>,] [max_iteration=<i>,]
             [run_initial_doe=<b>,] [doe_method=<k>,]
             [doe_size=<i>,] [plot_history=<b>,]
             [print_targets=<b>,] [save_parameters=<b>,]
             [outfile_prefix =<s>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`evaluate_only`	`<b>`	Specifies whether to run the evaluation only with the initial parameters without calibration. Default: `False`
`tolerance_cv`	`<f>`	Sets the RMS error tolerance for a `CV` calibration. Default: `0.0`
`tolerance_qg`	`<f>`	Sets the RMS error tolerance for a `Qg` calibration. Default: `0.0`
`tolerance_switching`	`<f>`	Sets the RMS error tolerance for a `Switching` calibration. Default: `0.0`
`max_iteration`	`<i>`	Sets the maximum number of final calibration loops. Default: `None` (uses the `max_iteration` set by `set_option(max_iteration=<i>)` or its default, 50)
`run_initial_doe`	`<b>`	Specifies whether to run an initial design-of-experiments (DoE) search before calibration. Default: `False`

Argument	Type	Description
`doe_method`	`<k>`	Sets the DoE method for the initial search before calibration. Options are: • `"latin-hypercube"` • `"uniform"` • `"sobol"` (default)
`doe_size`	`<i>`	Sets the number of samples for the DoE search before calibration. Default: `None` (uses the recommended `doe_size` by the `igbtcal` module based on the number of parameters)
`plot_history`	`<b>`	Specifies whether to plot the RMS error history. The file name is `{outfile_prefix}{type}_rmse.png`. Default: `True`
`print_targets`	`<b>`	Specifies whether to print the target value (TCAD data file name or scalar value) in Sentaurus Workbench. Variable names are: • `{target.id}_initial` for the TCAD file path or value with initial parameters • `{target.id}_calibrated` for the TCAD file path or value with calibrated parameters Default: `True`
`save_parameters`	`<b>`	Specifies whether to save `sdevice.par` with the calibrated parameters. The file name is `{outfile_prefix}scw_sdevice.par`. Default: `True`
`outfile_prefix`	`<s>`	Sets the prefix of the output file names for `plot_history` and `save_parameters`. Default: `n@node@_`

Returns

None

Examples

```
igbtcal.calibrate_ac(parameters=parameters, targets=targets)

igbtcal.calibrate_ac(parameters=parameters, targets=targets,
                     evaluate_only=True)

igbtcal.calibrate_ac(parameters=parameters, targets=targets,
                     run_initial_doe=True, max_iteration=20)
```

calibrate_dc

This command sets the DC characteristics calibration strategy, specific to the `igbtcal` calibration module (see IGBT DC Characteristics Calibration on page 113).

Syntax

```
calibrate_dc(parameters=<t>, targets=<t>,
             [parent=<s>,] [child_dir=<s>,] [evaluate_only=<b>,]
             [tolerance_icvg=<f>,] [tolerance_icvc=<f>,]
             [tolerance_bv=<f>,] [max_iteration=<i>,]
             [run_initial_doe=<b>,] [doe_method=<k>,]
             [doe_size=<i>,] [plot_history=<b>,]
             [print_targets=<b>,] [save_parameters=<b>,]
             [outfile_prefix =<s>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`evaluate_only`	`<b>`	Specifies whether to run the evaluation only with the initial parameters without calibration. Default: `False`
`tolerance_icvg`	`<f>`	Sets the RMS error tolerance for an `IcVg` calibration. Default: `0.0`
`tolerance_icvc`	`<f>`	Sets the RMS error tolerance for an `IcVc` calibration. Default: `0.0`
`tolerance_bv`	`<f>`	Sets the RMS error tolerance for a `BV` calibration. Default: `0.1`
`max_iteration`	`<i>`	Sets the maximum number of final calibration loops. Default: `None` (uses the `max_iteration` set by `set_option(max_iteration=<i>)` or its default, 50)
`run_initial_doe`	`<b>`	Specifies whether to run an initial design-of-experiments (DoE) search before calibration. For BV, the initial DoE is always `True` regardless of the value of `run_initial_doe`. Default: `False`

Argument	Type	Description
`doe_method`	`<k>`	Sets the DoE method for the initial search before calibration. Options are: • `"latin-hypercube"` • `"uniform"` • `"sobol"` (default)
`doe_size`	`<i>`	Sets the number of samples for the DoE search before calibration. Default: `None` (uses the recommended `doe_size` by the `igbtcal` module based on the number of parameters)
`plot_history`	`<b>`	Specifies whether to plot the RMS error history. The file name is `{outfile_prefix}{type}_rmse.png`. Default: `True`
`print_targets`	`<b>`	Specifies whether to print the target value (TCAD data file name or scalar value) in Sentaurus Workbench. Variable names are: • `{target.id}_initial` for the TCAD file path or value with initial parameters • `{target.id}_calibrated` for the TCAD file path or value with calibrated parameters Default: `True`
`save_parameters`	`<b>`	Specifies whether to save `sdevice.par` with the calibrated parameters. The file name is `{outfile_prefix}scw_sdevice.par`. Default: `True`
`outfile_prefix`	`<s>`	Sets the prefix of the output file names for `plot_history` and `save_parameters`. Default: `n@node@_`

Returns

None

Examples

```
igbtcal.calibrate_dc(parameters=parameters, targets=targets)

igbtcal.calibrate_dc(parameters=parameters, targets=targets,
                     evaluate_only=True)

igbtcal.calibrate_dc(parameters=parameters, targets=targets,
                     run_initial_doe=True, max_iteration=20)
```

calibrate_har_etch

This command sets the calibration strategy, specific to the `pmccal` calibration module, based on a gradient-based local calibration algorithm for Sentaurus Topography high aspect ratio etching (see HARC Etching Calibration on page 117).

Syntax

```
calibrate_har_etch(parameters=<t>, targets=<t>,
                   [parent=<s>,] [child_dir=<s>,]
                   [tolerance_end=<f>,] [tolerance_mid=<f>,]
                   [tolerance_init=<f>,] [max_iteration_end=<i>,]
                   [max_iteration_mid=<i>,] [max_iteration_init=<i>,]
                   [weight_factor=<f>,] [reevaluate=<b>,]
                   [use_bi_sensitivity=<b>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`tolerance_end`	`<f>`	Sets the average relative error tolerance for the final calibration. Default: `0.1`
`tolerance_mid`	`<f>`	Sets the average relative error tolerance for the middle calibration. Default: `0.14`
`tolerance_init`	`<f>`	Sets the average relative error tolerance for the initial calibration. Default: `0.05`
`max_iteration_end`	`<i>`	Sets the maximum number of final calibration loops. Default: `4`
`max_iteration_mid`	`<i>`	Sets the maximum number of middle calibration loops. Default: `2`
`max_iteration_init`	`<i>`	Sets the maximum number of initial calibration loops. Default: `5`
`weight_factor`	`<f>`	Specifies the factor by which the predefined weights of the `BottomZ` and `RemainingHM` targets are multiplied. Default: `2.0`

Argument	Type	Description
`reevaluate`	`<b>`	Specifies whether to reevaluate by rerunning the simulation due to variability.
`use_bi_sensitivity`	`<b>`	Specifies whether to switch on the bidirectional sensitivity calculation for the initial calibration. Default: `True`

Returns

None

Examples

```
pmccal.calibrate_har_etch(parameters=parameters, targets=targets,
                          tolerance_end=0.15)
```

calibrate_har_etch_ml

This command sets the calibration strategy, specific to the `pmccal` calibration module, based on machine learning for Sentaurus Topography high aspect ratio etching (see HARC Etching Calibration on page 117).

Syntax

```
calibrate_har_etch_ml(parameters=<t>, targets=<t>,
                      [parent=<s>,] [child_dir=<s>,]
                      [task_name=<s>,] [tolerance=<f>,]
                      [num_iterations=<i>,]
                      [num_validations=<i>,]
                      [num_parameters=<i>,]
                      [num_explore_parameters=<i>,]
                      [num_refine_parameters=<i>,]
                      [explore_doe_range=<f|s>,]
                      [explore_doe_method=<k>,]
                      [explore_doe_size=<i>,]
                      [tolerance_refine=<f>,]
                      [strategy=<s>,]
                      [validate_range=<b>,]
                      [sensitivity=<b|s|d>,]
                      [constraint=<s>,]
                      [weight_factor=<f>,]
                      [priority_targets=<t>,]
                      [num_retries=<i>,]
                      [num_evaluations=<i>)]
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the machine-learning task. Default: `"PMC_HAR"`
`tolerance`	`<f>`	Sets the total relative RMS error tolerance. Default: `0.12`
`num_iterations`	`<i>`	Sets the maximum number of iterations. Default: `10` (based on the `pmccal` project)

Argument	Type	Description
num_validations	<i>	Sets the number of validations to adjust the range. Default: 13
num_parameters	<i>	Sets the number of primary parameters used for machine learning. If num_parameters=0, then all defined parameters are assumed to be primary parameters. Default: 0
num_explore_parameters	<i>	Sets the number of primary parameters used for exploration. If num_explore_parameters=0, then all defined parameters are assumed to be primary parameters. Default: None
num_refine_parameters	<i>	Sets the number of primary parameters used for refinement. If num_refine_parameters=0, then all defined parameters are assumed to be primary parameters. Default: None
explore_doe_range	<f|s>	Sets the parameter range scaling used for the DoE. If a file name is provided, then it sets the name of a CSV file that specifies the parameter ranges for the DoE. Default: None
explore_doe_method	<k>	Sets the DoE method. Options are "box-behnken", "full-factorial", "gaussian", "latin-hypercube", "uniform", and "sobol".
explore_doe_size	<i>	Sets the number of samples for the DoE in the case of the DoE methods "gaussian", "latin-hypercube", "uniform", and "sobol". Default: None
tolerance_refine	<f>	Sets the tolerance value for the machine-learning refinement algorithm. Default: tolerance + 0.03
strategy	<s>	Specifies the strategy. For good initial parameter values and ranges, use "standard". If the initial parameters are far from optimal or the nominal simulation failed, then use "validate". Default: "validate"
validate_range	<b>	Specifies whether to perform a parameter range validation analysis. Default: True

Argument	Type	Description
`sensitivity`	`<b|s|d>`	Specifies the type of sensitivity analysis to be performed: • If `True`, then a sensitivity analysis is performed. • If a task name is provided, then the argument specifies the existing task to be used for sensitivity. • If a pandas data frame is provided, then the argument specifies the sensitivity information to be used. Default: `None`
`constraint`	`<s>`	Specifies an optional constraint or query for an extraction variable of simulations to be included in the training dataset. Default: `""`
`weight_factor`	`<f>`	Sets the weight factor for the `BottomZ` and `RemainingHMZ` targets. Default: `2.0` for `float` type
`priority_targets`	`<t>`	Specifies a list of priority targets in order of priority, starting with the highest priority. Default: `None`
`num_retries`	`<i>`	Sets the number of simulation run retries if there is a simulation runtime failure. Default: `1`
`num_evaluations`	`<i>`	Sets the number of initial and final evaluations. Default: `5`

Returns

None

Examples

```
pmccal.calibrate_har_etch_ml(parameters=parameters, targets=targets,
                             tolerance=0.1, explore_doe_range=0.5,
                             num_parameters=10)
```

calibrate_hcsd_nmos

This command sets the calibration command for NMOS devices, specific to the `devcal` calibration module (see Hot-Carrier Stress Degradation Calibration on page 99).

Syntax

```
calibrate_hcsd_nmos(parameters=<t>, targets<t>,
                    [use_thermal=<b>,] [condition_variable=<s>,]
                    [parent=<s>,] [child_dir=<s>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of two calibration parameter objects, in the following order: 1. Sentaurus Device acceptor trap distribution central energy 2. Sentaurus Device acceptor trap distribution energy width
`targets`	`<t>`	Specifies a list of calibration target objects.
`use_thermal`	`<b>`	Specifies whether to use the field-enhanced thermal component of the HCS degradation model in Sentaurus Device. Default: `True`
`condition_variable`	`<s>`	Sets the condition variable of the targets, in particular, the transient final time. Default: `"time"`
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.

Returns

None

Examples

```
devcal.calibrate_hcsd_nmos(parameters=[EmidA, EsigA],
                       targets=[t_1_IdVg, t_2_IdVg, t_3_IdVg, t_e_IdVg],
                       condition_variable="Time")
```

calibrate_hcsd_pmos

This command sets the calibration command for PMOS devices, specific to the `devcal` calibration module (see Hot-Carrier Stress Degradation Calibration on page 99).

Syntax

```
calibrate_hcsd_pmos(parameters=<t>, targets<t>,
                    [condition_variable=<s>,]
                    [parent=<s>,] [child_dir=<s>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of four calibration parameter objects, in the following order: 1. Sentaurus Device acceptor trap distribution central energy 2. Sentaurus Device acceptor trap distribution energy width 3. Sentaurus Device donor trap distribution central energy 4. Sentaurus Device donor trap distribution energy width
`targets`	`<t>`	Specifies a list of calibration target objects.
`condition_variable`	`<s>`	Sets the condition variable of the targets, in particular, the transient final time. Default: `"time"`
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.

Returns

None

Examples

```
devcal.calibrate_hcsd_pmos(parameters=[EmidA, EsigA, EmidD, EsigD],
                    targets=[t_1_IdVg, t_2_IdVg, t_3_IdVg, t_e_IdVg],
                    condition_variable="Time")
```

calibrate_implant_dualpearson

This command sets the implantation dual-Pearson moments calibration strategy in the `procal` calibration module (see Implantation Dual-Pearson Moments Calibration on page 105).

Syntax

```
calibrate_implant_dualpearson(targets=<t>,
                              [parent=<s>,] [child_dir=<s>,]
                              [evaluate_only=<b>,]
                              [run_sensitivity_only=<b>,]
                              [num_parameters=<i>,]
                              [max_iteration_1=<i>,]
                              [max_iteration_2=<i>,]
                              [plot_history=<b>,]
                              [print_targets=<b>,]
                              [save_moments=<b>,]
                              [outfile_prefix=<s>])
```

Argument	Type	Description
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`evaluate_only`	`<b>`	Specifies whether to run the evaluation only with the initial parameters without calibration. Default: `False`
`run_sensitivity_only`	`<b>`	Specifies whether to run the sensitivity analysis only, without calibration. Default: `False`
`num_parameters`	`<i>`	Sets the number of primary parameters determined from the sensitivity analysis. The minimum number is 1. Default: `4`
`max_iteration_1`	`<i>`	Sets the maximum number of the first calibration step, with primary parameters selected from the sensitivity analysis. Default: `30`
`max_iteration_2`	`<i>`	Sets the maximum number of the second calibration step with all the parameters. Default: `100`

Argument	Type	Description
`plot_history`	`<b>`	Specifies whether to plot the RMS error history. The file names are: • `{outfile_prefix}rmse_1.png` for the first calibration step. • `{outfile_prefix}rmse_2.png` for the second calibration step. Default: `True`
`print_targets`	`<b>`	Specifies whether to print the target value (TCAD data file name or scalar value) in Sentaurus Workbench. Variable names are: • `{target.id}_initial` for the TCAD file path or value with initial parameters • `{target.id}_calibrated` for the TCAD file path or value with calibrated parameters Default: `True`
`save_moments`	`<b>`	Specifies whether to save the calibrated moments. The file name is `{outfile_prefix}implant_calib.fps`. Default: `True`
`outfile_prefix`	`<s>`	Sets the prefix of the output file names for `plot_history` and `save_moments`. Default: `n@node@_`

Returns

None

Examples

```
procal.calibrate_implant_dualpearson(targets=targets)

procal.calibrate_implant_dualpearson(targets=targets,
                                     evaluate_only=True)

procal.calibrate_implant_dualpearson(targets=targets,
                                     run_sensitivity_only=True)
```

calibrate_implant_legendre

This command sets the calibration strategy for the Legendre polynomial implantation model (L-PIM) in the `procal` calibration module (see Legendre Polynomial Implantation Model Calibration on page 109).

Syntax

```
calibrate_implant_legendre(material=<s>, species=<s>, parameters=<t>,
                           [targets=<t>,][parent=<s>,][child_dir=<s>,])
                           [table_dir=<s>,][atomic_species=<s>,]
                           [temperature=<f>,][notch_dir=<s>,]
                           [miscut_toward=<s>,][miscut_angle=<f>,]
                           [beam_div=<f>,][num_particles_MC=<i>,]
                           [num_threads_MC=<i>,][num_samples=<i|t|d>,]
                           [d_norm=<f>] [c_floor=<f>,]
                           [degree_primary=<i>,][degree_lateral=<i>,]
                           [calibration_file=<s>,][table_format=<s>])
```

Argument	Type	Description
`material`	`<s>`	Specifies the ion-implanted material. Options are `Silicon` or `SiliconCarbide`.
`species`	`<s>`	Specifies the ion implantation species.
`parameters`	`<t>`	Specifies a list of implantation parameter objects for the grid DoE (see Parameters on page 110).
`targets`	`<t>`	Specifies a list of calibration target objects. Default: `None`
`parent`	`<s>`	Specifies the parent project. If set to `None`, then a predefined internal parent project is used. Default: `None`
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used. Default: `None`
`table_dir`	`<s>`	Specifies the directory in which the output L-PIM tables are written. Default: `"./PIM"`
`atomic_species`	`<s>`	Specifies the atomic species. If set to `None`, then it is the same as species. Default: `None`
`temperature`	`<f>`	Sets the implant temperature (in degrees Celsius). Default: `25`
`notch_dir`	`<s>`	Specifies the wafer notch direction (Miller indices). Default: `110`

Argument	Type	Description
`miscut_toward`	`<s>`	Specifies the wafer miscut toward direction (Miller indices). Default: `110`
`miscut_angle`	`<f>`	Specifies the wafer miscut angle (in degree). Default: `0`
`beam_div`	`<f>`	Sets the implant beam divergence. Default: `0.5`
`num_particles_MC`	`<i>`	Sets the number of pseudo particles used for Sentaurus Process Monte Carlo implant simulation. Default: `20000`
`num_threads_MC`	`<i>`	Sets the number of threads used for Sentaurus Process Monte Carlo implant simulation. Default: `1`
`num_samples`	`<i|t|d>`	Sets the number of samples per parameter for the extraction grid DoE. Specify `<i>` for the same number for all parameters, `<t>` for the same order as the `parameters` list, or `<d>` for parameter-specific numbers. Default: `5`
`d_norm`	`<f>`	Sets the dose norm for the extraction. Default: `1e13`
`c_floor`	`<f>`	Sets the floor cut-off concentration for primary profiles. Default: `1e13`
`degree_primary`	`<i>`	Sets the Legendre polynomial degree for primary direction. Default: `16`
`degree_lateral`	`<i>`	Sets the Legendre polynomial degree for lateral direction. Default: `6`
`calibration_file`	`<s>`	Sets the Sentaurus Process parameter file for an optional user Monte Carlo calibration. Default: `none`
`table_format`	`<s>`	Table format, one of `"pim"` (for crystalline materials) or `"tab"` (for amorphous materials). Default: `"pim"`

Returns

None

Examples

```
procal.calibrate_implant_legendre(
      material=SiliconCarbide,
      species=Nitrogen,
      parameters=[E, D, T, R, S],
      num_samples=10)
```

```
procal.calibrate_implant_legendre(
      material=SiliconCarbide,
      species=Nitrogen,
      parameters=[E, D, T, R, S],
      num_samples={'E': 5, 'T': 1, 'R': 1})

procal.calibrate_implant_legendre(
      material=SiliconCarbide,
      species=Nitrogen,
      parameters=[E, D, T, R, S],
      notch_dir='1_-1_0',
      miscut_toward='-1_-1_0',
      miscut_angle=4.0)

procal.calibrate_implant_legendre(
      material=Oxide,
      species=Nitrogen,
      parameters=[E, D, T, R, S],
      num_samples=10,
      table_fromat="tab")
```

calibrate_ml

This command sets the calibration strategy based on machine learning.

Syntax

```
calibrate_ml(parameters=<t>, targets=<t>,
             [parent=<s>,] [child_dir=<s>,]
             [task_name=<s>,] [tolerance=<f>,]
             [num_iterations=<i>,]
             [num_validations=<i>,]
             [num_parameters=<i>,]
             [num_explore_parameters=<i>,]
             [num_refine_parameters=<i>,]
             [explore_doe_range=<f>,]
             [explore_doe_method=<k>,]
             [explore_doe_size=<i>,]
             [tolerance_refine=<f>,]
             [strategy=<s>,]
             [validate_range=<b>,]
             [sensitivity=<b|s|d>,]
             [constraint=<s>,]
             [priority_targets=<t>,]
             [num_retries=<i>,]
             [num_evaluations=<i>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the machine-learning task. Default: `"ML_calibrate"`
`tolerance`	`<f>`	Sets the total relative RMS error tolerance. Default: `0.12`
`num_iterations`	`<i>`	Sets the maximum number of iterations. Default: `1000`
`num_validations`	`<i>`	Sets the number of validations to adjust the range. Default: `13`

Argument	Type	Description
`num_parameters`	`<i>`	Sets the number of primary parameters used for machine learning. If `num_parameters=0`, then all defined parameters are assumed to be primary parameters. Default: `0`
`num_explore_parameters`	`<i>`	Sets the number of primary parameters used for exploration. If `num_explore_parameters=0`, then all defined parameters are assumed to be primary parameters. Default: `None`
`num_refine_parameters`	`<i>`	Sets the number of primary parameters used for refinement. If `num_refine_parameters=0`, then all defined parameters are assumed to be primary parameters. Default: `None`
`explore_doe_range`	`<f>`	Sets the parameter range scaling used for the DoE. If a file name is provided, then it sets the name of a CSV file that specifies the parameter ranges for the DoE. Default: `None`
`explore_doe_method`	`<k>`	Sets the DoE method. Options are `"box-behnken"`, `"full-factorial"`, `"gaussian"`, `"latin-hypercube"`, `"uniform"`, and `"sobol"`.
`explore_doe_size`	`<i>`	Sets the number of samples for the DoE in the case of the DoE methods `"gaussian"`, `"latin-hypercube"`, `"uniform"`, and `"sobol"`. Default: `None`
`tolerance_refine`	`<f>`	Sets the tolerance value for the machine-learning refinement algorithm. Default: tolerance + `0.03`
`strategy`	`<s>`	Specifies the strategy. For good initial parameter values and ranges, specify `"standard"`. If the initial parameters are far from optimal or the nominal simulation failed, then specify `"validate"`. Default: `"validate"`
`validate_range`	`<b>`	Specifies whether to perform a parameter range validation analysis. Default: `True`
`sensitivity`	`<b|s|d>`	Specifies the type of sensitivity analysis to be performed: • If `True`, then a sensitivity analysis is performed. • If a task name is provided, then the argument specifies the existing task to be used for sensitivity. • If a pandas data frame is provided, then the argument specifies the sensitivity information to be used. Default: `None`

Argument	Type	Description
`constraint`	`<s>`	Specifies an optional constraint or query for an extraction variable of simulations to be included in the training dataset. Default: `""`
`priority_targets`	`<t>`	Specifies a list of priority targets in order of priority, starting with the highest priority. Default: `None`
`num_retries`	`<i>`	Sets the number of simulation run retries if there is a simulation runtime failure. Default: `1`
`num_evaluations`	`<i>`	Sets the number of initial and final evaluations. Default: `1`

Returns

None

Examples

```
cal.calibrate_ml(parameters=parameters, targets=targets,
                 tolerance=0.1, num_parameters=10)
```

create_ml

This command creates one or more machine-learning models. Data for training is generated or loaded from a file. Models are not trained. This command is specific to the `mlcal` calibration module (see Machine Learning Calibration Module: mlcal on page 52).

Syntax

```
create_ml(models=<t>, parameters=<t>, targets=<t>,
          [parent=<s>,] [child_dir=<s>,]
          [task_name=<s>,] [train_data=<s|p|t>,]
          [merge_table=<d>,] [save_report=<b>,] [data_generation=<d>])
          [doe_method=<k>, [doe_size=<i|t|d>]]
```

Argument	Type	Description
`models`	`<t>`	Specifies a list of machine-learning model objects.
`parameters`	`<t>`	Specifies a list of machine-learning model parameter objects.
`targets`	`<t>`	Specifies a list of machine-learning model target objects.

Argument	Type	Description
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the machine-learning task. Default: `"ML_create"`
`train_data`	`<s|p|t>`	Specifies a file or pandas data frame containing training data. Default: `None` Specifies a file or pandas data frame or a list of them. In the case of a list of more than one training data set, all data sets are merged into one data set, with *doe_size* being determined as the total size. In this case of a multi-window DoE, each training data set can be generated by a different `doe_method` with a different `doe_size`, parameter ranges. Default: `None`
`merge_table`	`<d>`	Specifies the rules to merge multiple parameter or target names into a single parameter or target, so that one simulation can be split into different data entries. Default: `None`
`save_report`	`<b>`	Specifies whether to save a report in the `report` folder. Default: `True`
`doe_method`	`<k>`	Sets the DoE method. Options are `"adaptive"`, `"box-behnken"`, `"full-factorial"`, `"gaussian"`, `"grid"`, `"latin-hypercube"`, `"uniform"`, and `"sobol"`.
`doe_size`	`<i|t|d>`	Sets the number of samples for the DoE in the case of the DoE methods `"latin-hypercube"`, `"uniform"`, and `"sobol"`. Sets the number of samples per parameter in the case of the DoE method "grid", which is either the same for all in the case of an integer or per parameter in the case of a list or dictionary.

Argument	Type	Description
`data_generation`	`<d>`	Specifies data generation information (default: `None`). Dictionary keys are: • `doe_method=<k>`: Sets the DoE method. Options are `"adaptive"`, `"box-behnken"`, `"full-factorial"`, `"gaussian"`, `"grid"`, `"latin-hypercube"`, `"uniform"`, and `"sobol"`. • `doe_size=<i|t|d>`: Sets the number of samples for the DoE in the case of the DoE methods `"latin-hypercube"`, `"uniform"`, and `"sobol"`. Sets the number of samples per parameter in the case of the DoE method `"grid"`, which is either the same for all in the case of an integer or per parameter in the case of a list or dictionary. • `random_seed=<i>`: Sets the random seed for the DoE in the case of the DoE methods `"latin-hypercube"`, `"uniform"`, and `"sobol"`. • `use_mean=<b>`: Specifies whether the nominal value is the mean of the minimum and maximum values. If `False`, then the nominal value is the user-specified value. Default: `True` • `include_nominal=<b>`: If `True`, then the nominal value is included. Default: `False` • `batch_size=<i>`: Used only for adaptive generation. Sets the number of data points generated before each update on the parameter filter. Default: `1` • `pass_rate=<f>`: Used only for adaptive generation. Defines the required passing rate of the filter. Default: `1.0` • `threshold_num_pass=<i>`: Used only for adaptive generation. Sets the threshold number of passing cases to invoke the parameter filter, before which the filter is all-passing. Default: `1` • `initial_sampling_doe=<s>`: Used only for adaptive generation. The initial few batches can use a different DoE method other than `"uniform"`. Default: `"uniform"` • `initial_doe_batches=<i>`: Used only for adaptive generation. Specifies how many batches can use the different DoE method, before switching to uniform randomization. If it is negative, then `initial_sampling_doe` is used throughout the data generation session. Default: `-1` • `transformer=<m>`: Used only for adaptive generation. Sets the transformer model to use. Default: `None` • `classifier=<m>`: Used only for adaptive generation. Sets the classifier model to use. Default: `None` • `unadapted_size=<i>`: Sets the number of unadapted data points (batch size) generated before each update on the parameter filter, where 0 <= `unadapted_size` < `batch_size`. Default: `0`

Returns

None

Examples

```
mlcal.create_ml(models=models, parameters=parameters,
                targets=targets,
                train_data='train/data.csv')

mlcal.create_ml(
   models=models,
   parameters=parameters,
   targets=targets,
   task_name='ML_create',
   data_generation={'doe_method': 'adaptive', 'doe_size': 300,
    'batch_size': 5, 'pass_rate': 0.8, 'threshold_num_pass': 1,
    'initial_sampling_doe': 'random', 'initial_doe_batches': -1,
    'transformer': Standard, 'classifier': DecisionTree,
    'include_nominal': False, 'use_mean': True, 'random_seed': 1},
   save_report=True,
)
```

determine_primary_parameters

This command determines the primary parameters based on sensitivity analysis, following a compute sensitivity task `run_sensitivity_analysis()` (see run_sensitivity_analysis on page 275). Primary parameters are defined as parameters with high sensitivity. Secondary parameters are defined as parameters with low sensitivity.

Syntax

```
determine_primary_parameters(parameters=<t>,
                             [num_parameters=<i>,]
                             [min_sensitivity=<f>,]
                             [use_mean=<b>,] [measure=<s>,]
                             [plot_file=<s>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects for which a sensitivity analysis has been performed previously.
`num_parameters`	`<i>`	Sets the maximum number of primary parameters. The minimum number is 1. Default: `6`
`min_sensitivity`	`<f>`	Sets the sensitivity of only primary parameters, which must be greater than `min_sensitivity`. Default: `0.001`
`use_mean`	`<b>`	Specifies whether to use the mean sensitivity of individual targets instead of total sensitivity. Default: `False`
`measure`	`<s>`	Specifies the sensitivity measure as follows: • For local sensitivity analysis, specify `'absolute'` or `'relative'` (default). • For global sensitivity analysis, specify `'mu'`, `'mu_star'` (default), or `'sigma'`.
`plot_file`	`<s>`	Specifies the name of the PNG file of the sensitivity bar plot. If `None`, then no plot is created. Default: `None`

Returns

`<t>`: List of primary parameter objects only

Examples

```
cal.determine_primary_parameters(parameters=parameters,
                                 num_parameters=5,
                                 plot_file='sensitivity.png')

cal.determine_primary_parameters(parameters=parameters,
                                 use_mean=True, measure='absolute')
```

determine_primary_parameters_ml

This command determines the primary parameters based on sensitivity analysis, following a compute sensitivity task `run_sensitivity_analysis_ml()`. Primary parameters are defined as parameters with high sensitivity. Secondary parameters are defined as parameters with low sensitivity.

Syntax

```
determine_primary_parameters_ml(parameters=<t>,
                                [task_name=<s>,]
                                [child_dir=<s>,]
                                [num_parameters=<i>,]
                                [min_sensitivity=<f>,]
                                [targets=<t>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects for which a sensitivity analysis has been performed previously.
`task_name`	`<s>`	Sets the name of the task that is used in the earlier performed `run_sensitivity_analysis_ml(task_name=<s>)`. Default: `"ML_sensitivity"`
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`num_parameters`	`<i>`	Sets the maximum number of primary parameters. The minimum number is 1. Default: `8`
`min_sensitivity`	`<f>`	Sets the sensitivity of only primary parameters, which must be greater than `min_sensitivity`. Default: `0.001`
`targets`	`<t>`	Specifies a list of targets to be included. Default: `None`

Returns

<t>: List of primary parameter objects only

Examples

```
cal.determine_primary_parameters_ml(parameters=parameters,
                                    num_parameters=5)
```

evaluate

This command performs a simple evaluation to calculate errors, without calibration. See Simple Evaluation on page 36.

Syntax

```
evaluate(parameters=<t>, targets=<t>, [parent=<s>,] [child_dir=<s>,]
         [extra_tool_parameters=<s|d>])
```

Syntax for pmccal Module

```
evaluate(parameters=<t>, targets=<t>, [parent=<s>,] [child_dir=<s>,]
         [extra_tool_parameters=<s|d>,] [reevaluate=<b>])
```

Argument	Type	Description
parameters	<t>	Specifies a list of calibration parameter objects.
targets	<t>	Specifies a list of calibration target objects.
parent	<s>	Specifies the global parent project. If set to None, then option("parent") is used.
child_dir	<s>	Specifies the directory of the child projects. If set to None, then option("child_dir") is used.
extra_tool_parameters	<s|d>	Specifies extra tool parameters to be added to the child project. If set to None, then option("extra_tool_parameters") is applied. If still set to None (default), then no parameters are added. If it specifies a string, then all parameters are added to this tool. If it specifies a dictionary, with tool labels as keys and lists of parameter objects as values, then parameters are added to individual tools. Default: None

Argument	Type	Description
`reevaluate`	`<b>`	(For `pmccal` module only) Specifies whether to reevaluate by rerunning the simulation due to variability. Default: `True`

Returns

None

Examples

```
cal.evaluate(parameters=[p1, p2, p3], targets=[t1, t2, t3])

cal.evaluate(parameters=[p1, p2, p3], targets=[t1, t2, t3]
             parent="../parent/example")
```

explore_har_etch_ml

This command explores a machine-learning model, specific to the `pmccal` calibration module, for Sentaurus Topography high aspect ratio etching (see HARC Etching Calibration on page 117).

Syntax

```
explore_har_etch_ml(parameters=<t>, targets=<t>,
                    [parent=<s>,] [child_dir=<s>,]
                    [task_name=<s>,] [tolerance=<f>,]
                    [num_iterations=<i>,]
                    [num_parameters=<i>,]
                    [explore_doe_range=<f|i>,]
                    [explore_doe_method=<k>,]
                    [explore_doe_size=<i>,]
                    [explore_doe=<s|p>,]
                    [explore_doe_sep=<s>,]
                    [sensitivity=<b|s|d>,]
                    [constraint=<s>,]
                    [weight_factor=<f>,]
                    [priority_targets=<t>,]
                    [num_retries=<i>,]
                    [num_evaluations=<i>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the machine-learning task. Default: `"ML_explore"`
`tolerance`	`<f>`	Sets the total relative RMS error tolerance. Default: `0.12`
`num_iterations`	`<i>`	Specifies how many exploration steps are performed. Default: `1`
`num_parameters`	`<i>`	Sets the number of primary parameters used for machine learning. If `num_parameters=0`, then all defined parameters are assumed to be primary parameters. Default: `0`

Argument	Type	Description
`explore_doe_range`	`<f|s>`	Sets the parameter range scaling used for the DoE. If a file name is provided, then it sets the name of a CSV file that specifies the parameter ranges for the DoE. Default: `None`
`explore_doe_method`	`<k>`	Sets the DoE method. Options are `"box-behnken"`, `"full-factorial"`, `"gaussian"`, `"latin-hypercube"`, `"uniform"`, `"sobol"`, and `"custom"`.
`explore_doe_size`	`<i>`	Sets the number of samples for the DoE in the case of the DoE methods `"gaussian"`, `"latin-hypercube"`, `"uniform"`, and `"sobol"`. Default: `None`
`explore_doe`	`<s|p>`	Sets the DoE of the CSV file path or pandas data frame when `explore_doe_method` is `"custom"`. Default: `None`
`explore_doe_sep`	`<s>`	Specifies the delimiter to read the DoE. Default: `','`
`sensitivity`	`<b|s|d>`	Specifies the type of sensitivity analysis to be performed: • If `True`, then a sensitivity analysis is performed. • If a task name is provided, then the argument specifies the existing task to be used for sensitivity. • If a pandas data frame is provided, then the argument specifies the sensitivity information to be used. Default: `None`
`constraint`	`<s>`	Specifies an optional constraint or query for an extraction variable of simulations to be included in the training dataset. Default: `""`
`weight_factor`	`<f>`	Sets the weight factor for the `BottomZ` and `RemainingHMZ` targets. Default: `2`
`priority_targets`	`<t>`	Specifies a list of priority targets in order of priority, starting with the highest priority. Default: `None`
`num_retries`	`<i>`	Sets the number of simulation run retries if there is a simulation runtime failure. Default: `1`
`num_evaluations`	`<i>`	Sets the number of initial and final evaluations. Default: `5`

Returns

None

Examples

```
pmccal.explore_har_etch_ml(parameters=parameters, targets=targets,
                           tolerance=0.1, explore_doe_range=0.5,
                           num_parameters=10)
```

explore_ml

This command explores a machine-learning model.

Syntax

```
explore_ml(parameters=<t>, targets=<t>,
           [parent=<s>,] [child_dir=<s>,]
           [task_name=<s>,] [tolerance=<f>,]
           [num_iterations=<i>,]
           [num_parameters=<i>,]
           [explore_doe_range=<f|i>,]
           [explore_doe_method=<k>,]
           [explore_doe_size=<i>,]
           [explore_doe=<s|p>,]
           [explore_doe_sep=<s>,]
           [sensitivity=<b|s|d>,]
           [constraint=<s>,]
           [priority_targets=<t>,]
           [num_retries=<i>,]
           [num_evaluations=<i>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the machine-learning task. Default: `"ML_explore"`
`tolerance`	`<f>`	Sets the total relative RMS error tolerance. Default: `0.12`

Argument	Type	Description
`num_iterations`	`<i>`	Specifies how many exploration steps are performed. Default: `1`
`num_parameters`	`<i>`	Sets the number of primary parameters used for machine learning. If `num_parameters=0`, then all defined parameters are assumed to be primary parameters. Default: `0`
`explore_doe_range`	`<f|s>`	Sets the parameter range scaling used for the DoE. If a file name is provided, then it sets the name of a CSV file that specifies the parameter ranges for the DoE. Default: `None`
`explore_doe_method`	`<k>`	Sets the DoE method. Options are `"box-behnken"`, `"full-factorial"`, `"gaussian"`, `"latin-hypercube"`, `"uniform"`, `"sobol"`, and `"custom"`. Default: `None`
`explore_doe_size`	`<i>`	Sets the number of samples for the DoE in the case of the DoE methods `"gaussian"`, `"latin-hypercube"`, `"uniform"`, and `"sobol"`. Default: `None`
`explore_doe`	`<s|p>`	Sets the DoE of the CSV file path or pandas data frame when `explore_doe_method` is `"custom"`. Default: `None`
`explore_doe_sep`	`<s>`	Specifies the delimiter to read the DoE. Default: `','`
`sensitivity`	`<b|s|d>`	Specifies the type of sensitivity analysis to be performed: • If `True`, then a sensitivity analysis is performed. • If a task name is provided, then the argument specifies the existing task to be used for sensitivity. • If a pandas data frame is provided, then the argument specifies the sensitivity information to be used. Default: `None`
`constraint`	`<s>`	Specifies an optional constraint or query for an extraction variable of simulations to be included in the training dataset. Default: `""`
`priority_targets`	`<t>`	Specifies a list of priority targets in order of priority, starting with the highest priority. Default: `None`
`num_retries`	`<i>`	Sets the number of simulation run retries if there is a simulation runtime failure. Default: `1`
`num_evaluations`	`<i>`	Sets the number of initial and final evaluations. Default: `1`

Returns

None

Examples

```
cal.explore_ml(parameters=parameters, targets=targets, tolerance=0.1,
               num_parameters=10, num_iterations=5)
```

generate_data_ml

This command creates projects and runs simulations that can be used to validate a reverse search analysis, specific to the `mlcal` calibration module (see Machine Learning Calibration Module: mlcal on page 52).

Syntax

```
generate_data_ml(parameters=<t>, targets=<t>,
                 [parent=<s>,] [child_dir=<s>,]
                 [task_name=<s>,] [doe=<s|p>,]
                 [doe_method=<k>,] [doe_size=<i|t|d>,]
                 [data_generation=<d>,]
                 [num_retries=<i>,] [save_report=<b>,]
                 [update_parameters=<b>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of machine-learning model parameter objects.
`targets`	`<t>`	Specifies a list of machine-learning model target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the machine-learning task. Default: `"ML_generate"`
`doe`	`<s|p>`	Sets the CSV file path or a pandas data frame. Default: `None`

Argument	Type	Description
`doe_method`	`<k>`	Sets the DoE method. Options are `"adaptive"`, `"box-behnken"`, `"full-factorial"`, `"gaussian"`, `"grid"`, `"latin-hypercube"`, `"uniform"`, and `"sobol"`.
`doe_size`	`<i|t|d>`	Sets the number of samples for the DoE in the case of the DoE methods `"latin-hypercube"`, `"uniform"`, and `"sobol"`. Sets the number of samples per parameter in the case of the DoE method `"grid"`, which is either the same for all in the case of an integer or per parameter in the case of a list or dictionary.
`data_generation`	`<d>`	Specifies data generation information (default: `None`). Dictionary keys are: • `doe_method=<k>`: Sets the DoE method. Options are `"adaptive"`, `"box-behnken"`, `"full-factorial"`, `"gaussian"`, `"grid"`, `"latin-hypercube"`, `"uniform"`, and `"sobol"`. • `doe_size=<i|t|d>`: Sets the number of samples for the DoE in the case of the DoE methods `"latin-hypercube"`, `"uniform"`, and `"sobol"`. Sets the number of samples per parameter in the case of the DoE method `"grid"`, which is either the same for all in the case of an integer or per parameter in the case of a list or dictionary. • `random_seed=<i>`: Sets the random seed for the DoE in the case of the DoE methods `"latin-hypercube"`, `"uniform"`, and `"sobol"`. • `use_mean=<b>`: Specifies whether the nominal value is the mean of the minimum and maximum values. If `False`, then the nominal value is the user-specified value. Default: `True` • `include_nominal=<b>`: If `True`, then the nominal value is included. Default: `False` • `batch_size=<i>`: Used only for adaptive generation. Sets the number of data points generated before each update on the parameter filter. Default: `1` • `pass_rate=<f>`: Used only for adaptive generation. Defines the required passing rate of the filter. Default: `1.0` • `threshold_num_pass=<i>`: Used only for adaptive generation. Sets the threshold number of passing cases to invoke the parameter filter, before which the filter is all-passing. Default: `1` • `initial_sampling_doe=<s>`: Used only for adaptive generation. The initial few batches can use a different DoE method other than `"uniform"`. Default: `"uniform"`

Argument	Type	Description
		• `initial_doe_batches=<i>`: Used only for adaptive generation. Specifies how many batches can use the different DoE method, before switching to uniform randomization. If it is negative, then `initial_sampling_doe` is used throughout the data generation session. Default: `-1` • `transformer=<m>`: Used only for adaptive generation. Sets the transformer model to use. Default: `None` • `classifier=<m>`: Used only for adaptive generation. Sets the classifier model to use. Default: `None` • `unadapted_size=<i>`: Sets the number of unadapted data points (batch size) generated before each update on the parameter filter, where 0 <= `unadapted_size` < `batch_size`. Default: `0`
`num_retries`	`<i>`	Sets the number of simulation run retries if there is a simulation runtime failure. Default: `1`
`save_report`	`<b>`	Specifies whether to save a report in the `report` folder. Default: `True`
`update_parameters`	`<b>`	Specifies whether the parameter values are updated after running the task. Default: `False`

Returns

None

Examples

```
mlcal.generate_data_ml(parameters=parameters, targets=targets,
                       data_generation={'doe_method': 'grid',
                                        'include_nominal': False,
                                        'doe_size': 3})
```

generate_pairplot

This command as an epilogue generates a correlation pairplot with parameter or target data for commands specific to the `mlcal` calibration module (see Machine Learning Calibration Module: mlcal on page 52). It conducts three representative analyses.

First, the distribution characteristics of the sample and the presence of outliers are checked through a histogram of the parameter or target data.

In addition, through the scatter plot, you can check the shape of the target space and check the correlation between the two paired variables through regression fitting. To this end, the initial fitting model can be checked through options such as `fit_order` and `confidence_interval`.

On the other hand, if multiwindow DoE is used, labels can be given for each DoE group and samples can be distinguished by color. Regression is performed for each DoE group.

Only for scalar target data, Mahalanobis distance data is added to make you aware of the distance between samples considering covariance in the target space. Outliers observed in the histogram of this distance data mean that it is likely less trained in training. The formula is given by:

$$d_M = \sqrt{(x-\mu)\sum^{-1}(x-\mu)^T}$$

where *x* is the vector of the single observation (row in a dataset), μ is the mean vector of all samples, and ∑ is the covariance matrix of targets.

Syntax

```
generate_pairplot(targets=<t>, [parameters=<t>,]
                  [combine_plots=<b>,][fit_regression=<b>,]
                  [fit_order=<i>,][confidence_interval=<i>,]
                  [outfile_prefix=<s>,][outfile_suffix=<s>])
```

Argument	Type	Description
`targets`	`<t>`	Specifies a list of targets.
`parameters`	`<t>`	Specifies a list of parameters.
`combine_plots`	`<b>`	If `False`, separate pairplots for parameters and targets are generated, respectively. If `True`, a combined pairplot is generated additionally. Default: `False`
`fit_regression`	`<b>`	Specifies whether to estimate and to plot a regression model relating the x and y variables. Default: `True`
`fit_order`	`<i>`	Order of regression fitting. Default: `1`

Argument	Type	Description
confidence_interval	<i>	Specifies an integer between 0 and 100 confidence intervals for regression estimate. If None, it is not displayed. Default: None
outfile_prefix	<s>	Specifies the prefix of the output image file. Default: "n@node@_pairplot"
outfile_suffix	<s>	Specifies the suffix of the output image file. Default: ""

Returns

None

Examples

```
mlcal.generate_pairplot(parameters=parameters, targets=targets)
mlcal.generate_pairplot(parameters=parameters, targets=targets,
          combine_plots=True, fit_regression=True, fit_order=2,
          confidence_interval=90, outfile_prefix="pairplot",
          outfile_suffix="Sobol")
```

get_rms_error

This command returns the current RMS error for a specific target or for a weighted total.

Syntax

```
get_rms_error([target=<s>,] [iteration=<i>])
```

Argument	Type	Description
target	<s>	Specifies the name of a target. If set to None, then the total weighted RMS error is returned. Default: None
iteration	<i>	Specifies the iteration number for which the RMS error is returned. The default, iteration=0, means the optimum iteration with minimum RMS error.

Returns

<f>: RMS error

Examples

```
cal.get_rms_error()
```

```
cal.get_rms_error(target="t_BottomZ")
```

get_sensitivity

This command returns the current pandas data frame including the parameter sensitivities.

Syntax

```
get_sensitivity()
```

Returns

<p>: pandas data frame including the parameter sensitivities

get_step_signature

This command returns the current calibration step or task signature.

Syntax

```
get_step_signature()
```

Returns

<i>: Calibration step signature or step number

infer_ml_models

This command sets up the machine-learning model inference.

Syntax

```
infer_ml_models(child_dir=<s>, [task_name=<s>,]
                [doe=<s|d|p>,] [doe_size=<i>,]
                [doe_type=<s>,] [doe_levels=<i>,]
                [doe_sep=<s>,] [file_prefix=<s>])
```

Argument	Type	Description
`child_dir`	`<s>`	Sets the directory of the child projects.
`task_name`	`<s>`	Specifies the name of a task from which the models used for inference are loaded. Default: `"ML_Calibrate"`
`doe`	`<s|d|p>`	Sets the parameters range that overrides the model defaults. The value can be a string specifying the file name to use, a dictionary, or a pandas data frame. Default: `None`
`doe_size`	`<i>`	Sets the number of rows in the generated DoE for the inference. Default: `10000`
`doe_type`	`<s>`	Sets the type of DoE. Options are: • `"uniform"`: Random uniform distribution • `"box-behnken"` • `"centered_uniform"` • `"custom"` • `"full-factorial"`: Full-factorial • `"nominal"` • `"OFAT"`: One-factor-at-a-time Default: `"uniform"`
`doe_levels`	`<i>`	Sets the parameter levels used in a full-factorial DoE (`doe_type="factorial"`). It defines the levels used for each model feature (two levels mean `min` and `max`). Default: `2`
`doe_sep`	`<s>`	Specifies the separator to be used when creating files. Default: `None`
`file_prefix`	`<s>`	Sets the prefix to be used in the created file names. Default: `""`

Returns

None

Examples

```
cal.infer_ml_models(child_dir="../path/to/child/directory",
                    task_name="ML_explore", doe_type="nominal")
```

load_parameters

This command reads in a CSV file including current parameter values and attributes written by `save_parameters` (see save_parameters on page 278), updates the corresponding parameter objects accordingly, and returns a pandas data frame.

Syntax

```
load_parameters(parameters=<t>, file=<s>, [deliminator=<s>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`file`	`<s>`	Specifies a path of a CSV file where the parameter data is included.
`deliminator`	`<s>`	Specifies the deliminator. Default: `','`

Returns

`<p>`: pandas data frame including the parameters

Examples

```
cal.load_parameters(parameters=[p1, p2, p3], file='./parameters.csv')
```

merge_tables

This epilogue command merges two Legendre polynomial implantation model (L-PIM) tables in the `procal` calibration module (see Legendre Polynomial Implantation Model Calibration on page 109).

Syntax

```
merge_tables(table=<s>, table_damage=<s>, [table_2=<s>,]
             [table_damage_2=<s>,] [table_dir=<s>,]
             [regularize=<b>,] [interpolator=<k>]
```

Argument	Type	Description
`table`	`<s>`	Specifies the base L-PIM dopant table.
`table_damage`	`<s>`	Specifies the base L-PIM damage table.
`table_2`	`<s>`	Specifies the added L-PIM dopant table. If `None`, the table from the previous task is taken. Default: `None`
`table_damage_2`	`<s>`	Specifies the added L-PIM damage table. If `None`, table from the previous task is taken. Default: `None`
`table_dir`	`<s>`	Specifies the directory of output L-PIM tables. Default: `"./PIM"`
`regularize`	`<b>`	Specifies whether to regularize the grid table. If `True`, add virtual points for regular grid table. If `False`, keep unstructured grid table. Default: `True`
`interpolator`	`<k>`	Specifies the Interpolator: `"RGLI"` for Regular Grid Linear Interpolator (default), `"RBF"` for Radial Basis Function

Returns

None

Examples

```
procal.merge_tables(table='../PIM/nitrogen_in_siliconcarbide.pim',
      table_damage='../PIM/nitrogen_damage_in_siliconcarbide.pim')
```

Model

This class represents a machine learning model.

Syntax

```
ml_0 = Model(name=<s>, hyperparameters=<d>, framework=<k>,
             conditions=<d>, load=<b>, validate=<b>)
```

Argument	Type	Description
`name`	`<s>`	The name of the model. This is the name used during automatic file saving and loading.
`hyperparameters`	`<d>`	Sets the hyperparameters for the `Model` object. For the Mx1 framework, the dictionary requires entries for each target name (for example, `{"Target 1": …, "Target 2": …, }`). For the MxN framework, use "All" (for example, `{"All": …}`) All models have the following possible hyperparameter entries: • `model_type <k>`: Specifies the machine learning model type • `model_hyperparameters <d>`: Specifies the hyperparameters for the machine learning model selected in `model_type`. These hyperparameters are unique to each model. • `parameter_scaler <k>`: Specifies the scaling to apply to input parameters • `target_scaler <k>`: Specifies the scaling to apply to output targets • `feature_interaction_type <k>`: Specifies the type of feature interaction terms to be considered • `feature_interaction_params <d>`: Specifies the feature interaction settings • `feature_selection_type <k>`: Specifies the type of feature selection algorithm to use • `feature_selection_parameters <d>`: Specifies the feature selection settings • `retrain <b>`: Specifies whether to retrain the model every time `train_ml` is called • `encoder_type <k>`: Specifies the type of encoder to use for profile targets • `encoder_params <d>`: Specifies the profile encoder settings • `optimizer_type <k>`: Specifies the optimizer to use when optimizing `model_hyperparameters` • `optimizer_params <d>`: Specifies the hyperparameter optimizer settings

Argument	Type	Description
		The specifics for each entry can be quite detailed and depend on the specifics of each model. See Model Hyperparameters on page 54 for more details on how to set up a hyperparameter dictionary for your specific case. Default: `None`
`framework`	`<k>`	Sets which model framework to use. Options are Mx1 or MxN. High dimensional targets can be used only with the Mx1 framework. Default: Mx1
`condition`	`<d>`	Specifies conditional parameter of the model. Default: None.
`load`	`<b>`	Specifies whether to load an existing model or not. If `True`, it searches `<name>_untrained.ml` or `<name>_trained.ml` files under the project directory while creating an instance. Default: `False`
`validate`	`<b>`	Specifies whether to validate attributes of the `Model` while loading an existing model. Default: `True`

Returns

An object of type `Model`

Examples

```
ml_0 = Model(name="ml_0", hyperparameters={"Y0": {"model_type": MLPR,
             "model_hyperparameters": {"hidden_layer_sizes": [[64, 64,
             64], [128,128,128]], "alpha": [0.1, 0.3, 1.0], "max_iter':
             [5000]}, "optimizer_type": GridSearch, "optimizer_params":
             {"cv": 10}}}, framework=Mx1, validate=True, load=False)

ml_1 = Model(name="ml_1_test", hyperparameters={"profile_1":
             {"model_type": MLPR, "model_hyperparameters":
             {"hidden_layer_sizes": [256, 256, 256], "alpha": 1.0},
             "parameter_scaler": MinMax, "target_scaler": Identity,
             "encoder_type": PointSamplingEncoder, "encoder_params":
             {"num_data_points": 100, "use_floating_range": True}}},
             framework=Mx1, conditions={"cond1": "cond_value"}
             validate=False, load=True)
```

optimize

This command performs the gradient-based local calibration task.

Syntax

```
optimize(parameters=<t>, targets=<t>, [parent=<s>,] [child_dir=<s>,]
         [max_iteration=<i>])
```

Syntax for pmccal Module

```
optimize(parameters=<t>, targets=<t>, [parent=<s>,] [child_dir=<s>,]
         [max_iteration=<i>,] [reevaluate=<b>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`max_iteration`	`<i>`	Sets a limit to the maximum number of iterations for the calibration. Default: `None`, which specifies to set to `max_iteration` in `set_option()`
`reevaluate`	`<b>`	(For `pmccal` module only) Specifies whether to reevaluate by rerunning the simulation due to variability. Default: `True`

Returns

None

Examples

```
cal.optimize(parameters=[p1, p2, p3], targets=[t1, t2, t3])
```

```
cal.optimize(parameters=parameters, targets=targets, max_iteration=25)
```

Parameter

This class represents a model parameter to be calibrated.

General Syntax

```
Parameter(name=<s>, value=<f>, minimum=<f>, maximum=<f>,
          [transform=<o>,] [choices=<t>,] [sort=<b>,] [type=<k>,]
          [dtype=<k>,] [precision_rel=<i>,]
          [precision_abs=<i>,] [force_pass=<b>])
```

Argument	Type	Description
`name`	`<s>`	Specifies the name of a calibration parameter. This must be the name of a Sentaurus Workbench parameter to be calibrated.
`value`	`<f>`	Specifies the initial value or the default of the parameter.
`minimum`	`<f>`	Specifies the minimum-allowed value for the parameter.
`maximum`	`<f>`	Specifies the maximum-allowed value for the parameter.
`transform`	`<o>`	Specifies the transform object for the parameter value. The transformed value is `"transform(value)"`. The predefined transformers are `Linear`, `Log10`, `Log`, `Square`, `Neg`, and `Sqrt` (see Table 2 on page 23).
`choices`	`<t>`	Specifies the candidate values for a discrete parameter.
`sort`	`<b>`	Specifies whether to sort the discrete parameter values before encoding. Default: `False`
`type`	`<k>`	Specifies an optional type for parameter classification. Available types for the `igbtcal` module are `IcVg`, `IcVc`, `BV`, `CV`, `Qg`, and `Switching`.
`dtype`	`<k>`	Specifies the Python datatype of the parameter. Options are `bool`, `float`, `int`, or `str`. Default: `float`
`precision_rel`	`<i>`	Specifies the relative precision given as a number of significant digits. Default: `8`
`precision_abs`	`<i>`	Specifies the absolute precision given as a number of digits after the decimal point (or before the decimal point if it is negative).
`force_pass`	`<b>`	(Discrete parameters only) Specifies whether samples are forced to pass all the values given by `choices`. Default: `False`

Returns

<o>: Parameter object

Examples

```
Parameter(name="p1", value=1.0, minimum=-10, maximum=10)

Parameter(name="p2", value=1e18, minimum=1e15, maximum=1e21,
          transform=Log10)

Parameter(name="p1", value=2.0, minimum=1.5, maximum=2.5, type=IcVg)

Parameter(name="p2", value=1.0, minimum=1E-4, maximum=1E4,
          transform=Log10, type=IcVc)
```

Parent

This class represents a parent project to be used for calibration.

Syntax

```
Parent(name=<s>, path=<s|o>, tools=<t>, [experiment=<i>,]
      [files_to_copy=<t>,] [extra_tool_parameters=<s|d>])
```

Argument	Type	Description
`name`	`<s>`	Specifies the name of a parent project.
`path`	`<s|o>`	Specifies the path (as string or Path object) to the Sentaurus Workbench project.
`tools`	`<t>`	Specifies the list of tools in the parent project tool flow.
`experiment`	`<i>`	Sets the number of the nominal experiment of the parent project. Default: `1`
`files_to_copy`	`<t>`	Specifies the Sentaurus Workbench tool steps to be included in the calibration. Default: `None`
`extra_tool_parameters`	`<s|d>`	Specifies the extra tool parameters to be added to the child project. If set to `None` (default), then no parameters are added. If it specifies a string, then all parameters are added to this tool. If it specifies a dictionary, with tool labels as keys and lists of parameter objects as values, then parameters are added to individual tools. Default: `None`

Returns

`<o>`: Parent object

Examples

```
Parent(name = "myparent", path = "../parent/myparent",
      tools = ['sprocess'], files_to_copy = ['./calib.fps'],
      extra_tool_parameters = 'sprocess')
```

plot_data

As a task epilogue, this command acts as the plot function for `mlcal` tasks. The supported plot types are `curve`, `hist`, `line`, `parallel_coordinates`, `scatter`, and `surface` plots. The generated graphs are based on two different backends, `mpl` (for matplotlib), and `plotly`. The mpl produces static images, while the plotly generates interactive graphs, which allow user interaction.

After completion of an `mlcal` task, the graph is generated using the internal data frame containing parameters and targets data and helps to analyze the results.

Syntax

```
plot_data(targets=<t>, parameters=<t>, x=<s|t>, y=<s|t>, [z=<s>,]
          [plot_type=<k>,][backend=<k>,][width=<i>,][height=<i>,]
          [y_axis=<k>,][reference_data=<t>,][reference_data=<t>,]
          [outfile_prefix=<s>,][outfile_suffix=<s>,][**kwargs=<d>])
```

Argument	Type	Description
`targets`	`<t>`	Specifies the list of targets.
`parameters`	`<t>`	Specifies the list of parameters.
`x`	`<s|t>`	Specifies the *x* variable or the list of *x* variables.
`y`	`<s|t>`	Specifies the *y* variable or the list of *y* variables.
`z`	`<s>`	Specifies the *z* variable. Default: `None`
`plot_type`	`<k>`	Selects the type of visualization. Options are: • `"curve"`: Curve plot for profile target. • `"hist"`: Histogram plot for single variable. • `"line"`: Line plot with *x* and *y* variables. • `"parallel_coordinates"`: Parallel coordinate plot for more than two variables only for `"plotly"` backend. • `"scatter"`: Scatter plot with *x* and *y* variables. • `"surface"`: Surface plot with *x*, *y* and *z* variables. Default: `"scatter"`.
`backend`	`<k>`	Selects the backend between `"plotly"` and `"mpl"` (matplotlib). Default: `"plotly"`
`width`	`<i>`	Figure width. Default: `800`
`height`	`<i>`	Figure height. Default: `600`

Argument	Type	Description
`y_axis`	`<s>`	Y-axis setting. Options are: • `"dual"`: Dual y-axes plot. • `"multi"`: Subplots with rows (number of *y* variables) and cols (number of *x* variables). • `"None"`: Single y-axis plot (Default).
`reference_data`	`<t>`	Data to compare. Default: `None`.
`reference_label`	`<t>`	Label for reference data. Default: `None`.
`outfile_prefix`	`<s>`	The prefix of output image file. Default: `"n@node@_@step@"`
`outfile_suffix`	`<s>`	The suffix of output image file. Default: `""`
`**kwargs`	`<d>`	The plot options for each `plot_type`. Default: `None`.

Returns

None

Examples

```
mlcal.plot_data(targets=targets, parameters=parameters,
                x="PWELL", y=["Ron_at200A", "Eoff_mJ],
                plot_type="scatter", backend="plotly",
                width=800, height=600, y_axis="dual",
                reference_data="retrieval.csv",
                reference_label="retrieval",
                outfile_prefix="MLoptimization")
```

plot_error_history

This command generates a plot PNG image with RMS error history after gradient-based local calibration.

Syntax

```
plot_error_history([file=<s>,] [targets=<t>,] [verbose=<b>])
```

Argument	Type	Description
`file`	`<s>`	Sets the name of an output PNG file to be saved. Default: `"error.png"`
`targets`	`<t>`	Specifies a list of target objects for the error history plot. If set to `None`, then only the total error history is plotted. Default: `None`
`verbose`	`<b>`	Sets the verbosity level. Default: `False`

Returns

None

Examples

```
cal.plot_error_history(file="n@node@_error.png")
```

plot_sensitivity

This command generates a bar plot PNG image with parameter sensitivities after a sensitivity analysis.

Syntax

```
plot_sensitivity([file=<s>,] [sensitivity=<p>,] [measure=<s>,]
                 [verbose=<b>])
```

Argument	Type	Description
`file`	`<s>`	Sets the name of an output PNG file to be saved. Default: `"sensitivity.png"`
`sensitivity`	`<p>`	Specifies a pandas data frame including the parameter sensitivities. If set to `None`, then the latest sensitivity is used. Default: `None`
`measure`	`<s>`	Specifies the sensitivity measure as follows: • For local sensitivity analysis, specify `'absolute'` or `'relative'` (default). • For global sensitivity analysis, specify `'mu'`, `'mu_star'` (default), or `'sigma'`.
`verbose`	`<b>`	Sets the verbosity level. Default: `False`

Returns

None

Examples

```
cal.plot_sensitivity(file="n@node@_sensitivity.png")
```

predict_ml

This command returns the prediction from one or more trained machine-learning models, specific to the `mlcal` calibration module (see Machine Learning Calibration Module: mlcal on page 52).

Syntax

```
predict_ml(models=<t>, parameters=<t>, targets=<t>,
           [parent=<s>,] [child_dir=<s>,]
           [task_name=<s>,] [doe_method=<k>,]
           [doe_size=<i|t|d>,] [doe=<s|p>,]
           [check_boundary=<b>,] [check_adaptive=<b>,]
           [save_report=<b>,] [save_report_figures=<b>])
```

Argument	Type	Description
`models`	`<t>`	Specifies a list of trained machine-learning model objects.
`parameters`	`<t>`	Specifies a list of machine-learning model parameter objects.
`targets`	`<t>`	Specifies a list of machine-learning model target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the machine-learning task. Default: `"ML_predict"`
`doe_method`	`<k>`	Sets the DoE method. Options are `"box-behnken"`, `"full-factorial"`, `"gaussian"`, `"grid"`, `"latin-hypercube"`, `"sobol"`, `"uniform"` , `"model_train_data"`, `"model_validate_data"`, and `"model_data"`. Default: `"model_data"`
`doe_size`	`<i|t|d>`	Sets the number of samples for the DoE in the case of the DoE methods `"latin-hypercube"`, `"uniform"`, and `"sobol"`. Sets the number of samples per parameter in the case of the DoE method `"grid"`, which is either the same for all in the case of an integer or per parameter in the case of a list or dictionary.

Argument	Type	Description
`doe`	`<s|p>`	Sets the CSV file path or a pandas data frame. Default: `None`
`check_boundary`	`<b>`	If set to `True`, when out-of-bounds parameters are given, the command raises an error. If set to `False`, then the command issues a warning. Default: `True`
`check_adaptive`	`<b>`	If set to `True`, when the parameter filter prediction fails for a parameter set, the command raises an error. If set to `False`, then the command issues a warning. Default: `False`
`save_report`	`<b>`	Specifies whether to save a report in the `report` folder. Default: `True`
`save_report_figures`	`<b>`	Specifies whether to save figures of high-dimensional targets in the `report` folder. The argument `save_report` must be set to `True` as well. Default: `False`

Returns

None

Examples

```
mlcal.predict_ml(models=models, parameters=parameters,
                 targets=targets)

mlcal.predict_ml(
     models=models, parameters=parameters, targets=targets,
     task_name='ML_predict', doe='user_input.csv',
     check_boundary=True, check_adaptive=True,
     save_report=True, save_report_figures=True)
```

print_parameters

This command logs the calibrated parameters for Sentaurus Workbench variable extraction.

Syntax

```
print_parameters(parameters=<t>, [suffix=<s>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of parameter objects.
`suffix`	`<s>`	Specifies a suffix to append to the parameter name for the Sentaurus Workbench variable extraction. Default: `'_cal'`

Returns

None

Examples

```
cal.print_parameters(parameters=parameters)
```

print_targets

This command logs the calibrated targets for Sentaurus Workbench variable extraction.

Syntax

```
print_targets(targets=<t>, [suffix=<s>,] [include_path=<b>])
```

Argument	Type	Description
`targets`	`<t>`	Specifies a list of target objects.
`suffix`	`<s>`	Specifies a suffix to append to the target name for the Sentaurus Workbench variable extraction. Default: `'_cal'`
`include_path`	`<b>`	Specifies whether to include the path for targets linked to files. If set to `True`, then the path is included for targets linked to files.

Returns

None

Examples

```
cal.print_targets(targets=targets, include_path=True)
```

refine_har_etch_ml

This command refines a machine-learning model, specific to the `pmccal` calibration module, for Sentaurus Topography etching (see Machine-Learning Model Refinement on page 122).

Syntax

```
refine_har_etch_ml(parameters=<t>, targets=<t>,
                   [parent=<s>,] [child_dir=<s>,]
                   [task_name=<s>,] [tolerance=<f>,]
                   [num_iterations=<i>,]
                   [num_parameters=<i>,]
                   [sensitivity=<b|s|d>,]
                   [constraint=<s>,]
                   [weight_factor=<f>,]
                   [num_retries=<i>,]
                   [num_evaluations=<i>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the machine-learning task. Default: `"ML_refine"`
`tolerance`	`<f>`	Sets the total relative RMS error tolerance. Default: `0.12`
`num_iterations`	`<i>`	Specifies how many refinement steps are performed. Default: `1`
`num_parameters`	`<i>`	Sets the number of primary parameters used for machine learning. If `num_parameters=0`, then all defined parameters are assumed to be primary parameters. Default: `0`
`sensitivity`	`<b|s|d>`	Specifies the type of sensitivity analysis to be performed: • If `True`, then a sensitivity analysis is performed. • If a task name is provided, then the argument specifies the existing task to be used for sensitivity. • If a pandas data frame is provided, then the argument specifies the sensitivity information to be used. Default: `None`

Argument	Type	Description
`constraint`	`<s>`	Specifies an optional constraint or query for an extraction variable of simulations to be included in the training dataset. Default: `""`
`weight_factor`	`<f>`	Sets the weight factor for the `BottomZ` and `RemainingHMZ` targets. Default: `2`
`num_retries`	`<i>`	Sets the number of simulation run retries if there is a simulation runtime failure. Default: `1`
`num_evaluations`	`<i>`	Sets the number of initial and final evaluations. Default: `5`

Returns

None

Examples

```
pmccal.refine_har_etch_ml(parameters=parameters, targets=targets,
                          tolerance=0.1, num_parameters=10)
```

refine_ml

This command refines a machine-learning model.

Syntax

```
refine_ml(parameters=<t>, targets=<t>,
          [parent=<s>,] [child_dir=<s>,]
          [task_name=<s>,] [tolerance=<f>,]
          [num_iterations=<i>,]
          [num_parameters=<i>,]
          [sensitivity=<b|s|d>,]
          [constraint=<s>,]
          [num_retries=<i>,]
          [num_evaluations=<i>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the machine-learning task. Default: `"ML_refine"`
`tolerance`	`<f>`	Sets the total relative RMS error tolerance. Default: `0.12`
`num_iterations`	`<i>`	Specifies how many refinement steps are performed. Default: `1`
`num_parameters`	`<i>`	Sets the number of primary parameters used for machine learning. If `num_parameters=0`, then all defined parameters are assumed to be primary parameters. Default: `0`
`sensitivity`	`<b|s|d>`	Specifies the type of sensitivity analysis to be performed: • If `True`, then a sensitivity analysis is performed. • If a task name is provided, then the argument specifies the existing task to be used for sensitivity. • If a pandas data frame is provided, then the argument specifies the sensitivity information to be used. Default: `None`
`constraint`	`<s>`	Specifies an optional constraint or query for an extraction variable of simulations to be included in the training dataset. Default: `""`

Argument	Type	Description
num_retries	<i>	Sets the number of simulation run retries if there is a simulation runtime failure. Default: 1
num_evaluations	<i>	Sets the number of initial and final evaluations. Default: 1

Returns

None

Examples

```
cal.refine_ml(parameters=parameters, targets=targets, tolerance=0.1,
              num_parameters=10, num_iterations=5)
```

reverse_search_ml

This command performs a reverse search analysis, specific to the mlcal calibration module (see Machine Learning Calibration Module: mlcal on page 52).

Syntax

```
reverse_search_ml(models=<t>, parameters=<t>, targets=<t>,
                  [parent=<s>,] [child_dir=<s>,]
                  [task_name=<s>,] [num_candidates=<i>,]
                  [doe_method=<k>,] [doe_size=<i|t|d>,] [doe=<s|p>,]
                  [range=<s|p>,] [cluster=<s>,] [optimize=<b>,]
                  [enable_parameter_filter=<b>,]
                  [extract_search_result=<b>,]
                  [return_output=<b>,]
                  [save_report=<b>,] [priority_targets=<t>,]
                  [update_parameters=<s>,] [reset_results=<b>,]
                  [args_nsgaIII=<d>])
```

Argument	Type	Description
models	<t>	Specifies a list of machine-learning models. If more than one model is provided, then reverse search is combined.
parameters	<t>	Specifies a list of machine-learning model parameter objects.
targets	<t>	Specifies a list of machine-learning model target objects.
parent	<s>	Specifies the global parent project. If set to None, then option("parent") is used.

Argument	Type	Description
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the machine-learning task. Default: `"ML_reverse"`
`num_candidates`	`<i>`	Sets the number of search results to store. Default: `1`
`doe_method`	`<k>`	Sets the DoE method. Options are `"adaptive"`, `"box_behnken"`, `"full_factorial"`, `"gaussian"`, `"grid"`, `"latin_hypercube"`, `"sobol"`, `"uniform"`, `"NSGAIII_S"` (non-dominated sorting genetic algorithm for multi-modal optimization), and `"NSGAIII_M"` (NSGAIII for multi-objective optimization). Default: `"uniform"`.
`doe_size`	`<i|t|d>`	Sets the number of samples for the DoE in the case of the DoE methods `"gaussian"`, `"latin-hypercube"`, `"uniform"`, and `"sobol"`. Sets the number of samples per parameter in the case of the DoE method `"grid"`, which is either the same for all in the case of an integer or per parameter in the case of a list or dictionary. Default: 1000
`doe`	`<s|p>`	Specifies the path of the DoE CSV file or pandas data frame. The generated DoE is updated using columns from the specified `doe`. Default: `None`
`range`	`<s|p>`	Specifies the path of the DoE CSV file or pandas data frame. Parameter ranges are updated based on the provided range. Default: `None`
`cluster`	`<s>`	Applies K-Means clustering. • `"optimal_number"`: Finds optimal cluster number by using silhouette score and elbow method. • `"fixed_number"`: Clusters the search results into `"num_candidates"` number of clusters. • `"off"`: Turns off clustering. Default: `"off"`
`optimize`	`<b>`	Specifies whether to perform a gradient-based optimization after candidates are found. Certain DoE methods, `"adaptive"`, `"NSGAIII_S"`, and `"NSGAIII_M"`, do not support optimization. Default: `False`

Argument	Type	Description
`enable_parameter_filter`	`<b>`	Specifies whether the generated entries pass through the parameter filter. Default: `False`
`extract_search_result`	`<b>`	Specifies whether to print the extraction strings, like those for Sentaurus Workbench, for the predicted search result. Default: `False`
`return_output`	`<b>`	Specifies whether to return the predicted output. Default: `False`
`save_report`	`<b>`	Specifies whether to save a report in the `report` folder. Default: `True`
`priority_targets`	`<t>`	Specifies a list of priority targets in order of priority, starting with the highest priority. Default: `None`
`update_parameters`	`<s>`	Specifies which values are updated. Options are: • `"range"` means both the parameter value and the range are updated based on reverse search results. • `"value"` means only the parameter value is updated based on reverse search results. Default: `None`
`reset_results`	`<b>`	Specifies whether to not load results from the previous search. If set to `True`, then results from the previous search are *not* loaded. Default: `False`

Argument	Type	Description
`args_nsgaIII`	`<d>`	Arguments for NSGAIII_S(M). Available keys are: • `"pop_size" <i>`: Population size • `"n_points" <i>`: Number of points for reference direction • `"xtol" <f>`: Tolerance of design space • `"ftol" <f>`: Tolerance of objective space • `"period" <i>`: Period to compare the tolerances • `"n_max_gen" <i>`: Maximum number of generations • `"n_max_evals" <i>`: Maximum number of evaluations • `"eps_neighbor_count" <f>`: Epsilon for neighbor count for NSGAIII_S • `"abs_tolerance" <f>`: Absolute tolerance to find multiple minima for NSGAIII_S • `"rel_tolerance" <f>`: Relative tolerance to find multiple minima for NSGAIII_M • `"mutation_prob" <f>`: Polynomial mutation probability • `"mutation_eta" <f>`: Distribution parameter of Polynomial mutation • `"crossover_prob" <f>`: Simulated binary crossover probability • `"crossover_eta" <f>`: Distribution parameter of simulated binary crossover

Returns

None

Examples

```
mlcal.reverse_search_ml(models=models, parameters=parameters,
                        targets=targets)

mlcal.reverse_search_ml(
   models=models,
   parameters=parameters,
   targets=targets,
   num_candidates=10,
   doe_method='sobol',
   doe_size=10000,
   optimize=False,
   extract_search_result=True,
   return_output=True,
   save_report=True,
   update_parameters='value'
)
```

run_sensitivity_analysis

This command computes local or global parameter sensitivities.

Syntax

```
run_sensitivity_analysis(parameters=<t>, targets=<t>,
                         [parent=<s>,] [child_dir=<s>,]
                         [method=<s>,] [percentage_range=<f>,]
                         [num_points=<i>,] [direction=<s>,]
                         [scale_sensitivity=<b>,]
                         [num_levels=<i>,] [num_trajectories=<i>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`method`	`<s>`	Specifies the type of sensitivity analysis to perform, either `"local"` (default) or `"global"`.
`percentage_range`	`<f>`	Specifies the range of the domain to be used for the local sensitivity analysis. Default: `None`, taking `sensdx` from `set_backend_control()`
`num_points`	`<i>`	Specifies the number of values per parameter to simulate within the given range for the local sensitivity analysis. Default: `3`
`direction`	`<s>`	Specifies the direction for the increment of each parameter for the local sensitivity analysis. Options are `"both"`, `"forward"`, and `"backward"`. Default: `"both"`
`scale_sensitivity`	`<b>`	Specifies whether to calculate the sensitivity measures in the scaled or transformed space. Default: `False`
`num_levels`	`<i>`	Specifies the number of sampling levels per parameter for the global sensitivity analysis.
`num_trajectories`	`<i>`	Specifies the number of different trajectories used for the global sensitivity analysis.

Returns

None

Examples

```
cal.run_sensitivity_analysis(parameters=[p1, p2, p3],
                             targets=[t1, t2, t3])

cal.run_sensitivity_analysis(parameters=parameters, targets=targets,
                             percentage_range=0.2)

cal.run_sensitivity_analysis(parameters=parameters, targets=targets,
                             method="global", num_levels=5,
                             num_trajectories=10)
```

run_sensitivity_analysis_ml

This command computes parameter sensitivity.

Syntax

```
run_sensitivity_analysis_ml(parameters=<t>, targets=<t>,
                            [parent=<s>,] [child_dir=<s>,]
                            [task_name=<s>,] [sensitivity=<s>,]
                            [constraint=<s>,]
                            [num_retries=<i>,]
                            [num_evaluations=<i>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the task. Default: `"ML_sensitivity"`
`sensitivity`	`<s>`	Specifies the existing task to be used for sensitivity. Default: `None`
`constraint`	`<s>`	Specifies an optional constraint or query for an extraction variable of simulations to be included in the training dataset. Default: `""`

Argument	Type	Description
`num_retries`	`<i>`	Sets the number of simulation run retries if there is a simulation runtime failure. Default: `1`
`num_evaluations`	`<i>`	Sets the number of initial evaluations. Default: `1`

Returns

None

Examples

```
cal.run_sensitivity_analysis_ml(parameters=[p1, p2, p3],
                                targets=[t1, t2, t3])
```

save_doe

This command saves the parameter DoE and corresponding target responses of previous tasks to a CSV file.

Syntax

```
save_doe([file=<s>])
```

Argument	Type	Description
`file`	`<s>`	Specifies a path of a CSV file where the DoE data is saved. Default: `'./n@node@_doe.csv'`

Returns

None

Examples

```
cal.save_doe('./doe.csv')
```

save_parameters

This command saves the current parameter values and attributes to a CSV file and returns a pandas data frame.

Syntax

```
save_parameters([parameters=<t>,] [file=<s>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`file`	`<s>`	Specifies a path of a CSV file where the parameter data is saved. Default: `'./n@node@_parameters.csv'`

Returns

`<p>`: pandas data frame including the parameters

Examples

```
cal.save_parameters(parameters=[p1, p2, p3], file='./parameters.csv')
```

save_sensitivity

This command saves the sensitivity of parameters to a CSV file.

Syntax

```
save_sensitivity([file=<s>])
```

Argument	Type	Description
`file`	`<s>`	Specifies a path of a csv file where sensitivity results are saved. Default: `'./n@node@_sensitivity.csv'`

Returns

None

Examples

```
cal.save_sensitivity('./sensitivity.csv')
```

search

This command performs an evaluation of different designs-of-experiments (DoE) to explore the parameter space.

Syntax

```
search(parameters=<t>, targets=<t>, method=<s>,
       [options=<d>,] [parent=<s>,] [child_dir=<s>,]
       [include_points=<t>,] [include_nominal=<b>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`method`	`<s>`	Sets the method used to perform the search (see Design-of-Experiments Search on page 39). Options are: • `"bayesian_optimizer"` creates experiments adaptively, starting from an initial set of experiments defined by Latin hypercube sampling. In each iteration, the algorithm selects some parameter sets that have the greatest potential to minimize the response. • `"grid"` creates experiments with parameter values defined on a regular grid. • `"latin"` creates experiments with parameter values using Latin hypercube sampling. • `"uniform"` creates experiments with random parameter values. • `"sobol"` creates experiments with parameter values using a Sobol sequence. • `"turbo"` creates experiments adaptively and builds on top of Bayesian optimization by keeping separate models that can change the focus of their search as the algorithm progresses. This makes it more likely to converge to a better local minimum. • The name of a Sentaurus Workbench DoE creates experiments according to the given Sentaurus Workbench DoE. • "`MOEAD`", "`NSGAII`", and "`NSGAIII`" create experiments adaptively and are multi-objective evolutionary algorithms, which use evolutionary methods to solve the optimization problem. For details about these algorithms, see *Sentaurus™ Workbench Optimization Framework User Guide*, Multi-Objective Optimization. Some of these methods have additional options that can be specified using the `options` argument. For details about these options, see *Sentaurus™ Workbench Optimization Framework User Guide*, Searching the Parameter Space.

Argument	Type	Description
`options`	`<d>`	Specifies a dictionary of specific options for the search method.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`include_points`	`<t>`	Specifies a list of dictionaries defining points to be included in the search. Default: `None`
`include_nominal`	`<b>`	Specifies whether to include the current nominal in the search. Default: `True`

Returns

None

Examples

```
cal.search(parameters=[p1, p2], targets=[t1, t2], method="grid",
           options={"Ns": (2, 2), "midpoint": True, "boundary": True})

cal.search(parameters=parameters, targets=targets, method="sobol",
           options={"Ns": 10})

cal.search(parameters=parameters, targets=targets,
           method="bayesian_optimizer",
           options={"Ns": 5, "max_iter": 3})

cal.search(parameters=parameters, targets=targets,
           method="turbo",
           options={"Ns": 15, "boundary": True, "trust_regions": 3,
           "batch_size": 5, "max_iter": 15, "disp": 1})

cal.search(parameters=parameters, targets=targets,
           method="NSGAII",
           options={"termination": {"n_gen": 50}, "pop_size": 40,
           "n_offsprings": 10, "disp": True})
```

set_backend_control

This command defines the control parameters for the calibration backend.

Syntax

```
set_backend_control([initdx=<f>,] [sensdx=<f>,]
                    [mindx=<f>,] [maxdx=<f>,]
                    [ssq_error_tolerance=<f>,]
                    [parameter_tolerance=<f>,]
                    [target_tolerance=<f>,] [jacobian_options=<d>,]
                    [method=<s>,] [options=<d>])
```

Argument	Type	Description
`initdx`	`<f>`	Sets the initial increment of the normalized value for the calibration by the `scalib` backend. Default: `4e-2`
`sensdx`	`<f>`	Sets the increment of the normalized value for sensitivity analysis. Default: `0.1`
`mindx`	`<f>`	Sets the minimum increment of the normalized value for the Jacobian calculation by the `scalib` backend. Default: `1e-7`
`maxdx`	`<f>`	Sets the maximum increment of the normalized value for the Jacobian calculation by the `scalib` backend. Default: `1.0`
`ssq_error_tolerance`	`<f>`	Sets the tolerance of the normalized update of the sum of squares (ssq) error by the `scalib` backend. Default: `1e-5`
`parameter_tolerance`	`<f>`	Sets the tolerance of the normalized parameter update by the `scalib` backend, or sets the absolute tolerance for the backends `"least_squares"`, `"minimize"`, and `"mkl_nls"`. Default: `2e-3`
`target_tolerance`	`<f>`	Sets the tolerance of the total target error. Default: `0.0`
`jacobian_options`	`<d>`	Specifies a dictionary of keyword arguments passed to the Jacobian evaluation for backends other than the `scalib` backend. For details, see *Sentaurus™ Workbench Optimization Framework User Guide*, Method-Specific Optimization Options.
`method`	`<s>`	Sets the calibration method used by backends other than the `scalib` backend. For details, see *Sentaurus™ Workbench Optimization Framework User Guide*, Method-Specific Optimization Options.

Argument	Type	Description
`options`	`<d>`	Specifies a dictionary of backend-specific options for backends other than the `scalib` backend. For details, see *Sentaurus™ Workbench Optimization Framework User Guide*, Method-Specific Optimization Options.

Returns

None

Examples

```
cal.set_backend_control(initdx=0.005)

cal.set_backend_control(sensdx=0.1, maxdx=1.0)

cal.set_backend_control(ssq_error_tolerance=1e-4)

cal.set_backend_control(parameter_tolerance=0.01)

cal.set_backend_control(method="trf",
   options={'max_nfev': 50, 'ftol': 1e-2, 'gtol': 1e-2, 'xtol': 1e-2})
```

set_backend_method

This command defines the calibration backend. It is only needed for backends other than the `scalib` backend, which is the default.

Syntax

```
set_backend_method(name=<s>)
```

Argument	Type	Description
`name`	`<s>`	Specifies the name of the calibration backend to be used. Options are: • `"scalib"` (default) • `"least_squares"` • `"minimize"` • `"mkl_nls"` • `"shgo"`

Returns

None

Examples

```
cal.set_backend_method(name="minimize")
```

```
cal.set_backend_method("minimize")
```

set_option

This command sets user-specified option values. The options listed here are general ones available for all calibration modules.

Note:

The `set_option()` command must be written using the format `<argument>=<value>`. This command does not accept value-only input.

Syntax

```
set_option([attempt_failed=<b>,]
           [check_child_project=<b>,] [child_dir=<s>,]
           [extra_tool_parameters=<s|d>,] [files_to_copy=<t>,]
           [max_iteration=<i>,] [ml_num_points_of_encoder=<i>,]
           [num_threads=<i>,] [parent=<s>,]
           [parent_experiment=<i>,] [preprocess_new_project=<b>,]
           [queue=<s>,] [run_limits=<d>,] [runtime_limits=<d>,]
           [scheduler=<s>,] [split_child=<b>,]
           [split_child_max_experiments=<i>,]
           [submit_bridge_tool=<b>,] [submit_virtual=<b>,]
           [tools=<t>,] [use_scalib_weights=<b>])
```

Argument	Type	Description
`attempt_failed`	`<b>`	Specifies whether to re-submit failed and terminated nodes (for example, in restarts). Default: `True`
`check_child_project`	`<b>`	Specifies whether to check the child project consistency by running an initial experiment after creating the child project. Default: `False`
`child_dir`	`<s>`	Specifies the child project directory. Default: `"../child/"`
`error_precision`	`<i>`	Specifies the internal error precision. Default: `8`
`extra_tool_parameters`	`<s|d>`	Specifies the extra tool parameters to be added to the child project. If set to `None` (default), then no parameters are added. If it specifies a string, then all parameters are added to this tool. If it specifies a dictionary, with tool labels as keys and lists of parameter objects as values, then parameters are added to individual tools. Default: `None`

Argument	Type	Description
`files_to_copy`	`<t>`	Specifies a list of files and folders to be copied from the parent project to the child projects. Default: `None`
`max_iteration`	`<i>`	Sets the maximum number of iterations of the calibration. Default: `50`
`ml_num_points_of_encoder`	`<i>`	Sets the number of points of the encoder of the profile target, which is used in machine learning–based commands. By default, the value is either 100 or the number of points of a reference profile, whichever is the smallest. Default: `None`
`num_threads`	`<i>`	Sets the number of threads. The value cannot be greater than the maximum number of threads. Default: `None`
`parent`	`<s>`	Specifies the global parent project. Default: `None`
`parent_experiment`	`<i>`	Sets the number of the nominal experiment of the parent project. Default: `None`
`preprocess_new_project`	`<b>`	Specifies whether to force preprocessing of the child project at the start. Default: `False`
`queue`	`<s>`	Sets the queue name for executing the child projects.
`run_limits`	`<d>`	Sets the optional run limit per tool by a dictionary, with tool names as keys and number of concurrent runs as value. Effective only if global run limits are set for Sentaurus Workbench. Default: `None`
`runtime_limits`	`<d>`	Sets the optional runtime limit per tool by a dictionary, with tool names as keys and runtime limit in hours as value. If runtime is exceeded, the tool run is terminated. Effective only if global run limits are set for Sentaurus Workbench. Default: `None`
`scheduler`	`<s>`	Sets the scheduler command. Default: `"gsub"`
`split_child`	`<b>`	Specifies whether to enable child project splitting by number. Default: `False`
`split_child_max_experiments`	`<i>`	Sets the maximum number of experiments per child project in case of `split_child` being `True`. Default: `250`

Argument	Type	Description
`submit_bridge_tool`	`<b>`	Specifies whether to submit the bridge tool node to the queue. Default: `True`
`submit_virtual`	`<b>`	Specifies whether to include virtual nodes in the list of nodes passed to `gsub`. Default: `False`
`tools`	`<t>`	Specifies the Sentaurus Workbench tool steps to be included in the calibration. Default: `None`
`use_scalib_weights`	`<b>`	Specifies whether to use the total weighted RMS error based on the `scalib` target weights. See Calculating the Total Weighted Root Mean Square Error on page 35. Default: `True`

Returns

None

Examples

```
cal.set_option(max_iteration=50)

cal.set_option(files_to_copy=['*.fps', 'sdevice.par'])

cal.set_option(extra_tool_parameters='sprocess')

cal.set_option(extra_tool_parameters={'sprocess': [p_a, p_b],
                                      'sdevice': [p_c]})

cal.set_option(runtime_limits={'sprocess': 1, 'sdevice': 0.5})
```

set_step_signature

This command changes the calibration step signature.

Syntax

```
set_step_signature(signature=<s>)
```

Argument	Type	Description
`signature`	`<s>`	Sets the calibration step signature.

Returns

None

Examples

```
cal.set_step_signature(signature="1.0")
```

Target

This class represents a model target against which it is calibrated. The target can be a scalar or an xy profile.

Syntax for Scalar Targets

```
Target(name=<s>, value=<f>,
       [id=<s>,] [type=<k>,] [dtype=<k>,] [conditions=<d>,]
       [weight=<f>,] [scale=<f>,] [penalty=<o>,] [range=<t>,]
       [abs_value_range=<t>,] [values=<t>,]
       [dimension=<i>,] [error_method=<s>,] [parent=<o>])
```

Argument	Type	Description
`name`	`<s>`	Specifies the name of a Sentaurus Workbench parameter or variable in which the simulation response will be available (as a scalar value).
`value`	`<f>`	Specifies the value to be matched by the simulation response. It must be a single scalar value. Default: `0.0`
`id`	`<s>`	Specifies a unique name or an ID of the target.
`type`	`<k>`	Specifies the type for target classification. Available types for the `igbtcal` module are `IcVg`, `IcVc`, `BV`, `CV`, `Qg`, and `Switching`.
`dtype`	`<k>`	Specifies the datatype of the target. Options are `bool`, `float`, or `int`.
`conditions`	`<d>`	Specifies the condition or split parameters that must be set to calculate the calibration target.
`weight`	`<f>`	Specifies the weight to be applied to the calibration target. Default: `1.0`
`scale`	`<f>`	Sets the scaling factor used to normalize the target value, with *value_scaled* = *value* / `scale`. Default: `1.0`
`penalty`	`<o>`	Specifies the penalty to be applied to the target in case of violation of the constraints. It must be of type `TargetPenalty` (see TargetPenalty on page 296). Default: `None`
`range`	`<t>`	Sets the constraint range [low, high] into which the passing target value falls. One of the bound can be `None`.
`abs_value_range`	`<t>`	Sets the range [low, high] into which the absolute value of the passing target falls. One of the bound can be `None`.

Argument	Type	Description
`values`	`<t>`	Specifies a list of passing discrete target values.
`dimension`	`<i>`	Specifies the dimension of a high-dimensional (> 2) target.
`error_method`	`<s>`	Specifies the method used to calculate the pointwise error between the simulation response and the reference value. Options are: • `"abs"`: Absolute error: $e_i = \left\|out_i - target_i\right\|$ • `"log_abs"`: Absolute logarithmic error: $e_i = \left\|\log_{10}\left\|out_i\right\| - \log_{10}\left\|target_i\right\|\right\|$ • `"log_rel"`: Relative logarithmic error: $e_i = \left\|\frac{\log_{10}\left\|out_i\right\| - \log_{10}\left\|target_i\right\|}{\log_{10}\left\|target_i\right\|}\right\|$ • `"rel"`: Relative error: $e_i = \left\|\frac{out_i - target_i}{target_i}\right\|$ • `"signed"`: Signed error: $e_i = out_i - target_i$ • `"signed_relative"`: Signed relative error: $e_i = \frac{out_i - target_i}{target_i}$ • `"square"`: Squared error: $e_i = (out_i - target_i)^2$ The default applies as follows: • `"abs"` for `type=IcVc` or `type=Qg` or `type=Switching` • `"log_abs"` for `type=IcVg` or `type=CV` • `"rel"` for `type=None` or `type=BV`
`parent`	`<o>`	Specifies the parent project of the target. If `None`, the global parent specified in the options is used. Default: `None`

Syntax for XY Profile Targets

```
Target(name=<s>, value=<s>,
       [id=<s>,] [type=<s>,] [dtype=<k>,] [conditions=<d>,]
       [weight=<f>,]
       [simulation_variables=<t>,] [reference_variables=<t>,]
       [simulation_options=<d>,] [reference_options=<d>,]
       [x_range=<t>,] [y_range=<t>,] [x_scale=<f>,] [y_scale=<f>,]
       [error_method=<s>,] [interpolation_options=<d>,]
```

```
        [duplicates=<s>])
```

Argument	Type	Description
`name`	`<s>`	Specifies the name of a Sentaurus Workbench parameter or variable in which the simulation response is available (as a file name).
`value`	`<s>`	Specifies the value to be matched by the simulation response. It must be a valid path to a file. The supported file formats are CSV, PLT, and PLX. The data must be sorted.
`id`	`<s>`	Specifies a unique name or an ID of the target.
`type`	`<s>`	Specifies the type for target classification.
`dtype`	`<k>`	Specifies the datatype of the target. For files, the only option is `str`.
`conditions`	`<d>`	Specifies the condition or split parameters that must be set to calculate the calibration target.
`weight`	`<f>`	Specifies the weight to be applied to the calibration target. Default: `1.0`
`simulation_variables`	`<t>`	Specifies a tuple with the names of the independent (x) and dependent (y) variables (in that order) in the simulation file.
`reference_variables`	`<t>`	Specifies a tuple with the names of the independent (x) and dependent (y) variables (in that order) in the reference file.
`simulation_options`	`<d>`	Sets additional options needed to read the values of the simulation file.
`reference_options`	`<d>`	Sets additional options needed to read the values of the reference file.
`x_range`	`<t>`	Sets the range of interest for the independent variable (x) of the reference target xy profile.
`y_range`	`<t>`	Sets the range of interest for the dependent variable (y) of the reference target xy profile.
`x_scale`	`<f>`	Sets the scaling factor to multiply by the independent variable (x) of the reference target xy profile, with *value_scaled* = *value* × `x_scale`.

Argument	Type	Description
`y_scale`	`<f>`	Sets the scaling factor to multiply by the dependent variable (y) of the reference target xy profile, with *value_scaled* = *value* × `y_scale`.
`error_method`	`<s>`	Specifies the method used to calculate the pointwise error between the simulation response and the reference value. Options are: • `"abs"`: Absolute error: $e_i = \left\|out_i - target_i\right\|$ • `"log_abs"`: Absolute logarithmic error: $e_i = \left\|\log_{10}\left\|out_i\right\| - \log_{10}\left\|target_i\right\|\right\|$ • `"log_rel"`: Relative logarithmic error: $e_i = \left\|\frac{\log_{10}\left\|out_i\right\| - \log_{10}\left\|target_i\right\|}{\log_{10}\left\|target_i\right\|}\right\|$ • `"rel"`: Relative error: $e_i = \left\|\frac{out_i - target_i}{target_i}\right\|$ • `"signed"`: Signed error: $e_i = out_i - target_i$ • `"signed_relative"`: Signed relative error: $e_i = \frac{out_i - target_i}{target_i}$ • `"square"`: Squared error: $e_i = (out_i - target_i)^2$ The default applies as follows: • `"abs"` for `type=IcVc` or `type=Qg` or `type=Switching` • `"log_abs"` for `type=IcVg` or `type=CV` • `"rel"` for `type=None` or `type=BV`
`interpolation_options`	`<d>`	Sets the interpolation options for the simulation data. If simulation data and reference values are in a file but evaluated at different points, then the simulation data is interpolated onto the locations of the target values using the following options: • `"clip"` limits oscillations resulting from interpolation [min, max]. • `"extra"` limits the value taken by the data when extrapolation is necessary [min, max]. • `"log"` determines whether to use logarithmic interpolation (default: `False`). • `"method"` determines the order of the interpolating spline (default: `1`).

Argument	Type	Description
`duplicates`	`<s>`	Specifies how to treat duplicated independent variable (x) values with different dependent variable (y) values. Options are: • `None` raises an error if there are duplicates. • `"all"` includes every occurrence of each x-value with the same weight as unique ones. • `"average"` includes the average of the target values for each of the duplicated x-values. • `"first"` includes only the first occurrence of each x-value. • `"last"` includes only the last occurrence of each x-value. Default: `None`
`parent`	`<o>`	Specifies the parent project of the target. If `None`, the global parent specified in the options is used. Default: `None`

Returns

<o>: Target object

Examples

```
Target(name='Error', value=0)

Target(name='t2', id='t_t2', value=10.0, weight=2.0)

Target(name='BottomZ', value=-3.3, conditions={'time': '10'})

Target(name='BTotalFile', id='t_B_SIMS', value='/path/b_sims.plx',
       simulation_variables=['', 'BTotal'],
       error_method="log_abs", x_range=[0.005, 0.7])

Target(name='IdVgFile', id='t_IdVgLinPlt',
     value='/path/reference_VdLin_nMOS.plt',
     conditions={'Domain': 'NMOS', 'IdVg': 'Lin'},
     simulation_variables=['gate OuterVoltage', 'drain TotalCurrent'],
     reference_variables=['gate OuterVoltage', 'drain TotalCurrent'],
     error_method="log_abs", x_range=[0.0, 1.3],
     interpolation_options={"method": 3})

Target(name='XYBowingCD', id='t_XYBowingCD',
       value='/path/reference_30.csv',
       conditions={'time': '30'},
       simulation_variables=['Z', 'Radius_Avg'],
       reference_variables=['Z', 'Radius_Avg'],
       error_method="rel", y_range=[1e-10, 1e30])
```

```
Target(name='sIcVc', id="tIcVc", value="IcVc_reference.plt",
       conditions={"IcVg": "0",'IcVc': "1", "BV":"0"},
       reference_variables=["x", "y"],
       simulation_variables=["collector OuterVoltage",
                             "collector TotalCurrent"],
       x_range=[1.0, 3.0],
       type=IcVc)
```

TargetDelta

This class represents a delta target, which can be used as calibration target for the difference (the delta) between two scalar targets.

Syntax

```
TargetPenalty(id=<s>, value=<f>, targets=<t>, [weight=<f>])
```

Argument	Type	Description
`id`	`<s>`	Specifies a unique name or an ID of the target.
`value`	`<f>`	Specifies the value of the difference to be matched by the simulation response. It must be a single scalar value.
`targets`	`<t>`	Specifies a list of two targets to be used for the delta calculation.
`weight`	`<f>`	Specifies the weight to be applied to the calibration target. Default: `1.0`

Returns

`<o>`: `TargetDelta` object

Examples

```
t1_BottomZ = Target(name="BottomZ", value=2.0,
                    conditions={"time": "10"})

t2_BottomZ = Target(name="BottomZ", value=3.0,
                    conditions={"time": "20"})

t_BottomZ_Delta = TargetDelta(id="t_BottomZ_Delta", value=1.0,
                              targets= [t2_BottomZ, t1_BottomZ],
                              weight=2.0)
```

TargetFunction

This class represents a function target.

Syntax

```
TargetFunction(id=<s>, value=<f>, targets=<t>, transform=<c>,
               [weight=<f>,] [error_method=<s>])
```

Argument	Type	Description
`id`	`<s>`	Specifies a unique name or an ID of the target.
`value`	`<f>`	Specifies the value to be matched by the simulation response. It must be a single scalar value. Default: `0.0`
`targets`	`<t>`	Specifies a list of targets to be used during the post-evaluation.
`transform`	`<c>`	Specifies a callable transform function to be applied to the targets.
`weight`	`<f>`	Specifies the weight to be applied to the calibration target. Default: `1.0`
`error_method`	`<s>`	Specifies the method used to calculate the pointwise error between the simulation response and the reference value. Options are: • `"abs"`: Absolute error: $e_i = \left\|out_i - target_i\right\|$ • `"log_abs"`: Absolute logarithmic error: $e_i = \left\|\log_{10}\left\|out_i\right\| - \log_{10}\left\|target_i\right\|\right\|$ • `"log_rel"`: Relative logarithmic error: $e_i = \left\|\frac{\log_{10}\left\|out_i\right\| - \log_{10}\left\|target_i\right\|}{\log_{10}\left\|target_i\right\|}\right\|$ • `"rel"`: Relative error: $e_i = \left\|\frac{out_i - target_i}{target_i}\right\|$ • `"signed"`: Signed error: $e_i = out_i - target_i$ • `"signed_relative"`: Signed relative error: $e_i = \frac{out_i - target_i}{target_i}$ • `"square"`: Squared error: $e_i = (out_i - target_i)^2$ Default: `"rel"`

Returns

`<o>`: `TargetFunction` object

Examples

```
t1_BottomZ = Target(name="BottomZ", value=2.0,
                    conditions={"time": "10"})

t2_BottomZ = Target(name="BottomZ", value=3.0,
                    conditions={"time": "20"})

t3_BottomZ = Target(name="BottomZ", value=4.0,
                    conditions={"time": "30"})

def calculate_distance(targets):
   a = targets[0]
   b = targets[1]
   distance = a.eval - b.eval
   return distance

t1_BottomZ_Eval = TargetFunction(id="t1_BottomZ_Eval", value=1.0,
                                 targets=[t2_BottomZ, t1_BottomZ],
                                 transform=calculate_distance,
                                 weight=1.0)

t2_BottomZ_Eval = TargetFunction(id="t2_BottomZ_Eval", value=2.0,
                                 targets=[t3_BottomZ, t1_BottomZ],
                                 transform=calculate_distance,
                                 weight=2.0)

t3_BottomZ_Eval = TargetFunction(id="t3_BottomZ_Eval", value=1.0,
                                 targets=[t3_BottomZ, t2_BottomZ],
                                 transform=calculate_distance,
                                 weight=1.0)
```

TargetPenalty

This class represents a function which can be used for the penalty argument of Target objects (see Target on page 288). If the constraint defined in `TargetPenalty` is not satisfied, then a penalty is applied to the target.

Syntax

```
TargetPenalty(filters=<t>, penalty=<c>, [ignore_value=<b>])
```

Argument	Type	Description
`filters`	`<t>`	Specifies a list of tuples defining the feasible region.
`penalty`	`<c>`	Specifies a callable penalty function that calculates a penalty, given the difference between the target and the threshold of the constraint.
`ignore_value`	`<b>`	Specifies whether to ignore the actual value of the target, keeping only the penalty. Default: `False`

Returns

<o>: `TargetPenalty` object

Examples

```
def quadratic_penalty(delta):
    return delta**2
TargetPenalty([("lt", 1.0), ("gt", 0.5)], quadratic_penalty)

TargetPenalty([("lt", 1.0)], quadratic_penalty)
```

Targetr2

This class represents an r^2 target. r^2 is the square of the Pearson correlation coefficient (r), which is a linear correlation between the reference values and the evaluated values. (It is not a coefficient of determination.) For details, see r^2 Targets on page 26.

Syntax

```
Targetr2(id=<s>, value=<f>, targets=<t>, [weight=<f>])
```

Argument	Type	Description
`id`	`<s>`	Specifies a unique name or an ID of the target.
`value`	`<f>`	Specifies the value to be matched by the simulation response. It must be a single scalar value.
`targets`	`<t>`	Specifies a list of targets to be used during the r^2 calculation.
`weight`	`<f>`	Specifies the weight to be applied to the calibration target. Default: `1.0`

Returns

`<o>`: `Targetr2` object

Examples

```
t1_BottomZ = Target(name="BottomZ", value=2.0,
                    conditions={"time": "10"})

t2_BottomZ = Target(name="BottomZ", value=3.0,
                    conditions={"time": "20"})

t3_BottomZ = Target(name="BottomZ", value=4.0,
                    conditions={"time": "30"})

t_BottomZ_r2 = Targetr2(id="t_BottomZ_r2", value=1.0,
                    targets = [t1_BottomZ, t2_BottomZ, t3_BottomZ],
                    weight=1.0)
```

TargetR2

This class represents an R^2 (coefficient of determination) target. For details, see R^2 Targets on page 27.

Syntax

```
TargetR2(id=<s>, value=<f>, targets=<t>, [weight=<f>])
```

Argument	Type	Description
`id`	`<s>`	Specifies a unique name or an ID of the target.
`value`	`<f>`	Specifies the value to be matched by the simulation response. It must be a single scalar value.
`targets`	`<t>`	Specifies a list of targets to be used during the R^2 calculation.
`weight`	`<f>`	Specifies the weight to be applied to the calibration target. Default: `1.0`

Returns

`<o>`: `TargetR2` object

Examples

```
t1_BottomZ = Target(name="BottomZ", value=2.0,
                    conditions={"time": "10"})

t2_BottomZ = Target(name="BottomZ", value=3.0,
                    conditions={"time": "20"})

t3_BottomZ = Target(name="BottomZ", value=4.0,
                    conditions={"time": "30"})

t_BottomZ_R2 = TargetR2(id="t_BottomZ_R2", value=1.0,
                        targets= [t1_BottomZ, t2_BottomZ, t3_BottomZ],
                        weight=1.0)
```

train_ml

This command trains one or more untrained machine-learning models, specific to the `mlcal` calibration module (see Machine Learning Calibration Module: mlcal on page 52).

Syntax

```
train_ml(models=<t>, parameters=<t>, targets=<t>,
         [parent=<s>,] [child_dir=<s>,]
         [task_name=<s>,] [save_report=<b>,]
         [retrain=<b>,] [get_cv_result=<b>,] [data_set=<s>])
```

Argument	Type	Description
`models`	`<t>`	Specifies a list of untrained machine-learning model objects.
`parameters`	`<t>`	Specifies a list of machine-learning model parameter objects. A parameter defined in the machine-learning model during creation and not defined in this list is excluded from the model during training.
`targets`	`<t>`	Specifies a list of machine-learning model target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the name of the machine-learning task. Default: `"ML_train"`
`save_report`	`<b>`	Specifies whether to save a report in the `report` folder. Default: `True`
`retrain`	`<b>`	Specifies whether to discard existing training results to retrain a new model if applicable. Default: `False`
`get_cv_result`	`<b>`	Specifies whether to save output images from cross-references. Default: `False`

Argument	Type	Description
`data_set`	`<s>`	Specifies the subset of data on which the model is trained. Options are: • `"full"`: Entire dataset • `"passed"`: Subset that passes the target criteria before data generation • `"passedtrimmed"`: Subset that passes both target criteria and data trimming criteria • `"trimmed"`: Subset that passes the data trimming criteria after data generation Default: `"full"`

Returns

None

Examples

```
mlcal.train_ml(models=models, parameters=parameters,
               targets=targets)

mlcal.train_ml(
   models=models,
   parameters=parameters,
   targets=targets,
   task_name='ML_train',
   save_report=True,
   retrain=True,
   get_cv_result=True,
   data_set='passedtrimmed',
)
```

validate_range_ml

This command validates the parameter ranges and adjusts them for subsequent tasks.

Syntax

```
validate_range_ml(parameters=<t>, targets=<t>,
                  [parent=<s>,] [child_dir=<s>,]
                  [task_name=<s>,]
                  [num_validations=<i>,]
                  [reducing_ratio=<f>,]
                  [constraint=<s>,]
                  [num_retries=<i>,]
                  [num_evaluations=<i>])
```

Argument	Type	Description
`parameters`	`<t>`	Specifies a list of calibration parameter objects.
`targets`	`<t>`	Specifies a list of calibration target objects.
`parent`	`<s>`	Specifies the global parent project. If set to `None`, then `option("parent")` is used.
`child_dir`	`<s>`	Specifies the directory of the child projects. If set to `None`, then `option("child_dir")` is used.
`task_name`	`<s>`	Sets the task name. Default: `"ML_validate_range"`
`num_validations`	`<i>`	Sets the number of validations to adjust the range. Default: `13`
`reducing_ratio`	`<f>`	Specifies the ratio used to reduce the parameter range. Default: `0.2`
`constraint`	`<s>`	Specifies an optional constraint or query for an extraction variable of simulations to be included in the training dataset. Default: `""`
`num_retries`	`<i>`	Sets the number of simulation run retries if there is a simulation runtime failure. Default: `1`
`num_evaluations`	`<i>`	Sets the number of initial evaluations. Default: `1`

Returns

None

Examples

```
cal.validate_range_ml(parameters=parameters, targets=targets)
```